HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

HỌC PHẦN PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH

BÁO CÁO TIỂU LUẬN MÔN HỌC

ĐỀ TÀI: ĐỒ THỊ TRI THỨC (KNOWLEDGE GRAPH) VÀ RAG, TRIỂN KHAI TÁC TỬ THÔNG MINH (AI AGENT) CHO Y TẾ VÀ TÀI CHÍNH VIỆT NAM

Lớp:	D23CTPM01
Nhóm:	Nhóm lớp CT – Nhóm 07
Họ và tên:	Nguyễn Duy Nghĩa
Mã sinh viên:	B23DCCN600
Giảng viên hướng dẫn:	PGS.TS Trần Đình Quế
Học kỳ:	Học kỳ 1 năm học 2026 – 2027

Hà Nội, Tháng 10/2026

Mục lục

Danh mục Hình vẽ **v**

Danh mục Bảng biểu **xi**

Danh mục Đoạn mã nguồn **xiii**

Tổng quan báo cáo **1**

1 Mở đầu về Knowledge Graph và RAG **4**
1.1 Khái niệm đồ thị tri thức và RAG 4
1.1.1 Khái niệm đồ thị tri thức (Knowledge Graph, KG) . . . 5
1.1.2 Khái niệm sinh tăng cường truy xuất (RAG) 6
1.1.3 Quá trình phát triển 7
1.2 Vai trò và ý nghĩa của Knowledge Base và RAG 7
1.3 Bài toán và cách tiếp cận của tiểu luận 9

2 Kiến trúc Knowledge Graph và cơ sở tri thức **10**
2.1 Các thành phần cấu tạo của Knowledge Graph 10
2.2 Mô hình biểu diễn bộ ba RDF 15
2.3 Bản thể học và biểu diễn tri thức hình thức 17
2.3.1 Suy diễn tiến trên RDF 19
2.3.2 Truy vấn SPARQL 21
2.3.3 Kiểm tra ràng buộc 23
2.4 Xây KB và KG từ dữ liệu bảng 24
2.5 So sánh chi tiết: Graph Database và Vector Database 26

3 Các thế hệ kiến trúc Retrieval-Augmented Generation **29**
3.1 Naive RAG và các điểm nghẽn 29
3.1.1 Quy trình hoạt động của Naive RAG 30
3.1.2 Các điểm nghẽn của Naive RAG 30
3.2 Thiết lập thực nghiệm truy xuất 31

3.2.1 Kho tri thức và bộ câu hỏi 31
3.2.2 Độ đo 33
3.3 Các chiến lược phân mảnh (chunking) 34
3.3.1 Bài toán đánh đổi: độ dài đoạn và khoảng chồng lấn . . 35
3.3.2 Năm chiến lược phân mảnh 36
3.3.3 Ý nghĩa của chunking trong dữ liệu y tế và tài chính Việt Nam 41
3.4 Truy xuất: thưa, dày, lai và xếp lại 41
3.4.1 Truy xuất thưa: TF-IDF và BM25 41
3.4.2 Truy xuất dày: mô hình nhúng đa ngôn ngữ 42
3.4.3 Truy xuất lai bằng Reciprocal Rank Fusion 45
3.4.4 Xếp lại bằng cross-encoder 46
3.4.5 Kết quả: chiến lược chia đoạn × bộ truy xuất 47
3.5 Advanced RAG: tối ưu đa chặng 53
3.6 GraphRAG: tổng hợp tri thức trên đồ thị 56
3.6.1 Ý tưởng và quy trình 56
3.6.2 Cộng đồng tri thức 61
3.6.3 So sánh bốn kiến trúc trên câu hỏi đa bước 63
3.7 Chiến lược lựa chọn kiến trúc trong hệ thống thực tế 66

4 Hệ thống y tế: bệnh đái tháo đường típ 2 tại Việt Nam 67
4.1 Bối cảnh dịch tễ và phác đồ của Bộ Y tế 67
4.2 Thiết kế bản thể học và đồ thị tri thức y tế 68
4.2.1 Hệ thống thực thể y tế 68
4.2.2 Mối quan hệ tri thức và ràng buộc lâm sàng 72
4.3 Mô hình học sâu chẩn đoán nguy cơ 78
4.3.1 Dữ liệu 78
4.3.2 Kiến trúc và huấn luyện 80
4.3.3 Kết quả và phân tích 83
4.4 Phân tích các tình huống lâm sàng thực tế 88
4.4.1 Bài toán 1: tương tác giữa metformin và rượu bia 89
4.4.2 Bài toán 2: dinh dưỡng thịt gà 91
4.4.3 Bài toán 3: định lượng tinh bột và ẩm thực truyền thống 92

4.4.4 Cơ chế phản ứng đa tầng trước tình huống phức tạp . . . 92

5 Hệ thống tài chính: VN30 và giá vàng SJC 96
5.1 Bối cảnh vĩ mô và dữ liệu thị trường . . . 96
5.1.1 Bốn cổ phiếu, bốn kênh truyền dẫn . . . 96
5.1.2 Tương quan giữa các tài sản . . . 99
5.1.3 Thị trường vàng Việt Nam: giá quy đổi và phần bù SJC . 101
5.2 Đồ thị tri thức tài chính `VietnamFinanceKG` . . . 103
5.2.1 Bản thể học . . . 103
5.2.2 Thực thể và quan hệ nhân quả . . . 103
5.3 Suy luận tác động đa bước . . . 109
5.3.1 Dấu của đường đi và tác động ròng . . . 110
5.3.2 Lan truyền cú sốc . . . 112
5.3.3 Kiểm chứng dấu cạnh bằng dữ liệu . . . 114
5.4 Dự báo chuỗi thời gian bằng BiLSTM . . . 116
5.4.1 Phát biểu bài toán . . . 116
5.4.2 Kiến trúc LSTM hai chiều . . . 118
5.4.3 Huấn luyện . . . 120
5.4.4 Kết quả và diễn giải . . . 123
5.4.5 Khoảng dự báo cho tác tử . . . 125
5.5 Tình huống minh họa: tác tử trả lời nhà đầu tư . . . 126
5.5.1 Câu hỏi tác động vĩ mô . . . 128
5.5.2 Câu hỏi dự báo . . . 132
5.5.3 Khám phá đồ thị tri thức trên giao diện . . . 133
5.6 Tổng kết chương . . . 133

6 Triển khai hệ thống tác tử thông minh 135
6.1 Mô hình ReAct . . . 135
6.1.1 Suy luận đan xen hành động . . . 135
6.1.2 Bộ công cụ . . . 136
6.2 Kiến trúc hệ thống . . . 137
6.2.1 Năm tầng . . . 138
6.2.2 Luồng xử lý một câu hỏi . . . 138
6.3 Hiện thực tác tử . . . 139

6.3.1 Bộ lập kế hoạch tất định . . . 139
6.3.2 Vòng lặp và các công cụ . . . 142
6.3.3 Tổng hợp có căn cứ và bộ kiểm chứng . . . 144
6.4 Đánh giá tác tử . . . 145
6.4.1 Thiết kế thí nghiệm . . . 147
6.4.2 Kết quả tổng hợp . . . 147
6.4.3 Vì sao LLM trả lời một mình là nguy hiểm . . . 149
6.4.4 Vì sao bản nháp LLM bị từ chối . . . 151
6.4.5 Ba kịch bản chưa đạt độ phủ đầy đủ . . . 152
6.4.6 Độ trễ . . . 152
6.5 Ứng dụng Web . . . 153
6.5.1 Kiến trúc triển khai . . . 153
6.5.2 Giao diện . . . 154
6.6 Tổng kết chương . . . 158

7 Kết luận và hướng phát triển **159**
7.1 Kết luận . . . 159
7.2 Hạn chế . . . 160
7.3 Hướng phát triển . . . 161

Tài liệu tham khảo **162**

Danh mục Hình vẽ

1.1 Tổng quan hệ sinh thái tri thức đa miền của tiểu luận: nguồn tri thức thô được tổ chức thành hai kho bổ sung cho nhau là đồ thị tri thức và chỉ mục vector; tác tử ReAct phối hợp hai kho với các mô hình học sâu để trả lời người dùng 4

1.2 Hai loại bộ nhớ của một hệ RAG: bộ nhớ tham số nằm trong trọng số LLM, bộ nhớ phi tham số là kho tri thức ngoài; bộ truy xuất là cầu nối 6

1.3 Dòng thời gian của biểu diễn tri thức và RAG: từ mạng ngữ nghĩa, Web ngữ nghĩa, Google Knowledge Graph tới Transformer, RAG, ReAct và GraphRAG 7

2.1 Ba thành phần của một KG, minh họa trên `VietnamDiabetesKG`: thực thể là nút có kiểu và thuộc tính, quan hệ là cạnh có nhãn vị từ, thuộc tính có thể nằm trên cả nút lẫn cạnh 11

2.2 Cùng một sự kiện "metformin tương tác nguy hiểm với rượu bia, mức độ cao" trong mô hình LPG (thuộc tính nằm trên cạnh) và mô hình RDF (cạnh được reification thành một `rdf:Statement`) 12

2.3 Bộ ba RDF (chủ thể, vị từ, đối tượng) viết bằng IRI đầy đủ trong không gian tên `ex:` của bài, và đồ thị RDF như một tập các bộ ba 15

2.4 Cây phân cấp lớp (`rdfs:subClassOf`) của bản thể học y tế: đúng 23 lớp khai báo trong `health_ontology()` 18

2.5 Ba tiên đề tính chất của vị từ: bắc cầu, đối xứng, nghịch đảo; cạnh suy ra vẽ nét đứt 18

2.6 Suy diễn tiến trên `VietnamFinanceKG`: cạnh tường minh nét liền, bộ ba suy ra nét đứt đỏ 21

2.7 Ảnh hưởng của suy diễn: số bộ ba RDF trước, sau suy diễn (trái) và số dòng kết quả của từng truy vấn SPARQL (phải) 23

2.8 Xây KB và KG từ dữ liệu bảng: mỗi dòng CSV được văn bản hóa thành một tài liệu (KB) và tách thành các bộ ba (KG), phục vụ hai kiểu truy vấn khác nhau 25

2.9 Câu hỏi có điều kiện số trên dữ liệu bảng: truy vấn KG trả đúng toàn bộ, tìm văn bản TF-IDF gần như bỏ sót 26
2.10 Graph DB duyệt quan hệ tường minh giữa các thực thể; Vector DB tìm láng giềng gần nhất theo khoảng cách trong không gian nhúng nhiều chiều 27

3.1 Ba thế hệ RAG xếp tầng: Naive RAG, Advanced RAG với tối ưu tiền và hậu truy xuất, GraphRAG với đồ thị thực thể và tóm tắt cộng đồng 29
3.2 Naive RAG ba pha: lập chỉ mục ngoại tuyến, truy xuất và tạo sinh trực tuyến, kèm ba điểm nghẽn chính 30
3.3 Cách tính Hit@k, MRR và nDCG@10 trên một danh sách xếp hạng 34
3.4 Quét cỡ đoạn: tỉ lệ trúng tăng theo cỡ đoạn trong khi số ký tự ngữ cảnh top-5 tăng tuyến tính 35
3.5 Năm chiến lược chia đoạn trên cùng một tài liệu: dải ngang là tài liệu, mỗi khối màu là một đoạn 37
3.6 Truy xuất cha – con: khớp trên đoạn con nhỏ, đưa đoạn cha đủ ngữ cảnh cho LLM 37
3.7 Phân bố độ dài đoạn của năm chiến lược 40
3.8 Chia đoạn ngữ nghĩa: khoảng cách cosin giữa các câu liền nhau và điểm cắt 41
3.9 Ba mô hình nhúng đa ngôn ngữ trên cùng bộ câu hỏi 44
3.10 Các đoạn trong không gian nhúng e5-base chiếu PCA hai chiều . 45
3.11 Độ tương đồng cosin giữa câu hỏi và đoạn văn 45
3.12 Bi-encoder mã hóa riêng câu hỏi và đoạn văn rồi so cosin; cross-encoder mã hóa chung cả cặp và cho điểm trực tiếp 47
3.13 Phễu truy xuất lai: BM25 và dense chạy song song, hợp nhất RRF, cross-encoder xếp lại 30 ứng viên và giữ top-k 47
3.14 Bản đồ nhiệt Hit@1: chiến lược chia đoạn (hàng) $\times$ bộ truy xuất (cột) 49
3.15 Đường Hit@k của năm bộ truy xuất 51
3.16 Hit@1 theo miền y tế và tài chính 52

3.17 Chất lượng và độ trễ của 25 tổ hợp: xếp lại đắt hơn nhiều nhưng chính xác hơn . . . 52
3.18 Hai kỹ thuật tối ưu tiền truy xuất bằng LLM cục bộ: HyDE tìm bằng câu trả lời giả định, Multi-Query hợp nhất nhiều cách hỏi bằng RRF . . . 53
3.19 Ảnh hưởng của HyDE và Multi-Query: Hit@1, MRR và số câu thắng, thua so với dense gốc . . . 55
3.20 Đường ống GraphRAG: đồ thị thực thể, cộng đồng Louvain, báo cáo cộng đồng do LLM viết, tìm kiếm cục bộ và toàn cục . . . 57
3.21 Các cộng đồng Louvain trên đồ thị hợp nhất hai KG; nhãn ghi kích thước và thực thể trung tâm, cạnh đứt nối hai cộng đồng khác nhau . . . 62
3.22 Độ phủ dữ kiện trong ngữ cảnh và trong câu trả lời, cùng độ dài ngữ cảnh của bốn kiến trúc . . . 64

4.1 Lược đồ `VietnamDiabetesKG`: các lớp chính và vị từ nối chúng theo ràng buộc miền → miền giá trị của `health_ontology()` . . 69
4.2 Toàn cảnh `VietnamDiabetesKG`: 46 thực thể xếp vòng theo lớp, 59 quan hệ . . . 72
4.3 Phân bố quan hệ theo vị từ của hai KG . . . 74
4.4 Phân bố các đặc trưng chính theo nhãn mắc bệnh . . . 79
4.5 Kiến trúc `DiabetesMLP`: 13 đặc trưng → 128 → 64 → 1 logit, mỗi tầng ẩn gồm BatchNorm, ReLU và Dropout . . . 81
4.6 Đường học của MLP: mất mát BCE và chỉ số trên tập kiểm định qua các epoch . . . 83
4.7 Đường ROC và Precision – Recall của hai mô hình trên tập kiểm thử . . . 84
4.8 Ma trận nhầm lẫn tại ngưỡng đã chọn . . . 84
4.9 So sánh MLP và Logistic Regression theo từng chỉ số . . . 85
4.10 Độ quan trọng hoán vị của các đặc trưng . . . 86
4.11 Xác suất mắc bệnh theo HbA1c ở ba mức glucose . . . 86
4.12 Biểu đồ tin cậy: xác suất dự đoán bị đẩy cao so với tần suất thực tế 87

4.13 Cơ chế tương tác metformin – rượu bia: hai con đường cùng làm giảm sử dụng lactat ở gan, dẫn tới nguy cơ nhiễm toan lactic . . 89
4.14 Thang chỉ số đường huyết GI của các món trong KG và mô hình đĩa ăn: nửa đĩa rau, một phần tư đạm, một phần tư tinh bột . . . 92
4.15 Trợ lý y tế trên Web App: hồ sơ xét nghiệm, vết suy luận ReAct, câu trả lời có trích dẫn và danh sách bằng chứng 94
4.16 Câu hỏi về cơm trắng không kèm hồ sơ: tác tử chỉ gọi KG và RAG 95

5.1 Diễn biến giá điều chỉnh của bốn cổ phiếu, ETF VN30 và vàng quy đổi, chuẩn hóa về 100 ở đầu kỳ (thang log) 98
5.2 Lợi suất – rủi ro năm hóa; diện tích bong bóng tỷ lệ với mức sụt giảm tối đa 99
5.3 Ma trận tương quan Pearson của lợi suất ngày (trái) và lợi suất 20 phiên (phải) 100
5.4 Tương quan trượt một năm của các cặp tương ứng với cạnh trong `VietnamFinanceKG` 100
5.5 Cấu thành giá vàng miếng SJC: giá thế giới, tỷ giá và hệ số đổi đơn vị tạo nên giá quy đổi; phần bù trong nước phụ thuộc chính sách 101
5.6 XAU/USD, USD/VND và giá vàng quy đổi theo (5.1) 102
5.7 Đồ thị nhân quả của `VietnamFinanceKG`: (a) kênh vàng, (b) kênh tỷ giá – lãi suất – cổ phiếu; xanh liền là cùng chiều, đỏ đứt là ngược chiều 105
5.8 Toàn cảnh `VietnamFinanceKG`: 31 thực thể, 40 quan hệ; màu nút theo lớp, màu cạnh theo dấu tác động 109
5.9 Ba đường nhân quả từ lãi suất Fed tới vàng SJC mà `impact_paths` tìm được, dấu của từng đường và tác động ròng . 110
5.10 Lan truyền cú sốc "DXY tăng" trong `VietnamFinanceKG`: xanh là tăng, đỏ là giảm 113
5.11 Tương quan thực nghiệm (thanh) và dấu lý thuyết trong KG (hình thoi) của bảy cạnh có chuỗi giá ở hai đầu 115
5.12 Tương quan giữa lợi suất 20 phiên của USD/VND và của cổ phiếu sau k phiên 116

5.13 Chia dữ liệu theo thời gian và cửa sổ trượt 30 phiên: mô hình không bao giờ nhìn thấy tương lai của mẫu nó dự báo 118
5.14 Bộ dự báo BiLSTM: tenxơ cửa sổ 30×7, hai tầng LSTM xuôi và ngược thời gian, đầu tuyến tính cho lợi suất phiên kế tiếp và MC Dropout cho khoảng dự báo 119
5.15 Đường cong học của BiLSTM cho FPT và vàng quy đổi 122
5.16 So sánh BiLSTM, LSTM và Ridge trên năm tài sản 123
5.17 Giá thực tế và giá dự báo một phiên của BiLSTM trên tập kiểm thử 124
5.18 Lợi suất dự báo và lợi suất thực tế trên tập kiểm thử: FPT dự báo mạnh nhưng sai chiều, vàng quy đổi co về 0 125
5.19 Trợ lý tài chính trên Web App: câu hỏi tác động tỷ giá, vết suy luận và câu trả lời có trích dẫn 131
5.20 Câu hỏi dự báo FPT: tác tử gọi BiLSTM và trả kèm khoảng dự báo, xác suất tăng và độ tin cậy trên tập kiểm thử 132
5.21 Trang khám phá `VietnamFinanceKG` trên Web App 133

6.1 Vòng lặp ReAct: Thought $\rightarrow$ Action $\rightarrow$ Observation lặp quanh bộ nhớ vết suy luận, kết thúc bằng bước tổng hợp và kiểm chứng 136
6.2 Kiến trúc năm tầng của tác tử và lớp, hàm hiện thực tương ứng trong gói `tl2` 138
6.3 Biểu đồ tuần tự cho một câu hỏi y tế có hồ sơ xét nghiệm: trình duyệt, Flask, tác tử, các công cụ, LLM và bộ kiểm chứng 139
6.4 Bộ kiểm chứng: trích dẫn phải trỏ tới bằng chứng có thật và con số phải có trong bằng chứng; không đạt thì dùng bản tổng hợp theo khuôn 145
6.5 Các chỉ số của tác tử theo miền 148
6.6 Độ phủ dữ kiện theo từng kịch bản: tác tử có công cụ so với LLM trả lời một mình 150
6.7 Số lần gọi và độ trễ trung vị của từng công cụ 153
6.8 Kiến trúc triển khai: trình duyệt, máy chủ Flask với bốn API, tác tử và phân bổ CPU/GPU cho từng thành phần 154
6.9 Màn hình chính của Web App: bốn thẻ chức năng và trợ lý y tế với hồ sơ xét nghiệm 156

6.10 Thẻ Khám phá KG với `VietnamDiabetesKG`: nút tô theo lớp, cạnh nguy cơ màu đỏ, khung bên liệt kê các cạnh của nút đang chọn 157
6.11 Thẻ Tra cứu RAG: ba bộ truy xuất trên cùng một câu hỏi 158

Danh mục Bảng biểu

1 Môi trường thực nghiệm và phân bổ tính toán 2

1.1 So sánh LLM đơn lẻ, tinh chỉnh (fine-tuning) và hệ thống tích hợp KB + RAG 8

2.1 Quy mô hai đồ thị tri thức của tiểu luận 17
2.2 Kết quả 7 truy vấn SPARQL trước và sau suy diễn 22
2.3 Bốn bộ ba cố tình sai và thông báo của bộ kiểm tra ràng buộc . . 24
2.4 Chuyển bảng chẩn đoán đái tháo đường thành KB văn bản và KG 26
2.5 Bảng đối chuẩn kỹ thuật Graph Database và Vector Database . . 28

3.1 Kho tri thức 17 tài liệu của tiểu luận 32
3.2 Trích bộ câu hỏi đánh giá truy xuất 33
3.3 Ảnh hưởng của cỡ đoạn tới truy xuất BM25 và dense 36
3.4 Thống kê đoạn của năm chiến lược trên toàn kho 40
3.5 So sánh ba mô hình nhúng đa ngôn ngữ (truy xuất dense, đoạn theo cấu trúc) 44
3.6 Hit@1 của 25 tổ hợp chiến lược chia đoạn và bộ truy xuất . . . 49
3.7 MRR của 25 tổ hợp chiến lược chia đoạn và bộ truy xuất 50
3.8 Chi tiết năm bộ truy xuất trên đoạn chia theo cấu trúc 51
3.9 Tối ưu tiền truy xuất bằng LLM so với truy xuất dense gốc . . . 54
3.10 Ví dụ văn bản do Qwen2.5-1.5B sinh trong HyDE và Multi-Query 56
3.11 Các cộng đồng tri thức và báo cáo cộng đồng 63
3.12 Bốn kiến trúc RAG trên 12 câu hỏi đa bước 64
3.13 Độ phủ ngữ cảnh / câu trả lời theo từng câu hỏi đa bước 65
3.14 Hướng dẫn chọn kiến trúc RAG theo loại câu hỏi và điều kiện triển khai 66

4.1 Tiêu chuẩn chẩn đoán đái tháo đường và tiền đái tháo đường theo Quyết định 5481/QĐ-BYT 68
4.2 Số thực thể theo lớp của hai đồ thị tri thức 71
4.3 Số cạnh theo vị từ của hai đồ thị tri thức 73

4.4 Bộ dữ liệu chẩn đoán đái tháo đường sau tiền xử lý 79
4.5 MLP và Logistic Regression trên tập kiểm thử 83
4.6 Kết quả truy vấn `VietnamDiabetesKG` cho ba bài toán lâm sàng 88

5.1 Thống kê mô tả các tài sản trong giai đoạn nghiên cứu 97
5.2 Kết quả suy luận đa bước cho bảy cặp nguồn – đích 112
5.3 Chiều tác động lên các thực thể khi DXY tăng, theo đường ngắn nhất 114
5.4 Đối chiếu dấu lý thuyết của cạnh KG với tương quan thực nghiệm 115
5.5 Cấu hình bộ dự báo 120
5.6 Kết quả dự báo trên tập kiểm thử của năm tài sản và bốn mô hình 123
5.7 Dự báo phiên tới của năm tài sản: bất định mô hình so với sai số dự báo thực tế 126

6.1 Bảy công cụ của tác tử, số lần được gọi và độ trễ trung vị trên 16 kịch bản 137
6.2 Kết quả tổng hợp của tác tử theo miền 147
6.3 Kết quả từng kịch bản 149

Danh mục Đoạn mã nguồn

2.1 Lớp `Predicate`: vị từ kèm miền, miền giá trị và tính chất logic . 12
2.2 Lớp `Ontology`: cây phân cấp lớp và phép kiểm tra `is_a` 13
2.3 Kiểm tra ràng buộc miền và miền giá trị khi thêm một bộ ba . . 14
2.4 Diễn đạt bộ ba thành câu tiếng Việt để đưa vào ngữ cảnh LLM . 14
2.5 Xuất KG sang RDF bằng rdflib, giữ thuộc tính cạnh bằng reification 16
2.6 Suy diễn tiến năm luật RDFS/OWL tới điểm bất động 19
2.7 Q1: thuốc tương tác nguy hiểm với rượu bia, đọc mức độ và hậu quả qua reification 21
2.8 Q5: mọi biến chịu tác động trực tiếp hay gián tiếp của lãi suất Fed (đường thuộc tính) 22
2.9 Q2: món ăn và chỉ số GI, truy vấn theo lớp cha `ex:Food` 22
2.10 Văn bản hóa mỗi hồ sơ thành một câu tiếng Việt 25
2.11 Tách mỗi hồ sơ thành các bộ ba thuộc tính 25
3.1 Tính Hit@k, MRR và nDCG@10 cho một câu hỏi 34
3.2 Lõi của chia đoạn đệ quy: cắt ở dấu phân cách cấp cao nhất có thể 38
3.3 Chia đoạn ngữ nghĩa theo phân vị khoảng cách cosin giữa các câu liền nhau 39
3.4 Chia đoạn cha – con: cha là mục, con là mảnh đệ quy nhỏ bên trong 39
3.5 Bộ truy xuất BM25 trên các âm tiết tiếng Việt 42
3.6 Bọc SentenceTransformer cho mô hình e5: tiền tố, chuẩn hóa L2, chạy GPU 43
3.7 Truy xuất lai: hợp nhất thứ hạng BM25 và dense bằng RRF . . . 46
3.8 Cross-encoder bge-reranker-v2-m3 chạy bf16 trên GPU 48
3.9 Hợp nhất nhiều danh sách xếp hạng bằng RRF trong thí nghiệm Advanced RAG 54
3.10 Liên kết thực thể: khớp bí danh dài nhất trước, không dấu, nguyên từ 58
3.11 Tóm tắt cộng đồng bằng LLM cục bộ từ các bộ ba nằm trong cộng đồng 59
3.12 Lớp `GraphRAG`: tìm kiếm cục bộ trên KG và tìm kiếm toàn cục trên báo cáo cộng đồng 60

4.1 Bản thể học y tế: 23 lớp và 17 vị từ kèm ràng buộc 70
4.2 Dữ liệu của `VietnamDiabetesKG`: thực thể có thuộc tính và bộ ba có nguồn 74
4.3 Mã hóa hồ sơ thành 13 đặc trưng 80
4.4 Mạng `DiabetesMLP` trong PyTorch 81
4.5 Vòng huấn luyện MLP: BCE có `pos_weight`, AdamW, giảm tốc độ học và dừng sớm theo PR-AUC 81
4.6 Suy luận nguy cơ từ hồ sơ trong tác tử 87
4.7 Cảnh báo tương tác: cạnh trực tiếp và nguy cơ chung qua đường hai bước 90
4.8 Lời khuyên dinh dưỡng: duyệt nhóm thực phẩm xuống các dạng cụ thể 91
5.1 Bản thể học tài chính: 16 lớp và 6 vị từ, hai vị từ nhân quả mang dấu 104
5.2 Xây `VietnamFinanceKG`: thực thể kèm bí danh, chuỗi giá, và các cạnh nhân quả có nguồn, có cơ chế 105
5.3 Suy luận đa bước: liệt kê đường nhân quả, tính dấu từng đường và tác động ròng (5.2) 111
5.4 Lan truyền cú sốc theo chiều rộng, giữ dấu của đường ngắn nhất 113
5.5 Đặc trưng dự báo: lợi suất của đích và biến ngoại sinh, độ lệch MA20, độ biến động 20 phiên; `keep_last` giữ phiên mới nhất khi suy luận 117
5.6 Cắt chuỗi đặc trưng thành các cửa sổ trượt cho mô hình tuần tự . 117
5.7 Mô hình tuần tự dùng chung cho BiLSTM và LSTM một chiều . 119
5.8 Vòng huấn luyện: Huber, AdamW, cắt gradient, dừng sớm theo mất mát kiểm định 120
5.9 Đánh giá trên tập kiểm thử: RMSE lợi suất, tỉ số với dự báo ngây thơ, MAE giá, độ chính xác hướng 122
5.10 Công cụ dự báo của tác tử: MC Dropout cộng nhiễu phần dư cho khoảng dự báo 90% và xác suất tăng 127
6.1 Đặc tả một công cụ: tên, loại, mô tả và hàm thực thi 137
6.2 Bộ lập kế hoạch: nhận miền từ thực thể liên kết, chọn chuỗi công cụ theo loại thực thể và ý định 140

6.3 Trích hồ sơ xét nghiệm từ câu hỏi tiếng Việt tự do 141
6.4 Vòng lặp ReAct: mỗi bước một Thought, một Action đo thời gian và một Observation 142
6.5 Công cụ `kg_tai_chinh`: suy luận đa bước cho mọi cặp nguồn – đích và diễn đạt thành bằng chứng có nguồn 143
6.6 Chỉ dẫn hệ thống cho LLM: chỉ dùng bằng chứng, trích dẫn [n], không bịa số liệu 144
6.7 Lời nhắc có căn cứ: bằng chứng đánh số, câu hỏi, yêu cầu trích dẫn 144
6.8 Tổng hợp: LLM viết từ bằng chứng, bộ kiểm chứng chọn bản LLM hoặc bản khuôn 146
6.9 Bộ kiểm chứng: trích dẫn hợp lệ và độ trung thực số học (6.1) . 146
6.10 API tác tử: nhận câu hỏi và hồ sơ tùy chọn, trả vết suy luận và câu trả lời đã kiểm chứng 154
6.11 API đồ thị tri thức: bố cục Kamada-Kawai tính sẵn, cạnh diễn đạt thành câu kèm nguồn 155

Tổng quan báo cáo

Mô hình ngôn ngữ lớn (LLM) viết trôi chảy nhưng không biết mình đang sai. Hỏi một trợ lý thông thường *"đang uống metformin thì tối nay uống bia được không"*, nó có thể trả lời bằng một đoạn văn nghe rất hợp lý mà không nêu được căn cứ nào, hoặc nêu một con số không có trong bất kỳ tài liệu nào. Trong y tế và tài chính, một câu trả lời như vậy còn nguy hiểm hơn là không trả lời. Tiểu luận này nghiên cứu ba kỹ thuật bổ sung cho LLM chính phần "biết" và "kiểm chứng" mà nó thiếu: **đồ thị tri thức** (Knowledge Graph, KG) lưu tri thức có cấu trúc và có nguồn; **sinh tăng cường truy xuất** (Retrieval-Augmented Generation, RAG) đưa văn bản gốc vào ngữ cảnh trước khi sinh; và **tác tử thông minh** theo vòng lặp ReAct phối hợp KG, RAG với các mô hình học sâu chuyên biệt để trả lời câu hỏi nhiều bước.

Mục tiêu và phạm vi

Đề bài yêu cầu trình bày khái niệm và kiến trúc của KG, các thế hệ RAG, rồi áp dụng vào hai miền của Việt Nam: quản lý **bệnh đái tháo đường típ 2** và phân tích **rổ cổ phiếu VN30 cùng giá vàng SJC**, cuối cùng triển khai một AI Agent có giao diện web. Báo cáo này làm trọn chuỗi đó trên máy cá nhân, không gọi một API LLM bên ngoài nào:

- Viết kho tri thức 17 tài liệu Markdown (38.359 ký tự) tổng hợp từ Quyết định 5481/QĐ-BYT, ADA, các Nghị định về vàng và báo cáo doanh nghiệp, kèm bộ 114 cặp hỏi đáp có đáp án nằm nguyên văn trong kho để chấm điểm truy xuất.
- Dựng hai đồ thị tri thức có bản thể học: `VietnamDiabetesKG` (46 thực thể, 59 quan hệ) và `VietnamFinanceKG` (31 thực thể, 40 quan hệ), xuất RDF, suy diễn RDFS và truy vấn SPARQL.
- So sánh định lượng 5 chiến lược chia đoạn × 5 bộ truy xuất, 3 mô hình nhúng, 7 cỡ đoạn, hai kỹ thuật tiền truy xuất bằng LLM (HyDE, Multi-Query) và bốn kiến trúc từ Naive RAG tới GraphRAG.

- Huấn luyện trên GPU hai mô hình học sâu làm công cụ cho tác tử: MLP chẩn đoán nguy cơ đái tháo đường (96.128 hồ sơ) và BiLSTM dự báo lợi suất cho bốn cổ phiếu và vàng quy đổi.
- Ghép tất cả thành tác tử ReAct có bảy công cụ, bộ kiểm chứng trích dẫn và con số, đánh giá trên 16 kịch bản, và đóng gói thành Web App Flask.

Phân bổ CPU và GPU

Máy thực nghiệm có một GPU laptop 8 GB và một CPU 32 luồng. Nguyên tắc phân bổ là: việc nào là phép nhân ma trận lớn lặp lại nhiều lần thì chạy GPU, việc nào là tra cứu, duyệt đồ thị hay tính toán nhỏ thì để CPU, vì chi phí chép dữ liệu qua PCIe lớn hơn phần tính toán. Bảng 1 ghi cụ thể từng thành phần.

Bảng 1. Môi trường thực nghiệm và phân bổ tính toán

Thành phần	Thiết bị	Lý do
Qwen2.5-1.5B-Instruct (bf16)	GPU	sinh văn bản tự hồi quy, mỗi token một lượt qua 28 tầng Transformer
bge-reranker-v2-m3 (bf16)	GPU	chấm 30 cặp (câu hỏi, đoạn) mỗi truy vấn bằng cross-encoder 568 triệu tham số
multilingual-e5 (fp16 khi chung tiến trình với LLM)	GPU	mã hóa câu hỏi và toàn bộ đoạn khi lập chỉ mục
Huấn luyện MLP và BiLSTM	GPU	lô 1024 và 128 mẫu, lan truyền ngược
BM25, TF-IDF, chia đoạn	CPU	tính trên chỉ mục thưa, dưới 1 ms mỗi truy vấn
Hai KG (NetworkX), RDF, SPARQL, suy diễn	CPU	duyệt đồ thị vài chục nút, rdflib chỉ chạy CPU
Suy luận MLP, BiLSTM trong tác tử	GPU	đã nạp sẵn trên GPU cùng LLM; MC Dropout chạy 200 mẫu một lô
Vẽ hình, dựng báo cáo	CPU	matplotlib và LuaLaTeX

GPU: NVIDIA GeForce RTX 4060 Laptop 8 GB; CPU: AMD Ryzen 9 7945HX, 32 luồng; RAM 40 GB. Python 3.13, PyTorch 2.13 + CUDA 12.6, sentence-transformers, NetworkX, rdflib, Flask.

Một ràng buộc thực tế đã định hình kiến trúc: trên Windows, trình điều khiển WDDM tính cả bộ nhớ GPU đã cấp phát vào hạn mức *commit* của hệ điều hành. Nạp LLM sau embedder và reranker khiến tiến trình vượt hạn mức và dừng đột ngột khi đọc tệp trọng số. Cách khắc phục được giữ lại trong mã: nạp LLM trước, lưu sẵn reranker ở dạng bf16 trên đĩa và dùng embedder fp16 khi chung tiến trình với LLM; cả ba mô hình khi đó cùng chạy trong khoảng 8 GB.

Cấu trúc báo cáo

Báo cáo bám theo bảy phần của đề bài. **Chương 1** nêu khái niệm KG, RAG và vai trò của cơ sở tri thức. **Chương 2** đi vào cấu tạo KG: thành phần, mô hình bộ ba RDF, bản thể học, suy diễn và so sánh Graph DB với Vector DB. **Chương 3** trình bày ba thế hệ RAG, từ Naive RAG, các chiến lược chia đoạn, Advanced RAG tới GraphRAG, mỗi phần đều có thực nghiệm trên kho tri thức của bài. **Chương 4** xây hệ thống y tế cho bệnh đái tháo đường típ 2, **Chương 5** xây hệ thống tài chính cho VN30 và vàng SJC. **Chương 6** ghép thành tác tử ReAct, đánh giá và triển khai Web App. **Chương 7** kết luận và nêu hướng phát triển.

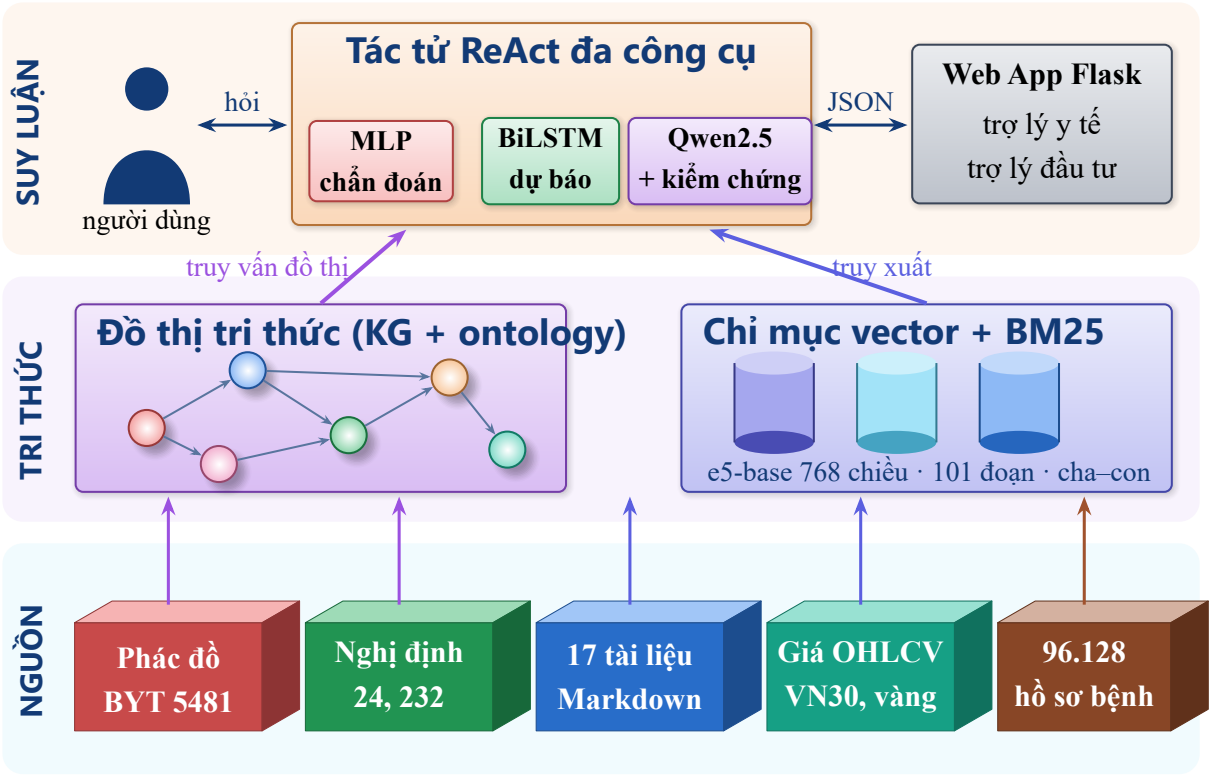
Quy ước về số liệu và mã nguồn

Không con số thực nghiệm nào trong báo cáo được gõ tay. Mỗi kịch bản trong `scripts/` ghi kết quả ra `outputs/metrics/*.json`; `build.py` đọc các tệp đó để sinh macro số liệu, bảng và vết suy luận, rồi trích nguyên văn từng hàm được trích dẫn từ mã nguồn trong `src/TieuLuan 02`. Hình dữ liệu vẽ bằng matplotlib từ cùng các tệp JSON; sơ đồ lý thuyết vẽ bằng TikZ. Chạy lại thực nghiệm rồi dựng lại báo cáo thì mọi con số, bảng và hình đổi theo.

Chương 1

Mở đầu về Knowledge Graph và RAG

Chương này đặt nền cho toàn bài: đồ thị tri thức là gì, RAG là gì, vì sao hai kỹ thuật này bổ sung đúng chỗ yếu của LLM, và chúng được ghép với nhau như thế nào trong hệ thống của tiểu luận. Hình 1.1 là bản đồ của cả báo cáo; mỗi khối trên hình tương ứng một chương phía sau.



Hình 1.1. Tổng quan hệ sinh thái tri thức đa miền của tiểu luận: nguồn tri thức thô được tổ chức thành hai kho bổ sung cho nhau là đồ thị tri thức và chỉ mục vector; tác tử ReAct phối hợp hai kho với các mô hình học sâu để trả lời người dùng

1.1 Khái niệm đồ thị tri thức và RAG

1.1.1 Khái niệm đồ thị tri thức (Knowledge Graph, KG)

Đồ thị tri thức là một đồ thị dữ liệu dùng để tích lũy và truyền đạt tri thức về thế giới thực, trong đó **nút** biểu diễn thực thể và **cạnh** biểu diễn quan hệ giữa các thực thể [1]. Khác một cơ sở dữ liệu quan hệ, KG không cố định lược đồ thành các bảng: muốn thêm một loại quan hệ mới chỉ cần thêm một nhãn cạnh mới. Khác một văn bản, mỗi mẩu tri thức trong KG là một đơn vị nguyên tử, máy đọc được và có thể truy vấn chính xác.

Về hình thức, gọi $\mathcal{E}$ là tập thực thể, $\mathcal{R}$ là tập quan hệ (vị từ). Một KG là tập các **bộ ba** (chủ thể, vị từ, đối tượng):

$$\mathcal{G} = \{(s, p, o) \mid s, o \in \mathcal{E},\ p \in \mathcal{R}\} \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}. \tag{1.1}$$

Ví dụ, sự kiện "metformin tương tác nguy hiểm với rượu bia" trong `VietnamDiabetesKG` được lưu thành bộ ba

$$(\text{Metformin},\ \text{TUONG_TAC_NGUY_HIEM},\ \text{Ruou_Bia}).$$

Đi kèm tập bộ ba là một **bản thể học** (ontology) quy định thực thể thuộc lớp nào, vị từ nào được nối những lớp nào; bản thể học biến một đồ thị nhãn tự do thành tri thức có ngữ nghĩa, để máy kiểm tra được và suy ra được tri thức mới (Chương 2).

Ba tính chất làm KG có giá trị với hệ thống thông minh:

- **Tường minh và truy vết được.** Mỗi cạnh trong hai KG của bài mang thuộc tính `source` trỏ về tài liệu gốc; câu trả lời dựa trên KG luôn nêu được căn cứ.
- **Đa bước.** Câu hỏi "Fed tăng lãi suất thì vàng SJC ra sao" không có câu trả lời trong bất kỳ đoạn văn đơn lẻ nào, nhưng là một đường đi qua 3–4 cạnh trên đồ thị nhân quả tài chính.
- **Có ràng buộc.** Bản thể học chặn được bộ ba vô nghĩa như "rau xanh là thuốc đầu tay của metformin" ngay khi nạp dữ liệu, trước khi nó lan vào câu trả lời.

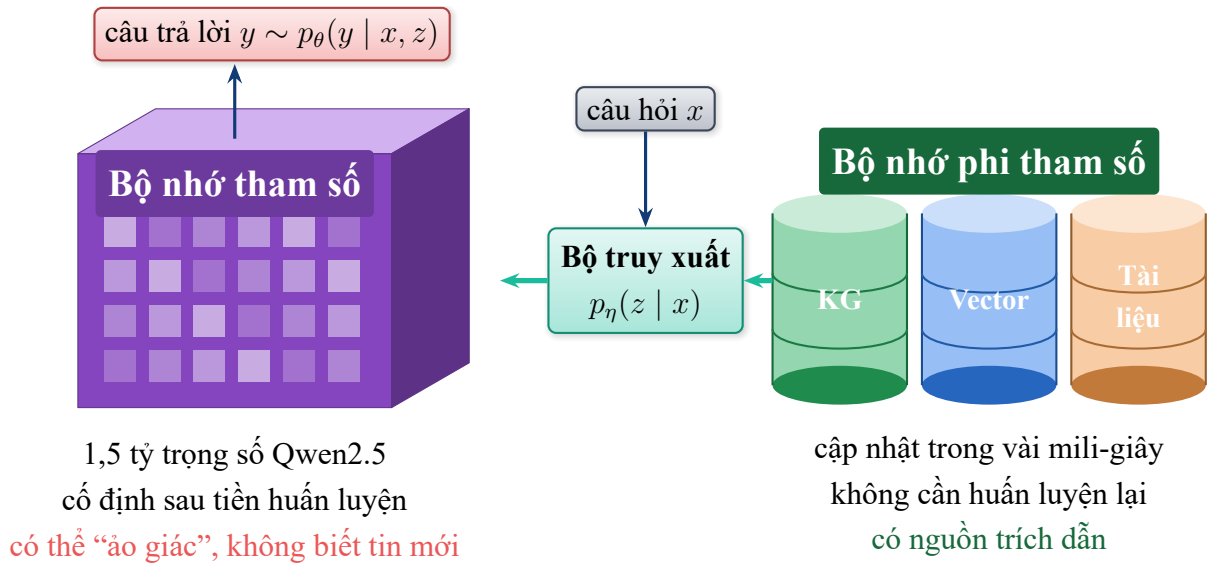
1.1.2 Khái niệm sinh tăng cường truy xuất (RAG)

RAG được Lewis và cộng sự đề xuất năm 2020 [2] để giải bài toán tri thức chuyên sâu: thay vì bắt mô hình nhớ mọi thứ trong tham số, ta cho nó *tra cứu* một kho văn bản ngoài trước khi sinh. Một hệ RAG có hai thành phần: bộ truy xuất $p_\eta(z \mid x)$ chọn các đoạn văn z liên quan tới câu hỏi x, và bộ sinh $p_\theta(y \mid x, z)$ viết câu trả lời y dựa trên cả câu hỏi lẫn đoạn văn. Xác suất câu trả lời là tổng theo các đoạn văn được truy xuất:

$$p(y \mid x) \approx \sum_{z \in \text{top-}k\, p_\eta(\cdot \mid x)} p_\eta(z \mid x)\, p_\theta(y \mid x, z). \tag{1.2}$$

Trong thực hành hiện nay, các đoạn top-k được nối vào prompt và LLM sinh một lần; công thức (1.2) vẫn là cách hiểu đúng: chất lượng câu trả lời bị chặn trên bởi chất lượng truy xuất. Nếu đoạn văn chứa đáp án không nằm trong top-k, bộ sinh dù mạnh tới đâu cũng chỉ còn cách đoán.

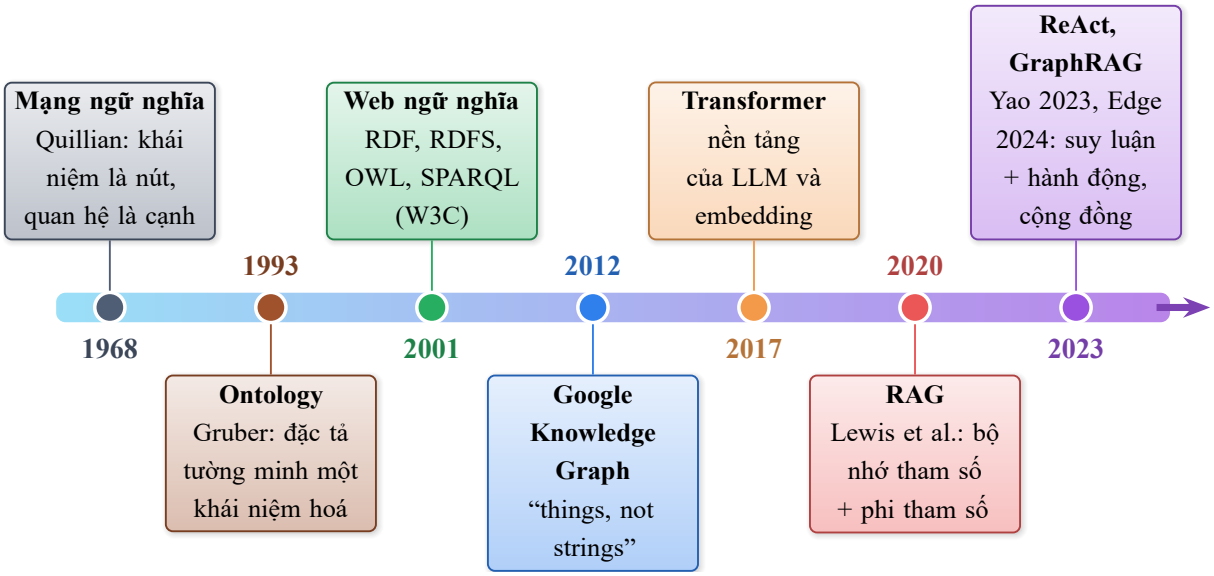
Hình 1.2 đặt RAG trong khung hai loại bộ nhớ. **Bộ nhớ tham số** là trọng số LLM: rộng, nén, học một lần, không sửa được từng mẩu và không chỉ ra được nguồn. **Bộ nhớ phi tham số** là kho tri thức ngoài: cập nhật bằng cách thêm hay sửa tài liệu, mỗi câu trả lời trỏ được về đoạn văn cụ thể. RAG nối hai bộ nhớ qua bộ truy xuất.



Hình 1.2. Hai loại bộ nhớ của một hệ RAG: bộ nhớ tham số nằm trong trọng số LLM, bộ nhớ phi tham số là kho tri thức ngoài; bộ truy xuất là cầu nối

1.1.3 Quá trình phát triển

Biểu diễn tri thức bằng đồ thị không mới. Mạng ngữ nghĩa có từ thập niên 1960; Web ngữ nghĩa của Berners-Lee năm 2001 [3] chuẩn hóa nó thành RDF, OWL và SPARQL; năm 2012 Google đưa thuật ngữ "Knowledge Graph" vào sản phẩm tìm kiếm. Nhánh thứ hai là mô hình ngôn ngữ: kiến trúc Transformer năm 2017 [4] mở đường cho LLM, rồi RAG năm 2020 nối LLM với kho văn bản. Hai nhánh gặp nhau từ 2023 [5]: ReAct [6] cho LLM gọi công cụ trong vòng suy luận, GraphRAG năm 2024 [7] dùng đồ thị và cộng đồng để trả lời câu hỏi tổng hợp (Hình 1.3).



Hình 1.3. Dòng thời gian của biểu diễn tri thức và RAG: từ mạng ngữ nghĩa, Web ngữ nghĩa, Google Knowledge Graph tới Transformer, RAG, ReAct và GraphRAG

1.2 Vai trò và ý nghĩa của Knowledge Base và RAG

Cơ sở tri thức (Knowledge Base, KB) là tập tri thức của một miền được tổ chức để máy truy cập được; trong bài này KB có hai dạng song song: kho văn bản Markdown (dạng phi cấu trúc, phục vụ RAG) và hai KG (dạng có cấu trúc). Giá trị của KB với một hệ thống dùng LLM nằm ở năm điểm.

1. **Giảm ảo giác.** Ảo giác (hallucination) là khi mô hình sinh nội dung trôi chảy nhưng sai hoặc không có căn cứ [8]. Ở Mục 6.4, cùng mô hình Qwen2.5-1.5B

trả lời 16 câu hỏi kiểm thử: khi không có công cụ, câu trả lời chỉ chứa 15,62% dữ kiện bắt buộc; khi có KB qua tác tử, tỉ lệ này là 90,62%.

2. **Cập nhật không cần huấn luyện lại.** Nghị định 232/2025/NĐ-CP sửa Nghị định 24/2012 về vàng; với KB, chỉ cần thêm một thực thể và một cạnh `SUA_DOI`, không phải tinh chỉnh mô hình.
3. **Truy vết nguồn.** Trong y tế, một khuyến nghị không có nguồn thì không dùng được. Mọi cạnh trong hai KG đều mang mã tài liệu nguồn (59 trên 59 cạnh y tế, 40 trên 40 cạnh tài chính).
4. **Bảo mật dữ liệu.** Hồ sơ xét nghiệm của người bệnh không rời máy: LLM, embedder, reranker và cả hai KG chạy cục bộ.
5. **Suy luận có kiểm soát.** Quy tắc nghiệp vụ (chống chỉ định, dấu tác động vĩ mô) nằm trong KG dưới dạng tường minh; LLM chỉ diễn đạt lại, không phải tự suy ra.

Bảng 1.1 so sánh ba cách đưa tri thức miền vào một hệ thống dùng LLM.

Bảng 1.1. So sánh LLM đơn lẻ, tinh chỉnh (fine-tuning) và hệ thống tích hợp KB + RAG

Tiêu chí	LLM đơn lẻ	Tinh chỉnh	KB + RAG (+ KG)
Nguồn tri thức	tham số, cố định lúc huấn luyện	tham số, học thêm dữ liệu miền	kho ngoài, đọc lúc trả lời
Cập nhật	không	huấn luyện lại, tốn GPU	thêm, sửa tài liệu hoặc bộ ba
Truy vết nguồn	không	không	có, tới từng đoạn hoặc cạnh
Ảo giác	cao với tri thức hiếm	giảm nhưng vẫn còn	thấp nếu có bộ kiểm chứng
Câu hỏi đa bước	tự suy, khó kiểm soát	tự suy	duyệt đường trên KG
Chi phí triển khai	thấp	cao	trung bình: chỉ mục + truy xuất
Độ trễ	một lượt sinh	một lượt sinh	thêm truy xuất, rerank
Phù hợp	hội thoại chung	đổi văn phong, định dạng	y tế, pháp lý, tài chính

Tinh chỉnh và RAG không loại trừ nhau: tinh chỉnh dạy mô hình *cách* trả lời

(văn phong, định dạng trích dẫn), RAG cung cấp *nội dung* để trả lời. Với một mô hình nhỏ như Qwen2.5-1.5B, Chương 6 cho thấy mô hình hiểu nội dung bằng chứng tốt nhưng thường quên ghi số hiệu trích dẫn; đó chính là phần việc mà tinh chỉnh có thể bù.

1.3 Bài toán và cách tiếp cận của tiểu luận

Đề bài đặt hai bài toán ứng dụng có tính chất rất khác nhau, và chính sự khác nhau đó làm rõ khi nào cần KG, khi nào cần RAG, khi nào cần mô hình học sâu.

- **Y tế** là miền của *ràng buộc*: tiêu chuẩn chẩn đoán có ngưỡng cụ thể, thuốc có chống chỉ định, thực phẩm có chỉ số đường huyết. Câu trả lời phải đúng từng con số và có nguồn. KG lưu ràng buộc, RAG cung cấp lời giải thích, MLP ước lượng nguy cơ từ hồ sơ xét nghiệm.
- **Tài chính** là miền của *quan hệ nhân quả và bất định*: tỷ giá tác động lên cổ phiếu qua nhiều kênh, có kênh trái dấu nhau; giá tương lai không biết trước. KG lưu chiều tác động, dữ liệu thị trường kiểm chứng lại chiều đó, BiLSTM dự báo kèm khoảng bất định.

Cả hai miền dùng chung một lõi: lớp `KnowledgeGraph` với bản thể học, cùng một kho văn bản và đường ống truy xuất, cùng một LLM cục bộ và cùng một tác tử ReAct. Mỗi thành phần được đánh giá riêng trước khi ghép: truy xuất ở Chương 3, KG ở Chương 2, 4, 5, mô hình học sâu ở Chương 4, 5, và toàn hệ thống ở Chương 6.

Chương 2

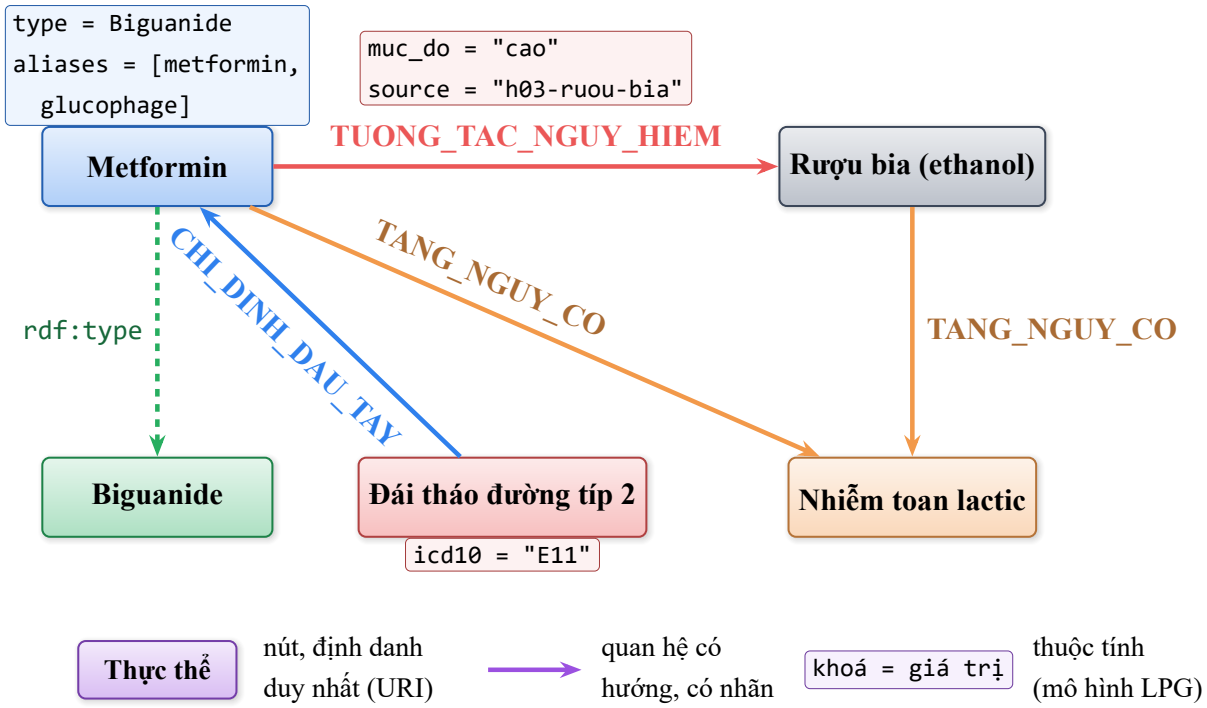
Kiến trúc Knowledge Graph và cơ sở tri thức

Chương 1 định nghĩa KG là một tập bộ ba. Chương này mở tập bộ ba đó ra: một KG được cấu tạo từ những gì, được viết ra theo chuẩn RDF thế nào, bản thể học thêm vào đó những ràng buộc và luật suy diễn ra sao, và khi nào nên lưu tri thức trong một cơ sở dữ liệu đồ thị thay vì một cơ sở dữ liệu vector. Mọi ví dụ đều lấy từ hai KG thật của bài, cài trong `tl2/kg_base.py`.

2.1 Các thành phần cấu tạo của Knowledge Graph

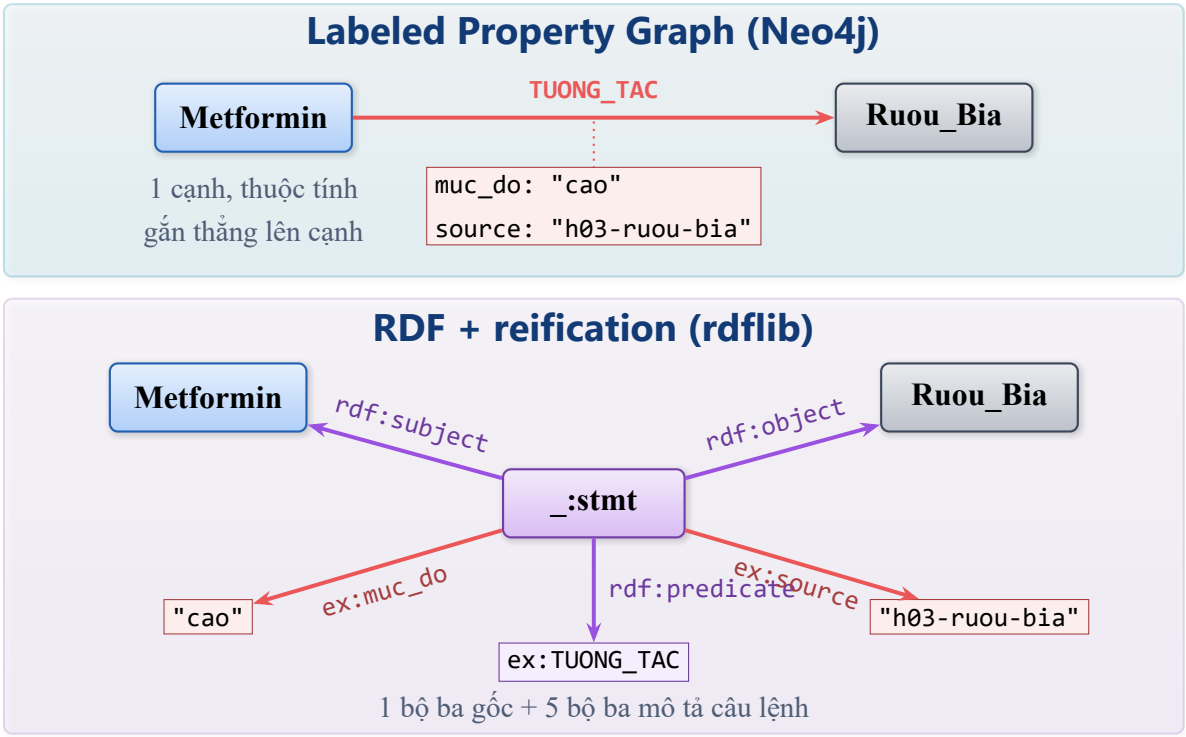
Một KG có ba loại thành phần (Hình 2.1):

- **Thực thể (entity)** là nút của đồ thị, đại diện cho một đối tượng có danh tính: một bệnh (DTD_Tip2), một thuốc (Metformin), một món ăn (Com_Trang), một chính sách (ND_232_2025), một cổ phiếu (FPT). Mỗi thực thể có mã định danh duy nhất, một **kiểu** là một lớp trong bản thể học, nhãn tiếng Việt để hiển thị và danh sách bí danh để liên kết với câu hỏi tự nhiên ("tiểu đường", "đtđ", "type 2" đều trỏ về DTD_Tip2).
- **Quan hệ (relation)** là cạnh có hướng, gán nhãn bằng một vị từ: Metformin $\xrightarrow{\text{TUONG_TAC_NGUY_HIEM}}$ Ruou_Bia. Giữa hai thực thể có thể có nhiều cạnh với vị từ khác nhau, nên cấu trúc dữ liệu phù hợp là *đa đồ thị có hướng* (`networkx.MultiDiGraph` [9]).
- **Thuộc tính (attribute)** gắn lên nút (mã ICD-10 `E11`, chỉ số GI 73 của cơm trắng, ngưỡng 126 mg/dL) hoặc lên cạnh (mức độ `cao`, hậu quả, nguồn trích dẫn `h03-ruou-bia`).



Hình 2.1. Ba thành phần của một KG, minh họa trên `VietnamDiabetesKG`: thực thể là nút có kiểu và thuộc tính, quan hệ là cạnh có nhãn vị từ, thuộc tính có thể nằm trên cả nút lẫn cạnh

Thuộc tính trên cạnh là chỗ hai mô hình dữ liệu đồ thị phổ biến khác nhau. **Đồ thị thuộc tính có nhãn** (Labeled Property Graph, LPG), mô hình của Neo4j, cho phép gắn trực tiếp cặp khóa–giá trị lên cạnh. **RDF** chỉ có bộ ba, nên muốn nói "cạnh này có mức độ cao" phải biến chính cạnh thành một thực thể, kỹ thuật gọi là *reification* (Hình 2.2). Lớp `KnowledgeGraph` của bài làm việc theo LPG khi xây dựng và truy vấn, rồi xuất sang RDF có reification khi cần suy diễn và SPARQL; nhờ vậy dùng được thế mạnh của cả hai.



Hình 2.2. Cùng một sự kiện "metformin tương tác nguy hiểm với rượu bia, mức độ cao" trong mô hình LPG (thuộc tính nằm trên cạnh) và mô hình RDF (cạnh được reification thành một `rdf:Statement`)

Bản thể học được cài bằng hai lớp dữ liệu nhỏ. `Predicate` mô tả một vị từ: miền (domain) là lớp của chủ thể, miền giá trị (range) là lớp của đối tượng, cùng ba tính chất logic đối xứng, bắc cầu, nghịch đảo. `Ontology` giữ cây lớp dưới dạng từ điển *lớp* $\rightarrow$ *lớp cha* và trả lời câu hỏi "lớp A có phải là một loại B không" bằng cách đi ngược lên gốc (Đoạn mã 2.2).

```python
# tl2/kg_base.py
@dataclass
class Predicate:
    name: str
    domain: str
    range: str
    label: str
    symmetric: bool = False
    transitive: bool = False
    inverse: str = None
```

Đoạn mã 2.1. Lớp `Predicate`: vị từ kèm miền, miền giá trị và tính chất logic

```python
# tl2/kg_base.py
@dataclass
class Ontology:
    classes: dict = field(default_factory=dict)        # lớp -> lớp cha (None nếu là gốc)
    predicates: dict = field(default_factory=dict)     # tên -> Predicate

    def add_class(self, name, parent=None):
        self.classes[name] = parent

    def add_predicate(self, *args, **kw):
        p = Predicate(*args, **kw)
        self.predicates[p.name] = p

    def ancestors(self, cls):
        """Bao đóng bắc cầu của subClassOf: SafeFood -> Food -> MedicalEntity -> Thing."""
        out = []
        while cls is not None:
            out.append(cls)
            cls = self.classes.get(cls)
        return out

    def is_a(self, cls, target):
        return target in self.ancestors(cls)
```

Đoạn mã 2.2. Lớp `Ontology`: cây phân cấp lớp và phép kiểm tra `is_a`

Điểm quan trọng nhất của lõi KG là mọi bộ ba đều được **kiểm tra ràng buộc trước khi nạp** (Đoạn mã 2.3): chủ thể phải thuộc miền của vị từ, đối tượng phải thuộc miền giá trị, xét theo cả lớp con. Thêm (Rau_Xanh, CHI_DINH_DAU_TAY, Metformin) bị từ chối ngay vì rau xanh không phải một bệnh. Mục 2.3.3 thử bốn bộ ba cố tình sai như vậy.

```python
# tl2/kg_base.py
def validate(self, s, p, o):
    pred = self.onto.predicates.get(p)
    if pred is None:
        raise ValueError(f"Vị từ '{p}' chưa khai báo")
    for node, want, role in ((s, pred.domain, "chủ thể"), (o, pred.range,
    ↪ "đối tượng")):
        if node not in self.G:
            raise ValueError(f"Thực thể '{node}' chưa tồn tại")
        have = self.G.nodes[node]["type"]
        if not self.onto.is_a(have, want):
            raise ValueError(f"Vi phạm ràng buộc: {role} của {p} phải thuộc
            ↪ lớp {want}, '{node}' là {have}")
```

Đoạn mã 2.3. Kiểm tra ràng buộc miền và miền giá trị khi thêm một bộ ba

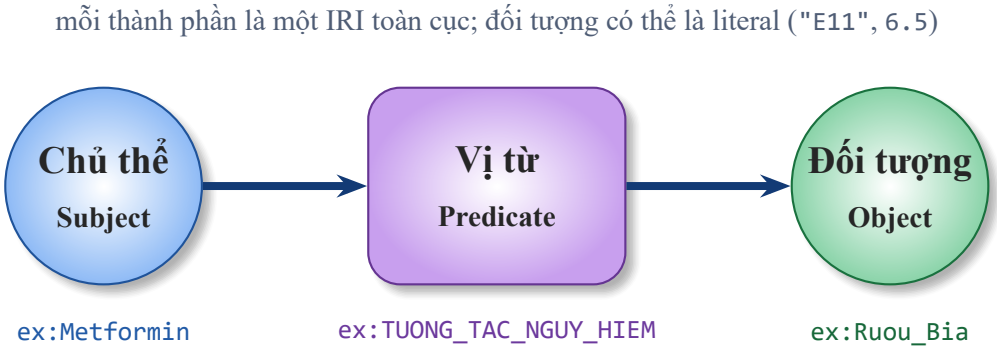
Để LLM đọc được tri thức, mỗi bộ ba được **diễn đạt thành câu** tiếng Việt kèm các thuộc tính cạnh (Đoạn mã 2.4). Đây là định dạng mà tác tử đưa vào prompt ở Chương 6, ví dụ "Metformin —[tương tác nguy hiểm với]→ Rượu bia (ethanol) (severity: cao; effect: tăng nguy cơ nhiễm toan lactic…)".

```python
# tl2/kg_base.py
def verbalize(self, fact):
    """Chuyển bộ ba thành câu tiếng Việt để đưa vào ngữ cảnh của LLM."""
    s, p, o, d = fact
    pl = self.onto.predicates[p].label
    extra = "; ".join(f"{k}: {v}" for k, v in d.items() if k not in
    ↪ ("relation", "source") and v is not None)
    return f"{self.label(s)} —[{pl}]→ {self.label(o)}" + (f" ({extra})" if
    ↪ extra else "")
```

Đoạn mã 2.4. Diễn đạt bộ ba thành câu tiếng Việt để đưa vào ngữ cảnh LLM

2.2 Mô hình biểu diễn bộ ba RDF

RDF (Resource Description Framework) là chuẩn W3C để mô tả tài nguyên trên Web bằng bộ ba [10]. Mỗi thành phần của bộ ba là một **IRI** (định danh tài nguyên toàn cục), một **literal** (giá trị có kiểu như số, chuỗi có nhãn ngôn ngữ) hoặc một **nút trống**. Chủ thể là IRI hoặc nút trống, vị từ luôn là IRI, đối tượng là bất kỳ loại nào. Một đồ thị RDF đơn giản là *tập hợp* các bộ ba (Hình 2.3); hai đồ thị gộp lại bằng phép hợp tập hợp, điều làm RDF đặc biệt hợp để tích hợp dữ liệu từ nhiều nguồn.




```
@prefix ex: <http://tl02.ptit.local/kg#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
ex:Metformin a ex:Biguanide ;
  rdfs:label "Metformin"@vi ;
  ex:TUONG_TAC_NGUY_HIEM ex:Ruou_Bia .
ex:Biguanide rdfs:subClassOf ex:OralDrug .  # tri thức lược đồ (TBox)
```

Hình 2.3. Bộ ba RDF (chủ thể, vị từ, đối tượng) viết bằng IRI đầy đủ trong không gian tên `ex:` của bài, và đồ thị RDF như một tập các bộ ba

Phép ánh xạ từ KG của bài sang RDF (Đoạn mã 2.5) đi theo bốn bước:

1. Mỗi lớp thành một `rdfs:Class`, quan hệ lớp cha thành `rdfs:subClassOf`.
2. Mỗi vị từ thành một `rdf:Property` kèm `rdfs:domain`, `rdfs:range`, nhãn tiếng Việt và các tiên đề OWL `owl:SymmetricProperty`, `owl:TransitiveProperty`, `owl:inverseOf`.
3. Mỗi thực thể có bộ ba `rdf:type` trỏ tới lớp, `rdfs:label` và một bộ ba cho mỗi thuộc tính nút.
4. Mỗi cạnh thành một bộ ba; nếu cạnh có thuộc tính thì thêm một

`rdf:Statement` mang `rdf:subject`, `rdf:predicate`, `rdf:object` và từng thuộc tính.

Đoạn mã 2.5. Xuất KG sang RDF bằng rdflib, giữ thuộc tính cạnh bằng reification

```python
# tl2/kg_base.py
def to_rdf(self):
    """Ánh xạ sang RDF: lớp -> rdfs:Class, vị từ -> rdf:Property kèm rdfs:domain/range, cạnh -> bộ ba.

    Thuộc tính của cạnh (LPG) không có chỗ trong một bộ ba RDF, nên được giữ bằng reification
    dạng rdf:Statement — đúng điểm khác biệt giữa mô hình LPG và RDF.
    """
    g = Graph()
    g.bind("ex", EX)
    for c, parent in self.onto.classes.items():
        g.add((EX[c], RDF.type, RDFS.Class))
        if parent:
            g.add((EX[c], RDFS.subClassOf, EX[parent]))
    for p in self.onto.predicates.values():
        g.add((EX[p.name], RDF.type, RDF.Property))
        g.add((EX[p.name], RDFS.domain, EX[p.domain]))
        g.add((EX[p.name], RDFS.range, EX[p.range]))
        g.add((EX[p.name], RDFS.label, Literal(p.label, lang="vi")))
        if p.symmetric:
            g.add((EX[p.name], RDF.type, OWL.SymmetricProperty))
        if p.transitive:
            g.add((EX[p.name], RDF.type, OWL.TransitiveProperty))
        if p.inverse:
            g.add((EX[p.name], OWL.inverseOf, EX[p.inverse]))
    for n, d in self.G.nodes(data=True):
        g.add((EX[n], RDF.type, EX[d["type"]]))
        g.add((EX[n], RDFS.label, Literal(d["label"], lang="vi")))
        for k, v in d.items():
            if k in ("type", "label", "aliases") or v is None:
                continue
            g.add((EX[n], EX[k], _lit(v)))
    for i, (s, p, o, d) in enumerate(self.facts()):
        g.add((EX[s], EX[p], EX[o]))
        extra = {k: v for k, v in d.items() if k != "relation" and v is not None}
        if extra:
            st = EX[f"stmt{i}"]
            g.add((st, RDF.type, RDF.Statement))
            g.add((st, RDF.subject, EX[s]))
            g.add((st, RDF.predicate, EX[p]))
```

```
            g.add((st, RDF.object, EX[o]))
            for k, v in extra.items():
                g.add((st, EX[k], _lit(v)))
    return g
```

Bảng 2.1 tổng hợp kích thước hai KG. Số bộ ba RDF lớn hơn nhiều số cạnh vì mỗi thực thể sinh thêm bộ ba kiểu và nhãn, mỗi cạnh có thuộc tính sinh thêm khối reification.

Bảng 2.1. Quy mô hai đồ thị tri thức của tiểu luận

Chỉ số	VietnamDiabetesKG	VietnamFinanceKG
Số nút (thực thể)	46	31
Số cạnh (quan hệ)	59	40
Số lớp trong ontology	23	16
Số vị từ (predicate)	17	6
Mật độ đồ thị	0,0570	0,0860
Bậc trung bình	2,565	2,581
Thành phần liên thông	1	1
Cạnh có trích nguồn	59	40
Bộ ba RDF xuất ra	618	402
Bộ ba suy diễn thêm	172	111

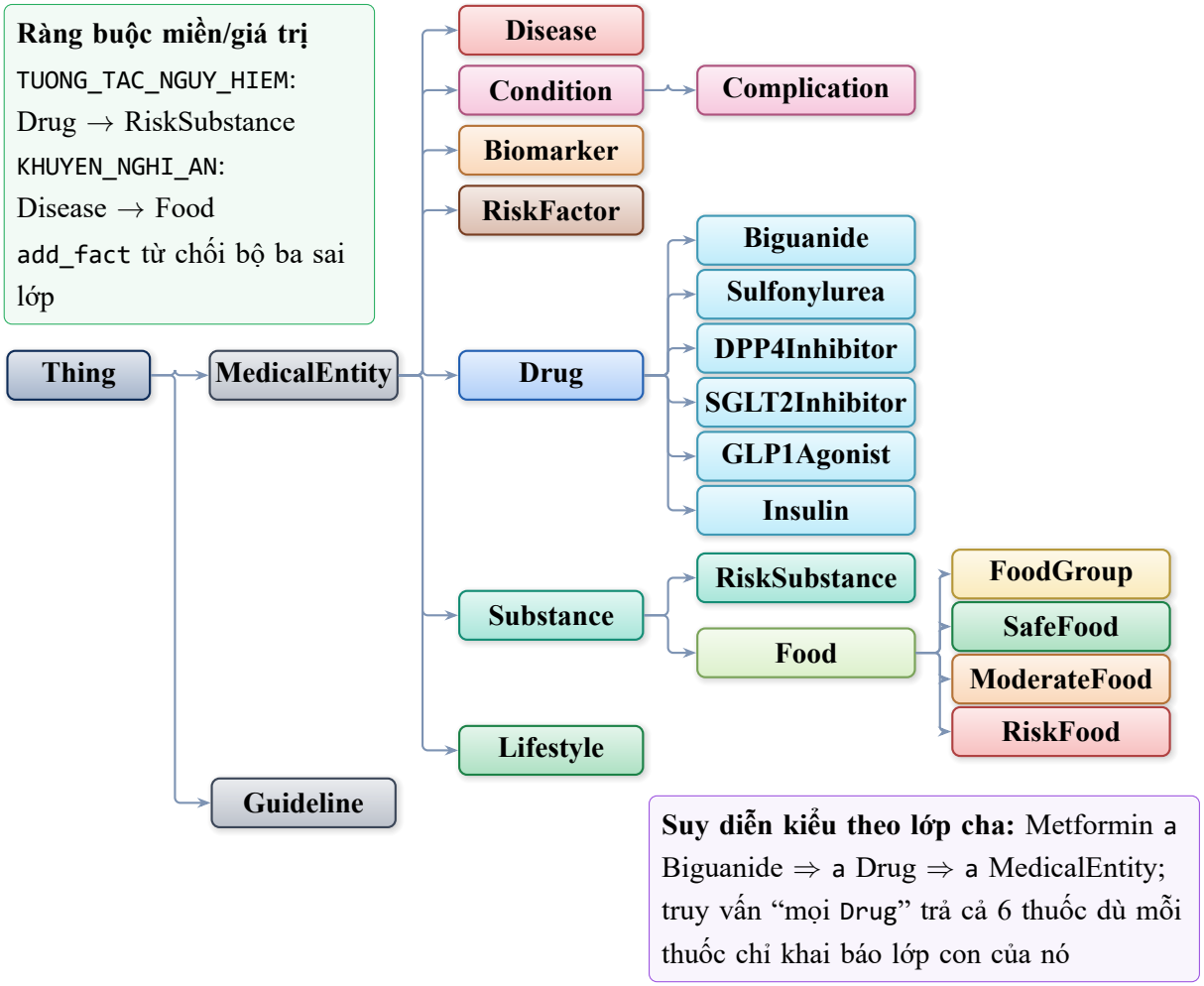
Bộ ba RDF gồm cả nhãn, kiểu và thuộc tính cạnh (reification); suy diễn: bao đóng RDFS và các luật đối xứng, bắc cầu, nghịch đảo.

2.3 Bản thể học và biểu diễn tri thức hình thức

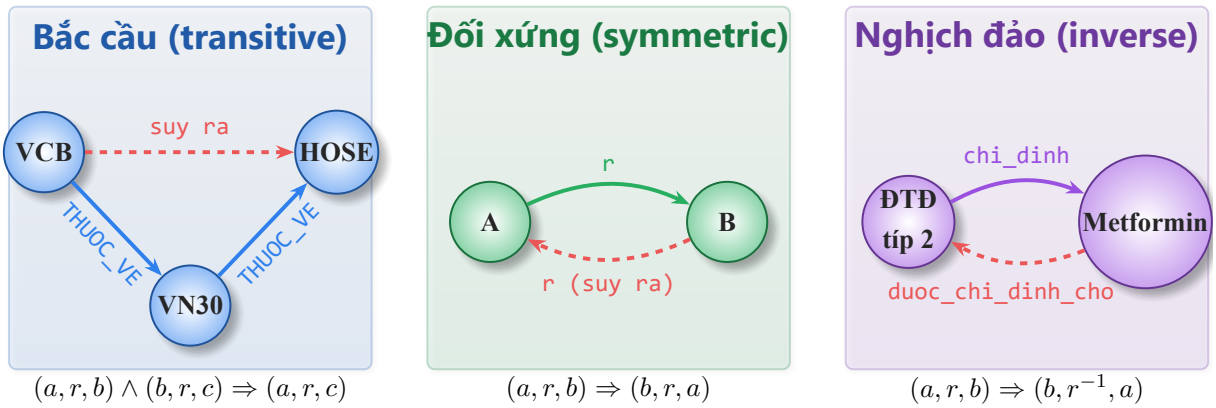
Theo định nghĩa kinh điển của Gruber, bản thể học là *"một đặc tả tường minh của một khái niệm hóa"* [11]: nó liệt kê các loại sự vật tồn tại trong một miền và các quan hệ được phép giữa chúng. Trong thực hành Web ngữ nghĩa, bản thể học được viết bằng RDFS và OWL [12] và gồm ba lớp tri thức:

- **Phân cấp lớp**: SafeFood $\sqsubseteq$ Food $\sqsubseteq$ Substance $\sqsubseteq$ MedicalEntity. Hình 2.4 là cây lớp của `VietnamDiabetesKG` với đúng 23 lớp khai báo trong mã.
- **Ràng buộc vị từ**: TUONG_TAC_NGUY_HIEM có miền Drug và miền giá trị RiskSubstance; KHUYEN_NGHI_AN nối Disease với Food.

- **Tiên đề tính chất**: vị từ có thể đối xứng, bắc cầu hoặc là nghịch đảo của vị từ khác (Hình 2.5).



Hình 2.4. Cây phân cấp lớp (`rdfs:subClassOf`) của bản thể học y tế: đúng 23 lớp khai báo trong `health_ontology()`



$(a, r, b) \wedge (b, r, c) \Rightarrow (a, r, c)$ $(a, r, b) \Rightarrow (b, r, a)$ $(a, r, b) \Rightarrow (b, r^{-1}, a)$

Hình 2.5. Ba tiên đề tính chất của vị từ: bắc cầu, đối xứng, nghịch đảo; cạnh suy ra vẽ nét đứt

2.3.1 Suy diễn tiến trên RDF

Bản thể học không chỉ để kiểm tra mà còn để **suy ra** tri thức không được ghi tường minh. Hàm `rdfs_closure` cài suy diễn tiến (forward chaining): áp lặp lại năm luật cho tới khi không sinh thêm bộ ba mới nào (điểm bất động). Với $\sqsubseteq$ là `rdfs:subClassOf` và `a` là `rdf:type`:

$$(A \sqsubseteq B) \wedge (B \sqsubseteq C) \;\Rightarrow\; A \sqsubseteq C \qquad \text{(phân cấp bắc cầu)} \qquad (2.1)$$

$$(x \;\texttt{a}\; A) \wedge (A \sqsubseteq B) \;\Rightarrow\; x \;\texttt{a}\; B \qquad \text{(kế thừa kiểu)} \qquad (2.2)$$

$$p \text{ đối xứng} \wedge (x\,p\,y) \;\Rightarrow\; (y\,p\,x) \qquad \text{(đối xứng)} \qquad (2.3)$$

$$p \text{ bắc cầu} \wedge (x\,p\,y) \wedge (y\,p\,z) \;\Rightarrow\; (x\,p\,z) \qquad \text{(bắc cầu)} \qquad (2.4)$$

$$p \equiv q^{-1} \wedge (x\,p\,y) \;\Rightarrow\; (y\,q\,x) \qquad \text{(nghịch đảo)} \qquad (2.5)$$

Năm luật này là tập con của ngữ nghĩa RDFS và OWL 2 RL đủ cho hai KG của bài. Luật `rdfs:domain` và `rdfs:range` (suy ra kiểu của chủ thể từ vị từ) cố ý không bật: ở đây domain và range được dùng như *ràng buộc* để chặn dữ liệu sai, không phải để suy ra kiểu mới.

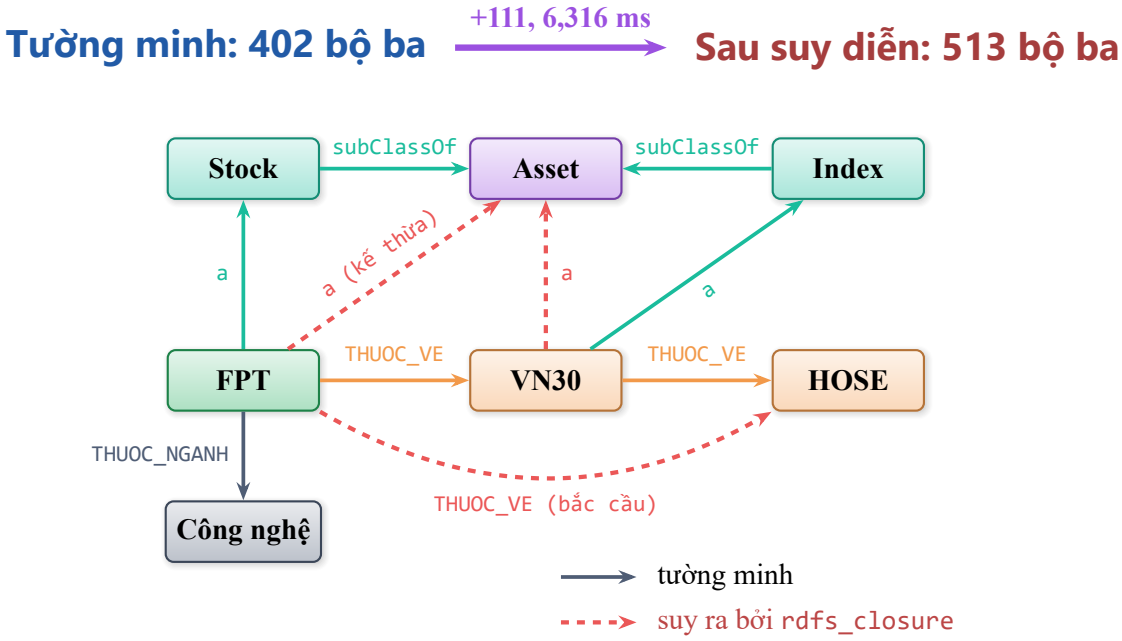
Đoạn mã 2.6. Suy diễn tiến năm luật RDFS/OWL tới điểm bất động

```python
# tl2/kg_base.py
def rdfs_closure(g):
    """Suy luận tiến (forward chaining) các luật RDFS/OWL cần thiết cho đến khi không sinh thêm bộ ba mới.

    Luật: (1) subClassOf bắc cầu; (2) kiểu kế thừa: x a C, C ⊑ D ⇒ x a D; (3) vị từ đối xứng;
    (4) vị từ bắc cầu; (5) nghịch đảo: p inverseOf q, (x p y) ⇒ (y q x). Trả về đồ thị mới và số bộ ba suy ra.
    """
    g2 = Graph()
    for t in g:
        g2.add(t)
    sym = set(g2.subjects(RDF.type, OWL.SymmetricProperty))
    trans = set(g2.subjects(RDF.type, OWL.TransitiveProperty))
    inv = {(p, q) for p, q in g2.subject_objects(OWL.inverseOf)}
    base = len(g2)
    while True:
        new = set()
        sub = list(g2.subject_objects(RDFS.subClassOf))
        for a, b in sub:
```

```
            for c in g2.objects(b, RDFS.subClassOf):
                new.add((a, RDFS.subClassOf, c))
        for x, c in list(g2.subject_objects(RDF.type)):
            for d in g2.objects(c, RDFS.subClassOf):
                new.add((x, RDF.type, d))
        for p in sym:
            for x, y in g2.subject_objects(p):
                new.add((y, p, x))
        for p in trans:
            for x, y in g2.subject_objects(p):
                for z in g2.objects(y, p):
                    new.add((x, p, z))
        for p, q in inv:
            for x, y in g2.subject_objects(p):
                new.add((y, q, x))
            for x, y in g2.subject_objects(q):
                new.add((y, p, x))
        new = {t for t in new if t not in g2}
        if not new:
            return g2, len(g2) - base
        for t in new:
            g2.add(t)
```

Trên đồ thị y tế, suy diễn thêm 172 bộ ba vào 618 bộ ba gốc; trên đồ thị tài chính thêm 111 bộ ba vào 402. Hình 2.6 minh họa trên một phần của `VietnamFinanceKG`: vị từ THUOC_VE bắc cầu nên từ "FPT thuộc VN30" và "VN30 thuộc HOSE" suy ra "FPT thuộc HOSE".



Hình 2.6. Suy diễn tiến trên `VietnamFinanceKG`: cạnh tường minh nét liền, bộ ba suy ra nét đứt đỏ

2.3.2 Truy vấn SPARQL

SPARQL là ngôn ngữ truy vấn chuẩn của RDF [13]: một truy vấn là một *mẫu đồ thị* chứa biến (`?thuoc`), và kết quả là mọi cách gán biến làm mẫu khớp với đồ thị. Bài chạy 7 truy vấn trên cả đồ thị gốc lẫn đồ thị đã suy diễn (Bảng 2.2). Truy vấn Q1 đọc mức độ tương tác qua khối reification; Q5 dùng đường thuộc tính `(p|q)+` để tìm mọi biến chịu tác động từ lãi suất Fed sau bất kỳ số bước nào.

```
# Q1 -- q1_tuong_tac_ruou
SELECT ?thuoc ?mucdo ?hauqua WHERE {
  ?thuoc ex:TUONG_TAC_NGUY_HIEM ex:Ruou_Bia .
  ?st rdf:subject ?thuoc ; rdf:predicate ex:TUONG_TAC_NGUY_HIEM ; rdf:object
  ↪ ex:Ruou_Bia ;
      ex:severity ?mucdo ; ex:effect ?hauqua .
}
```

Đoạn mã 2.7. Q1: thuốc tương tác nguy hiểm với rượu bia, đọc mức độ và hậu quả qua reification

```
# Q5 -- q5_lan_truyen_fed
SELECT DISTINCT ?bien WHERE {
  ex:Lai_Suat_Fed (ex:TAC_DONG_CUNG_CHIEU|ex:TAC_DONG_NGUOC_CHIEU)+ ?bien .
}
```

Đoạn mã 2.8. Q5: mọi biến chịu tác động trực tiếp hay gián tiếp của lãi suất Fed (đường thuộc tính)

```
# Q2 -- q2_thuc_pham_gi
SELECT ?mon ?lop ?gi WHERE {
  ?mon a ex:Food ; a ?lop ; ex:gi ?gi .
  ?lop rdfs:subClassOf ex:Food .
  FILTER(isNumeric(?gi))
} ORDER BY DESC(?gi)
```

Đoạn mã 2.9. Q2: món ăn và chỉ số GI, truy vấn theo lớp cha `ex:Food`

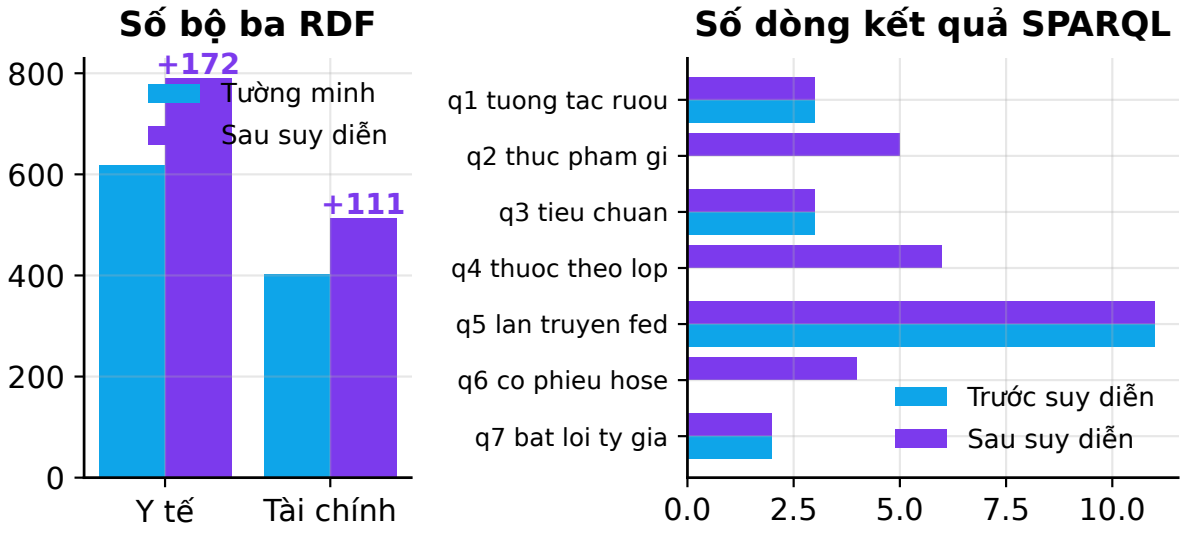
Bảng 2.2. Kết quả 7 truy vấn SPARQL trước và sau suy diễn

Mã	Miền	Nội dung truy vấn	Kết quả	Trước suy diễn	ms
Q1	Y tế	Thuốc tương tác nguy hiểm với rượu bia, đọc mức độ qua reification	3	3	3,9
Q2	Y tế	Món ăn và chỉ số GI theo lớp con của Food	5	0	6,4
Q3	Y tế	Tiêu chuẩn chẩn đoán ĐTĐ típ 2 và ngưỡng	3	3	2,2
Q4	Y tế	Đếm thuốc theo nhóm (lớp con của Drug)	6	0	6,0
Q5	Tài chính	Mọi biến chịu tác động từ lãi suất Fed (đường thuộc tính +)	11	11	2,3
Q6	Tài chính	Cổ phiếu niêm yết trên HOSE và ngành	4	0	2,8
Q7	Tài chính	Doanh nghiệp bất lợi khi USD/VND tăng, kèm cơ chế	2	2	3,1

Kết quả: số dòng trên đồ thị đã suy diễn; trước suy diễn: cùng truy vấn trên bộ ba gốc. Các truy vấn theo lớp cha (Food, Drug, Stock) chỉ có kết quả sau khi suy diễn kiểu.

Cột "trước suy diễn" cho thấy vai trò thực sự của bản thể học. Q2, Q4 và Q6 hỏi theo lớp cha (Food, Drug, Stock) trong khi dữ liệu chỉ ghi lớp lá

(ModerateFood, Biguanide…), nên trên đồ thị gốc chúng trả về rỗng; sau suy diễn kế thừa kiểu chúng trả về đầy đủ. Không có suy diễn, người viết truy vấn phải tự liệt kê mọi lớp con, và truy vấn hỏng mỗi khi bản thể học thêm một lớp mới. Hình 2.7 tổng hợp hai hiệu ứng: số bộ ba tăng thêm và số dòng kết quả của từng truy vấn.



Hình 2.7. Ảnh hưởng của suy diễn: số bộ ba RDF trước, sau suy diễn (trái) và số dòng kết quả của từng truy vấn SPARQL (phải)

2.3.3 Kiểm tra ràng buộc

Để chứng minh ràng buộc domain/range thực sự hoạt động, kịch bản `build_kgs.py` cố tình thêm 4 bộ ba sai. Cả bốn đều bị từ chối với thông báo nêu đúng lớp mong đợi và lớp thực tế (Bảng 2.3). Đây là lớp phòng thủ đầu tiên chống lại dữ liệu bẩn, đặt ngay ở thời điểm nạp, trước khi một bộ ba sai kịp lan vào câu trả lời của tác tử.

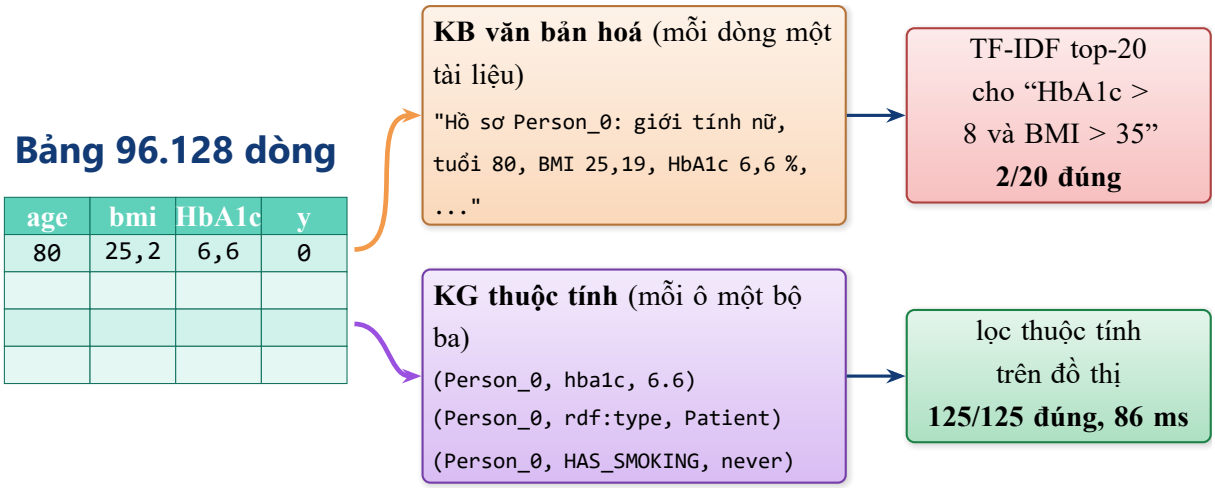
Bảng 2.3. Bốn bộ ba cố tình sai và thông báo của bộ kiểm tra ràng buộc

Bộ ba cố tình sai	Thông báo của bộ kiểm tra ràng buộc
`Rau_Xanh CHI_DINH_DAU_TAY Metformin`	chủ thể của CHI_DINH_DAU_TAY phải thuộc lớp Disease, 'Rau_Xanh' là SafeFood
`Metformin TIEU_CHUAN_CHAN_DOAN DTD_Tip2`	chủ thể của TIEU_CHUAN_CHAN_DOAN phải thuộc lớp Biomarker, 'Metformin' là Biguanide
`Com_Trang TUONG_TAC_NGUY_HIEM Ruou_Bia`	chủ thể của TUONG_TAC_NGUY_HIEM phải thuộc lớp Drug, 'Com_Trang' là ModerateFood
`DTD_Tip2 KHUYEN_NGHI_AN Metformin`	đối tượng của KHUYEN_NGHI_AN phải thuộc lớp Food, 'Metformin' là Biguanide

2.4 Xây KB và KG từ dữ liệu bảng

Không phải tri thức nào cũng có sẵn dạng văn bản. Bộ dữ liệu chẩn đoán đái tháo đường [14] là một bảng CSV 96.128 dòng; dữ liệu giá cổ phiếu cũng là bảng. Tiếp cận của bài tham khảo dungvt, đưa một bảng thành hai dạng tri thức song song (Hình 2.8):

- **KB văn bản**: mỗi dòng thành một câu mô tả, ví dụ "Hồ sơ Person_0: giới tính nữ, tuổi 80, tăng huyết áp không, bệnh tim có…". Dạng này đưa vào RAG được.
- **KG bộ ba**: mỗi ô thành một bộ ba $(\text{Person_0}, \text{has_age}, 80)$. Dạng này truy vấn chính xác theo điều kiện số được.



Hình 2.8. Xây KB và KG từ dữ liệu bảng: mỗi dòng CSV được văn bản hóa thành một tài liệu (KB) và tách thành các bộ ba (KG), phục vụ hai kiểu truy vấn khác nhau

```python
# tl2/kg_tabular.py
def diabetes_documents(df):
    """Hồ sơ -> câu mô tả có tên trường, phù hợp cho TF-IDF và embedding."""
    docs = []
    for i, r in df.iterrows():
        parts = [f"{DIAB_VI[c]} {_vi_value(c, r[c])}" for c in DIAB_REL]
        docs.append({"record_id": f"Person_{i}", "document": f"Hồ sơ Person_{i}: " + ", ".join(parts) + "."})
    return pd.DataFrame(docs)
```

Đoạn mã 2.10. Văn bản hóa mỗi hồ sơ thành một câu tiếng Việt

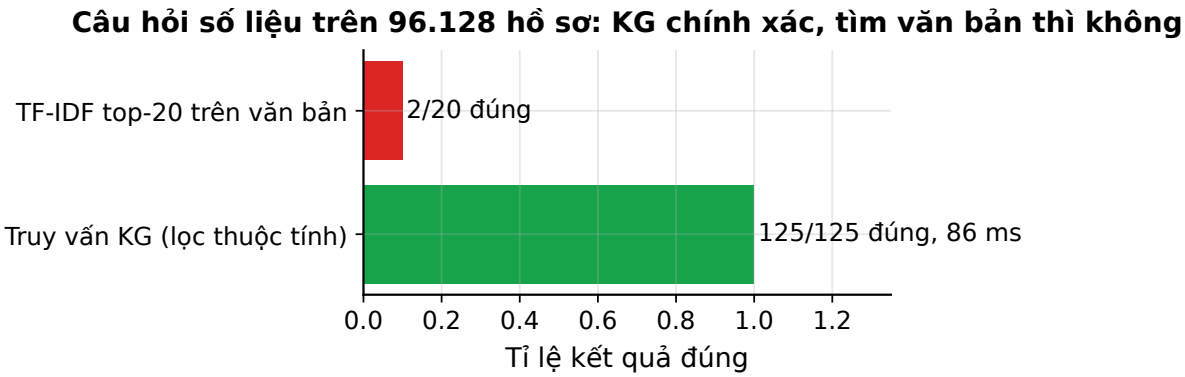
```python
# tl2/kg_tabular.py
def diabetes_triples(df):
    out = []
    for i, r in df.iterrows():
        s = f"Person_{i}"
        out.append((s, "rdf:type", "Patient"))
        for c, rel in DIAB_REL.items():
            out.append((s, rel, r[c]))
    return out
```

Đoạn mã 2.11. Tách mỗi hồ sơ thành các bộ ba thuộc tính

Bảng 2.4. Chuyển bảng chẩn đoán đái tháo đường thành KB văn bản và KG

Chỉ số	Giá trị
Bản ghi CSV	96.128
Văn bản hồ sơ sinh ra	96.128
Bộ ba (triple)	961.280
Nút	100.528
Cạnh	961.280
Thời gian sinh văn bản (s)	3,17
Thời gian dựng đồ thị (s)	5,35

Hai dạng tri thức trả lời hai loại câu hỏi khác nhau. Câu hỏi "có bao nhiêu người trên 70 tuổi, tăng huyết áp, HbA1c từ 8,8 trở lên" là một phép lọc trên thuộc tính số: KG lọc ba điều kiện, trả đúng 125 trên 125 hồ sơ trong 85,96 ms. Tìm kiếm văn bản bằng TF-IDF trên 20.000 tài liệu mẫu chỉ đưa được 2 hồ sơ đúng vào top-20 (Hình 2.9), vì độ tương đồng từ vựng không hiểu phép so sánh "$\geq$ 8,8". Bài học: RAG mạnh với câu hỏi diễn giải, KG mạnh với câu hỏi có điều kiện và số liệu; một hệ thống tốt cần cả hai.

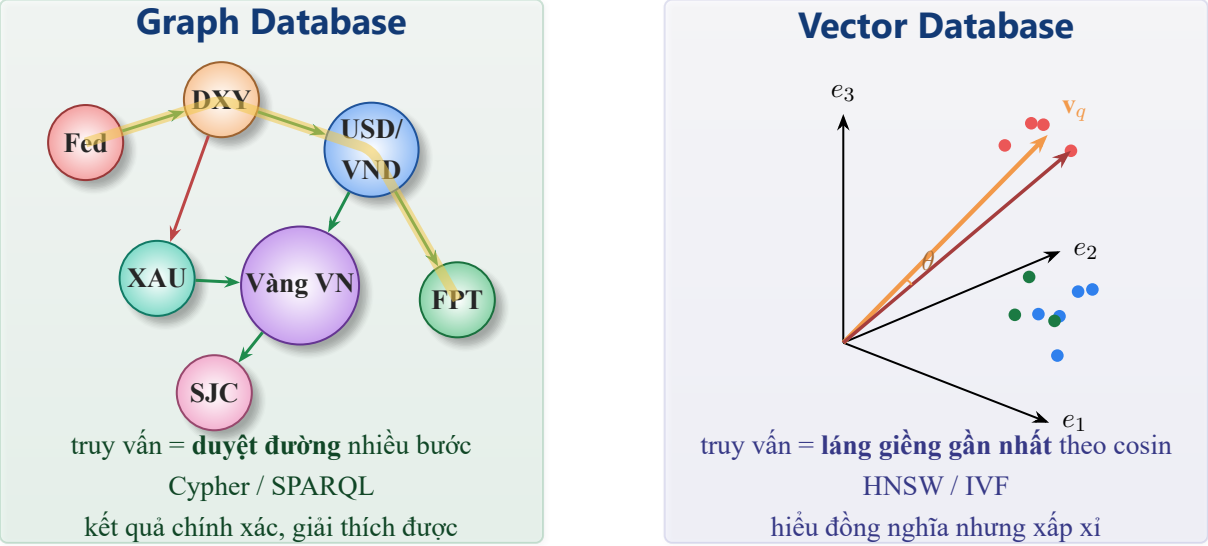


Hình 2.9. Câu hỏi có điều kiện số trên dữ liệu bảng: truy vấn KG trả đúng toàn bộ, tìm văn bản TF-IDF gần như bỏ sót

2.5 So sánh chi tiết: Graph Database và Vector Database

Hai loại kho dữ liệu chuyên dụng cho hệ thống dùng LLM là cơ sở dữ liệu đồ thị (Neo4j, GraphDB, Amazon Neptune) và cơ sở dữ liệu vector (FAISS, Milvus,

Qdrant, Chroma). Chúng trả lời hai câu hỏi khác nhau (Hình 2.10): Graph DB trả lời *"cái gì nối với cái gì, bằng quan hệ gì"*; Vector DB trả lời *"cái gì giống cái này nhất"*.



Hình 2.10. Graph DB duyệt quan hệ tường minh giữa các thực thể; Vector DB tìm láng giềng gần nhất theo khoảng cách trong không gian nhúng nhiều chiều

Bảng 2.5. Bảng đối chuẩn kỹ thuật Graph Database và Vector Database

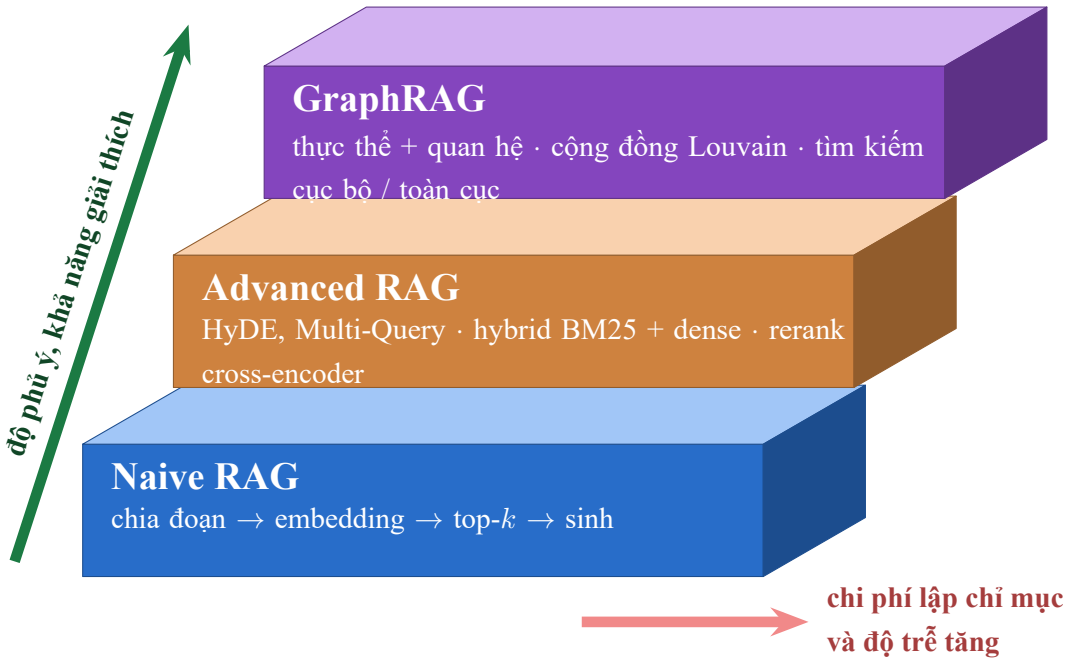
Tiêu chí	Graph Database	Vector Database
Đơn vị dữ liệu	nút, cạnh có nhãn, thuộc tính	vector dày $\mathbb{R}^d$ kèm metadata
Mô hình truy vấn	khớp mẫu đồ thị (Cypher, SPARQL), duyệt đường	láng giềng gần nhất (k-NN, ANN)
Chỉ mục	danh sách kề, chỉ mục theo nhãn	HNSW, IVF, lượng tử hóa PQ
Câu hỏi đa bước	tự nhiên: mỗi bước là một bước duyệt	không có khái niệm bước
Câu hỏi ngữ nghĩa mờ	kém: cần đúng tên thực thể	mạnh: hiểu diễn đạt khác nhau
Điều kiện số, phép đếm	chính xác	không hỗ trợ trực tiếp
Khả năng giải thích	đường đi và nguồn của từng cạnh	điểm tương đồng, khó giải thích
Chi phí xây dựng	cao: cần trích xuất, chuẩn hóa thực thể	thấp: nhúng văn bản thô
Cập nhật	thêm, sửa bộ ba cục bộ	nhúng lại đoạn văn đổi
Vai trò trong bài	`VietnamDiabetesKG`, `VietnamFinanceKG`	chỉ mục e5 trên 17 tài liệu

Hai loại kho không cạnh tranh mà bổ sung: Vector DB tìm *đoạn văn* giải thích, Graph DB cung cấp *quan hệ* chính xác và đường suy luận. GraphRAG ở Chương 3 và tác tử ở Chương 6 dùng cả hai cùng lúc.

Chương 3

Các thế hệ kiến trúc Retrieval-Augmented Generation

RAG không phải một kiến trúc mà là một họ kiến trúc tiến hóa qua ba thế hệ [15] (Hình 3.1). **Naive RAG** chia văn bản, nhúng, tìm top-k rồi sinh. **Advanced RAG** thêm các bước tối ưu trước truy xuất (viết lại câu hỏi) và sau truy xuất (xếp lại, nén ngữ cảnh). **GraphRAG** thay đơn vị tri thức từ đoạn văn rời rạc sang đồ thị thực thể và cộng đồng, để trả lời được câu hỏi đa bước và câu hỏi tổng hợp. Chương này đi qua cả ba thế hệ, và với mỗi thế hệ đo trên chính kho tri thức của bài xem nó cải thiện được gì, tốn thêm bao nhiêu.



Hình 3.1. Ba thế hệ RAG xếp tầng: Naive RAG, Advanced RAG với tối ưu tiền và hậu truy xuất, GraphRAG với đồ thị thực thể và tóm tắt cộng đồng

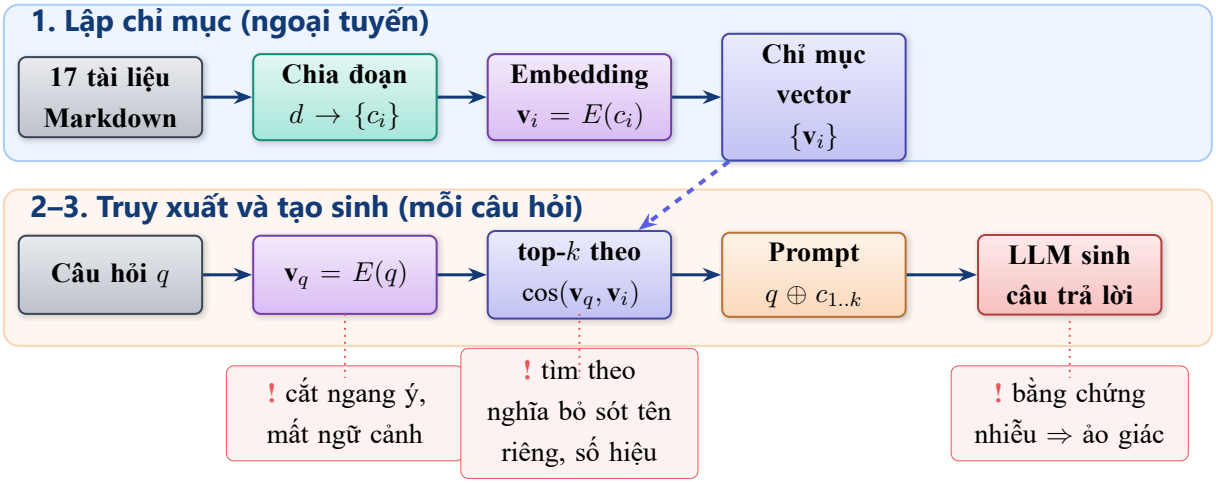
3.1 Naive RAG và các điểm nghẽn

3.1.1 Quy trình hoạt động của Naive RAG

Naive RAG có ba pha (Hình 3.2). Pha **lập chỉ mục** chạy ngoại tuyến: tài liệu được chia thành đoạn (chunk), mỗi đoạn được mã hóa thành vector $\mathbf{d}_j = f(\text{đoạn}_j) \in \mathbb{R}^{768}$ và lưu vào chỉ mục. Pha **truy xuất** chạy khi có câu hỏi: câu hỏi được mã hóa thành $\mathbf{q} = f(\text{câu hỏi})$ và k đoạn có độ tương đồng cosin lớn nhất được chọn:

$$\text{sim}(\mathbf{q}, \mathbf{d}_j) = \frac{\mathbf{q}^\top \mathbf{d}_j}{\|\mathbf{q}\| \, \|\mathbf{d}_j\|}, \qquad \mathcal{Z}_k(\mathbf{q}) = \arg\text{top-}k_j \;\; \text{sim}(\mathbf{q}, \mathbf{d}_j). \tag{3.1}$$

Pha **tạo sinh** nối k đoạn vào prompt theo khuôn "BẰNG CHỨNG: [1]…[k] — CÂU HỎI — TRẢ LỜI" và để LLM sinh câu trả lời.



Hình 3.2. Naive RAG ba pha: lập chỉ mục ngoại tuyến, truy xuất và tạo sinh trực tuyến, kèm ba điểm nghẽn chính

3.1.2 Các điểm nghẽn của Naive RAG

Mỗi pha có một điểm nghẽn, và lỗi ở pha trước không pha sau nào sửa được:

1. **Chia đoạn cắt rời ngữ cảnh.** Cửa sổ 400 ký tự có thể cắt giữa câu "chống chỉ định khi eGFR < 30". Đoạn chứa nửa đầu không còn con số, đoạn chứa nửa sau không còn tên thuốc. Mục 3.3 đo hiện tượng này.
2. **Truy xuất trượt từ khóa hoặc trượt ngữ nghĩa.** Truy xuất dày hiểu "tiểu đường" gần nghĩa "đái tháo đường" nhưng dễ bỏ qua một mã cụ thể như "Nghị định 232"; truy xuất thưa thì ngược lại. Với cách chia đoạn cố định và truy xuất dense thuần, chỉ 71,05% câu hỏi có đoạn đúng ở vị trí đầu.

3. **Câu hỏi đa bước không nằm trong đoạn nào.** "Fed tăng lãi suất thì vàng SJC ra sao" cần nối ít nhất ba mẩu tri thức nằm ở ba tài liệu khác nhau; top-k theo độ tương đồng với câu hỏi không bảo đảm lấy đủ cả ba.

3.2 Thiết lập thực nghiệm truy xuất

3.2.1 Kho tri thức và bộ câu hỏi

Kho tri thức gồm 17 tài liệu Markdown, tám tài liệu y tế và chín tài liệu tài chính (Bảng 3.1). Mỗi tài liệu tổng hợp từ một văn bản gốc có số hiệu, viết lại bằng lời của tác giả và có cấu trúc tiêu đề hai cấp, mô phỏng cách một phác đồ hay một báo cáo phân tích được tổ chức trong thực tế.

Bảng 3.1. Kho tri thức 17 tài liệu của tiểu luận

Mã tài liệu	Miền	Tiêu đề	Số mục	Ký tự
`h01-chan-doan`	Y tế	Chẩn đoán đái tháo đường típ 2 theo hướng dẫn của Bộ Y tế	6	3.778
`h02-thuoc-dieu-tri`	Y tế	Thuốc điều trị đái tháo đường típ 2	8	3.962
`h03-ruou-bia`	Y tế	Rượu bia và tương tác với thuốc điều trị đái tháo đường	5	2.871
`h04-dinh-duong`	Y tế	Dinh dưỡng cho người đái tháo đường và chỉ số đường huyết của thực phẩm Việt Nam	6	3.523
`h05-van-dong`	Y tế	Vận động, kiểm soát cân nặng và lối sống	6	2.088
`h06-bien-chung`	Y tế	Biến chứng mạn tính và theo dõi định kỳ	5	2.004
`h07-ha-duong-huyet`	Y tế	Hạ glucose huyết và các biến chứng cấp	5	1.871
`h08-tien-dai-thao-duong`	Y tế	Tiền đái tháo đường, dịch tễ và phòng ngừa tại Việt Nam	4	2.060
`f01-vang-mieng-sjc`	Tài chính	Thị trường vàng miếng SJC và Nghị định 24/2012/NĐ-CP	5	2.672
`f02-gia-vang-the-gioi`	Tài chính	Giá vàng thế giới và công thức quy đổi sang giá trong nước	5	2.075
`f03-ty-gia`	Tài chính	Tỷ giá USD/VND và cơ chế điều hành	4	1.979
`f04-chinh-sach-tien-te`	Tài chính	Chính sách tiền tệ của Ngân hàng Nhà nước	5	1.967
`f05-fpt`	Tài chính	Công ty Cổ phần FPT (mã FPT)	4	1.617
`f06-hpg`	Tài chính	Tập đoàn Hòa Phát (mã HPG)	4	1.489
`f07-vcb`	Tài chính	Ngân hàng TMCP Ngoại thương Việt Nam - Vietcombank (mã VCB)	4	1.455
`f08-vic`	Tài chính	Tập đoàn Vingroup (mã VIC)	4	1.263
`f09-vn30`	Tài chính	Chỉ số VN30 và thị trường chứng khoán Việt Nam	4	1.685
Tổng		17 tài liệu	84	38.359

Mỗi tài liệu Markdown ghi rõ văn bản gốc (Quyết định 5481/QĐ-BYT, ADA, Nghị định, báo cáo thường niên) ở dòng đầu; kho được viết lại bằng lời của tác giả để học tập.

Bộ đánh giá có 114 câu hỏi (62 y tế, 52 tài chính). Mỗi câu hỏi đi kèm một *chuỗi đáp án* nằm nguyên văn trong kho (Bảng 3.2); câu hỏi được viết bằng cách diễn đạt khác với văn bản (“tiểu đường” thay cho “đái tháo đường”, “uống bia” thay cho “rượu bia”) để không chấm điểm sự trùng từ đơn thuần.

Bảng 3.2. Trích bộ câu hỏi đánh giá truy xuất

Mã	Câu hỏi	Đáp án tham chiếu
h01	Ngưỡng đường huyết lúc đói bao nhiêu thì được chẩn đoán là tiểu đường?	Glucose huyết tương lúc đói ≥ 126 mg/dL (7,0 mmol/L)
h02	Xét nghiệm HbA1c bao nhiêu phần trăm thì khẳng định đái tháo đường?	HbA1c ≥ 6,5% (48 mmol/mol)
h03	Nghiệm pháp uống 75 gam glucose, sau 2 giờ đạt mức nào là bệnh?	75 g ≥ 200 mg/dL (11,1 mmol/L)
h04	Tiểu đường típ 2 chiếm bao nhiêu phần trăm số ca?	chiếm khoảng 90 đến 95% tổng số ca đái tháo đường
f01	Nghị định 24 năm 2012 quy định gì về vàng miếng?	Nhà nước độc quyền sản xuất vàng miếng
f02	Thương hiệu vàng miếng quốc gia là của công ty nào?	thương hiệu vàng miếng SJC của Công ty Vàng bạc Đá quý Sài Gòn
f03	Tại sao giá vàng SJC cao hơn giá thế giới?	Nguồn cung vàng miếng bị giới hạn trong khi nhu cầu tích trữ của người dân tăng mạnh
f04	Chênh lệch vàng SJC đầu năm 2024 lên tới bao nhiêu?	lên tới khoảng 15 đến 18 triệu đồng mỗi lượng

Trích 8 trong 114 cặp hỏi đáp; mỗi câu gắn tài liệu chứa đáp án để chấm hit@k.

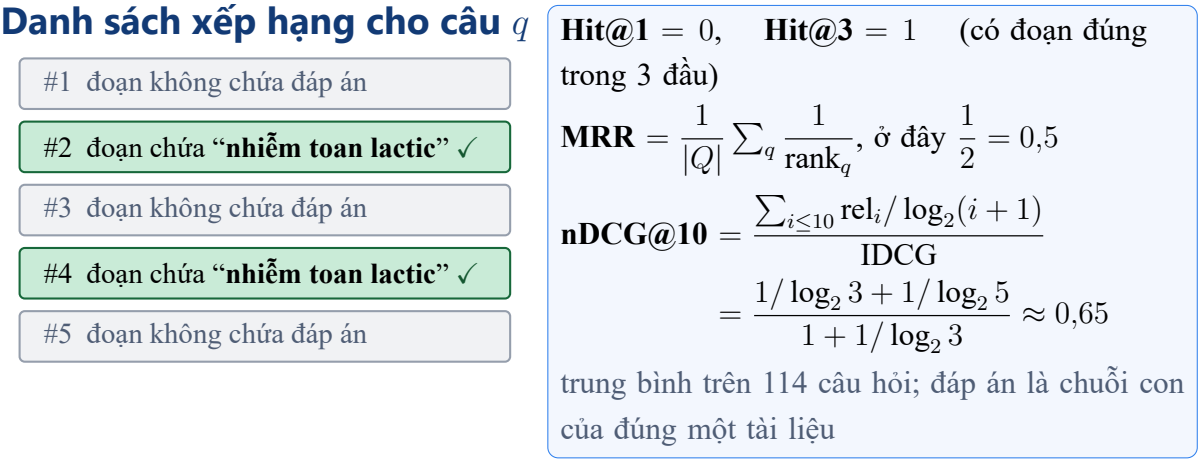
3.2.2 Độ đo

Một đoạn được coi là **liên quan** nếu nó chứa trọn chuỗi đáp án; định nghĩa này không cần gán nhãn tay và công bằng giữa các chiến lược chia đoạn, vì đoạn bị cắt ngang đáp án không được tính. Với r là thứ hạng của đoạn liên quan đầu tiên trong danh sách trả về, ba độ đo được dùng (Hình 3.3):

$$\text{Hit@}k = \mathbb{1}[r \le k], \qquad \text{MRR} = \frac{1}{|Q|}\sum_{q \in Q}\frac{1}{r_q}, \tag{3.2}$$

$$\text{nDCG@10} = \frac{1}{\text{IDCG@10}}\sum_{i=1}^{10}\frac{\text{rel}_i}{\log_2(i+1)}, \tag{3.3}$$

trong đó $\text{rel}_i \in \{0, 1\}$ cho biết đoạn ở vị trí i có liên quan không và IDCG là DCG của thứ tự lý tưởng. Hit@1 trả lời "đoạn đầu tiên có đúng không", câu hỏi quan trọng nhất khi chỉ đưa ít đoạn vào LLM; MRR phạt nhẹ khi đoạn đúng ở vị trí 2–3; nDCG thưởng việc đặt *mọi* đoạn liên quan lên trên.



Hình 3.3. Cách tính Hit@k, MRR và nDCG@10 trên một danh sách xếp hạng

```python
# tl2/retrieval.py
def rank_metrics(ranked, answer, n_relevant, ks=(1, 3, 5, 10)):
    """ranked: list[Chunk] theo thứ tự; n_relevant: số chunk trong kho chứa trọn câu trả lời.

    Trả hit@k, MRR và nDCG@10 nhị phân (nhiều chunk chồng lấn có thể cùng liên quan).
    """
    rel = [is_relevant(c, answer) for c in ranked]
    out = {f"hit@{k}": float(any(rel[:k])) for k in ks}
    first = next((i for i, r in enumerate(rel) if r), None)
    out["mrr"] = 0.0 if first is None else 1.0 / (first + 1)
    dcg = sum(r / np.log2(i + 2) for i, r in enumerate(rel[:10]))
    n_rel = max(1, min(10, n_relevant))
    idcg = sum(1 / np.log2(i + 2) for i in range(n_rel))
    out["ndcg@10"] = float(dcg / idcg) if any(rel[:10]) else 0.0
    return out
```

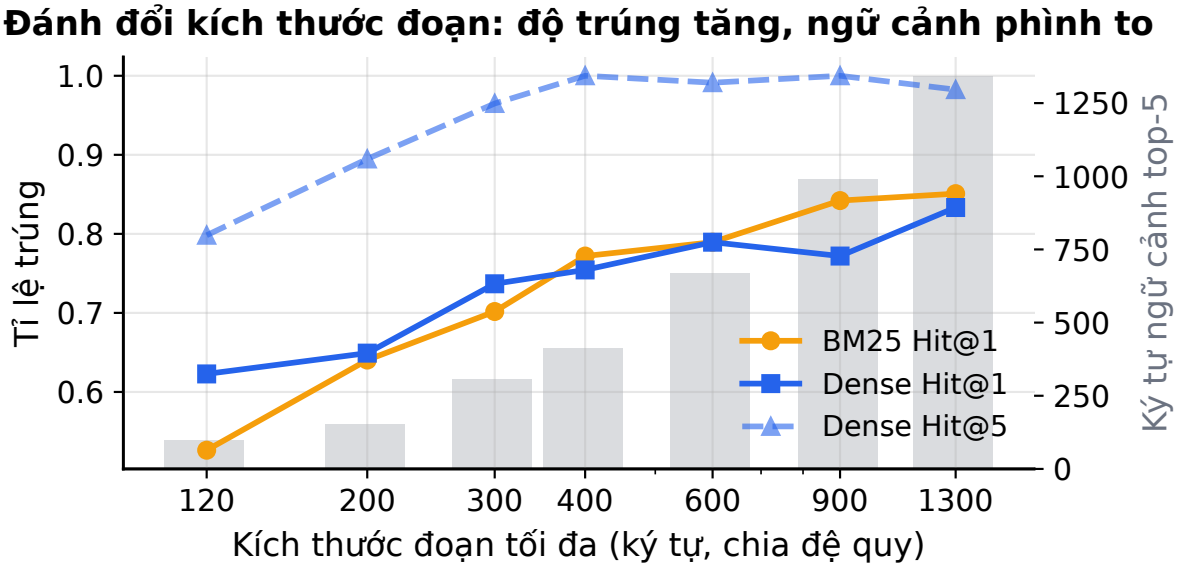
Đoạn mã 3.1. Tính Hit@k, MRR và nDCG@10 cho một câu hỏi

3.3 Các chiến lược phân mảnh (chunking)

3.3.1 Bài toán đánh đổi: độ dài đoạn và khoảng chồng lấn

Chọn cỡ đoạn là chọn giữa hai loại lỗi. Đoạn *ngắn* chính xác nhưng dễ cắt rời đáp án khỏi ngữ cảnh của nó; đoạn *dài* giữ trọn ngữ cảnh nhưng vector của nó là trung bình của nhiều ý, nên khó khớp với một câu hỏi cụ thể, và mỗi đoạn đưa vào LLM tốn nhiều token hơn. Chồng lấn (overlap) o giữa hai đoạn kề giảm xác suất cắt ngang câu: với cửa sổ w, bước trượt là $w - o$, số đoạn của tài liệu dài L là $\lceil (L - o)/(w - o) \rceil$.

Thí nghiệm quét bảy cỡ đoạn từ 120 tới 1300 ký tự (chia đệ quy, chồng lấn 15%) cho thấy cả hai mặt của đánh đổi (Hình 3.4, Bảng 3.3). Ở cỡ 120 ký tự, chỉ 95,61% câu hỏi còn đoạn chứa trọn đáp án, và dense Hit@1 chỉ đạt 62,28%. Tăng cỡ đoạn thì Hit@1 tăng, nhưng ngữ cảnh top-5 phình từ 95,94 lên 1.340 token. Lưu ý một hiệu ứng của cách chấm: ở cỡ 1300 ký tự kho chỉ còn 40 đoạn, gần với truy xuất cả tài liệu, nên tìm đúng "đoạn" dễ hơn nhiều; Hit@1 cao ở đó không có nghĩa LLM nhận được ngữ cảnh gọn. Vùng 400–900 ký tự là điểm cân bằng, và là cỡ được chọn cho các thí nghiệm sau.



Hình 3.4. Quét cỡ đoạn: tỉ lệ trúng tăng theo cỡ đoạn trong khi số ký tự ngữ cảnh top-5 tăng tuyến tính

Bảng 3.3. Ảnh hưởng của cỡ đoạn tới truy xuất BM25 và dense

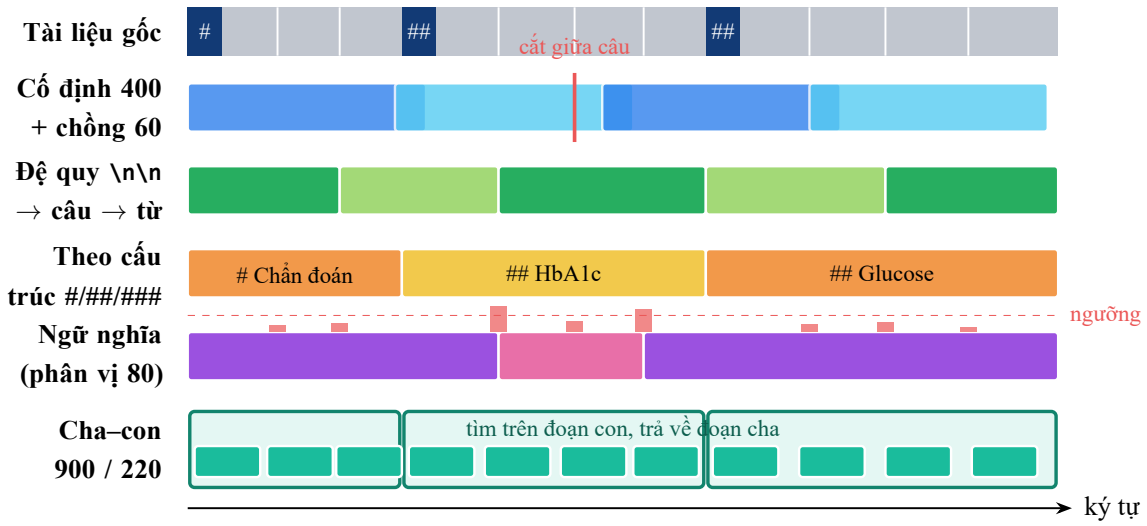
Cỡ đoạn	Số đoạn	Dài TB	Giữ trọn (%)	BM25 hit@1	BM25 MRR	Dense hit@1	Dense MRR	Token top-5
120	670	76,8	95,6	0,5263	0,6310	0,6228	0,6985	96
200	425	120,9	99,1	0,6404	0,7352	0,6491	0,7514	151
300	190	244,7	100,0	0,7018	0,8147	0,7368	0,8392	306
400	140	328,4	100,0	0,7719	0,8611	0,7544	0,8651	411
600	83	535,3	100,0	0,7895	0,8702	0,7895	0,8811	669
900	55	792,1	100,0	0,8421	0,9040	0,7719	0,8743	990
1.300	40	1.072,3	100,0	0,8509	0,9143	0,8333	0,9020	1.340

Chia đệ quy, chồng lấn 15% cỡ đoạn; token top-5: ước lượng 1 token ≈ 4 ký tự cho năm đoạn đầu đưa vào LLM.

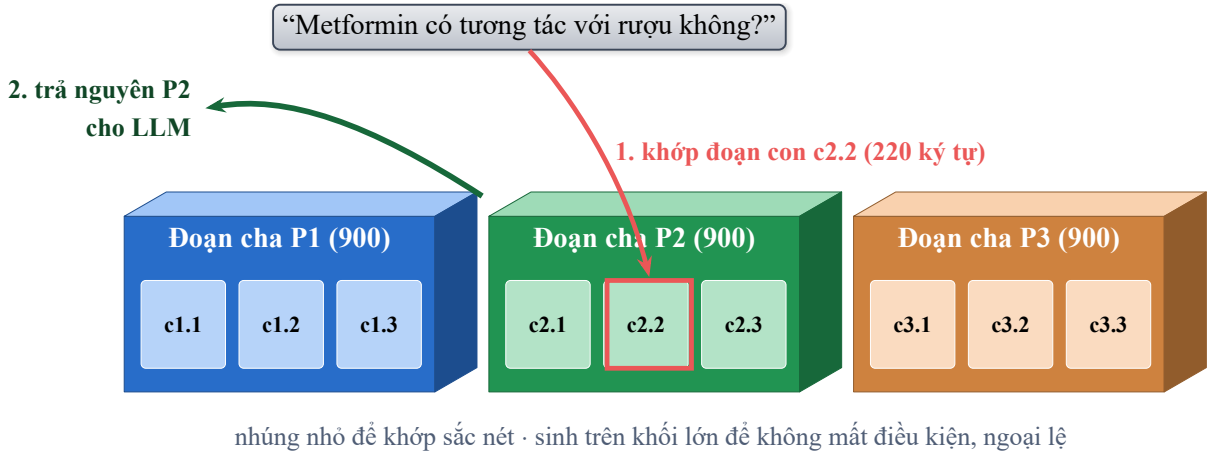
3.3.2 Năm chiến lược phân mảnh

Năm chiến lược được cài trong `tl2/chunking.py`, mọi chiến lược đều giữ vị trí ký tự gốc của đoạn để chấm điểm được (Hình 3.5):

1. **Cố định (fixed-size)**: cửa sổ trượt 400 ký tự, chồng lấn 60, không quan tâm ranh giới câu.
2. **Đệ quy (recursive)**: thử cắt ở dấu phân cách cấp cao nhất (đoạn trống, xuống dòng, dấu chấm, chấm phẩy, dấu phẩy, khoảng trắng) và chỉ xuống cấp thấp hơn khi mảnh vẫn quá dài; các mảnh nhỏ liền nhau được gộp tới giới hạn.
3. **Theo cấu trúc (structure-aware)**: mỗi mục Markdown là một đoạn, gắn chuỗi tiêu đề làm ngữ cảnh (“Thuốc điều trị > Metformin > Chống chỉ định”); mục quá 900 ký tự mới tách đệ quy bên trong.
4. **Ngữ nghĩa (semantic)**: nhúng từng câu, cắt khi khoảng cách cosin giữa hai câu liền nhau vượt phân vị thứ 80 của chuỗi khoảng cách trong tài liệu.
5. **Cha – con (parent-child, small-to-big)**: lập chỉ mục các đoạn con khoảng 220 ký tự để khớp chính xác, nhưng trả cho LLM đoạn cha là cả mục (Hình 3.6).



Hình 3.5. Năm chiến lược chia đoạn trên cùng một tài liệu: dải ngang là tài liệu, mỗi khối màu là một đoạn



Hình 3.6. Truy xuất cha – con: khớp trên đoạn con nhỏ, đưa đoạn cha đủ ngữ cảnh cho LLM

Bảng 3.4 và Hình 3.7 cho thấy hình dạng các đoạn sinh ra. Cố định cho độ dài gần như đều; theo cấu trúc dao động theo độ dài mục; ngữ nghĩa dao động mạnh nhất, có đoạn chỉ 2 ký tự (một dòng tiêu đề bị tách riêng) và có đoạn tới 1.018 ký tự. Chia cố định là chiến lược duy nhất làm mất đáp án: chỉ 99,12% câu hỏi còn đoạn chứa trọn chuỗi đáp án.

```python
# tl2/chunking.py
def _recursive_spans(text, s, e, size, seps):
    """Trả về các khoảng [s, e) dài ≤ size, ưu tiên cắt ở dấu phân cách cấp cao nhất."""
    if e - s <= size:
        return [(s, e)]
    for k, sep in enumerate(seps):
        cuts = [s + m.end() for m in re.finditer(re.escape(sep), text[s:e])]
        cuts = [c for c in cuts if c < e]
        if not cuts:
            continue
        pieces = list(zip([s] + cuts, cuts + [e]))
        # gộp các mảnh nhỏ liên tiếp cho tới khi chạm giới hạn size
        merged = []
        for a, b in pieces:
            if merged and b - merged[-1][0] <= size:
                merged[-1] = (merged[-1][0], b)
            else:
                merged.append((a, b))
        out = []
        for a, b in merged:
            out += _recursive_spans(text, a, b, size, seps[k + 1:]) if b - a > size else [(a, b)]
        return out
    return [(i, min(i + size, e)) for i in range(s, e, size)]
```

Đoạn mã 3.2. Lõi của chia đoạn đệ quy: cắt ở dấu phân cách cấp cao nhất có thể

```python
# tl2/chunking.py
def semantic(doc, embed, percentile=80, max_size=900):
    """Cắt khi khoảng cách cosin giữa hai câu liên tiếp vượt phân vị p của chuỗi khoảng cách.

    embed: hàm nhận list[str] trả về ma trận embedding đã chuẩn hóa L2.
    """
    sents = sentence_spans(doc.text)
    if len(sents) < 3:
        return [_mk(doc, 0, 0, len(doc.text), "se")]
    E = embed([doc.text[s:e] for s, e in sents])
    d = 1.0 - np.sum(E[:-1] * E[1:], axis=1)              # d_i = 1 - cos(e_i, e_{i+1})
    thr = np.percentile(d, percentile)
    out, cur = [], sents[0][0]
    for i in range(len(sents) - 1):
        nxt = sents[i + 1][0]
        if d[i] > thr or nxt - cur > max_size:
            out.append(_mk(doc, len(out), cur, sents[i][1], "se",
                           meta={"dist": float(d[i])}))
            cur = nxt
    out.append(_mk(doc, len(out), cur, sents[-1][1], "se"))
    return out, d, thr
```

Đoạn mã 3.3. Chia đoạn ngữ nghĩa theo phân vị khoảng cách cosin giữa các câu liền nhau

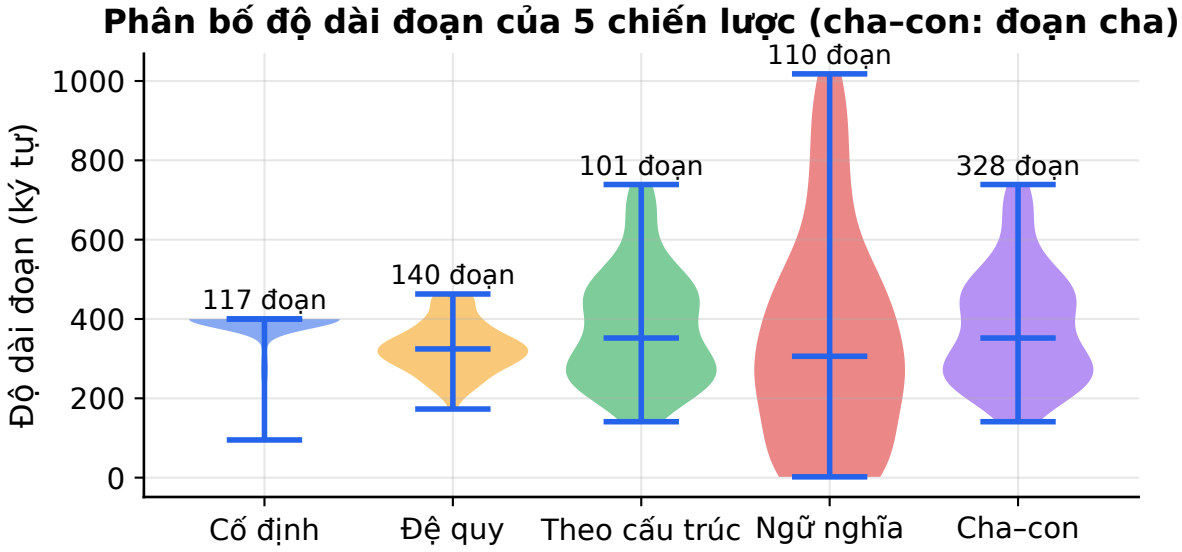
```python
# tl2/chunking.py
def parent_child(doc, parent_size=900, child_size=220):
    """Cha là mục theo cấu trúc; con là mảnh đệ quy nhỏ bên trong cha. Lập chỉ mục con, trả về cha."""
    parents = structure_aware(doc, size=parent_size)
    children = []
    for p in parents:
        for j, (s, e) in enumerate(_recursive_spans(doc.text, p.start, p.end,
                                   child_size, SEPARATORS)):
            if doc.text[s:e].strip():
                children.append(Chunk(f"{p.chunk_id}.c{j}", doc.doc_id,
                                      doc.domain, s, e, doc.text[s:e],
                                      heading=p.heading,
                                      parent_id=p.chunk_id))
    return parents, children
```

Đoạn mã 3.4. Chia đoạn cha – con: cha là mục, con là mảnh đệ quy nhỏ bên trong

Bảng 3.4. Thống kê đoạn của năm chiến lược trên toàn kho

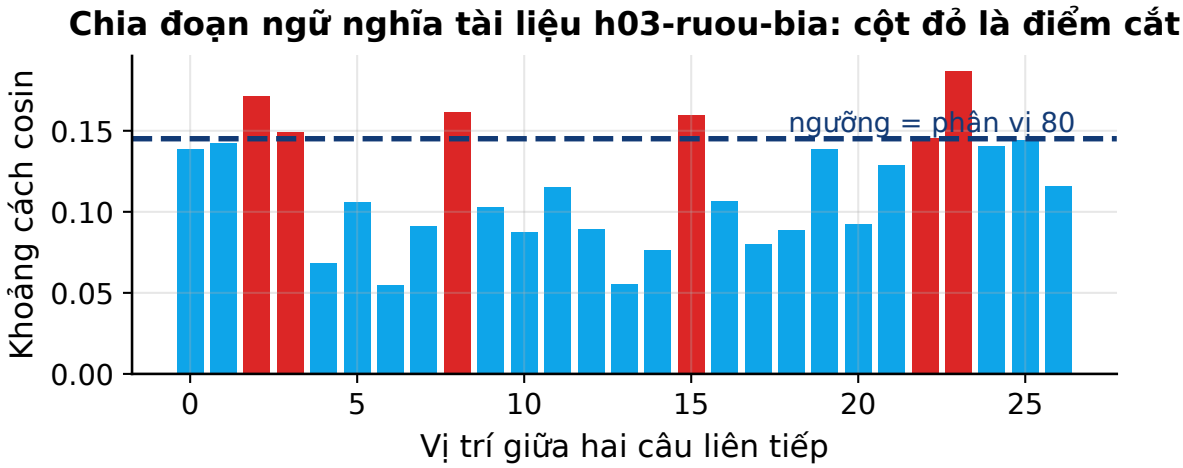
Chiến lược	Số đoạn	Đoạn cha	Dài TB	Ngắn nhất	Dài nhất	Giữ trọn đáp án (%)
Cố định 400 ký tự	117	–	379,1	95	400	99,1
Đệ quy (recursive)	140	–	328,4	173	463	100,0
Theo cấu trúc tiêu đề	101	–	379,8	141	739	100,0
Ngữ nghĩa (semantic)	110	–	347,4	2	1.018	100,0
Cha – con	328	101	379,8	141	739	100,0

Độ dài tính bằng ký tự. Giữ trọn đáp án: tỉ lệ câu hỏi có ít nhất một đoạn chứa trọn câu đáp án gốc. Với cha – con, đoạn con dùng để tìm, đoạn cha để đưa vào ngữ cảnh.



Hình 3.7. Phân bố độ dài đoạn của năm chiến lược

Hình 3.8 mở hộp chiến lược ngữ nghĩa trên tài liệu `h03-ruou-bia`: mỗi cột là khoảng cách giữa hai câu liền nhau, đường ngang là ngưỡng phân vị 80, cột đỏ là điểm cắt. Các điểm cắt rơi đúng vào chỗ văn bản chuyển chủ đề (từ cơ chế nhiễm toan lactic sang nguy cơ hạ glucose huyết), nhưng cũng có điểm cắt giữa một danh sách gạch đầu dòng vì các mục của danh sách ngắn và khác từ vựng.



Hình 3.8. Chia đoạn ngữ nghĩa: khoảng cách cosin giữa các câu liền nhau và điểm cắt

3.3.3 Ý nghĩa của chunking trong dữ liệu y tế và tài chính Việt Nam

Văn bản y tế và tài chính có hai đặc điểm làm chunking quan trọng hơn văn bản thông thường. Thứ nhất, *con số gắn chặt với ngữ cảnh*: "$\geq 6{,}5\%$" chỉ có nghĩa khi đi cùng "HbA1c" và "tiêu chuẩn chẩn đoán"; một đoạn chứa con số nhưng mất chủ ngữ là một đoạn nguy hiểm. Thứ hai, *tài liệu có cấu trúc chuẩn*: phác đồ Bộ Y tế, nghị định, báo cáo thường niên đều chia mục rõ ràng. Chiến lược theo cấu trúc khai thác đúng đặc điểm thứ hai để bảo vệ đặc điểm thứ nhất: tiêu đề mục được gắn vào đoạn, nên đoạn "Chống chỉ định" luôn biết nó thuộc mục "Metformin". Tiếng Việt còn có thêm một khó khăn: từ được ghép từ nhiều âm tiết cách nhau bởi khoảng trắng, nên cắt theo khoảng trắng có thể tách "đái tháo" khỏi "đường"; chia đệ quy ưu tiên dấu câu trước khoảng trắng để tránh điều này.

3.4 Truy xuất: thưa, dày, lai và xếp lại

3.4.1 Truy xuất thưa: TF-IDF và BM25

Truy xuất thưa biểu diễn văn bản bằng tần suất từ. BM25 [16] cho điểm tài liệu d với câu hỏi q gồm các từ t bằng

$$\mathrm{BM25}(q,d) = \sum_{t \in q} \mathrm{IDF}(t)\, \frac{f(t,d)\,(k_1+1)}{f(t,d) + k_1 \left(1 - b + b\,\frac{|d|}{\mathrm{avgdl}}\right)}, \tag{3.4}$$

với $f(t,d)$ là tần suất từ, $|d|$ là độ dài đoạn, $k_1 = 1{,}5$ điều tiết bão hòa tần suất và $b = 0{,}75$ chuẩn hóa độ dài. Tách từ ở mức âm tiết viết thường, giữ số và đơn vị như "mg", "dl".

```python
# tl2/retrieval.py
class BM25Retriever:
    name = "BM25"

    def __init__(self, chunks, k1=1.5, b=0.75):
        self.chunks = chunks
        self.bm25 = BM25Okapi([tokenize(c.index_text()) for c in chunks],
        ↪  k1=k1, b=b)

    def scores(self, query):
        return self.bm25.get_scores(tokenize(query))

    def search(self, query, k=5):
        return _topk(self.chunks, self.scores(query), k)
```

Đoạn mã 3.5. Bộ truy xuất BM25 trên các âm tiết tiếng Việt

3.4.2 Truy xuất dày: mô hình nhúng đa ngôn ngữ

Truy xuất dày [17] dùng một bộ mã hóa Transformer ánh xạ câu hỏi và đoạn văn vào cùng một không gian vector. Bài dùng `multilingual-e5-base` [18], mô hình được huấn luyện tương phản trên dữ liệu đa ngôn ngữ, yêu cầu tiền tố `query:` cho câu hỏi và `passage:` cho đoạn văn. Đây là kiến trúc **bi-encoder** [19]: câu hỏi và đoạn văn được mã hóa *riêng*, nên vector đoạn văn tính trước một lần khi lập chỉ mục (Hình 3.12).

So sánh ba mô hình nhúng (Bảng 3.5, Hình 3.9) cho thấy họ e5 vượt xa `paraphrase-multilingual-MiniLM`: Hit@1 84,21% so với 67,54%. Bản e5-small và e5-base có Hit@1 bằng nhau trên bộ câu hỏi này nhưng phân bổ khác theo miền: e5-base tốt hơn ở y tế, e5-small tốt hơn ở tài chính. Bài chọn e5-base vì y tế là miền đòi hỏi độ chính xác cao hơn.

```python
# tl2/retrieval.py
class Embedder:
    """Bọc SentenceTransformer; mô hình e5 cần tiền tố "query: " / "passage: "."""

    _cache = {}

    def __init__(self, key="e5-base", half=False):
        """half=True: trọng số fp16, dùng khi chung tiến trình với LLM (GPU qua WDDM tính cả vào commit của Windows)."""
        from sentence_transformers import SentenceTransformer
        self.key = key
        if (key, half) not in Embedder._cache:
            model = SentenceTransformer(hf_path(EMBEDDERS[key]), device=device())
            Embedder._cache[(key, half)] = model.half() if half else model
        self.model = Embedder._cache[(key, half)]
        self.is_e5 = key.startswith("e5")

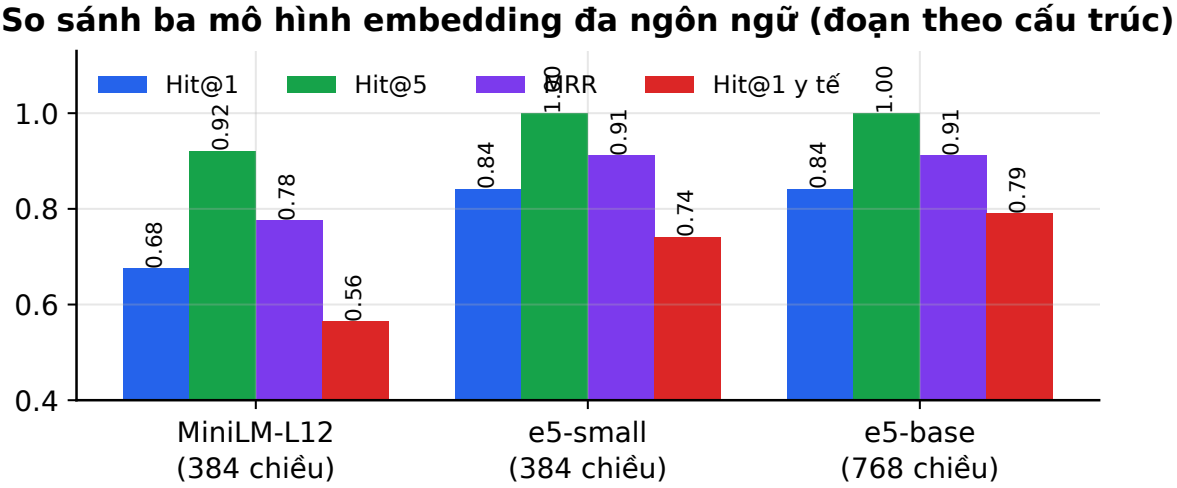
    def encode(self, texts, kind="passage", batch_size=64):
        if self.is_e5:
            texts = [f"{kind}: {t}" for t in texts]
        return self.model.encode(texts, batch_size=batch_size, normalize_embeddings=True, convert_to_numpy=True,
                                 show_progress_bar=False)
```

Đoạn mã 3.6. Bọc SentenceTransformer cho mô hình e5: tiền tố, chuẩn hóa L2, chạy GPU

Bảng 3.5. So sánh ba mô hình nhúng đa ngôn ngữ (truy xuất dense, đoạn theo cấu trúc)

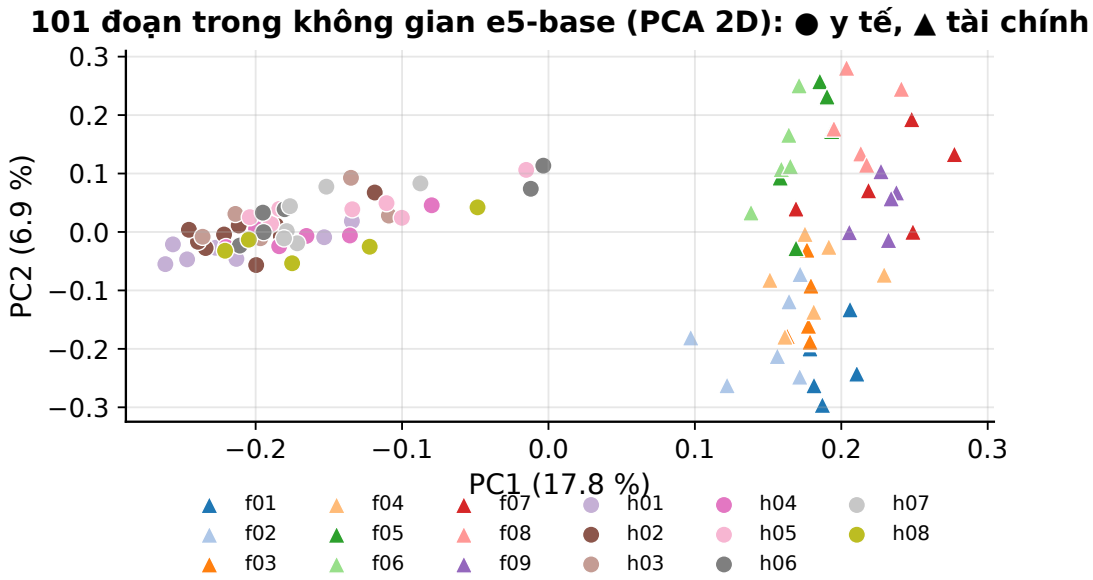
Mô hình nhúng	hit@1	hit@3	hit@5	MRR	nDCG@10	hit@1 y tế	hit@1 tài chính
`paraphrase-multilingual-MiniLM-L12-v2`	0,6754	0,8421	0,9211	0,7757	0,8181	0,5645	0,8077
`multilingual-e5-small`	**0,8421**	0,9825	**1,0000**	**0,9118**	0,9337	0,7419	**0,9615**
`multilingual-e5-base`	**0,8421**	**0,9912**	**1,0000**	0,9115	**0,9342**	**0,7903**	0,9038

Cùng chia đoạn theo cấu trúc, truy xuất dense thuần; e5 dùng tiền tố query:/passage:.

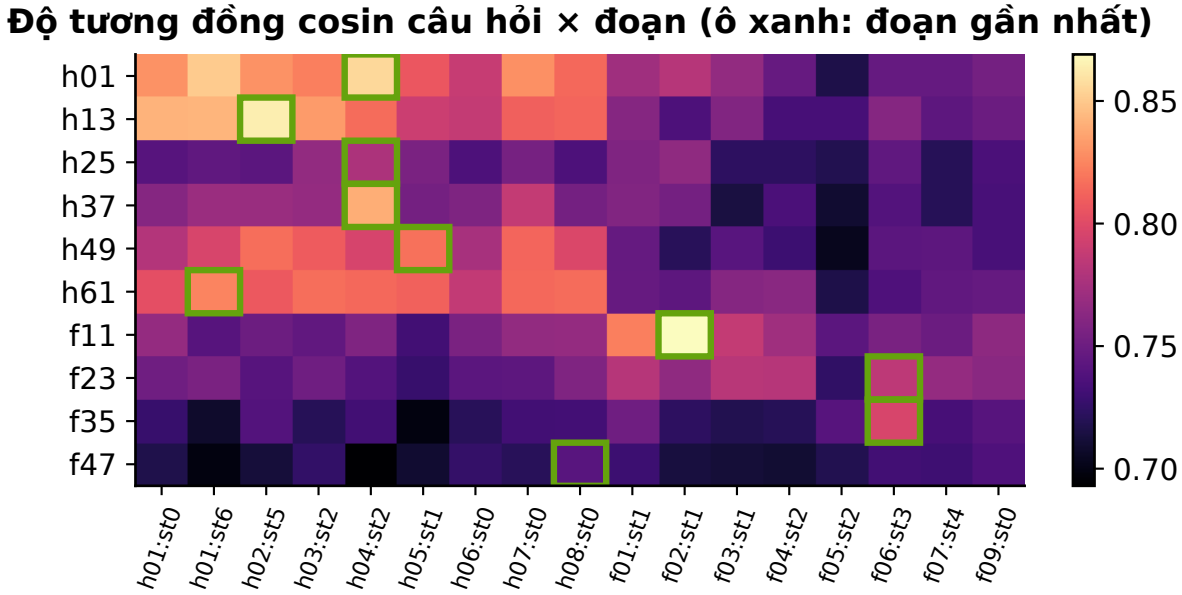


Hình 3.9. Ba mô hình nhúng đa ngôn ngữ trên cùng bộ câu hỏi

Hình 3.10 chiếu 101 đoạn theo cấu trúc xuống hai chiều bằng PCA: đoạn y tế và đoạn tài chính tách thành hai vùng rõ rệt, và trong mỗi vùng các đoạn cùng tài liệu nằm gần nhau. Hai thành phần đầu chỉ giữ 17,78% và 6,86% phương sai, nhắc rằng không gian 768 chiều phức tạp hơn nhiều so với những gì một hình hai chiều thể hiện. Hình 3.11 là ma trận cosin giữa mười câu hỏi và các đoạn: ô sáng nhất mỗi hàng thường là đoạn đúng, nhưng khoảng cách giữa ô đúng và các ô kế bên nhỏ, lý do cần thêm bước xếp lại.



Hình 3.10. Các đoạn trong không gian nhúng e5-base chiếu PCA hai chiều



Hình 3.11. Độ tương đồng cosin giữa câu hỏi và đoạn văn

3.4.3 Truy xuất lai bằng Reciprocal Rank Fusion

BM25 và dense mắc lỗi ở những câu hỏi khác nhau, nên gộp chúng lại có lợi. Điểm số của hai bộ truy xuất không cùng thang đo (BM25 không giới hạn, cosin trong $[-1, 1]$), nên thay vì cộng điểm, Reciprocal Rank Fusion (RRF) [20] cộng

nghịch đảo thứ hạng:

$$\mathrm{RRF}(d) = \sum_{r \in \{\mathrm{BM25},\ \mathrm{dense}\}} \frac{1}{k + \mathrm{rank}_r(d)}, \qquad k = 60. \tag{3.5}$$

Hằng số k làm phẳng đóng góp của các vị trí đầu, để một đoạn đứng hạng 1 ở một bộ truy xuất nhưng hạng 50 ở bộ kia không áp đảo một đoạn đứng hạng 3 ở cả hai.

```python
# tl2/retrieval.py
class HybridRRF:
    """Reciprocal Rank Fusion: s(d) = Σ_r 1 / (k + rank_r(d)), không cần chuẩn hóa thang điểm."""

    def __init__(self, retrievers, k=60):
        self.retrievers, self.k = retrievers, k
        self.chunks = retrievers[0].chunks
        self.name = "Hybrid RRF (" + " + ".join(r.name for r in retrievers) + ")"

    def scores(self, query):
        fused = np.zeros(len(self.chunks))
        for r in self.retrievers:
            rank = np.empty(len(self.chunks), dtype=int)
            rank[np.argsort(-r.scores(query))] = np.arange(1, len(self.chunks) + 1)
            fused += 1.0 / (self.k + rank)
        return fused

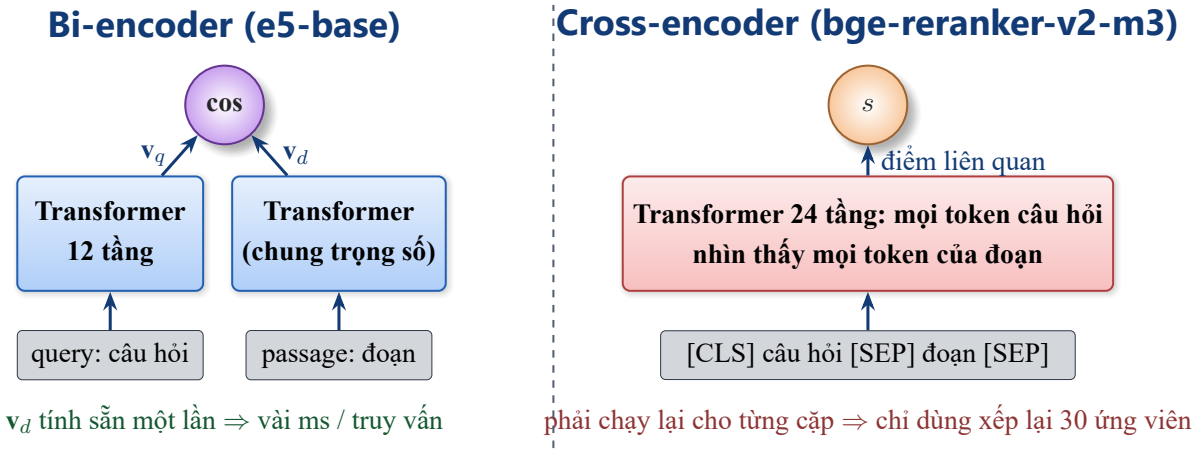
    def search(self, query, k=5):
        return _topk(self.chunks, self.scores(query), k)
```

Đoạn mã 3.7. Truy xuất lai: hợp nhất thứ hạng BM25 và dense bằng RRF

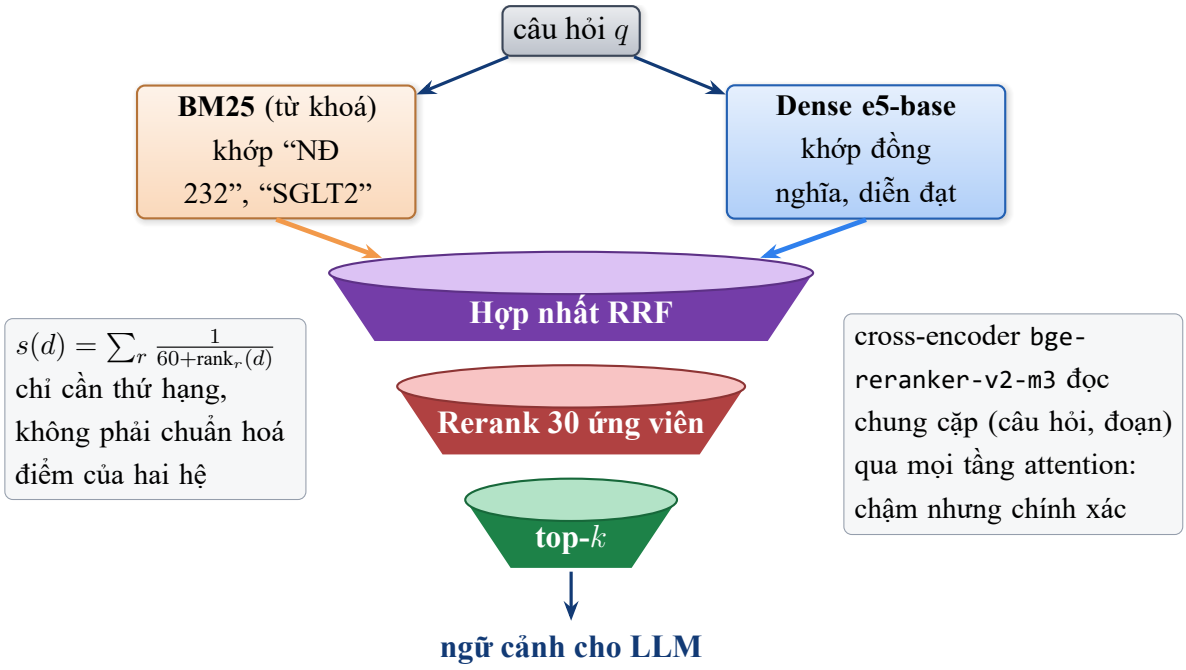
3.4.4 Xếp lại bằng cross-encoder

Bi-encoder nhanh nhưng không bao giờ nhìn câu hỏi và đoạn văn *cùng lúc*. Cross-encoder [21] nối hai chuỗi thành một đầu vào và cho Transformer chú ý chéo giữa chúng, đầu ra là một điểm liên quan trực tiếp $s(q,d) = \sigma\big(\mathbf{w}^{\top}\mathrm{Enc}([q;d])_{\texttt{[CLS]}}\big)$ (Hình 3.12). Cái giá là phải chạy mô hình một lần cho *mỗi* cặp, nên chỉ dùng được

ở giai đoạn hai trên một tập ứng viên nhỏ. Bài dùng `bge-reranker-v2-m3` [22], xếp lại 30 ứng viên của truy xuất lai (Hình 3.13).



Hình 3.12. Bi-encoder mã hóa riêng câu hỏi và đoạn văn rồi so cosin; cross-encoder mã hóa chung cả cặp và cho điểm trực tiếp



Hình 3.13. Phễu truy xuất lai: BM25 và dense chạy song song, hợp nhất RRF, cross-encoder xếp lại 30 ứng viên và giữ top-k

3.4.5 Kết quả: chiến lược chia đoạn × bộ truy xuất

Bảng 3.6 và Hình 3.14 là kết quả chính của chương: 25 tổ hợp chia đoạn và truy xuất trên 114 câu hỏi. Tổ hợp tốt nhất là **chia theo cấu trúc + truy xuất lai +**

```python
# tl2/retrieval.py
class Reranker:
    """Cross-encoder đọc đồng thời (câu hỏi, đoạn văn) và chấm mức liên quan; chậm nhưng chính xác."""

    _model = None

    def __init__(self):
        from sentence_transformers import CrossEncoder
        if Reranker._model is None:
            Reranker._model = CrossEncoder(hf_path(RERANKER), device=device(), max_length=512,
                                           model_kwargs={"torch_dtype": "bfloat16"})
        self.model = Reranker._model

    def rerank(self, query, hits, k=5):
        if not hits:
            return hits
        s = self.model.predict([(query, c.index_text()) for c, _ in hits], batch_size=32, show_progress_bar=False)
        order = np.argsort(-s)[:k]
        return [(hits[i][0], float(s[i])) for i in order]
```

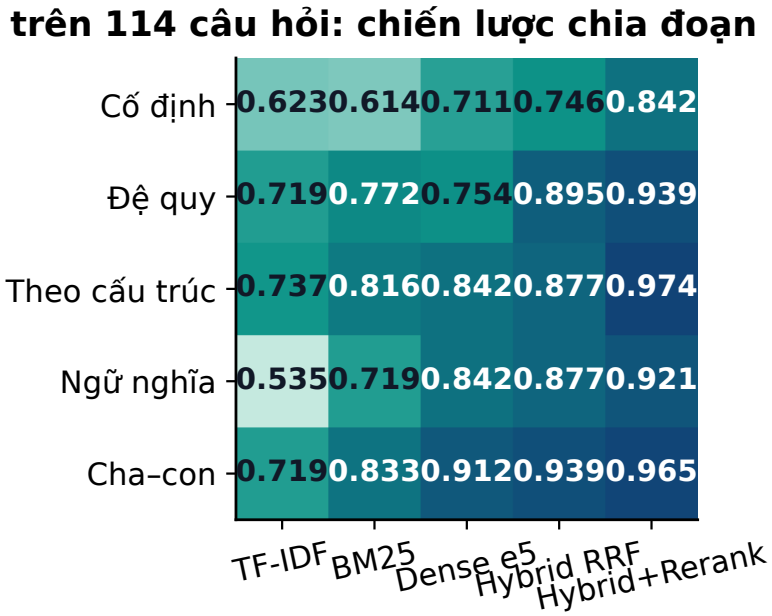
Đoạn mã 3.8. Cross-encoder bge-reranker-v2-m3 chạy bf16 trên GPU

xếp lại, Hit@1 97,37% và MRR 0,9854; tổ hợp Naive RAG (cố định + dense) chỉ đạt 71,05%.

Bảng 3.6. Hit@1 của 25 tổ hợp chiến lược chia đoạn và bộ truy xuất

Chiến lược chia đoạn	TF-IDF	BM25	Dense e5-base	Hybrid RRF	Hybrid + rerank
Cố định 400 ký tự	0,6228	0,6140	0,7105	0,7456	**0,8421**
Đệ quy (recursive)	0,7193	0,7719	0,7544	0,8947	**0,9386**
Theo cấu trúc tiêu đề	0,7368	0,8158	0,8421	0,8772	**0,9737**
Ngữ nghĩa (semantic)	0,5351	0,7193	0,8421	0,8772	**0,9211**
Cha – con	0,7193	0,8333	0,9123	0,9386	**0,9649**

114 câu hỏi trên 17 tài liệu; chữ đậm: bộ truy xuất tốt nhất của mỗi chiến lược.



Hình 3.14. Bản đồ nhiệt Hit@1: chiến lược chia đoạn (hàng) × bộ truy xuất (cột)

Bảng 3.7. MRR của 25 tổ hợp chiến lược chia đoạn và bộ truy xuất

Chiến lược chia đoạn	TF-IDF	BM25	Dense e5-base	Hybrid RRF	Hybrid + rerank
Cố định 400 ký tự	0,7650	0,7723	0,8382	0,8519	**0,9096**
Đệ quy (recursive)	0,8246	0,8611	0,8651	0,9375	**0,9667**
Theo cấu trúc tiêu đề	0,8423	0,8921	0,9115	0,9386	**0,9854**
Ngữ nghĩa (semantic)	0,7224	0,8341	0,9130	0,9298	**0,9547**
Cha – con	0,8290	0,8942	0,9547	0,9671	**0,9795**

114 câu hỏi trên 17 tài liệu; chữ đậm: bộ truy xuất tốt nhất của mỗi chiến lược.

Ba quan sát rút ra từ lưới:

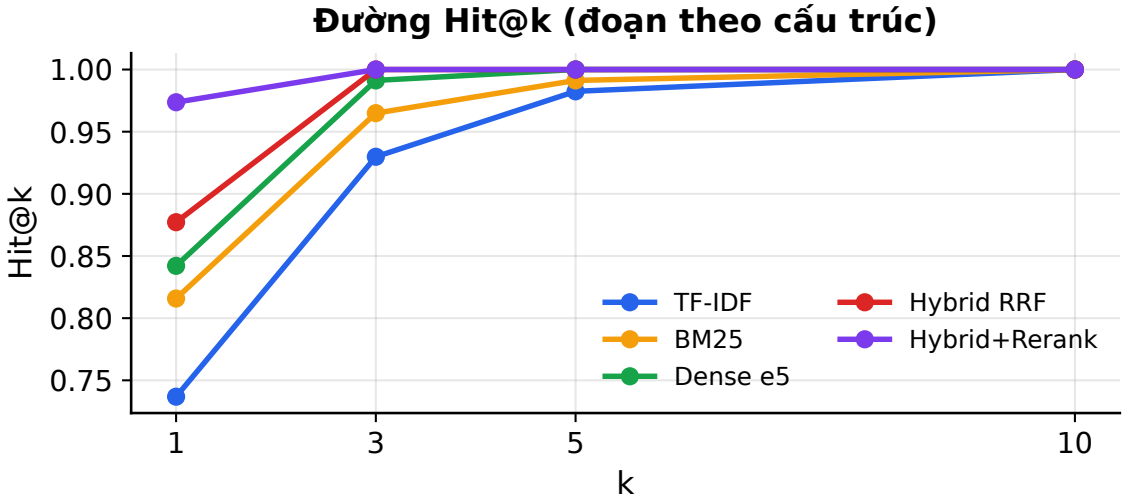
- **Chiến lược chia đoạn quan trọng ngang với bộ truy xuất.** Cùng bộ hybrid + rerank, đổi từ chia cố định sang chia theo cấu trúc nâng Hit@1 từ 84,21% lên 97,37%.
- **Mỗi tầng của phễu đều có ích.** Với chia theo cấu trúc: BM25 81,58%, dense 84,21%, lai 87,72%, lai + xếp lại 97,37%.
- **Cha – con mạnh nhất khi không có reranker.** Khớp trên đoạn con nhỏ cho dense Hit@1 91,23%, cao nhất cột dense; khi đã có reranker thì ưu thế này thu hẹp.

Bảng 3.8 đi sâu vào chiến lược theo cấu trúc. Hình 3.15 là đường Hit@k: mọi bộ truy xuất đều tìm được đoạn đúng trong top-10 ở gần như mọi câu hỏi; khác biệt nằm ở *thứ hạng*, đúng điều mà reranker cải thiện. Hình 3.16 tách theo miền: câu hỏi y tế khó hơn tài chính với mọi bộ truy xuất, vì nhiều tài liệu y tế cùng nói về một bệnh và dùng chung từ vựng (metformin xuất hiện trong bốn tài liệu).

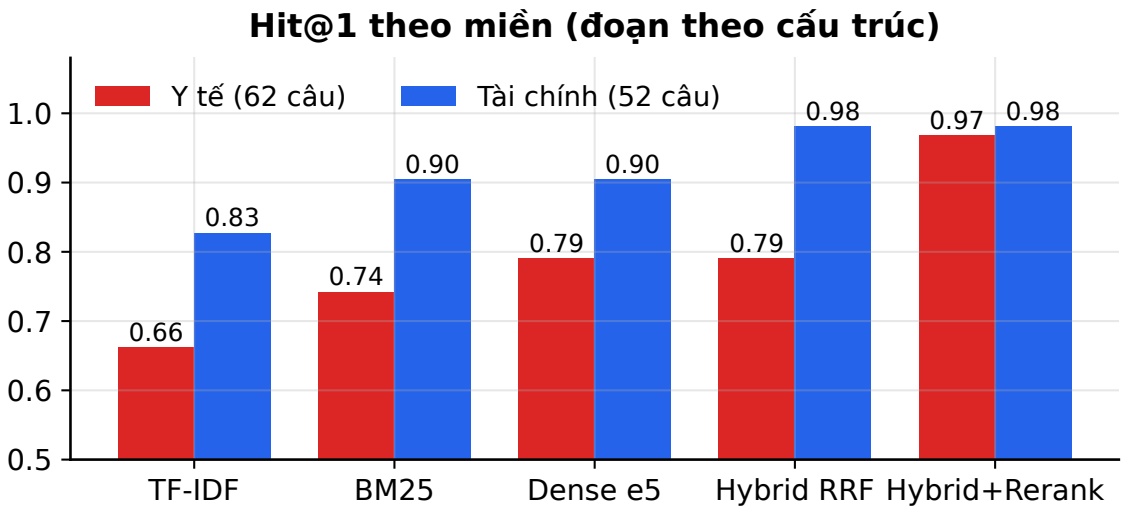
Bảng 3.8. Chi tiết năm bộ truy xuất trên đoạn chia theo cấu trúc

Bộ truy xuất	hit@1	hit@3	hit@5	MRR	nDCG@10	hit@1 y tế	hit@1 tài chính	Trễ (ms)
TF-IDF	0,7368	0,9298	0,9825	0,8423	0,8823	0,6613	0,8269	1,20
BM25	0,8158	0,9649	0,9912	0,8921	0,9187	0,7419	0,9038	**0,53**
Dense e5-base	0,8421	0,9912	**1,0000**	0,9115	0,9342	0,7903	0,9038	14,97
Hybrid RRF	0,8772	**1,0000**	**1,0000**	0,9386	0,9540	0,7903	**0,9808**	20,63
Hybrid + rerank	**0,9737**	**1,0000**	**1,0000**	**0,9854**	**0,9891**	**0,9677**	**0,9808**	439,50

Chia đoạn theo cấu trúc tiêu đề. Trễ: trung vị mỗi câu hỏi, gồm mã hóa câu hỏi trên GPU; rerank chấm lại 30 ứng viên của hybrid.

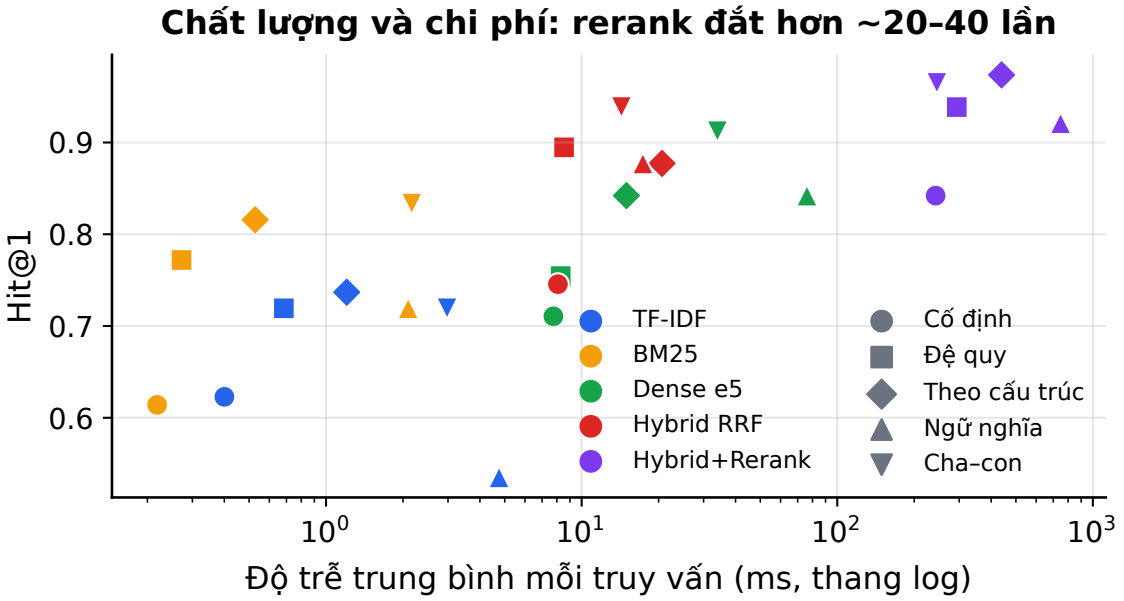


Hình 3.15. Đường Hit@k của năm bộ truy xuất



Hình 3.16. Hit@1 theo miền y tế và tài chính

Chất lượng có giá. Hình 3.17 đặt Hit@1 cạnh độ trễ: BM25 trả lời trong 0,5281 ms, dense trong 14,97 ms (gồm mã hóa câu hỏi trên GPU), còn hybrid + rerank mất 439,50 ms vì phải chạy cross-encoder 30 lần. Với một tác tử hội thoại, nửa giây là chấp nhận được; với tìm kiếm thời gian thực trên hàng triệu đoạn thì cần giảm số ứng viên hoặc dùng reranker nhỏ hơn.



Hình 3.17. Chất lượng và độ trễ của 25 tổ hợp: xếp lại đắt hơn nhiều nhưng chính xác hơn

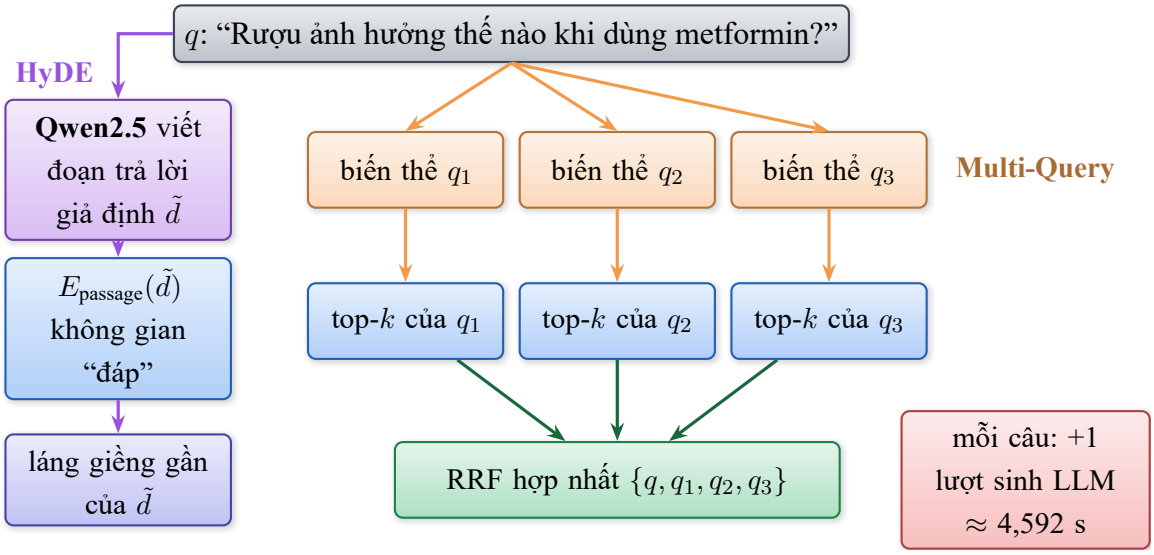
3.5 Advanced RAG: tối ưu đa chặng

Advanced RAG giữ khung truy xuất – sinh của Naive RAG nhưng thêm bước tối ưu ở hai đầu:

- **Tiền truy xuất** sửa *câu hỏi*: viết lại câu hỏi cho rõ, tách câu hỏi phức thành câu hỏi con, sinh nhiều cách hỏi (Multi-Query), hoặc sinh một câu trả lời giả định rồi dùng nó để tìm (HyDE).
- **Truy xuất** cải tiến bằng chia đoạn tốt hơn, truy xuất lai, lọc theo metadata (miền, tài liệu).
- **Hậu truy xuất** sửa *ngữ cảnh*: xếp lại bằng cross-encoder, nén hoặc cắt bớt đoạn, sắp xếp lại để thông tin quan trọng nằm ở đầu và cuối prompt.

Phần truy xuất lai và xếp lại đã được đo ở mục trước. Mục này đo hai kỹ thuật tiền truy xuất dùng LLM cục bộ (Hình 3.18).

HyDE (Hypothetical Document Embeddings) [23] dựa trên nhận xét: câu hỏi và đoạn trả lời thường không giống nhau về hình thức, nhưng hai đoạn trả lời thì giống nhau. Thay vì nhúng câu hỏi q, HyDE cho LLM viết một đoạn trả lời giả định $\tilde{d} = \text{LLM}(q)$ rồi tìm bằng $f(\tilde{d})$. Biến thể *HyDE + câu hỏi* hợp nhất bằng RRF hai danh sách tìm bằng $f(q)$ và $f(\tilde{d})$. **Multi-Query** cho LLM viết ba cách hỏi khác q_1, q_2, q_3, truy xuất cho từng câu và hợp nhất bốn danh sách bằng RRF.



Hình 3.18. Hai kỹ thuật tối ưu tiền truy xuất bằng LLM cục bộ: HyDE tìm bằng câu trả lời giả định, Multi-Query hợp nhất nhiều cách hỏi bằng RRF

```python
# scripts/run_advanced.py
def rrf(score_lists, k=60):
    fused = np.zeros(len(score_lists[0]))
    for s in score_lists:
        rank = np.empty(len(s), dtype=int)
        rank[np.argsort(-s)] = np.arange(1, len(s) + 1)
        fused += 1.0 / (k + rank)
    return fused
```

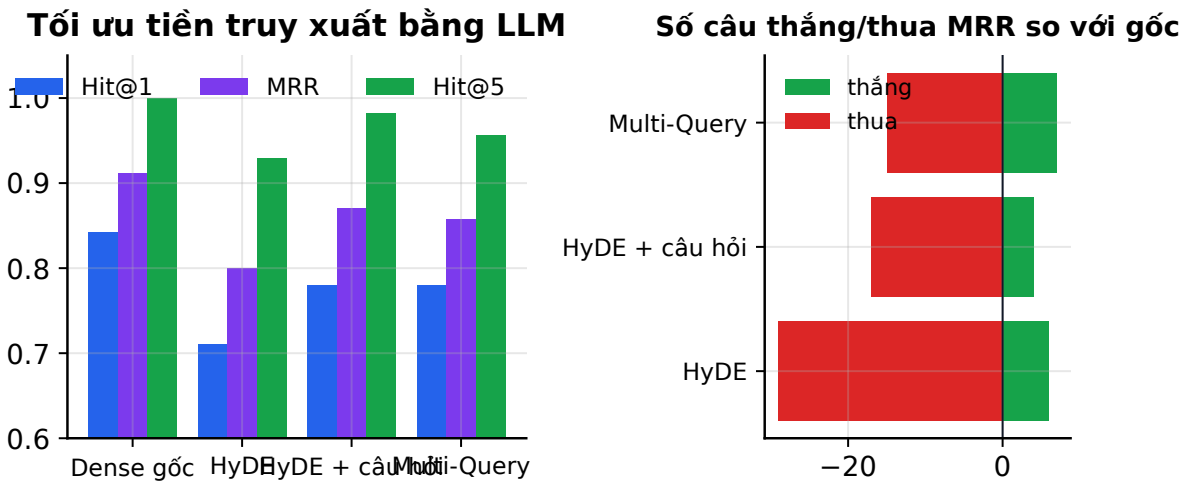
Đoạn mã 3.9. Hợp nhất nhiều danh sách xếp hạng bằng RRF trong thí nghiệm Advanced RAG

Kết quả (Bảng 3.9, Hình 3.19) là một kết quả âm có giá trị. Với Qwen2.5-1.5B, HyDE *làm giảm* Hit@1 từ 84,21% xuống 71,05%, thua dense gốc ở 29 câu và chỉ thắng ở 6 câu. Multi-Query giảm ít hơn (78,07%) nhưng vẫn giảm. Bảng 3.10 giải thích lý do: đoạn giả định của mô hình nhỏ chứa các chi tiết *bịa*, ví dụ nói metformin cùng rượu gây "tăng huyết áp", và vector của chi tiết bịa kéo truy xuất về các đoạn sai. HyDE chỉ có lợi khi LLM sinh đoạn giả định đủ đúng để trông giống đoạn thật; với một mô hình 1,5 tỉ tham số trên miền chuyên sâu tiếng Việt, điều kiện đó không thỏa. Mỗi câu hỏi còn tốn thêm 4,592 s sinh văn bản. Vì vậy tác tử ở Chương 6 không dùng HyDE hay Multi-Query mà dùng truy xuất lai + xếp lại.

Bảng 3.9. Tối ưu tiền truy xuất bằng LLM so với truy xuất dense gốc

Phương pháp	hit@1	hit@5	MRR	nDCG@10	Thắng	Thua	Sinh (s)
Dense e5-base (gốc)	**0,8421**	**1,0000**	**0,9115**	**0,9342**	–	–	–
HyDE (chỉ đoạn giả định)	0,7105	0,9298	0,7997	0,8414	6	29	4,59
HyDE + câu hỏi (RRF)	0,7807	0,9825	0,8703	0,9021	4	17	4,59
Multi-Query (4 truy vấn, RRF)	0,7807	0,9561	0,8579	0,8888	7	15	3,65

Thắng/thua: số câu hỏi có MRR cao/thấp hơn dense gốc. Sinh: trung vị thời gian Qwen2.5-1.5B viết đoạn giả định hoặc ba câu hỏi biến thể.



Hình 3.19. Ảnh hưởng của HyDE và Multi-Query: Hit@1, MRR và số câu thắng, thua so với dense gốc

Bảng 3.10. Ví dụ văn bản do Qwen2.5-1.5B sinh trong HyDE và Multi-Query

Mã	Câu hỏi	Đoạn giả định HyDE	Biến thể 1
h26	Đang dùng metformin mà uống bia rượu thì nguy hiểm gì?	Đối với người đang sử dụng metformin, việc uống bia rượu có thể gây ra các tác dụng phụ như tăng huyết áp, giảm đường huyết, và thậm chí gây ra các vấn đề về gan. Vì vậy, tốt nhất là kiêng uống bia rượu khi đang dùng thuốc này để đảm bảo an toàn và hiệu quả…	Sử dụng metformin cùng với bia rượu có thể gây ra những tác động nào không?
h41	Tại sao người tiểu đường nên tránh gà rán?	Người tiểu đường cần tránh ăn gà rán vì: 1. Gà rán thường chứa nhiều chất béo bão hòa và cholesterol cao, làm tăng nguy cơ bệnh tim mạch. 2. Chất béo trong gà rán không được chuyển hóa thành năng lượng như glucose, nhưng vẫn gây tăng cân và tăng huyết áp. 3.…	Người tiểu đường cần tránh ăn gà rán vì nó chứa nhiều đường và chất béo không tốt.
h56	Xử trí hạ đường huyết khi còn tỉnh theo quy tắc nào?	Khi còn tỉnh, xử trí hạ đường huyết bằng quy tắc “Điều chỉnh - Tăng cường - Nâng cao”. Điều chỉnh lượng carbohydrate ăn; tăng cường hoạt động thể chất; nâng cao chế độ ăn uống.	Cách thứ nhất:

Văn bản sinh nguyên văn (cắt bớt); đoạn HyDE chứa cả chi tiết đúng lẫn chi tiết bịa.

3.6 GraphRAG: tổng hợp tri thức trên đồ thị

3.6.1 Ý tưởng và quy trình

Advanced RAG vẫn coi tri thức là một tập đoạn văn rời rạc. GraphRAG [7] thay đơn vị tri thức bằng **đồ thị thực thể**, rồi thêm một tầng trừu tượng là **cộng đồng**: nhóm thực thể liên kết chặt được tóm tắt trước thành một “báo cáo cộng đồng”. Quy trình gồm bốn bước (Hình 3.20):

1. **Đồ thị thực thể**: ở bài báo gốc, LLM trích xuất thực thể và quan hệ từ văn

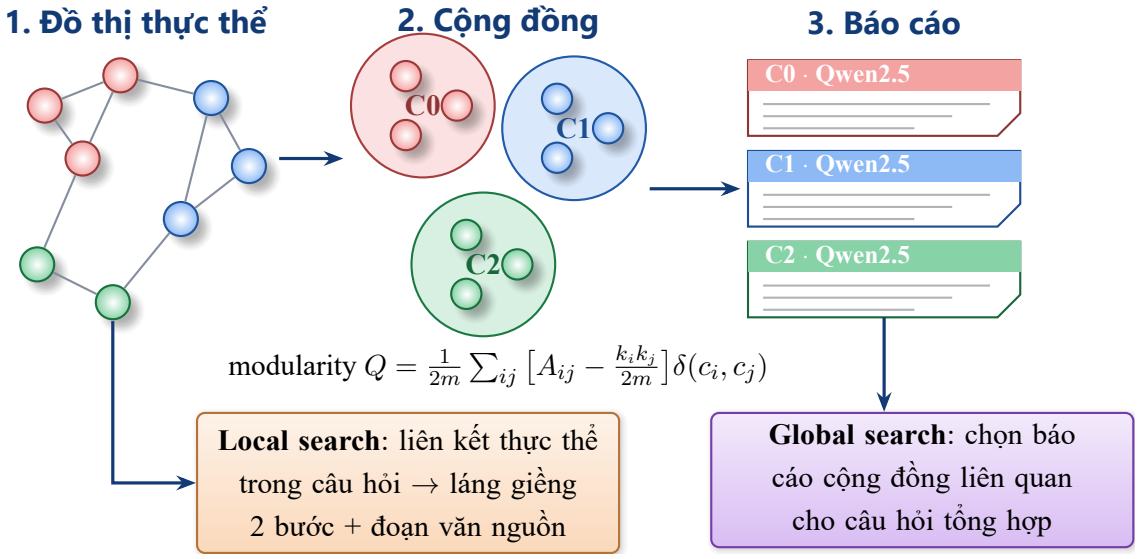
bản; ở đây hai KG đã có sẵn và được hợp nhất thành một đồ thị vô hướng có trọng số (cạnh nhiều quan hệ được cộng trọng số).

2. **Phát hiện cộng đồng**: thuật toán Louvain [24] tìm phân hoạch $\{c_i\}$ cực đại độ đo modularity

$$Q = \frac{1}{2m} \sum_{i,j} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j), \tag{3.6}$$

với A_{ij} là trọng số cạnh, k_i là bậc có trọng số, m là tổng trọng số. Q đo mật độ cạnh trong cộng đồng vượt bao nhiêu so với một đồ thị ngẫu nhiên cùng dãy bậc.

3. **Báo cáo cộng đồng**: các bộ ba nằm trọn trong một cộng đồng được diễn đạt thành câu và LLM viết một đoạn tóm tắt 3–4 câu.
4. **Tìm kiếm**: *cục bộ* liên kết thực thể trong câu hỏi rồi lấy các bộ ba quanh chúng và đoạn văn của tài liệu nguồn; *toàn cục* xếp hạng các báo cáo cộng đồng theo độ tương đồng với câu hỏi.



Hình 3.20. Đường ống GraphRAG: đồ thị thực thể, cộng đồng Louvain, báo cáo cộng đồng do LLM viết, tìm kiếm cục bộ và toàn cục

Bước khó nhất trong thực tế là **liên kết thực thể**: nhận ra "đang uống glucophage" nói về Metformin. Bộ liên kết của bài (Đoạn mã 3.10) bỏ dấu tiếng Việt, khớp nguyên từ với bảng bí danh và ưu tiên bí danh dài nhất trước, để "gà rán" không bị hiểu thành "gà" và một đoạn đã khớp không bị khớp lại.

```python
# tl2/graphrag.py
class EntityLinker:
    """Khớp bí danh dài nhất trước (khớp nguyên từ, không dấu) để "gà rán" không bị hiểu thành "gà"."""

    def __init__(self, kgs):
        self.table = []
        for kg in kgs:
            for n, d in kg.G.nodes(data=True):
                for a in set(d.get("aliases", [])) | {d["label"].lower()}:
                    if len(a) >= 2:
                        self.table.append((strip_accents(a), n, kg.name))
        self.table.sort(key=lambda t: -len(t[0]))

    def link(self, text):
        t = " " + re.sub(r"[^\w/%.-]+", " ", strip_accents(text)) + " "
        found, used = [], []
        for alias, node, kg in self.table:
            for m in re.finditer(r"(?<=\s)" + re.escape(alias) + r"(?=\s)", t):
                span = (m.start(), m.end())
                if any(a < span[1] and span[0] < b for a, b in used):
                    continue
                used.append(span)
                if node not in [f[0] for f in found]:
                    found.append((node, kg, alias))
        return found
```

Đoạn mã 3.10. Liên kết thực thể: khớp bí danh dài nhất trước, không dấu, nguyên từ

```python
# tl2/graphrag.py
def summarize_communities(kgs, comms, llm=None, max_facts=18):
    """Tóm tắt mỗi cộng đồng bằng LLM cục bộ; nếu không truyền LLM thì dùng bản tóm tắt trích xuất."""
    label = {n: d["label"] for kg in kgs for n, d in kg.G.nodes(data=True)}
    out = []
    for i, members in enumerate(comms):
        facts, sources = community_report(kgs, members)
        hub = sorted(members, key=lambda n: -sum(1 for kg in kgs if n in kg.G for _ in kg.G.edges(n)))
        title = ", ".join(label[n] for n in hub[:3])
        extractive = f"Cụm gồm {len(members)} thực thể, trung tâm là {title}. " + " ".join(f + "." for f in facts[:6])
        summary, tokens = extractive, 0
        if llm is not None and facts:
            bullets = "\n".join(f"- {f}" for f in facts[:max_facts])
            prompt = ("Viết một đoạn tóm tắt 3-4 câu bằng tiếng Việt cho cụm tri thức sau, nêu chủ đề chung và các quan "
                      "hệ quan trọng nhất, không thêm thông tin ngoài danh sách.\n" + bullets)
            summary, tokens, _ = llm.chat(prompt, system="Bạn là chuyên gia tóm tắt đồ thị tri thức.", max_new_tokens=220)
        out.append({"id": i, "size": len(members), "members": sorted(members), "title": title, "facts": facts,
                    "sources": sources, "summary": summary, "extractive": extractive, "tokens": tokens})
    return out
```

Đoạn mã 3.11. Tóm tắt cộng đồng bằng LLM cục bộ từ các bộ ba nằm trong cộng đồng

Đoạn mã 3.12. Lớp `GraphRAG`: tìm kiếm cục bộ trên KG và tìm kiếm toàn cục trên báo cáo cộng đồng

```python
# tl2/graphrag.py
class GraphRAG:
    def __init__(self, kgs, chunks, embedder=None, reports=None):
        self.kgs = {kg.name: kg for kg in kgs}
        self.linker = EntityLinker(kgs)
        self.chunks_by_doc = {}
        for c in chunks:
            self.chunks_by_doc.setdefault(c.doc_id, []).append(c)
        self.emb, self.reports = embedder, reports or []
        if embedder is not None and self.reports:
            self.R = embedder.encode([r["summary"] for r in self.reports])

    def local_search(self, question, hops=1, max_facts=24):
        """Trả (thực thể liên kết, bộ ba đã diễn đạt kèm nguồn, tài liệu nguồn)."""
        ents = self.linker.link(question)
        facts, docs, seen = [], [], set()
        for node, kgname, _ in ents:
            kg = self.kgs[kgname]
            hood = kg.neighbors(node, hops)
            for f in kg.facts():
                if (f[0] == node or f[2] == node or (f[0] in hood and f[2] in hood and hops > 1)) and \
                        (f[0], f[1], f[2]) not in seen:
                    seen.add((f[0], f[1], f[2]))
                    facts.append((kg.verbalize(f), f[3].get("source")))
                    if f[3].get("source") and f[3]["source"] not in docs:
                        docs.append(f[3]["source"])
        return ents, facts[:max_facts], docs

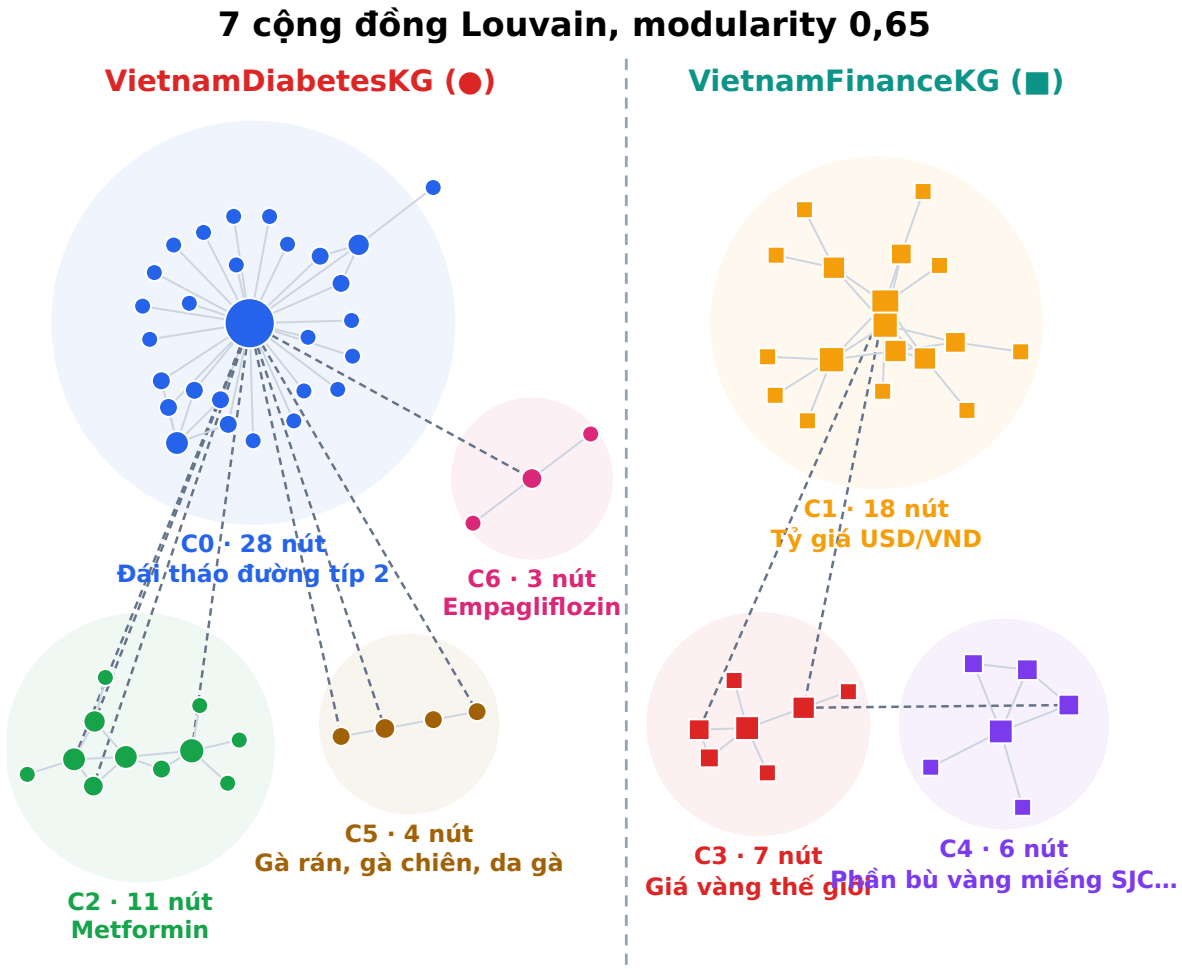
    def local_context(self, question, retriever=None, k_chunks=4, hops=1):
        """Ngữ cảnh lai: bộ ba KG + đoạn văn tốt nhất trong các tài liệu nguồn của bộ ba (hoặc toàn kho)."""
        ents, facts, docs = self.local_search(question, hops)
        passages = []
        if retriever is not None:
            pool = retriever.search(question, 40)
            ranked = [c for c, _ in pool if c.doc_id in docs] or [c for c, _ in pool]
            passages = ranked[:k_chunks]
        return {"entities": ents, "facts": facts, "docs": docs, "passages": passages}

    def global_search(self, question, top=3):
        q = self.emb.encode([question], kind="query")[0]
```

```
        s = self.R @ q
        order = s.argsort()[::-1][:top]
        return [(self.reports[i], float(s[i])) for i in order]
```

3.6.2 Cộng đồng tri thức

Louvain tìm được 7 cộng đồng với modularity 0,6517 (Hình 3.21, Bảng 3.11). Không cộng đồng nào chứa thực thể của cả hai miền, đúng như mong đợi vì hai KG không có cạnh nối nhau. Các cộng đồng có nghĩa rõ ràng mà không cần ai gán nhãn: C0 gom đái tháo đường típ 2 với tiêu chuẩn chẩn đoán, yếu tố nguy cơ, các món tinh bột và lối sống; C2 gom ba thuốc chính (metformin, insulin, gliclazide) với rượu bia, nhiễm toan lactic, hạ glucose huyết và các chống chỉ định, tức đúng phân hệ "an toàn thuốc"; C6 là bộ ba empagliflozin – suy tim – bệnh thận mạn, đúng chỉ định ưu tiên của nhóm SGLT2. Bên tài chính, C1 là kênh tỷ giá – lãi suất – cổ phiếu và ngành, C3 là kênh vàng thế giới (Fed, DXY, XAU/USD), C4 là chính sách vàng miếng trong nước. LLM viết 7 bản tóm tắt trong 52,23 s.



Hình 3.21. Các cộng đồng Louvain trên đồ thị hợp nhất hai KG; nhãn ghi kích thước và thực thể trung tâm, cạnh đứt nối hai cộng đồng khác nhau

Bảng 3.11. Các cộng đồng tri thức và báo cáo cộng đồng

Cụm	Đồ thị	Nút	Ba thực thể bậc cao nhất	Dữ kiện	Nguồn	Token tóm tắt
C0	Y tế	28	Đái tháo đường típ 2, Tiền đái tháo đường, HbA1c 5,7 - 6,4%	33	7	220
C1	Tài chính	18	Tỷ giá USD/VND, Lãi suất điều hành NHNN, VCB (Vietcombank)	23	7	220
C2	Y tế	11	Metformin, Insulin, Gliclazide	13	3	172
C3	Tài chính	7	Lãi suất Fed, Giá vàng thế giới quy đổi (VND/lượng), Chỉ số USD (DXY)	7	2	220
C4	Tài chính	6	Nghị định 232/2025/NĐ-CP, Nghị định 24/2012/NĐ-CP, NHNN đấu thầu và…	7	1	114
C5	Y tế	4	Gà rán, gà chiên, da gà, Ức gà bỏ da luộc/hấp, Bệnh tim mạch do xơ vữa	3	1	220
C6	Y tế	3	Empagliflozin, Suy tim, Bệnh thận mạn	2	1	68

Louvain trên đồ thị hợp nhất, modularity 0,652; tóm tắt cộng đồng do LLM viết từ các dữ kiện, tổng 52,2 s.

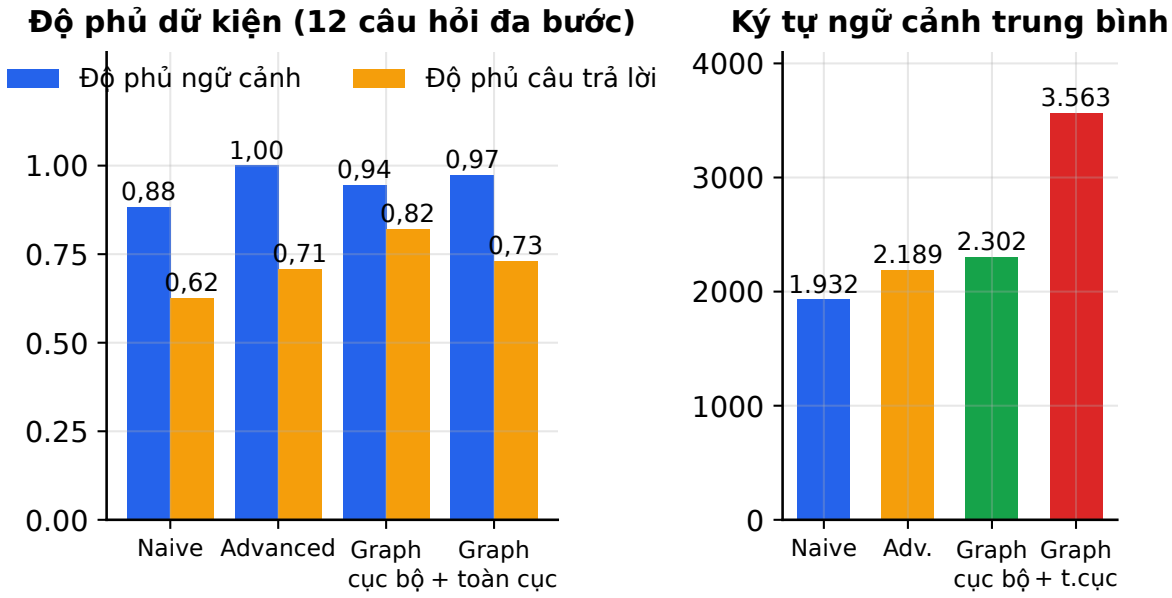
3.6.3 So sánh bốn kiến trúc trên câu hỏi đa bước

Để đo đúng thế mạnh của GraphRAG, bài dùng một bộ 12 câu hỏi đa bước riêng, mỗi câu kèm các nhóm từ khóa ứng với các dữ kiện bắt buộc phải có (ví dụ câu Fed – vàng SJC cần cả kênh DXY, kênh XAU, kênh tỷ giá và phần bù trong nước). Bốn kiến trúc dùng cùng LLM và cùng prompt có trích dẫn, chỉ khác ngữ cảnh: Naive (cố định + dense, top-5), Advanced (cấu trúc + lai + xếp lại, top-5), GraphRAG cục bộ (bộ ba quanh thực thể liên kết + 3 đoạn văn của tài liệu nguồn) và GraphRAG cục bộ + toàn cục (thêm 2 báo cáo cộng đồng gần nhất). **Độ phủ ngữ cảnh** là tỉ lệ dữ kiện có mặt trong ngữ cảnh đưa vào LLM; **độ phủ câu trả lời** là tỉ lệ dữ kiện có trong câu trả lời cuối.

Bảng 3.12. Bốn kiến trúc RAG trên 12 câu hỏi đa bước

Phương pháp	Độ phủ ngữ cảnh	Độ phủ câu trả lời	Ký tự ngữ cảnh	Sinh (s)
Naive RAG (cố định, dense)	0,8819	0,6250	1.932	9,98
Advanced RAG (hybrid + rerank)	**1,0000**	0,7083	2.189	12,18
GraphRAG cục bộ	0,9444	**0,8194**	2.302	12,74
GraphRAG cục bộ + toàn cục	0,9722	0,7292	3.563	18,12

Trung bình trên 12 câu hỏi đa bước. Độ phủ: tỉ lệ dữ kiện bắt buộc (nhóm từ khóa) có mặt; cùng LLM Qwen2.5-1.5B, cùng prompt có trích dẫn.



Hình 3.22. Độ phủ dữ kiện trong ngữ cảnh và trong câu trả lời, cùng độ dài ngữ cảnh của bốn kiến trúc

GraphRAG cục bộ cho câu trả lời đầy đủ nhất: độ phủ câu trả lời 81,94% so với 62,50% của Naive và 70,83% của Advanced. Điều đáng chú ý là Advanced có độ phủ *ngữ cảnh* cao nhất (100,00%) nhưng độ phủ câu trả lời thấp hơn GraphRAG: dữ kiện có trong năm đoạn văn dài, nhưng LLM nhỏ không gom đủ chúng lại. Bộ ba KG đã diễn đạt thành câu ngắn, mỗi câu một quan hệ, nên LLM dễ chép đúng vào câu trả lời hơn. Thêm báo cáo toàn cục đưa độ phủ ngữ cảnh lên 97,22% nhưng *giảm* độ phủ câu trả lời xuống 72,92%, vì ngữ cảnh dài thêm gần gấp rưỡi (3.563 ký tự) và mô hình nhỏ bị loãng sự chú ý. Tìm kiếm toàn cục

hợp với câu hỏi tổng hợp kiểu "chủ đề chính của kho là gì", không phải câu hỏi đa bước có đáp án cụ thể.

Bảng 3.13. Độ phủ ngữ cảnh / câu trả lời theo từng câu hỏi đa bước

STT	Câu hỏi	Naive	Advanced	Graph cục bộ	Graph + toàn cục
1	Fed tăng lãi suất thì giá vàng SJC trong nước chịu tác động qua những kênh nào?	0,75 / 0,25	1,00 / 0,25	1,00 / 0,50	1,00 / 1,00
2	Tỷ giá USD/VND tăng thì FPT và HPG bị ảnh hưởng ngược nhau ra sao?	1,00 / 0,33	1,00 / 0,33	1,00 / 1,00	1,00 / 0,67
3	Đồng USD mạnh lên tác động thế nào tới khối ngoại và chỉ số VN30?	1,00 / 1,00	1,00 / 1,00	1,00 / 0,67	1,00 / 0,67
4	Nghị định 232/2025 ảnh hưởng thế nào tới chênh lệch giá vàng SJC?	1,00 / 0,75	1,00 / 0,25	1,00 / 1,00	1,00 / 1,00
5	NHNN hạ lãi suất có lợi cho những cổ phiếu nào trong VN30?	1,00 / 0,00	1,00 / 1,00	1,00 / 0,67	1,00 / 0,67
6	Tôi đang dùng metformin và hay đi nhậu, có nguy cơ biến chứng gì?	0,50 / 0,50	1,00 / 0,50	1,00 / 1,00	1,00 / 0,50
7	Người tiểu đường ăn thịt gà và cơm thế nào cho đúng?	1,00 / 1,00	1,00 / 0,50	1,00 / 1,00	1,00 / 0,75
8	Bệnh nhân tiểu đường bị suy tim và suy thận nên dùng thuốc nào, tránh thuốc nào?	1,00 / 0,33	1,00 / 0,67	1,00 / 0,67	1,00 / 0,67
9	Dùng gliclazide mà uống rượu thì nguy hiểm thế nào và xử trí ra sao?	0,33 / 0,33	1,00 / 1,00	0,33 / 0,33	0,67 / 0,33
10	Tiền đái tháo đường phòng ngừa bằng những biện pháp nào và hiệu quả bao nhiêu?	1,00 / 1,00	1,00 / 1,00	1,00 / 1,00	1,00 / 1,00
11	Giá quặng sắt và đầu tư công tác động thế nào tới Hòa Phát?	1,00 / 1,00	1,00 / 1,00	1,00 / 1,00	1,00 / 1,00
12	Biến chứng của tiểu đường gồm những gì và cần theo dõi định kỳ ra sao?	1,00 / 1,00	1,00 / 1,00	1,00 / 1,00	1,00 / 0,50

Mỗi ô: độ phủ ngữ cảnh / độ phủ câu trả lời.

3.7 Chiến lược lựa chọn kiến trúc trong hệ thống thực tế

Không có kiến trúc RAG tốt nhất cho mọi trường hợp. Bảng 3.14 tóm tắt hướng dẫn chọn rút ra từ các thí nghiệm của chương.

Bảng 3.14. Hướng dẫn chọn kiến trúc RAG theo loại câu hỏi và điều kiện triển khai

Tình huống	Kiến trúc nên dùng	Căn cứ trong bài
Hỏi đáp tra cứu một dữ kiện	Advanced: cấu trúc + lai + xếp lại	Hit@1 cao nhất trên 114 câu
Không có GPU, cần độ trễ thấp	BM25 hoặc lai không xếp lại	nhanh hơn reranker hàng trăm lần
Câu hỏi đa bước, nhân quả	GraphRAG cục bộ	độ phủ câu trả lời cao nhất
Câu hỏi tổng hợp toàn kho	GraphRAG toàn cục	báo cáo cộng đồng tóm tắt sẵn
Câu hỏi có điều kiện số	truy vấn KG / SPARQL	TF-IDF bỏ sót gần hết (Mục 2.4)
LLM nhỏ ($\leq$ 2 tỉ tham số)	tránh HyDE, Multi-Query	cả hai làm giảm Hit@1

Tác tử của bài kết hợp ba hàng của bảng: truy xuất lai + xếp lại cho đoạn văn, KG cho quan hệ và ràng buộc, tính toán trực tiếp cho số liệu thị trường, và để bộ lập kế hoạch chọn công cụ theo loại câu hỏi (Chương 6).

Chương 4

Hệ thống y tế: bệnh đái tháo đường típ 2 tại Việt Nam

Chương này dựng phân hệ y tế của tác tử gồm ba lớp: kho tri thức văn bản về chẩn đoán, điều trị, dinh dưỡng; `VietnamDiabetesKG` lưu các ràng buộc lâm sàng có nguồn; và một mạng MLP ước lượng nguy cơ đái tháo đường từ hồ sơ xét nghiệm. Ba lớp được kiểm tra trên ba tình huống thường gặp ở người bệnh Việt Nam: uống rượu bia khi đang dùng metformin, ăn thịt gà, và ăn cơm, bún, xôi.

Lưu ý về phạm vi

Hệ thống được xây cho mục đích học tập. Nội dung y khoa tổng hợp từ Quyết định 5481/QĐ-BYT, tiêu chuẩn của ADA và các bảng chỉ số đường huyết đã công bố; mọi câu trả lời của tác tử đều kèm lời nhắc "không thay thế chỉ định của bác sĩ".

4.1 Bối cảnh dịch tễ và phác đồ của Bộ Y tế

Đái tháo đường đang tăng nhanh tại Việt Nam cùng đô thị hóa, thay đổi chế độ ăn và giảm vận động. Theo Liên đoàn Đái tháo đường Quốc tế, Việt Nam có khoảng 4 đến 5 triệu người trưởng thành mắc bệnh, tức khoảng 6 đến 7% dân số trưởng thành [25], và các điều tra quốc gia ghi nhận tới khoảng một nửa số người mắc chưa biết mình bị bệnh. Đái tháo đường típ 2 chiếm 90 đến 95% số ca; cơ chế chính là đề kháng insulin ở cơ, gan và mô mỡ kèm suy giảm dần chức năng tế bào beta của tụy.

Hướng dẫn chẩn đoán và điều trị đái tháo đường típ 2 hiện hành là tài liệu ban hành kèm Quyết định 5481/QĐ-BYT ngày 30/12/2020 [26], thống nhất với tiêu chuẩn của Hiệp hội Đái tháo đường Hoa Kỳ [27]. Bảng 4.1 ghi bốn tiêu chuẩn chẩn đoán; đây chính là các thực thể loại Biomarker trong KG.

Bảng 4.1. Tiêu chuẩn chẩn đoán đái tháo đường và tiền đái tháo đường theo Quyết định 5481/QĐ-BYT

Xét nghiệm	Đái tháo đường	Tiền đái tháo đường
Glucose huyết tương lúc đói (nhịn ăn ≥ 8 giờ)	≥ 126 mg/dL (7,0 mmol/L)	100 – 125 mg/dL
Glucose 2 giờ sau nghiệm pháp 75 g glucose	≥ 200 mg/dL (11,1 mmol/L)	140 – 199 mg/dL
HbA1c (phòng xét nghiệm chuẩn hóa)	≥ 6,5% (48 mmol/mol)	5,7 – 6,4%
Glucose bất kỳ kèm triệu chứng kinh điển	≥ 200 mg/dL	–

Không có triệu chứng kinh điển thì cần hai kết quả bất thường, cùng mẫu hoặc hai thời điểm.

Về điều trị, metformin là thuốc đầu tay; nhóm ức chế SGLT2 (empagliflozin) được ưu tiên khi người bệnh có suy tim hoặc bệnh thận mạn; insulin bắt buộc với típ 1 và dùng khi típ 2 không kiểm soát được bằng thuốc uống. Mục tiêu HbA1c cho đa số người trưởng thành là dưới 7%. Thay đổi lối sống đi cùng mọi bậc điều trị: vận động ít nhất 150 phút mỗi tuần và giảm 5 đến 7% cân nặng; ở người tiền đái tháo đường, chương trình DPP cho thấy can thiệp lối sống giảm 58% nguy cơ tiến triển thành bệnh [28].

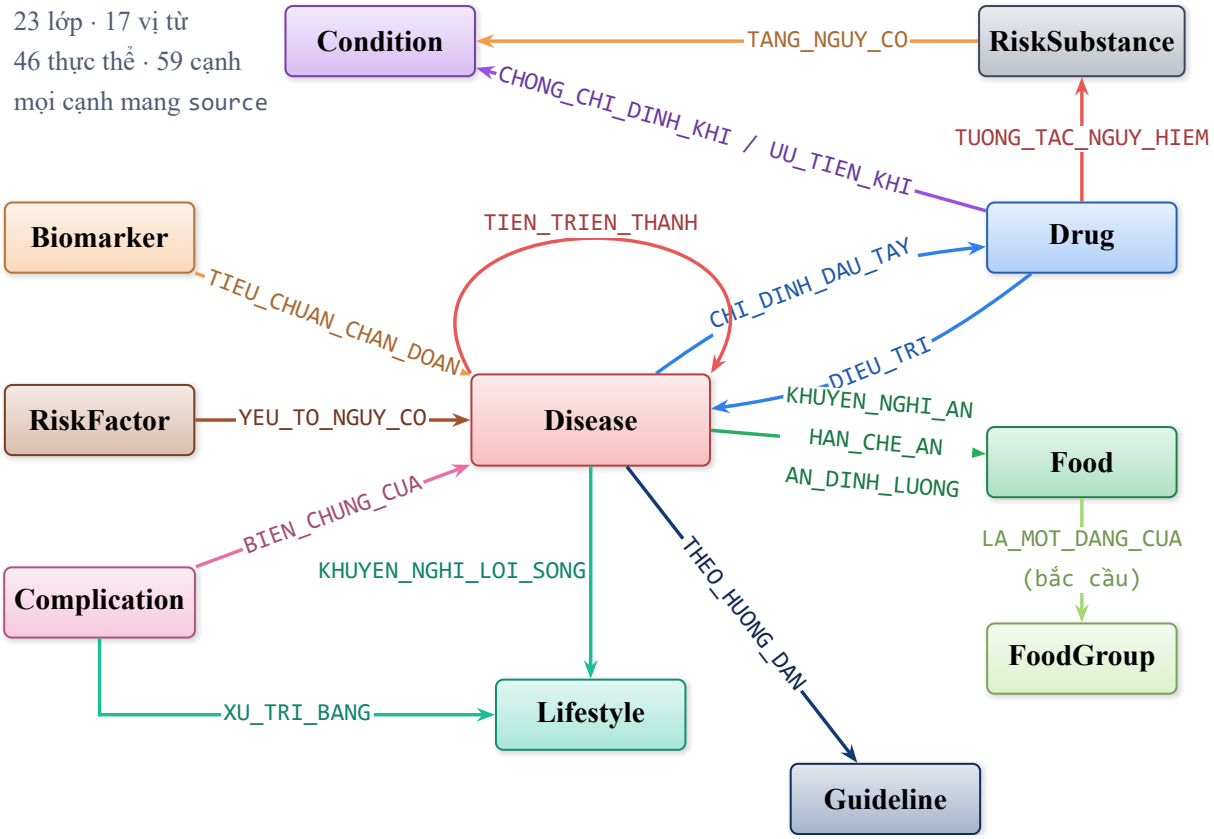
Ba đặc điểm của người bệnh Việt Nam khiến tri thức chung chưa đủ. Bữa ăn lấy *cơm trắng* làm nền, chỉ số đường huyết GI khoảng 73; các món xôi, bánh, bún, phở phổ biến; và *rượu bia* gắn với tiệc tùng, đám cưới, giỗ chạp, nên câu hỏi "đang dùng thuốc thì uống được không" là câu hỏi thật, thường xuyên. KG của bài vì vậy có một phân hệ dinh dưỡng với các món ăn Việt và một phân hệ tương tác thuốc – rượu.

4.2 Thiết kế bản thể học và đồ thị tri thức y tế

4.2.1 Hệ thống thực thể y tế

Bản thể học y tế có 23 lớp (Hình 2.4 ở Chương 2), xếp quanh lớp gốc MedicalEntity: bệnh, tình trạng và biến chứng, chỉ số sinh học, yếu tố nguy cơ, sáu nhóm thuốc, chất và thực phẩm phân loại theo mức an toàn (SafeFood,

ModerateFood, RiskFood), lối sống và hướng dẫn chuyên môn. Hình 4.1 là lược đồ: các lớp chính và vị từ được phép nối chúng.



Hình 4.1. Lược đồ `VietnamDiabetesKG`: các lớp chính và vị từ nối chúng theo ràng buộc miền → miền giá trị của `health_ontology()`

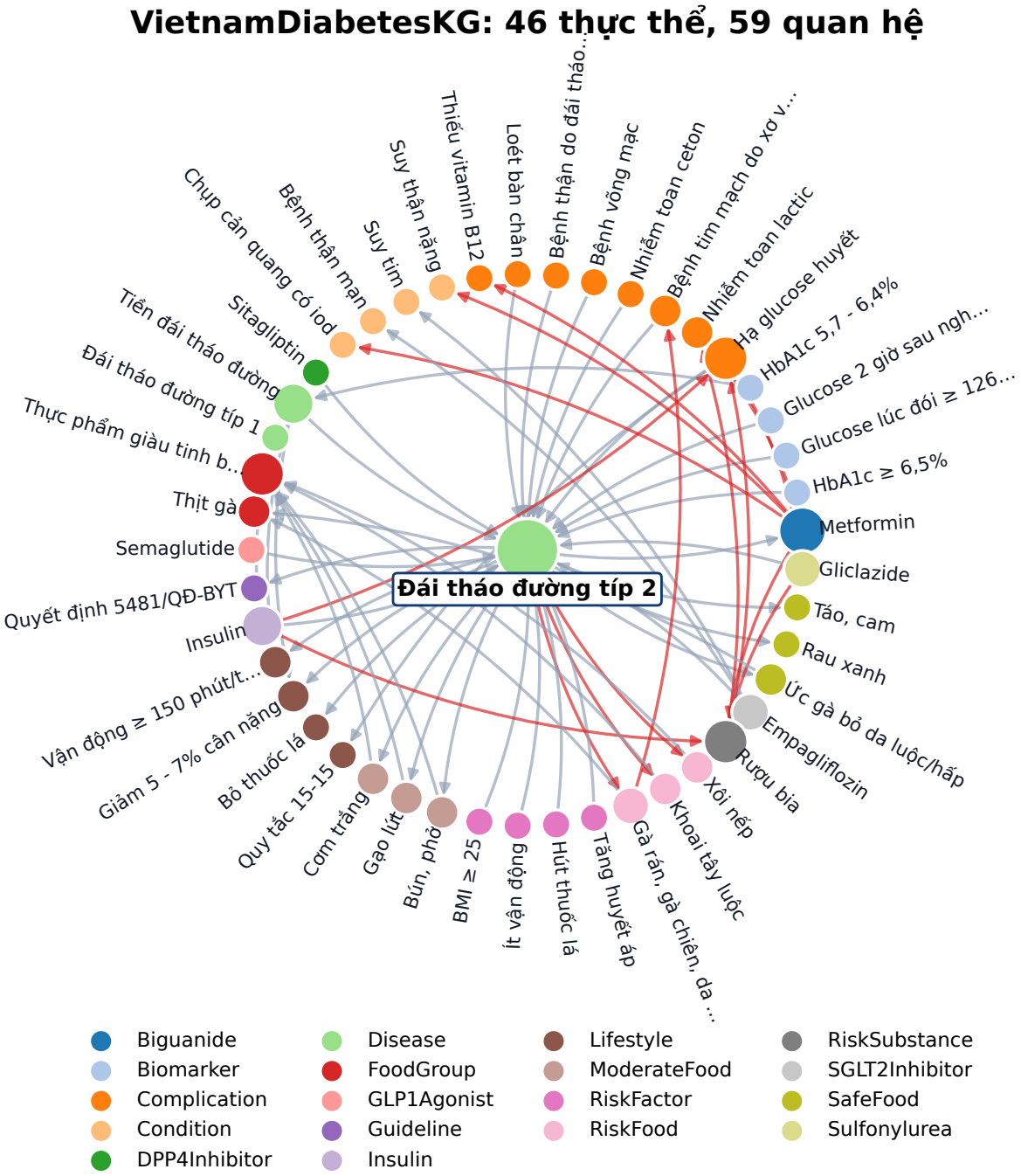
Mỗi thực thể có nhãn tiếng Việt, bí danh và thuộc tính lâm sàng: HbA1c_Cao có `threshold=6.5`, Suy_Than_Nang có `egfr_max=30`, Com_Trang có `gi=73, gl=29`. Bảng 4.2 đếm thực thể theo lớp của cả hai KG, Hình 4.2 vẽ toàn bộ đồ thị y tế.

```python
# tl2/kg_health.py
def health_ontology():
    o = Ontology()
    for c, parent in [
        ("Thing", None), ("MedicalEntity", "Thing"),
        ("Disease", "MedicalEntity"), ("Condition", "MedicalEntity"),
        ("Complication", "Condition"),
        ("Biomarker", "MedicalEntity"), ("RiskFactor", "MedicalEntity"),
        ("Drug", "MedicalEntity"), ("Biguanide", "Drug"), ("Sulfonylurea", "Drug"), ("DPP4Inhibitor", "Drug"),
        ("SGLT2Inhibitor", "Drug"), ("GLP1Agonist", "Drug"), ("Insulin", "Drug"),
        ("Substance", "MedicalEntity"), ("RiskSubstance", "Substance"),
        ("Food", "Substance"), ("FoodGroup", "Food"), ("SafeFood", "Food"), ("ModerateFood", "Food"),
        ("RiskFood", "Food"), ("Lifestyle", "MedicalEntity"), ("Guideline", "Thing"),
    ]:
        o.add_class(c, parent)
    P = o.add_predicate
    P("TIEU_CHUAN_CHAN_DOAN", "Biomarker", "Disease", "là tiêu chuẩn chẩn đoán")
    P("CHI_DINH_DAU_TAY", "Disease", "Drug", "chỉ định đầu tay", inverse="DIEU_TRI")
    P("DIEU_TRI", "Drug", "Disease", "điều trị")
    P("UU_TIEN_KHI", "Drug", "Condition", "ưu tiên khi có")
    P("CHONG_CHI_DINH_KHI", "Drug", "Condition", "chống chỉ định khi")
    P("TUONG_TAC_NGUY_HIEM", "Drug", "RiskSubstance", "tương tác nguy hiểm với")
    P("TANG_NGUY_CO", "MedicalEntity", "Condition", "làm tăng nguy cơ")
    P("YEU_TO_NGUY_CO", "RiskFactor", "Disease", "là yếu tố nguy cơ của")
    P("BIEN_CHUNG_CUA", "Complication", "Disease", "là biến chứng của")
    P("KHUYEN_NGHI_AN", "Disease", "Food", "khuyến nghị ăn")
    P("HAN_CHE_AN", "Disease", "Food", "khuyến nghị hạn chế")
    P("AN_DINH_LUONG", "Disease", "Food", "ăn có định lượng")
    P("KHUYEN_NGHI_LOI_SONG", "Disease", "Lifestyle", "khuyến nghị lối sống")
    P("LA_MOT_DANG_CUA", "Food", "FoodGroup", "là một dạng của", transitive=True)
    P("TIEN_TRIEN_THANH", "Disease", "Disease", "có thể tiến triển thành")
    P("THEO_HUONG_DAN", "Disease", "Guideline", "theo hướng dẫn")
    P("XU_TRI_BANG", "Complication", "Lifestyle", "xử trí bằng")
    return o
```

Đoạn mã 4.1. Bản thể học y tế: 23 lớp và 17 vị từ kèm ràng buộc

Bảng 4.2. Số thực thể theo lớp của hai đồ thị tri thức

Lớp (y tế)	Nút	Lớp (tài chính)	Nút
Complication	8	GlobalFactor	6
Biomarker	4	GoldAsset	4
RiskFactor	4	Stock	4
Condition	4	Sector	4
Lifestyle	4	MacroFactor	3
Disease	3	PolicyTool	3
SafeFood	3	Regulation	2
RiskFood	3	MarketFlow	1
ModerateFood	3	Event	1
FoodGroup	2	Indicator	1
Guideline	1	Index	1
Biguanide	1	Exchange	1
Sulfonylurea	1		
DPP4Inhibitor	1		
SGLT2Inhibitor	1		
GLP1Agonist	1		
Insulin	1		
RiskSubstance	1		



Hình 4.2. Toàn cảnh `VietnamDiabetesKG`: 46 thực thể xếp vòng theo lớp, 59 quan hệ

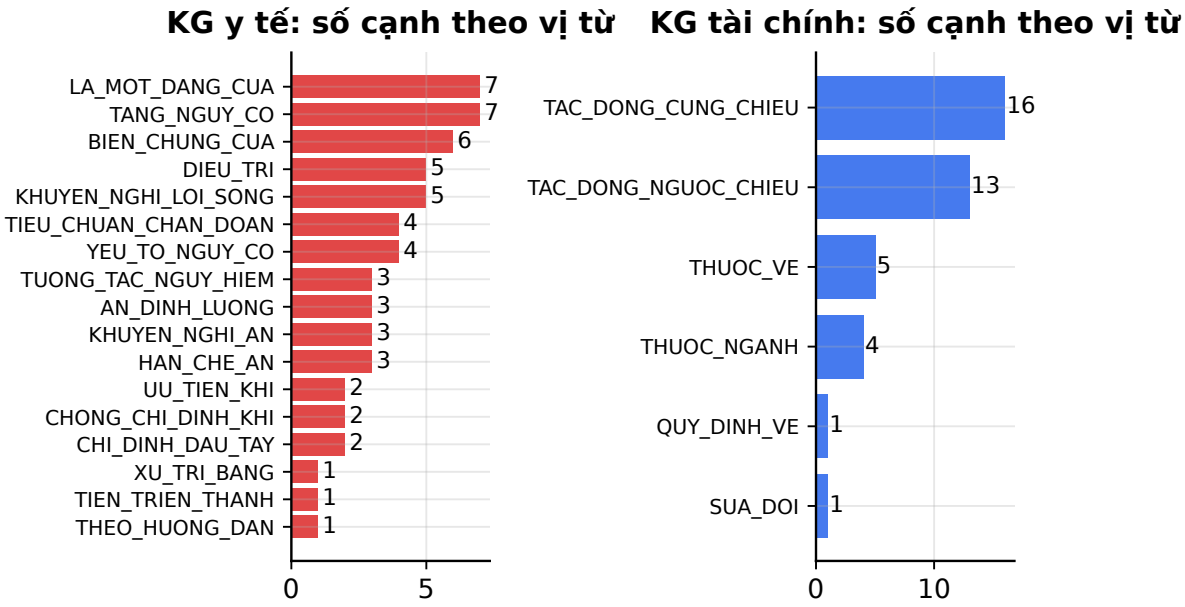
4.2.2 Mối quan hệ tri thức và ràng buộc lâm sàng

`VietnamDiabetesKG` có 17 vị từ, chia thành bốn phân hệ: *chẩn đoán* (TIEU_CHUAN_CHAN_DOAN, TIEN_TRIEN_THANH, YEU_TO_NGUY_CO), *điều trị* (CHI_DINH_DAU_TAY và nghịch đảo DIEU_TRI, UU_TIEN_KHI, CHONG_CHI_DINH_KHI), *an toàn thuốc* (TUONG_TAC_NGUY_HIEM,

TANG_NGUY_CO, BIEN_CHUNG_CUA, XU_TRI_BANG) và *dinh dưỡng, lối sống* (KHUYEN_NGHI_AN, HAN_CHE_AN, AN_DINH_LUONG, LA_MOT_DANG_CUA bắc cầu, KHUYEN_NGHI_LOI_SONG). Bảng 4.3 và Hình 4.3 đếm số cạnh mỗi vị từ.

Bảng 4.3. Số cạnh theo vị từ của hai đồ thị tri thức

Quan hệ (y tế)	Cạnh	Quan hệ (tài chính)	Cạnh
TANG_NGUY_CO	7	TAC_DONG_CUNG_CHIEU	16
LA_MOT_DANG_CUA	7	TAC_DONG_NGUOC_CHIEU	13
BIEN_CHUNG_CUA	6	THUOC_VE	5
KHUYEN_NGHI_LOI_SONG	5	THUOC_NGANH	4
DIEU_TRI	5	QUY_DINH_VE	1
TIEU_CHUAN_CHAN_DOAN	4	SUA_DOI	1
YEU_TO_NGUY_CO	4		
KHUYEN_NGHI_AN	3		
HAN_CHE_AN	3		
AN_DINH_LUONG	3		
TUONG_TAC_NGUY_HIEM	3		
CHI_DINH_DAU_TAY	2		
CHONG_CHI_DINH_KHI	2		
UU_TIEN_KHI	2		
THEO_HUONG_DAN	1		
TIEN_TRIEN_THANH	1		
XU_TRI_BANG	1		



Hình 4.3. Phân bố quan hệ theo vị từ của hai KG

Đoạn mã 4.2 là toàn bộ dữ liệu của KG y tế. Mỗi lời gọi `F(…)` thêm một bộ ba kèm `source` là mã tài liệu trong kho và các thuộc tính lâm sàng (mức độ, hậu quả, quy tắc, lý do). Mọi bộ ba đều qua bộ kiểm tra ràng buộc của Chương 2 trước khi vào đồ thị.

Đoạn mã 4.2. Dữ liệu của `VietnamDiabetesKG`: thực thể có thuộc tính và bộ ba có nguồn

```
# tl2/kg_health.py
def build_health_kg():
    kg = KnowledgeGraph("VietnamDiabetesKG", health_ontology())
    E = kg.add_entity
    # bệnh và hướng dẫn
    E("DTD_Tip2", "Disease", "Đái tháo đường típ 2", ["đái tháo đường",
     ↪ "tiểu đường", "đtđ", "típ 2", "type 2"], icd10="E11")
    E("DTD_Tip1", "Disease", "Đái tháo đường típ 1", ["típ 1", "type 1"],
     ↪ icd10="E10")
    E("Tien_DTD", "Disease", "Tiền đái tháo đường", ["tiền đái tháo đường",
     ↪ "tiền tiểu đường"], icd10="R73.03")
    E("QD_5481", "Guideline", "Quyết định 5481/QĐ-BYT", ["5481", "bộ y tế"],
     ↪ year=2020)
    # chỉ số chẩn đoán
    E("HbA1c_Cao", "Biomarker", "HbA1c ≥ 6,5%", ["hba1c"], threshold=6.5,
     ↪ unit="%")
    E("Glucose_Doi_Cao", "Biomarker", "Glucose lúc đói ≥ 126 mg/dL",
     ↪ ["glucose lúc đói", "đường huyết đói", "đường huyết lúc đói"],
     ↪ threshold=126, unit="mg/dL")
```

```
    E("OGTT_2h_Cao", "Biomarker", "Glucose 2 giờ sau nghiệm pháp ≥ 200 mg/dL", ["nghiệm pháp", "ogtt"], threshold=200, unit="mg/dL")
    E("HbA1c_Tien", "Biomarker", "HbA1c 5,7 - 6,4%", [], low=5.7, high=6.4, unit="%")
    # yếu tố nguy cơ
    E("Beo_Phi", "RiskFactor", "BMI ≥ 25 (béo phì, chuẩn châu Á)", ["béo phì", "thừa cân", "bmi"], threshold=25)
    E("It_Van_Dong", "RiskFactor", "Ít vận động", ["ít vận động"])
    E("Hut_Thuoc", "RiskFactor", "Hút thuốc lá", ["hút thuốc", "thuốc lá"])
    E("Tang_Huyet_Ap", "RiskFactor", "Tăng huyết áp", ["huyết áp", "tăng huyết áp"])
    # tình trạng, biến chứng
    E("Suy_Than_Nang", "Condition", "Suy thận nặng (eGFR < 30)", ["suy thận", "egfr"], egfr_max=30)
    E("Suy_Tim", "Condition", "Suy tim", ["suy tim"])
    E("Benh_Than_Man", "Condition", "Bệnh thận mạn", ["bệnh thận mạn"])
    E("Chup_Can_Quang", "Condition", "Chụp cản quang có iod", ["cản quang", "chụp ct"])
    E("Nhiem_Toan_Lactic", "Complication", "Nhiễm toan lactic", ["nhiễm toan lactic", "toan lactic", "lactic"], severity="nguy kịch")
    E("Ha_Duong_Huyet", "Complication", "Hạ glucose huyết (< 70 mg/dL)", ["hạ đường huyết", "hạ glucose"], threshold=70)
    E("Toan_Ceton", "Complication", "Nhiễm toan ceton", ["toan ceton", "ceton"])
    E("Benh_Tim_Mach", "Complication", "Bệnh tim mạch do xơ vữa", ["tim mạch", "xơ vữa", "đột quỵ"])
    E("Benh_Vong_Mac", "Complication", "Bệnh võng mạc", ["võng mạc", "mắt", "mù"])
    E("Benh_Than_DTD", "Complication", "Bệnh thận do đái tháo đường", ["albumin niệu"])
    E("Loet_Ban_Chan", "Complication", "Loét bàn chân", ["bàn chân", "loét"])
    E("Thieu_B12", "Complication", "Thiếu vitamin B12", ["b12"])
    # thuốc
    E("Metformin", "Biguanide", "Metformin", ["metformin", "glucophage"], first_line=True)
    E("Gliclazide", "Sulfonylurea", "Gliclazide", ["gliclazide", "diamicron", "sulfonylurea"])
    E("Sitagliptin", "DPP4Inhibitor", "Sitagliptin", ["sitagliptin", "januvia", "dpp-4"])
    E("Empagliflozin", "SGLT2Inhibitor", "Empagliflozin", ["empagliflozin", "dapagliflozin", "sglt2"])
    E("Semaglutide", "GLP1Agonist", "Semaglutide", ["semaglutide", "liraglutide", "glp-1"])
    E("Insulin_Nen", "Insulin", "Insulin", ["insulin"])
    # chất, thực phẩm
    E("Ruou_Bia", "RiskSubstance", "Rượu bia (ethanol)", ["rượu", "bia", "rượu bia", "cồn", "nhậu", "say"], kcal_per_g=7)
```

```
    E("Thit_Ga", "FoodGroup", "Thịt gà", ["thịt gà", "gà"])
    E("Uc_Ga_Bo_Da", "SafeFood", "Ức gà bỏ da luộc/hấp", ["ức gà", "gà luộc",
    ↪  "gà hấp"], protein_g=31, carb_g=0)
    E("Ga_Ran", "RiskFood", "Gà rán, gà chiên, da gà", ["gà rán", "gà chiên",
    ↪  "da gà"], saturated_fat="cao")
    E("Tinh_Bot", "FoodGroup", "Thực phẩm giàu tinh bột", ["tinh bột",
    ↪  "carbohydrate"])
    E("Com_Trang", "ModerateFood", "Cơm trắng", ["cơm", "cơm trắng", "gạo
    ↪  trắng"], gi=73, gl=29)
    E("Gao_Lut", "ModerateFood", "Gạo lứt", ["gạo lứt"], gi=68)
    E("Bun_Pho", "ModerateFood", "Bún, phở", ["bún", "phở"], gi=53)
    E("Khoai_Tay", "RiskFood", "Khoai tây luộc", ["khoai tây"], gi=78)
    E("Xoi_Nep", "RiskFood", "Xôi nếp", ["xôi", "nếp"], gi="rất cao")
    E("Rau_Xanh", "SafeFood", "Rau xanh (rau muống, cải, bông cải)", ["rau",
    ↪  "rau xanh", "rau muống"], gi="rất thấp")
    E("Trai_Cay_GI_Thap", "SafeFood", "Táo, cam (GI thấp)", ["táo", "cam",
    ↪  "trái cây"], gi=36)
    # lối sống
    E("Van_Dong_150", "Lifestyle", "Vận động ≥ 150 phút/tuần", ["vận động",
    ↪  "tập thể dục", "đi bộ"], minutes=150)
    E("Giam_Can", "Lifestyle", "Giảm 5 - 7% cân nặng", ["giảm cân"], pct=7)
    E("Bo_Thuoc_La", "Lifestyle", "Bỏ thuốc lá", ["bỏ thuốc"])
    E("Quy_Tac_15_15", "Lifestyle", "Quy tắc 15-15 (15 g carbohydrate, chờ
    ↪  15 phút)", ["15-15", "15 15"])

    F = kg.add_fact
    F("HbA1c_Cao", "TIEU_CHUAN_CHAN_DOAN", "DTD_Tip2",
    ↪  source="h01-chan-doan")
    F("Glucose_Doi_Cao", "TIEU_CHUAN_CHAN_DOAN", "DTD_Tip2",
    ↪  source="h01-chan-doan")
    F("OGTT_2h_Cao", "TIEU_CHUAN_CHAN_DOAN", "DTD_Tip2",
    ↪  source="h01-chan-doan")
    F("HbA1c_Tien", "TIEU_CHUAN_CHAN_DOAN", "Tien_DTD",
    ↪  source="h01-chan-doan")
    F("Tien_DTD", "TIEN_TRIEN_THANH", "DTD_Tip2",
    ↪  source="h08-tien-dai-thao-duong", rate="5-10%/năm")
    F("DTD_Tip2", "THEO_HUONG_DAN", "QD_5481", source="h01-chan-doan")
    for rf in ("Beo_Phi", "It_Van_Dong", "Hut_Thuoc", "Tang_Huyet_Ap"):
        F(rf, "YEU_TO_NGUY_CO", "DTD_Tip2",
        ↪  source="h08-tien-dai-thao-duong")
    F("DTD_Tip2", "CHI_DINH_DAU_TAY", "Metformin",
    ↪  source="h02-thuoc-dieu-tri", dose="500 mg x 1-2 lần/ngày")
    F("DTD_Tip1", "CHI_DINH_DAU_TAY", "Insulin_Nen",
    ↪  source="h02-thuoc-dieu-tri", note="bắt buộc suốt đời")
    for d in ("Gliclazide", "Sitagliptin", "Empagliflozin", "Semaglutide",
    ↪  "Insulin_Nen"):
        F(d, "DIEU_TRI", "DTD_Tip2", source="h02-thuoc-dieu-tri")
```

```
72  F("Metformin", "CHONG_CHI_DINH_KHI", "Suy_Than_Nang",
    ↪  source="h02-thuoc-dieu-tri", rule="eGFR < 30 mL/phút/1,73 m2")
73  F("Metformin", "CHONG_CHI_DINH_KHI", "Chup_Can_Quang",
    ↪  source="h02-thuoc-dieu-tri", rule="tạm ngưng trước chụp")
74  F("Empagliflozin", "UU_TIEN_KHI", "Suy_Tim",
    ↪  source="h02-thuoc-dieu-tri")
75  F("Empagliflozin", "UU_TIEN_KHI", "Benh_Than_Man",
    ↪  source="h02-thuoc-dieu-tri")
76  F("Metformin", "TUONG_TAC_NGUY_HIEM", "Ruou_Bia", source="h03-ruou-bia",
    ↪  severity="cao",
77    effect="tăng nguy cơ nhiễm toan lactic; tránh uống quá mức và say
      ↪  rượu")
78  F("Gliclazide", "TUONG_TAC_NGUY_HIEM", "Ruou_Bia", source="h03-ruou-bia",
    ↪  severity="cao",
79    effect="tăng mạnh nguy cơ hạ glucose huyết")
80  F("Insulin_Nen", "TUONG_TAC_NGUY_HIEM", "Ruou_Bia",
    ↪  source="h03-ruou-bia", severity="cao",
81    effect="tăng mạnh nguy cơ hạ glucose huyết")
82  F("Metformin", "TANG_NGUY_CO", "Nhiem_Toan_Lactic",
    ↪  source="h02-thuoc-dieu-tri", freq="hiếm")
83  F("Metformin", "TANG_NGUY_CO", "Thieu_B12", source="h02-thuoc-dieu-tri",
    ↪  freq="dùng lâu dài")
84  F("Ruou_Bia", "TANG_NGUY_CO", "Nhiem_Toan_Lactic",
    ↪  source="h03-ruou-bia")
85  F("Ruou_Bia", "TANG_NGUY_CO", "Ha_Duong_Huyet", source="h03-ruou-bia",
    ↪  window="tới 24 giờ")
86  F("Gliclazide", "TANG_NGUY_CO", "Ha_Duong_Huyet",
    ↪  source="h02-thuoc-dieu-tri")
87  F("Insulin_Nen", "TANG_NGUY_CO", "Ha_Duong_Huyet",
    ↪  source="h02-thuoc-dieu-tri")
88  F("Ga_Ran", "TANG_NGUY_CO", "Benh_Tim_Mach", source="h04-dinh-duong",
    ↪  reason="chất béo bão hòa, tăng LDL")
89  for c in ("Benh_Tim_Mach", "Benh_Vong_Mac", "Benh_Than_DTD",
    ↪  "Loet_Ban_Chan", "Ha_Duong_Huyet", "Toan_Ceton"):
90      F(c, "BIEN_CHUNG_CUA", "DTD_Tip2", source="h06-bien-chung" if c not
        ↪  in ("Ha_Duong_Huyet", "Toan_Ceton")
91        else "h07-ha-duong-huyet")
92  F("Ha_Duong_Huyet", "XU_TRI_BANG", "Quy_Tac_15_15",
    ↪  source="h07-ha-duong-huyet")
93  F("Uc_Ga_Bo_Da", "LA_MOT_DANG_CUA", "Thit_Ga", source="h04-dinh-duong")
94  F("Ga_Ran", "LA_MOT_DANG_CUA", "Thit_Ga", source="h04-dinh-duong")
95  for f in ("Com_Trang", "Gao_Lut", "Bun_Pho", "Khoai_Tay", "Xoi_Nep"):
96      F(f, "LA_MOT_DANG_CUA", "Tinh_Bot", source="h04-dinh-duong")
97  F("DTD_Tip2", "KHUYEN_NGHI_AN", "Uc_Ga_Bo_Da", source="h04-dinh-duong",
    ↪  reason="đạm nạc 31 g/100 g, không carbohydrate")
98  F("DTD_Tip2", "KHUYEN_NGHI_AN", "Rau_Xanh", source="h04-dinh-duong",
    ↪  reason="chất xơ làm chậm hấp thu; ít nhất nửa đĩa")
```

```
    F("DTD_Tip2", "KHUYEN_NGHI_AN", "Trai_Cay_GI_Thap",
    ↪  source="h04-dinh-duong", reason="GI thấp, chia nhỏ")
    F("DTD_Tip2", "HAN_CHE_AN", "Ga_Ran", source="h04-dinh-duong",
    ↪  reason="nhiều chất béo bão hòa, tăng đề kháng insulin")
    F("DTD_Tip2", "HAN_CHE_AN", "Khoai_Tay", source="h04-dinh-duong",
    ↪  reason="GI 78")
    F("DTD_Tip2", "HAN_CHE_AN", "Xoi_Nep", source="h04-dinh-duong",
    ↪  reason="GI rất cao")
    F("DTD_Tip2", "AN_DINH_LUONG", "Com_Trang", source="h04-dinh-duong",
    ↪  reason="GI 73, GL 29; tối đa ~1 bát con/bữa, ăn rau trước")
    F("DTD_Tip2", "AN_DINH_LUONG", "Gao_Lut", source="h04-dinh-duong",
    ↪  reason="GI 68, giàu chất xơ; thay một phần gạo trắng")
    F("DTD_Tip2", "AN_DINH_LUONG", "Bun_Pho", source="h04-dinh-duong",
    ↪  reason="GI 53; nước dùng trong, nhiều rau")
    F("DTD_Tip2", "KHUYEN_NGHI_LOI_SONG", "Van_Dong_150",
    ↪  source="h05-van-dong")
    F("DTD_Tip2", "KHUYEN_NGHI_LOI_SONG", "Giam_Can", source="h05-van-dong")
    F("DTD_Tip2", "KHUYEN_NGHI_LOI_SONG", "Bo_Thuoc_La",
    ↪  source="h05-van-dong")
    F("Tien_DTD", "KHUYEN_NGHI_LOI_SONG", "Van_Dong_150",
    ↪  source="h08-tien-dai-thao-duong", effect="giảm 58% nguy cơ (DPP)")
    F("Tien_DTD", "KHUYEN_NGHI_LOI_SONG", "Giam_Can",
    ↪  source="h08-tien-dai-thao-duong")
    return kg
```

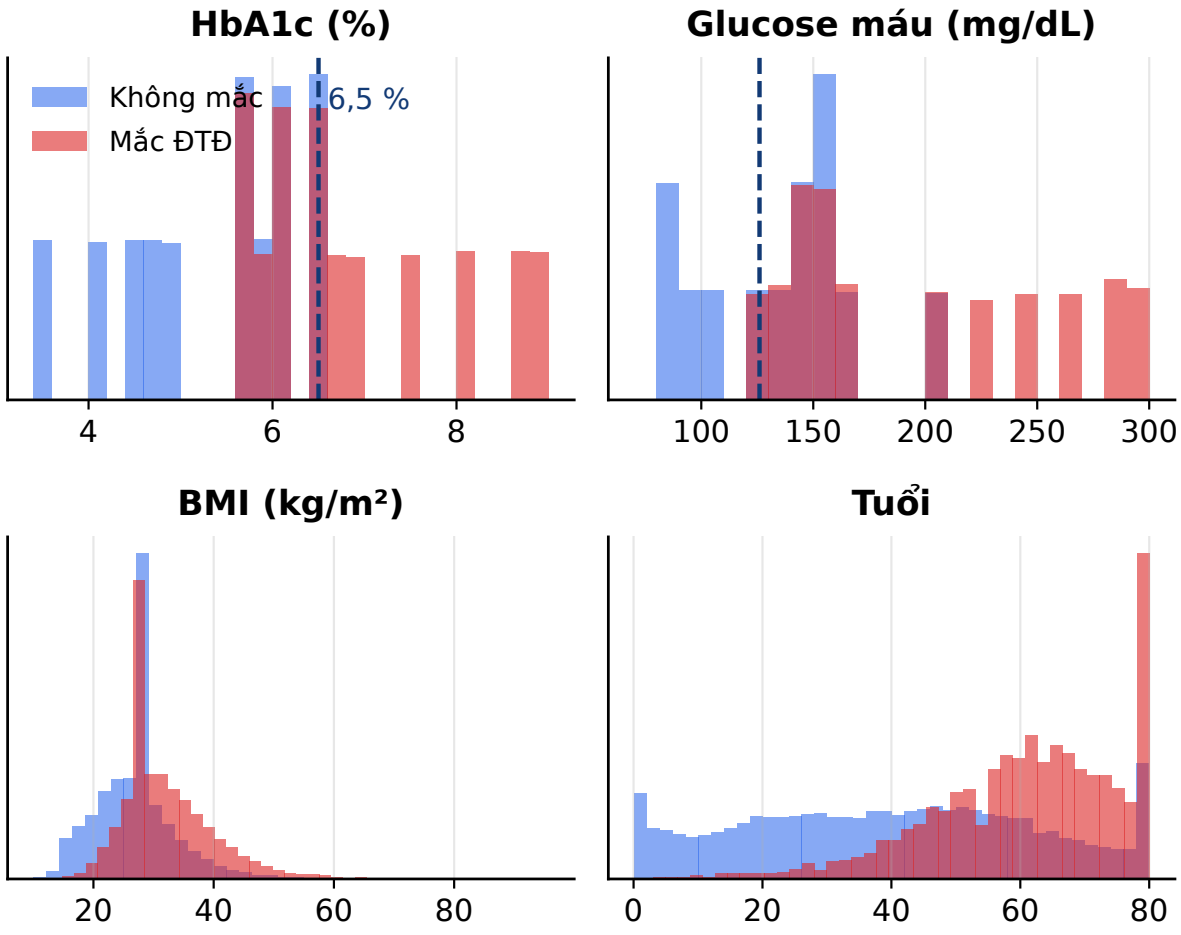
4.3 Mô hình học sâu chẩn đoán nguy cơ

4.3.1 Dữ liệu

Công cụ `chan_doan_nguy_co` của tác tử là một mạng MLP huấn luyện trên bộ dữ liệu dự báo đái tháo đường [14]: hồ sơ gồm giới tính, tuổi, tăng huyết áp, bệnh tim, tiền sử hút thuốc, BMI, HbA1c, glucose máu và nhãn mắc bệnh. Sau khi bỏ trùng còn 96.128 hồ sơ, tỉ lệ mắc bệnh 8,82%, chia phân tầng thành ba tập (Bảng 4.4). Hình 4.4 cho thấy HbA1c và glucose tách hai lớp rõ nhất, còn tuổi và BMI chồng lấn nhiều.

Bảng 4.4. Bộ dữ liệu chẩn đoán đái tháo đường sau tiền xử lý

Thuộc tính	Giá trị
Bản ghi sau khi bỏ trùng	96.128
Tỉ lệ mắc bệnh	8,82%
Huấn luyện / kiểm định / kiểm thử	67.289 / 14.419 / 14.420
Số đặc trưng sau mã hóa	13



Hình 4.4. Phân bố các đặc trưng chính theo nhãn mắc bệnh

Hồ sơ được mã hóa thành 13 đặc trưng: bốn cột số (tuổi, BMI, HbA1c, glucose), giới tính, hai cờ bệnh nền và sáu cột one-hot của tiền sử hút thuốc; mọi đặc trưng được chuẩn hóa z-score theo trung bình và độ lệch chuẩn của tập huấn luyện.

```python
# tl2/data.py
def diabetes_features(df):
    """Mã hóa hồ sơ: 4 cột số + giới tính + 2 cờ bệnh nền + one-hot hút thuốc (6) = 13 đặc trưng."""
    X = pd.DataFrame({c: df[c].astype(float) for c in NUMERIC})
    X["male"] = (df["gender"] == "Male").astype(float)
    X["hypertension"] = df["hypertension"].astype(float)
    X["heart_disease"] = df["heart_disease"].astype(float)
    for s in SMOKING:
        X[f"smoke_{s}"] = (df["smoking_history"] == s).astype(float)
    return X
```

Đoạn mã 4.3. Mã hóa hồ sơ thành 13 đặc trưng

4.3.2 Kiến trúc và huấn luyện

Mạng `DiabetesMLP` có hai tầng ẩn 128 và 64 nơ-ron (Hình 4.5). Với $\mathbf{x} \in \mathbb{R}^{13}$:

$$\mathbf{h}_1 = \mathrm{Dropout}_{0,2}\big(\mathrm{ReLU}(\mathrm{BN}(W_1\mathbf{x} + \mathbf{b}_1))\big) \in \mathbb{R}^{128}, \tag{4.1}$$

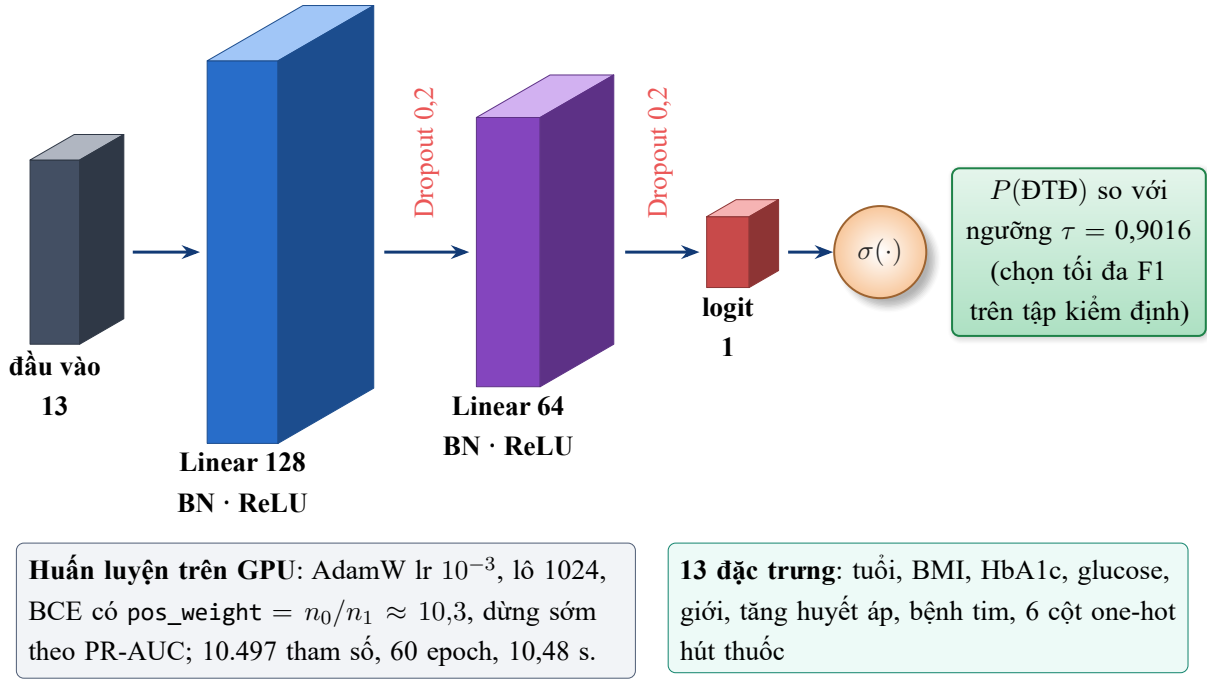
$$\mathbf{h}_2 = \mathrm{Dropout}_{0,2}\big(\mathrm{ReLU}(\mathrm{BN}(W_2\mathbf{h}_1 + \mathbf{b}_2))\big) \in \mathbb{R}^{64}, \tag{4.2}$$

$$z = \mathbf{w}_3^{\top}\mathbf{h}_2 + b_3, \qquad \hat{p} = \sigma(z). \tag{4.3}$$

Tổng cộng 10.497 tham số. Vì lớp dương chỉ chiếm chưa tới một phần mười, hàm mất mát là entropy chéo nhị phân có trọng số lớp dương $w_+ = N_-/N_+$:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\Big[w_+\, y_i \log\sigma(z_i) + (1-y_i)\log\big(1-\sigma(z_i)\big)\Big]. \tag{4.4}$$

Mạng được tối ưu bằng AdamW (tốc độ học 10^{-3}, suy giảm trọng số 10^{-4}), lô 1024 mẫu, giảm tốc độ học một nửa khi PR-AUC trên tập kiểm định không tăng sau 3 epoch, dừng sớm nếu không cải thiện sau 8 epoch, tối đa 60 epoch. Huấn luyện chạy trên GPU trong 10,48 s.



Hình 4.5. Kiến trúc `DiabetesMLP`: 13 đặc trưng → 128 → 64 → 1 logit, mỗi tầng ẩn gồm BatchNorm, ReLU và Dropout

```python
# tl2/dl_models.py
class DiabetesMLP(nn.Module):
    """13 -> 128 -> 64 -> 1, BatchNorm + ReLU + Dropout; đầu ra là logit của P(đái tháo đường)."""

    def __init__(self, d_in, hidden=(128, 64), dropout=0.2):
        super().__init__()
        layers, d = [], d_in
        for h in hidden:
            layers += [nn.Linear(d, h), nn.BatchNorm1d(h), nn.ReLU(), nn.Dropout(dropout)]
            d = h
        layers.append(nn.Linear(d, 1))
        self.net = nn.Sequential(*layers)

    def forward(self, x):
        return self.net(x).squeeze(-1)
```

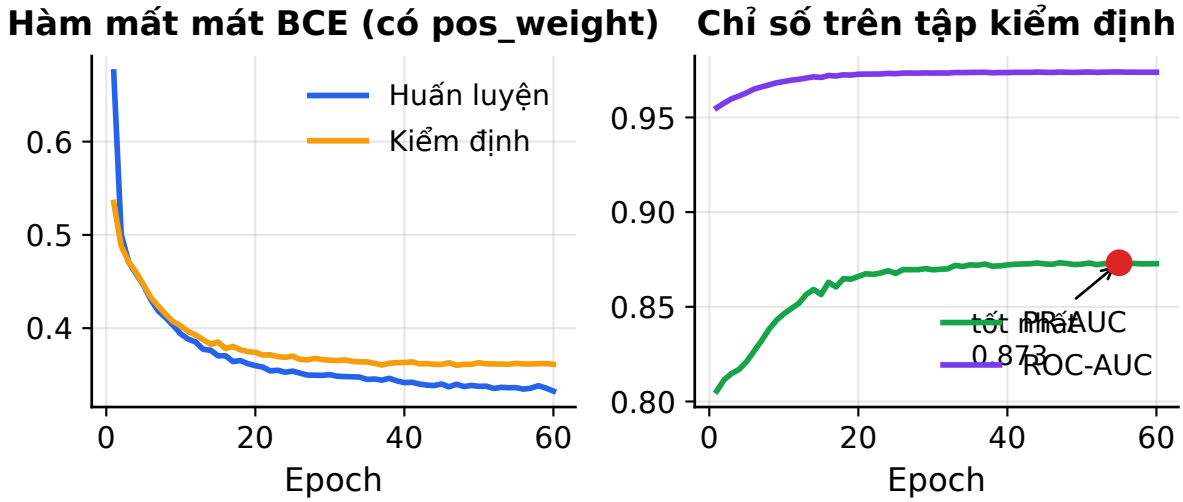
Đoạn mã 4.4. Mạng `DiabetesMLP` trong PyTorch

Đoạn mã 4.5. Vòng huấn luyện MLP: BCE có `pos_weight`, AdamW, giảm tốc độ học và dừng sớm theo PR-AUC

```python
# scripts/train_diabetes.py
```

```python
def fit_mlp(xtr, ytr, xva, yva, d_in, dev):
    """Huấn luyện MLP bằng BCE có pos_weight (bù lớp dương chỉ ~9%), dừng sớm theo PR-AUC trên tập kiểm định."""
    model = DiabetesMLP(d_in, HIDDEN, DROPOUT).to(dev)
    pos_weight = (ytr == 0).sum() / (ytr == 1).sum()
    loss_fn = nn.BCEWithLogitsLoss(pos_weight=pos_weight)
    opt = torch.optim.AdamW(model.parameters(), lr=LR, weight_decay=1e-4)
    sched = torch.optim.lr_scheduler.ReduceLROnPlateau(opt, mode="max", factor=0.5, patience=3)
    hist, best, best_state, wait = [], -1, None, 0
    for ep in range(1, EPOCHS + 1):
        model.train()
        perm = torch.randperm(len(xtr), device=dev)
        tot = 0.0
        for i in range(0, len(xtr), BATCH):
            b = perm[i:i + BATCH]
            opt.zero_grad()
            loss = loss_fn(model(xtr[b]), ytr[b])
            loss.backward()
            opt.step()
            tot += loss.item() * len(b)
        model.eval()
        with torch.no_grad():
            lva = model(xva)
            val_loss = loss_fn(lva, torch.tensor(yva, dtype=torch.float32, device=dev)).item()
            pva = torch.sigmoid(lva).cpu().numpy()
        ap = average_precision_score(yva, pva)
        sched.step(ap)
        hist.append({"epoch": ep, "train_loss": tot / len(xtr), "val_loss": val_loss, "val_pr_auc": ap,
                     "val_roc_auc": roc_auc_score(yva, pva), "lr": opt.param_groups[0]["lr"]})
        print(f"ep {ep:2d} loss {tot / len(xtr):.4f} val {val_loss:.4f} PR-AUC {ap:.4f}", flush=True)
        if ap > best:
            best, wait = ap, 0
            best_state = {k: v.detach().clone() for k, v in model.state_dict().items()}
        else:
            wait += 1
            if wait >= PATIENCE:
                break
    model.load_state_dict(best_state)
    model.eval()
    return model, hist
```

Đường học (Hình 4.6) hội tụ êm: mất mát huấn luyện và kiểm định giảm cùng nhịp, không có dấu hiệu quá khớp; mạng chạy đủ 60 epoch mà không kích hoạt dừng sớm, tốc độ học được giảm nhiều lần ở nửa sau khi PR-AUC kiểm định chững lại.



Hình 4.6. Đường học của MLP: mất mát BCE và chỉ số trên tập kiểm định qua các epoch

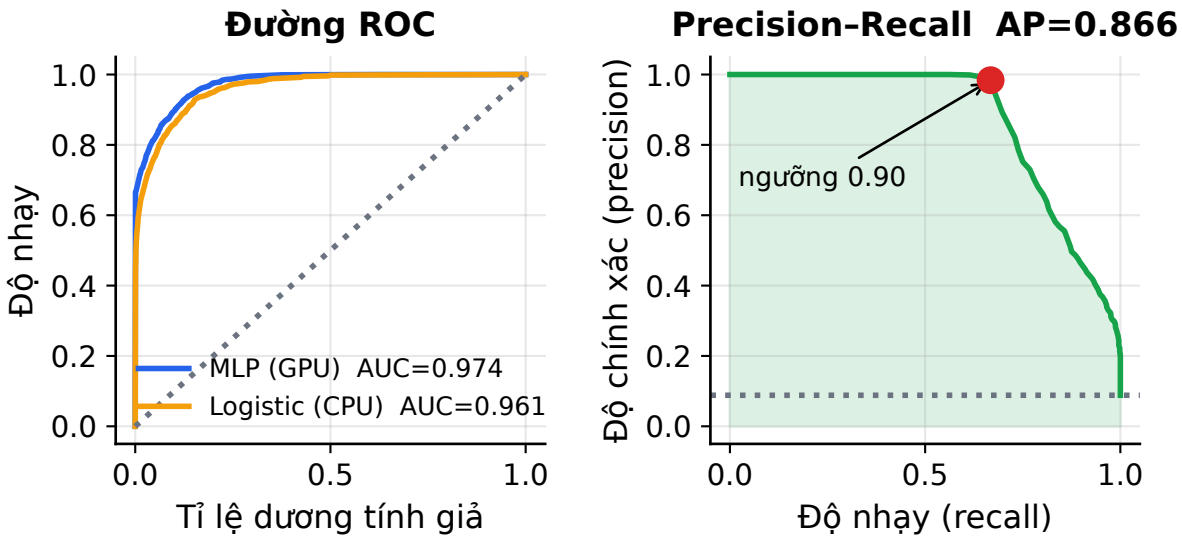
4.3.3 Kết quả và phân tích

Ngưỡng quyết định được chọn để F1 lớn nhất trên tập kiểm định, ra 0,9016. Ngưỡng cao như vậy là hệ quả trực tiếp của w_+: mất mát có trọng số đẩy mọi xác suất lên, nên 0,5 không còn là điểm cân bằng. Trên tập kiểm thử, MLP đạt F1 0,7955 và ROC-AUC 0,9737, vượt Logistic Regression ở mọi chỉ số (Bảng 4.5, Hình 4.9).

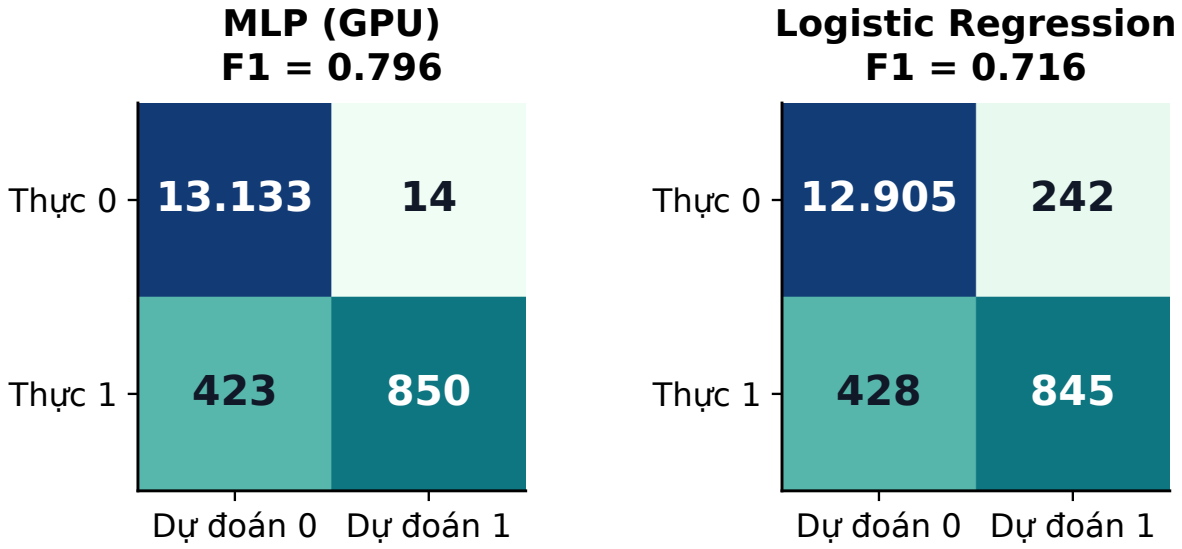
Bảng 4.5. MLP và Logistic Regression trên tập kiểm thử

Mô hình	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC	Ngưỡng	Huấn luyện (s)
MLP PyTorch (GPU)	**0,9697**	**0,9838**	**0,6677**	**0,7955**	**0,9737**	**0,8665**	0,902	10,48
Logistic Regression	0,9535	0,7774	0,6638	0,7161	0,9611	0,8114	0,878	0,05

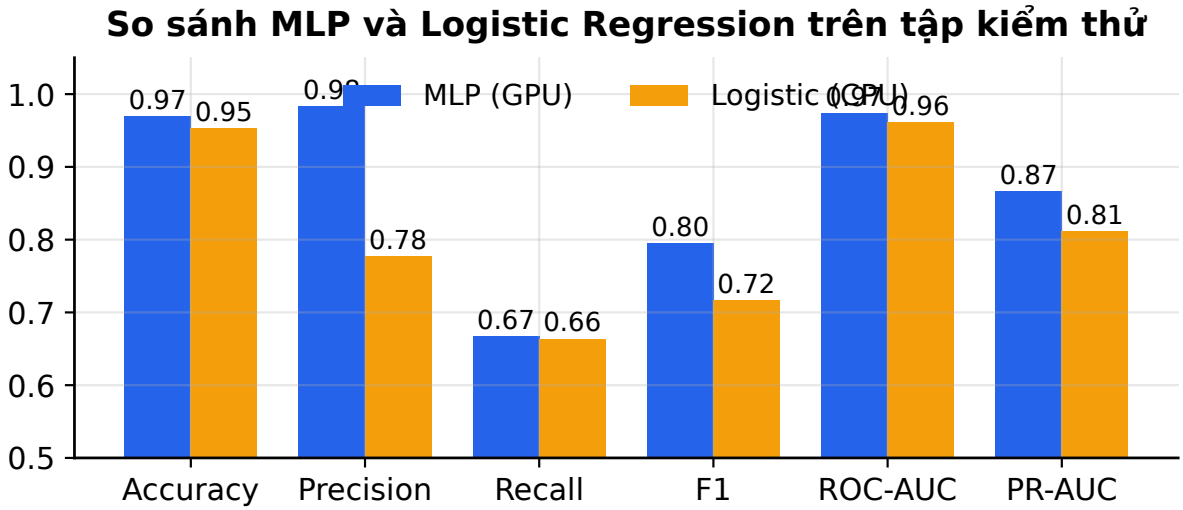
Tập kiểm thử 14.420 bản ghi; ngưỡng chọn để F1 lớn nhất trên tập kiểm định. MLP có 10.497 tham số, 60 epoch.



Hình 4.7. Đường ROC và Precision – Recall của hai mô hình trên tập kiểm thử



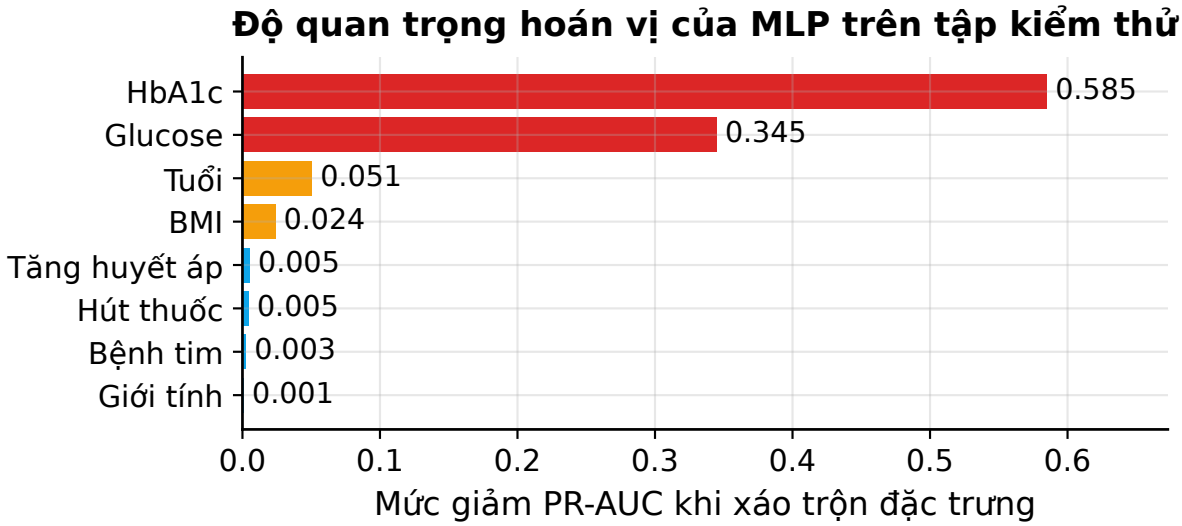
Hình 4.8. Ma trận nhầm lẫn tại ngưỡng đã chọn



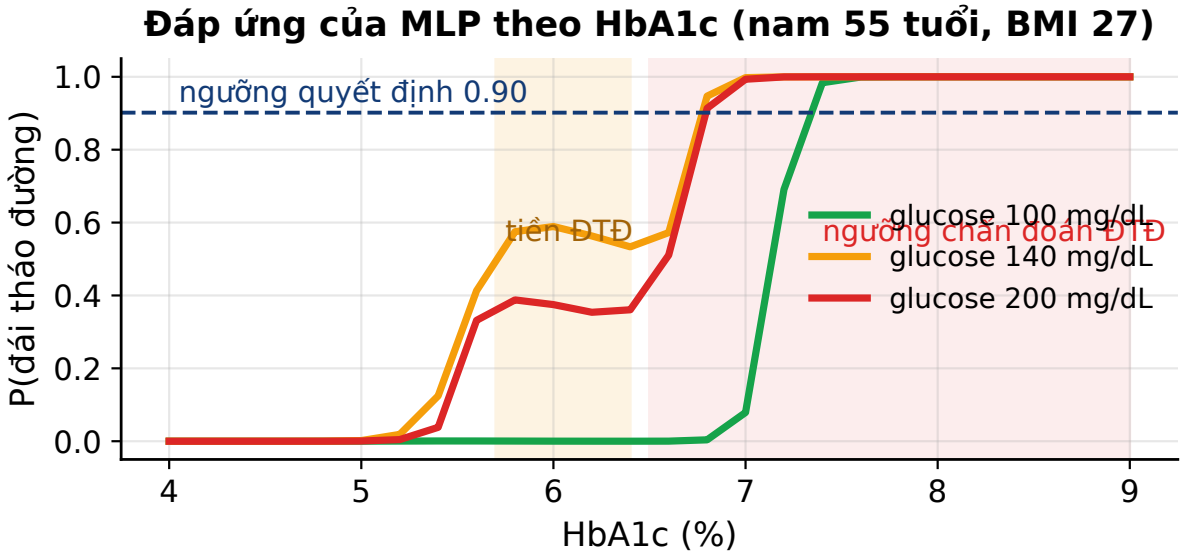
Hình 4.9. So sánh MLP và Logistic Regression theo từng chỉ số

Ma trận nhầm lẫn (Hình 4.8) cho thấy tính cách của mô hình: chỉ 14 ca dương tính giả trên 14.420 hồ sơ, precision 0,9838, nhưng bỏ sót 423 ca bệnh, recall 0,6677. Với một công cụ *sàng lọc*, ngưỡng nên hạ thấp để đổi precision lấy recall; với một công cụ trong tác tử trả lời người bệnh, ngưỡng cao giảm báo động giả. Bài giữ ngưỡng F1 và để tác tử trình bày xác suất cùng ngưỡng thay vì một kết luận có/không.

Hai phân tích sau giải thích mô hình đã học gì. Độ quan trọng hoán vị (Hình 4.10) đo PR-AUC giảm bao nhiêu khi xáo trộn một đặc trưng: HbA1c (0,5851) và glucose (0,3450) chiếm gần như toàn bộ, tuổi và BMI đóng góp nhỏ, các cờ còn lại gần bằng không. Đường đáp ứng theo HbA1c (Hình 4.11) cố định các đặc trưng khác và quét HbA1c: xác suất vượt ngưỡng quyết định ở HbA1c 6,8% khi glucose 140 mg/dL và 7,4% khi glucose 100 mg/dL, sát ngưỡng chẩn đoán 6,5% của Bộ Y tế. Mô hình tự học lại được tiêu chuẩn lâm sàng từ dữ liệu.



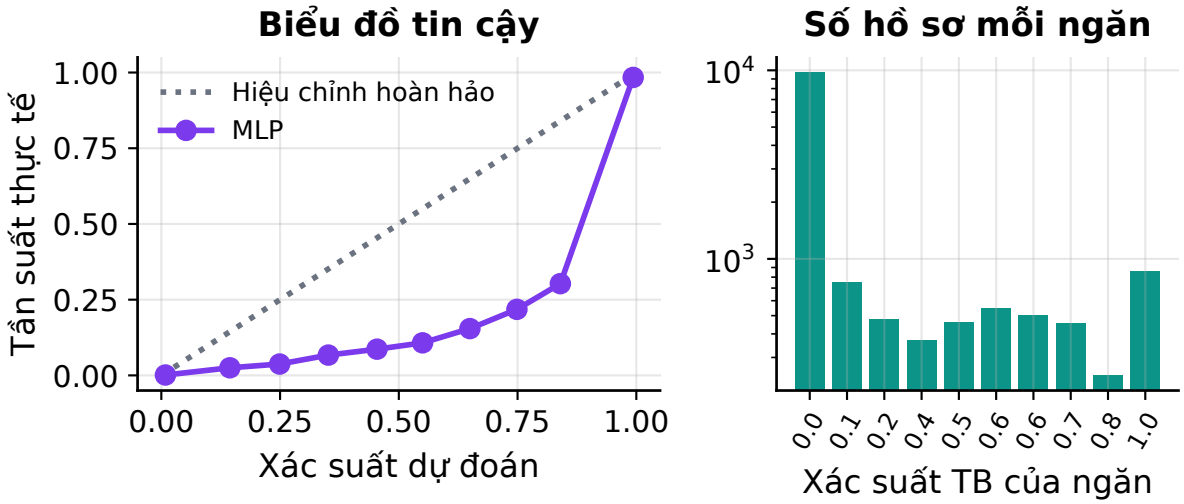
Hình 4.10. Độ quan trọng hoán vị của các đặc trưng



Hình 4.11. Xác suất mắc bệnh theo HbA1c ở ba mức glucose

Biểu đồ tin cậy (Hình 4.12) cho thấy giới hạn quan trọng nhất: xác suất của MLP *không hiệu chuẩn*. Ở ngăn có xác suất dự đoán trung bình 0,5500, tỉ lệ mắc thực tế chỉ 0,1075; chỉ ngăn trên cùng là khớp. Đây vẫn là hệ quả của w_+. Vì vậy con số mà tác tử và giao diện hiển thị phải đọc như một *điểm nguy cơ* so với ngưỡng, không phải xác suất mắc bệnh; muốn dùng như xác suất cần hiệu chuẩn thêm bằng Platt scaling hoặc hồi quy isotonic trên tập kiểm định. Bộ dữ liệu cũng là dữ liệu mô phỏng có quan hệ rất sắc giữa HbA1c và nhãn, nên hồ sơ có HbA1c trên 7% gần như luôn được chẩm sát 100%; mô hình không nên dùng

cho quyết định lâm sàng thật.



Hình 4.12. Biểu đồ tin cậy: xác suất dự đoán bị đẩy cao so với tần suất thực tế

Khi chạy trong tác tử, lớp `DiabetesRiskModel` nạp trọng số và tham số chuẩn hóa, điền giá trị mặc định cho trường người dùng không cung cấp và trả mức nguy cơ ba bậc (Đoạn mã 4.6).

```
# tl2/dl_models.py
def predict(self, profile):
    p = {"gender": "Male", "age": 50, "hypertension": 0, "heart_disease": 0,
     ↪  "smoking_history": "never",
          "bmi": 23.0, "HbA1c_level": 5.5, "blood_glucose_level": 100}
    p.update({k: v for k, v in profile.items() if v is not None})
    X = diabetes_features(pd.DataFrame([p]))[self.cols].to_numpy()
    x = torch.tensor((X - self.mean) / self.std, dtype=torch.float32,
     ↪  device=self.dev)
    with torch.no_grad():
        prob = float(torch.sigmoid(self.model(x))[0])
    # huấn luyện với pos_weight đẩy xác suất lên cao, nên mốc "cao" là
     ↪  ngưỡng tối ưu F1 chứ không phải 0,5
    level = "cao" if prob >= self.threshold else "trung bình" if prob >= 0.5
     ↪  else "thấp"
    return {"model": "DiabetesMLP (PyTorch)", "probability": prob,
     ↪  "threshold": self.threshold,
            "positive": prob >= self.threshold, "risk_level": level, "input":
             ↪  p}
```

Đoạn mã 4.6. Suy luận nguy cơ từ hồ sơ trong tác tử

4.4 Phân tích các tình huống lâm sàng thực tế

Ba bài toán dưới đây đều là câu hỏi người bệnh Việt Nam thường hỏi, và mỗi bài cần một loại suy luận khác nhau trên KG. Bảng 4.6 là kết quả nguyên văn của các truy vấn KG tương ứng.

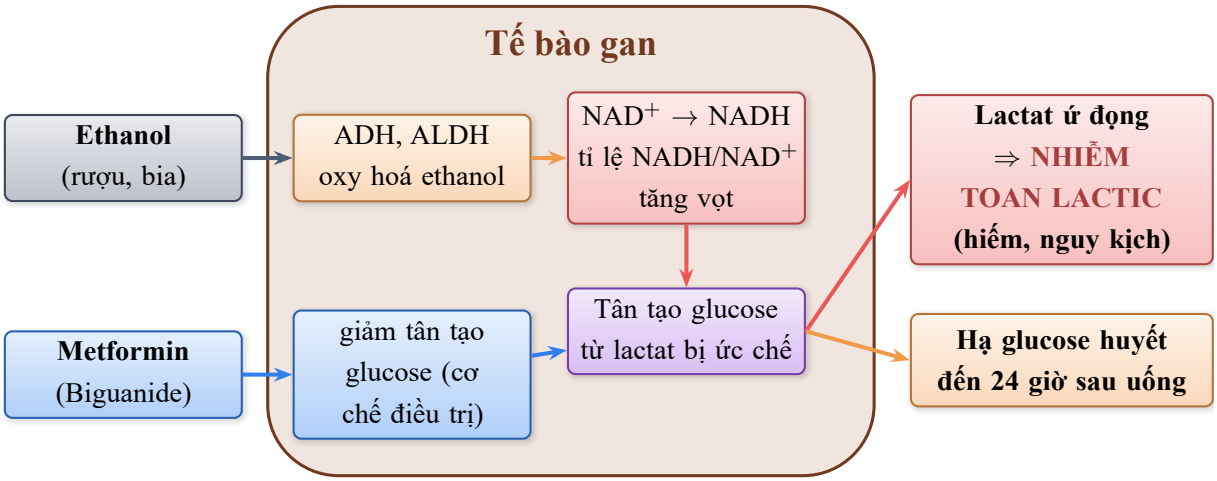
Bảng 4.6. Kết quả truy vấn `VietnamDiabetesKG` cho ba bài toán lâm sàng

Truy vấn trên KG	Kết quả (bộ ba đã diễn đạt)
Metformin + rượu bia: cạnh trực tiếp	Metformin —[tương tác nguy hiểm với]→ Rượu bia (ethanol) (severity: cao; effect: tăng nguy cơ nhiễm toan lactic; tránh uống quá mức và say rượu)
Metformin + rượu bia: nguy cơ chung (2 bước)	Nhiễm toan lactic
Thịt gà (nhóm thực phẩm)	Đái tháo đường típ 2 —[khuyến nghị ăn]→ Ức gà bỏ da luộc/hấp (reason: đạm nạc 31 g/100 g, không carbohydrate) Đái tháo đường típ 2 —[khuyến nghị hạn chế]→ Gà rán, gà chiên, da gà (reason: nhiều chất béo bão hòa, tăng đề kháng insulin)
Tinh bột	Đái tháo đường típ 2 —[ăn có định lượng]→ Cơm trắng (reason: GI 73, GL 29; tối đa ~1 bát con/bữa, ăn rau trước) Đái tháo đường típ 2 —[ăn có định lượng]→ Gạo lứt (reason: GI 68, giàu chất xơ; thay một phần gạo trắng) Đái tháo đường típ 2 —[ăn có định lượng]→ Bún, phở (reason: GI 53; nước dùng trong, nhiều rau) Đái tháo đường típ 2 —[khuyến nghị hạn chế]→ Khoai tây luộc (reason: GI 78) Đái tháo đường típ 2 —[khuyến nghị hạn chế]→ Xôi nếp (reason: GI rất cao)

Kết quả nguyên văn của `drug_warnings` và `food_advice` trên `VietnamDiabetesKG`; mỗi bộ ba mang mã tài liệu nguồn.

4.4.1 Bài toán 1: tương tác giữa metformin và rượu bia

Câu hỏi "đang uống metformin, tối nay đi tiệc uống bia được không" cần hai mẩu tri thức. Mẩu thứ nhất là một cạnh trực tiếp Metformin $\xrightarrow{\text{TUONG_TAC_NGUY_HIEM}}$ Ruou_Bia với mức độ `cao`. Mẩu thứ hai không được ghi tường minh mà là một *đường hai bước*: cả metformin và rượu bia đều có cạnh TANG_NGUY_CO tới Nhiem_Toan_Lactic. Hàm `drug_warnings` lấy cạnh trực tiếp và giao hai tập nguy cơ để tìm nguy cơ chung (Đoạn mã 4.7). Cơ chế sinh học đằng sau được vẽ ở Hình 4.13: ethanol tiêu thụ NAD^+ ở gan và ức chế tân tạo glucose từ lactat, metformin cũng giảm tân tạo glucose, hai tác động cộng lại làm lactat ứ đọng.



Khuyến nghị đưa ra từ KG (cạnh `TUONG_TAC_NGUY_HIEM`, nguồn `h03-ruou-bia`): tránh uống quá mức và say rượu; không uống khi đói; nếu có uống thì ăn kèm và theo dõi đường huyết. Đây là *cảnh báo mức cao*, không phải chống chỉ định tuyệt đối (tờ hướng dẫn metformin ghi "tránh uống rượu quá mức"); với sulfonylurea và insulin, rượu làm tăng mạnh nguy cơ hạ glucose huyết.

Hình 4.13. Cơ chế tương tác metformin – rượu bia: hai con đường cùng làm giảm sử dụng lactat ở gan, dẫn tới nguy cơ nhiễm toan lactic

Câu trả lời đúng không phải "tuyệt đối cấm" mà là cảnh báo có mức độ: tránh uống quá mức và tránh say, không uống khi đói, theo dõi dấu hiệu bất thường; với người dùng sulfonylurea hoặc insulin, rượu còn làm tăng mạnh nguy cơ hạ glucose huyết tới 24 giờ sau khi uống. Khung dưới đây là vết suy luận thật của tác tử cho câu hỏi này, do `build.py` sinh từ `agent.json`.

```python
# tl2/kg_health.py
def drug_warnings(kg, drug, substance):
    """Cảnh báo tương tác: cạnh trực tiếp drug -> substance và mọi nguy cơ
    ↪  mà cả hai cùng làm tăng (đường 2 bước)."""
    direct = list(kg.facts(drug, "TUONG_TAC_NGUY_HIEM", substance))
    risk_d = {o for _, _, o, _ in kg.facts(drug, "TANG_NGUY_CO")}
    risk_s = {o for _, _, o, _ in kg.facts(substance, "TANG_NGUY_CO")}
    return direct, sorted(risk_d & risk_s)
```

Đoạn mã 4.7. Cảnh báo tương tác: cạnh trực tiếp và nguy cơ chung qua đường hai bước

Vết suy luận ReAct thật cho câu hỏi y tế (kịch bản 1)

CÂU HỎI	*Tôi bị tiểu đường típ 2 đang uống Metformin 500mg, tối nay đi tiệc có uống bia được không?*
THOUGHT	Miền health; thực thể liên kết: Đái tháo đường típ 2, Metformin, Rượu bia (ethanol). Kế hoạch: kg_y_te → truy_xuat_rag.
THOUGHT	Câu hỏi nhắc tới thuốc, chất hoặc món ăn; tra đồ thị tri thức để lấy ràng buộc tương tác và khuyến nghị dinh dưỡng có nguồn.
ACTION	`kg_y_te`(DTD_Tip2, Metformin, Ruou_Bia) – loại KG, 0,2 ms
OBSERV.	Metformin —[tương tác nguy hiểm với]→ Rượu bia (ethanol) (severity: cao; effect: tăng nguy cơ nhiễm toan lactic; tránh uống quá mức và say rượu) | Metformin —[chống chỉ định khi]→ Suy thận nặng (eGFR < 30) (rule: eGFR < 30 mL/phút/1,73 m2) | Metformin —[chống chỉ định khi]→ Chụp cản quang có iod (rule: tạm ngưng trước chụp)
THOUGHT	Truy xuất văn bản gốc trong kho tri thức để trích dẫn căn cứ.
ACTION	`truy_xuat_rag`(Tôi bị tiểu đường típ 2 đang uống Metformin 500mg, tối nay đi tiệc có uống bia…) – loại RAG, 960,0 ms
OBSERV.	Rượu bia làm tăng nguy cơ nhiễm toan lactic ở người dùng metformin. Cơ chế là cả metformin và ethanol đều cản trở gan chuyển lactate thành glucose, khiến lactate tích tụ trong máu. Nguy cơ cao nhất khi uống nhiều, uống say hoặc uống kéo dài. Tờ hướng dẫn sử dụng metformin khuyến cáo người bệnh tránh uống rượu quá mức và tránh…
VERIFY	trích dẫn hợp lệ: 0; trung thực số học: 1,00; dùng **bản theo khuôn**; tổng thời gian 10,19 s

Bộ kiểm chứng đã từ chối bản nháp của LLM và trả bản tổng hợp theo khuôn. Bản nháp (dưới đây) đúng về nội dung, nhưng không ghi số hiệu bằng chứng [n] nào, nên không truy vết được từng ý về nguồn; Mục 6.4 phân tích hiện tượng này trên toàn bộ kịch bản.

Bản nháp của Qwen2.5-1.5B (bị bộ kiểm chứng từ chối vì thiếu trích dẫn)

[Từ câu hỏi và các chứng từ đã cung cấp, chúng ta có thể rút ra rằng]:
1. Người bệnh đang dùng Metformin 500mg.
2. Người bệnh cần tránh uống rượu bia vì có thể gây tăng nguy cơ nhiễm toan lactic.
3. Cụ thể hơn, Metformin có thể tương tác với rượu bia, tăng nguy cơ nhiễm toan lactic.
4. Theo hướng dẫn sử dụng Metformin, người bệnh nên chủ động từ chối rượu bia.
5. Nếu cần uống rượu, giới hạn khuyến cáo là không quá 1 đơn vị cồn mỗi ngày với nữ và không quá 2 đơn vị cồn mỗi ngày với nam.
6. Một đơn vị cồn tương đương khoảng 10g cồn nguyên chất, xấp xỉ một lon bia 330ml hoặc một chén rượu mạnh 30ml.
Tóm lại, theo khuyến nghị, bạn nên từ chối uống rượu bia tối nay. Nếu cần uống rượu, hãy tuân thủ giới hạn khuyến cáo.

4.4.2 Bài toán 2: dinh dưỡng thịt gà

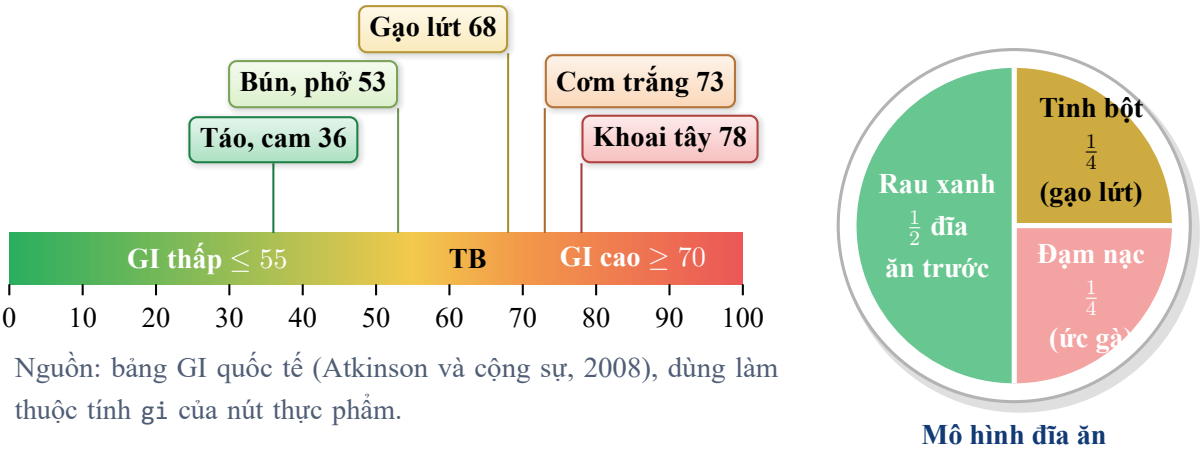
"Người tiểu đường ăn thịt gà được không" không có một câu trả lời: ức gà bỏ da luộc là đạm nạc 31 g/100 g, không carbohydrate, được khuyến nghị; gà rán và da gà nhiều chất béo bão hòa, làm tăng đề kháng insulin, nên hạn chế. Trong KG, Thit_Ga là một FoodGroup, các dạng cụ thể nối tới nó bằng LA_MOT_DANG_CUA. Hàm `food_advice` nhận ra câu hỏi về *nhóm* thực phẩm và duyệt xuống các dạng con trước khi tra lời khuyên (Đoạn mã 4.8), nên trả cả hai chiều khuyến nghị thay vì một câu "được" hay "không" sai một nửa.

```python
# tl2/kg_health.py
def food_advice(kg, disease, food):
    """Lời khuyên cho một món; nếu là nhóm thực phẩm (Thịt gà) thì duyệt các dạng cụ thể qua LA_MOT_DANG_CUA."""
    items = [food] + [s for s, _, _, _ in kg.facts(p="LA_MOT_DANG_CUA", o=food)]
    out = []
    for it in items:
        for rel in ("KHUYEN_NGHI_AN", "AN_DINH_LUONG", "HAN_CHE_AN"):
            out += list(kg.facts(disease, rel, it))
    return out
```

Đoạn mã 4.8. Lời khuyên dinh dưỡng: duyệt nhóm thực phẩm xuống các dạng cụ thể

4.4.3 Bài toán 3: định lượng tinh bột và ẩm thực truyền thống

Chỉ số đường huyết GI so sánh tốc độ một thực phẩm làm tăng glucose máu với glucose chuẩn (GI = 100); GI dưới 55 là thấp, trên 70 là cao [29]. Tải lượng đường huyết GL = GI × lượng carbohydrate (g) /100 tính cả khẩu phần. Các món tinh bột Việt Nam trải khắp thang GI (Hình 4.14): bún, phở GI khoảng 53, gạo lứt 68, cơm trắng 73, khoai tây luộc 78, xôi nếp rất cao. KG không cấm cơm mà mã hóa ba mức: AN_DINH_LUONG cho cơm trắng (tối đa khoảng một bát con mỗi bữa, ăn rau trước), HAN_CHE_AN cho khoai tây và xôi, KHUYEN_NGHI_AN cho rau xanh (ít nhất nửa đĩa). Truy vấn SPARQL Q2 ở Chương 2 trả đúng danh sách món có GI, sắp theo GI giảm dần, nhờ suy diễn kiểu từ các lớp ModerateFood, RiskFood lên Food.



Nguồn: bảng GI quốc tế (Atkinson và cộng sự, 2008), dùng làm thuộc tính `gi` của nút thực phẩm.

Hình 4.14. Thang chỉ số đường huyết GI của các món trong KG và mô hình đĩa ăn: nửa đĩa rau, một phần tư đạm, một phần tư tinh bột

4.4.4 Cơ chế phản ứng đa tầng trước tình huống phức tạp

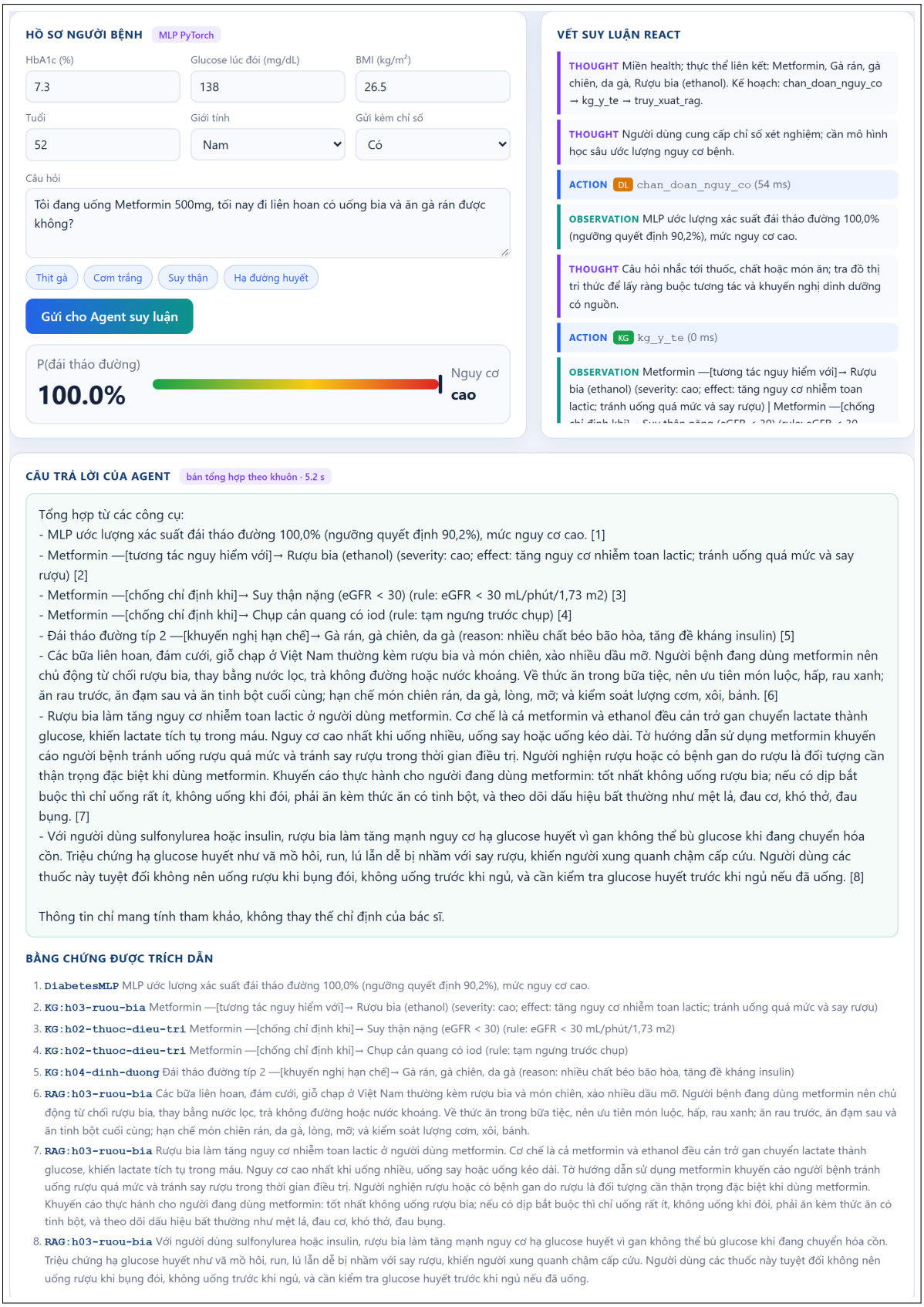
Câu hỏi thật hiếm khi chỉ một ý. Người bệnh nam 52 tuổi, HbA1c 7,3%, glucose lúc đói 138 mg/dL, BMI 26,5 hỏi "tôi đang uống metformin 500 mg, tối nay đi liên hoan có uống bia và ăn gà rán được không". Tác tử phản ứng theo ba tầng (Hình 4.15):

1. **Tầng học sâu**: hồ sơ xét nghiệm được gửi kèm, nên bộ lập kế hoạch gọi `chan_doan_nguy_co` trước; MLP chấm điểm nguy cơ vượt ngưỡng, mức "cao".
2. **Tầng đồ thị**: liên kết thực thể nhận ra Metformin, Ruou_Bia, Ga_Ran;

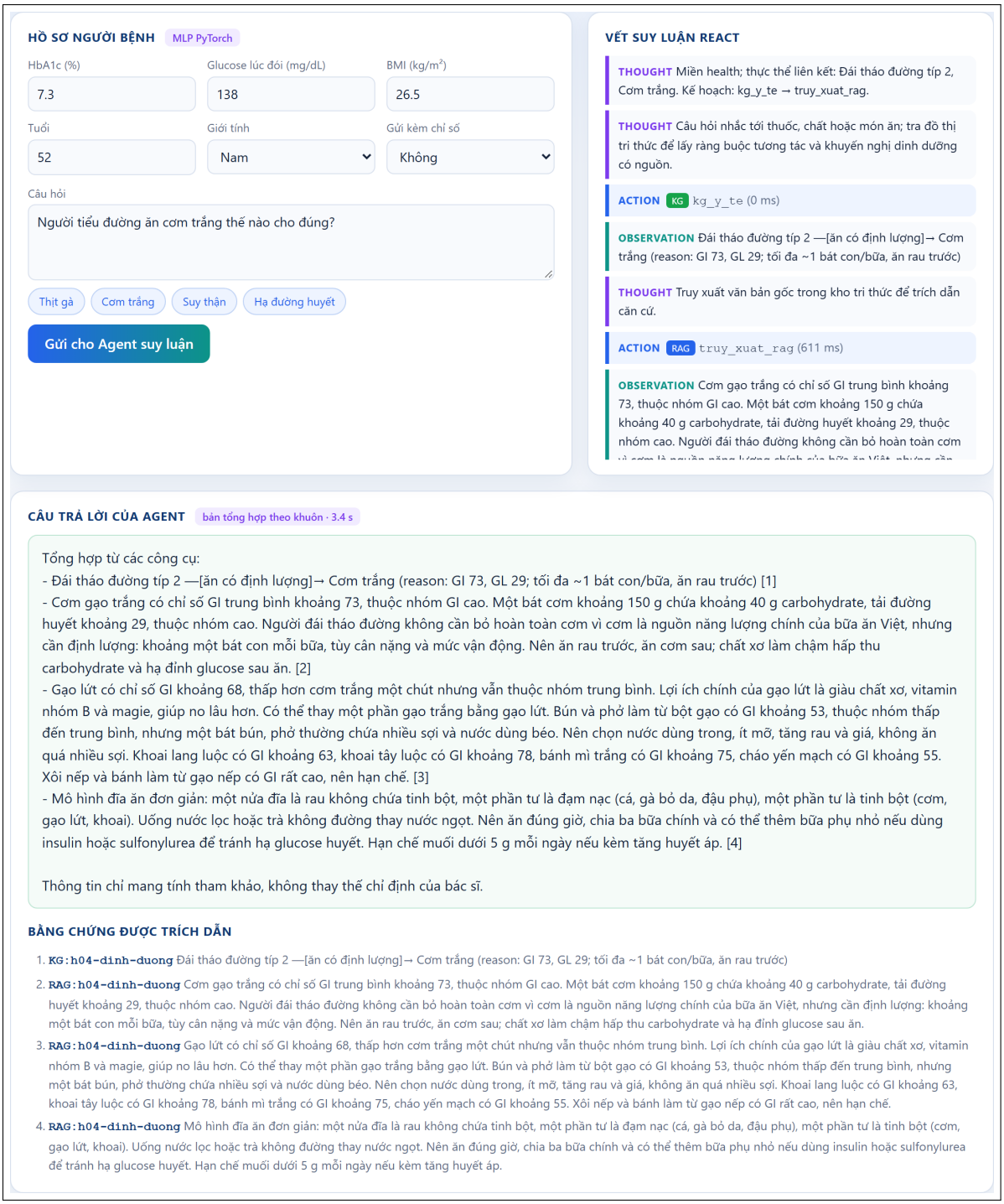
`kg_y_te` trả cảnh báo tương tác, các chống chỉ định của metformin và lời khuyên hạn chế gà rán.

3. **Tầng văn bản**: `truy_xuat_rag` lấy các đoạn về rượu bia trong bữa tiệc và cơ chế nhiễm toan lactic từ tài liệu `h03-ruou-bia` để giải thích.

Mỗi tầng trả lời một phần mà tầng khác không trả lời được: MLP không biết gì về bia, KG không biết người này có nguy cơ cao, văn bản không chấm điểm được hồ sơ. Hình 4.16 là câu hỏi về cơm trắng trên cùng giao diện, khi không gửi kèm hồ sơ: bộ lập kế hoạch bỏ qua MLP và chỉ dùng KG cùng RAG.



Hình 4.15. Trợ lý y tế trên Web App: hồ sơ xét nghiệm, vết suy luận ReAct, câu trả lời có trích dẫn và danh sách bằng chứng



Hình 4.16. Câu hỏi về cơm trắng không kèm hồ sơ: tác tử chỉ gọi KG và RAG

Chương 5

Hệ thống tài chính: VN30 và giá vàng SJC

Phân hệ tài chính trả lời loại câu hỏi mà nhà đầu tư cá nhân Việt Nam hay đặt ra: tỷ giá USD/VND tăng thì cổ phiếu nào hưởng lợi, Fed tăng lãi suất thì vàng SJC đi đâu, phiên tới FPT có thể ở mức giá nào. Ba câu hỏi đó cần ba loại tri thức khác nhau. Câu đầu cần *cơ chế* (doanh thu bằng ngoại tệ, chi phí bằng đồng); câu thứ hai cần *suy luận nhiều bước* qua các biến trung gian (DXY, giá vàng thế giới, giá quy đổi); câu thứ ba cần *dữ liệu chuỗi thời gian*. Chương này dựng ba lớp tương ứng: `VietnamFinanceKG` lưu đồ thị nhân quả có dấu và có nguồn, bộ suy luận đa bước tính chiều tác động ròng trên đồ thị đó, và mạng BiLSTM dự báo lợi suất phiên kế tiếp. Lớp thứ tư, ít hiển nhiên hơn, là bước *kiểm chứng KG bằng dữ liệu*: dấu của mỗi cạnh lý thuyết được đối chiếu với tương quan thực nghiệm trên gần 12 năm giao dịch.

Lưu ý về phạm vi

Hệ thống phục vụ mục đích học tập, không phải công cụ khuyến nghị đầu tư. Kho văn bản tài chính tổng hợp từ văn bản pháp luật (Nghị định 24/2012/NĐ-CP, Nghị định 232/2025/NĐ-CP), thông cáo của Ngân hàng Nhà nước và báo cáo thường niên doanh nghiệp; mọi câu trả lời của tác tử đều kết thúc bằng câu "Thông tin chỉ mang tính tham khảo, không phải khuyến nghị đầu tư".

5.1 Bối cảnh vĩ mô và dữ liệu thị trường

5.1.1 Bốn cổ phiếu, bốn kênh truyền dẫn

Rổ VN30 gồm 30 cổ phiếu vốn hóa và thanh khoản lớn nhất sàn HOSE. Bài chọn bốn mã không phải vì vốn hóa mà vì mỗi mã đại diện một kênh truyền dẫn vĩ mô khác nhau, để KG có đủ dấu dương và dấu âm cho cùng một biến nguồn:

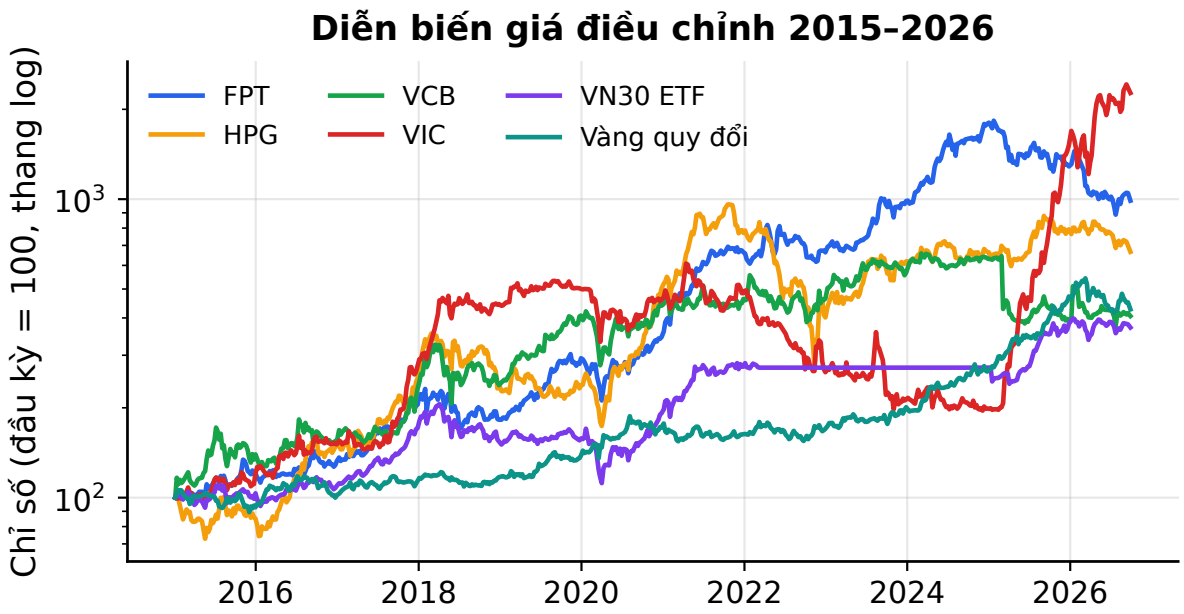
- **FPT** (công nghệ thông tin): doanh thu xuất khẩu phần mềm thanh toán bằng USD, JPY, EUR, còn chi phí chủ yếu là lương kỹ sư trả bằng đồng. Tỷ giá tăng làm doanh thu quy đổi tăng nhanh hơn chi phí, nên về mặt cơ bản FPT *hưởng lợi* khi USD/VND tăng; Nhật Bản là thị trường xuất khẩu lớn nhất nên FPT còn nhạy với đồng yên.
- **HPG** (thép): nhập quặng sắt và than cốc bằng USD và có nợ vay ngoại tệ, nên tỷ giá tăng làm giá vốn tăng và phát sinh lỗ tỷ giá; ngược lại, giải ngân đầu tư công làm tăng cầu thép xây dựng.
- **VCB** (ngân hàng): lợi nhuận phụ thuộc tăng trưởng tín dụng, mà tín dụng nhạy với lãi suất điều hành của Ngân hàng Nhà nước (NHNN).
- **VIC** (bất động sản, ô tô): chi phí tài chính lớn và có nợ vay ngoại tệ, chịu cả hai kênh lãi suất và tỷ giá.

Chỉ số VN30 được đại diện bằng giá chứng chỉ quỹ ETF mô phỏng VN30 (chuỗi `VN30ETF`). Ngoài bốn cổ phiếu, dữ liệu gồm giá vàng thế giới XAU/USD, tỷ giá USD/VND và chỉ số đồng đô la DXY. Hàm `market_frame` lấy lịch giao dịch của HOSE làm trục thời gian và điền tiến các chuỗi nước ngoài vào ngày giao dịch Việt Nam, nên mọi tài sản cùng có 2.863 phiên chung từ 2015-01-05 tới 2026-09-30.

Bảng 5.1. Thống kê mô tả các tài sản trong giai đoạn nghiên cứu

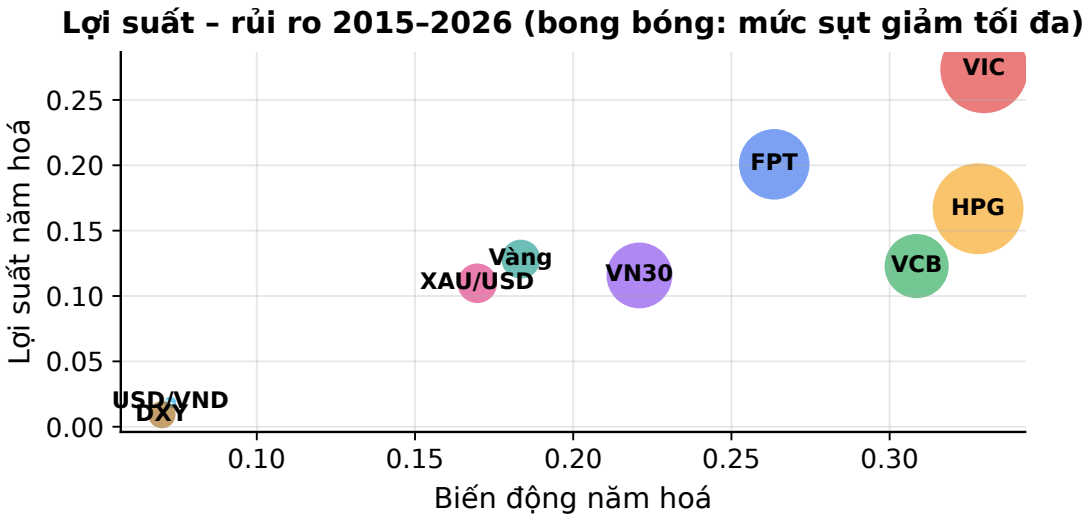
Tài sản	Giá cuối	Đơn vị	Lợi suất năm (%)	Biến động năm (%)	Sụt giảm lớn nhất (%)	Tăng cả kỳ (lần)
FPT	63.000	đồng	20,1	26,4	−52,1	8,78
HPG	20.150	đồng	16,7	32,8	−72,3	5,65
VCB	58.000	đồng	12,3	30,9	−46,1	3,04
VIC	227.900	đồng	27,3	33,0	−68,8	21,30
ETF E1VFVN30	34.550	đồng	11,6	22,1	−47,7	2,72
Vàng TG quy đổi	131,1	triệu đ/lượng	12,8	18,3	−24,4	3,29
XAU/USD	4.187	USD/ounce	11,0	17,0	−24,9	2,48
USD/VND	25.970	đồng	1,9	7,3	−5,8	0,23
DXY	101,4	điểm	0,9	7,0	−15,7	0,11

Dữ liệu ngày 2015-01-05 – 2026-09-30 (2.863 phiên chung); giá cổ phiếu đã điều chỉnh; lợi suất năm tính từ lợi suất log trung bình $\times 252$.



Hình 5.1. Diễn biến giá điều chỉnh của bốn cổ phiếu, ETF VN30 và vàng quy đổi, chuẩn hóa về 100 ở đầu kỳ (thang log)

Bảng 5.1 và Hình 5.2 cho thấy lợi suất cao luôn đi cùng rủi ro cao. VIC có lợi suất năm hóa cao nhất (27,33%) nhưng từng sụt −68,83% từ đỉnh; HPG sụt sâu nhất, −72,30%, đúng tính chu kỳ của ngành thép. Vàng quy đổi ra đồng có lợi suất 12,83% mỗi năm, ngang VCB, nhưng mức sụt tối đa chỉ −24,37%, vì vậy vàng đóng vai trò tài sản phòng thủ trong danh mục của người Việt. Tỷ giá USD/VND biến động chỉ 7,27% mỗi năm do NHNN điều hành trong biên độ, nhưng chính vì ít biến động nên mỗi lần tỷ giá tăng nhanh đều là tin lớn với thị trường.

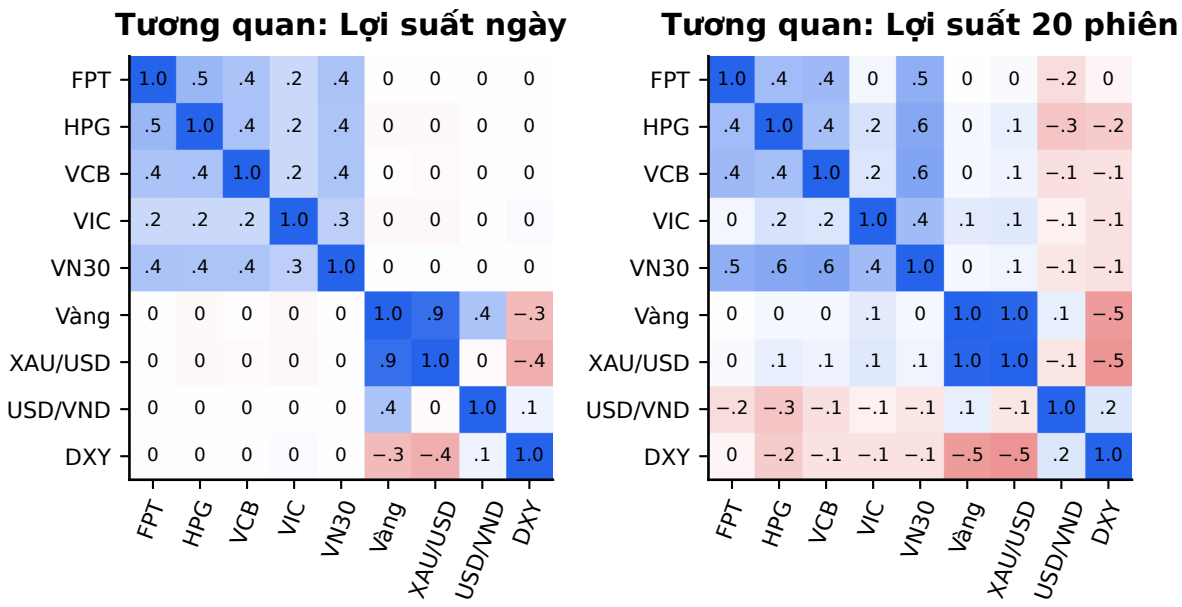


Hình 5.2. Lợi suất – rủi ro năm hóa; diện tích bong bóng tỷ lệ với mức sụt giảm tối đa

5.1.2 Tương quan giữa các tài sản

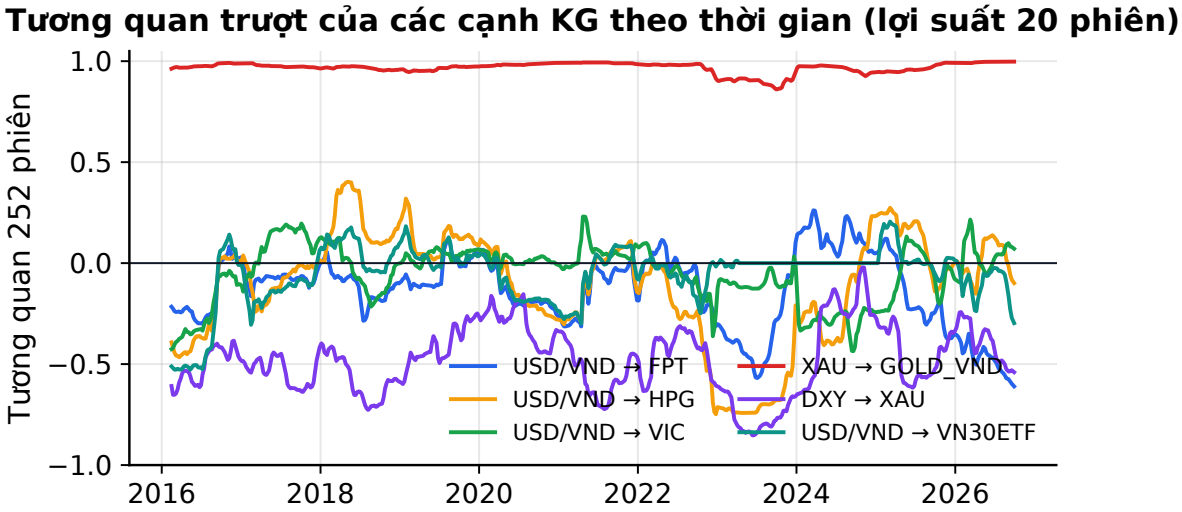
Ma trận tương quan (Hình 5.3) được tính trên hai thang: lợi suất ngày và lợi suất cộng dồn 20 phiên. Thang 20 phiên ít nhiễu hơn và gần với tầm nhìn của nhà đầu tư cá nhân. Ba nhóm quan hệ hiện ra rõ:

1. Bốn cổ phiếu cùng tương quan dương với ETF VN30 (từ 0,4353 với VIC tới 0,5781 với VCB): rủi ro chung của thị trường chiếm phần lớn biến động.
2. USD/VND tương quan *âm* với cả bốn cổ phiếu (FPT −0,1638, HPG −0,2669): tỷ giá tăng thường đi cùng khối ngoại bán ròng và tâm lý rủi ro.
3. Vàng thế giới gần như quyết định vàng quy đổi (0,9755), và ngược chiều với DXY (−0,4902) vì vàng được định giá bằng USD.



Hình 5.3. Ma trận tương quan Pearson của lợi suất ngày (trái) và lợi suất 20 phiên (phải)

Tương quan không đứng yên. Hình 5.4 vẽ tương quan trượt 252 phiên (khoảng một năm) của các cặp tương ứng với cạnh trong KG: cặp DXY – vàng luôn âm, còn cặp tỷ giá – cổ phiếu dao động quanh 0 và đổi dấu theo giai đoạn. Đây là cảnh báo đầu tiên rằng một cạnh có dấu cố định trong KG chỉ mô tả *xu hướng cơ bản*, không phải quy luật đúng ở mọi thời điểm.



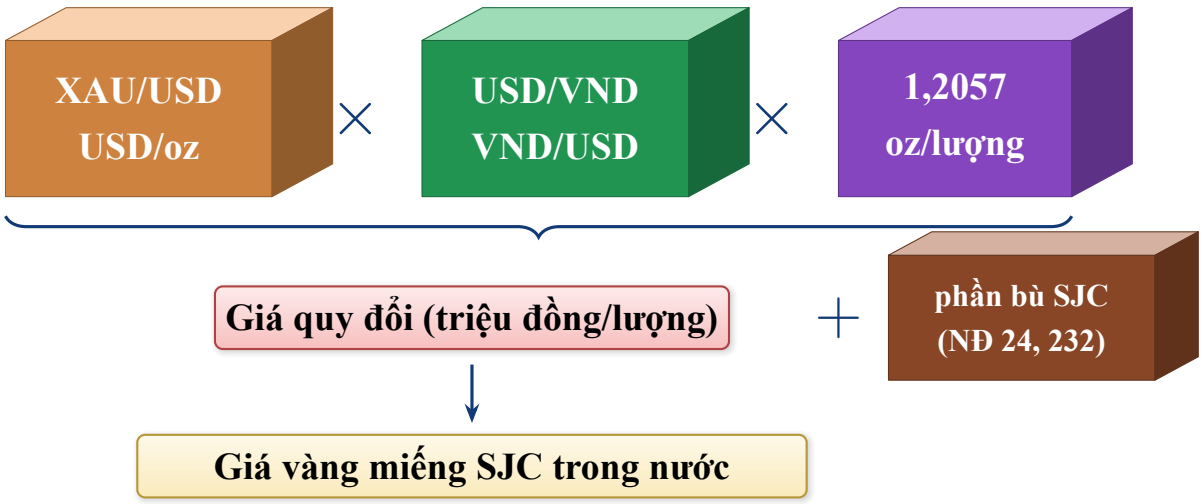
Hình 5.4. Tương quan trượt một năm của các cặp tương ứng với cạnh trong `VietnamFinanceKG`

5.1.3 Thị trường vàng Việt Nam: giá quy đổi và phần bù SJC

Giá vàng trong nước niêm yết theo *lượng* (37,5 g), còn giá thế giới theo *ounce troy* (31,1035 g). Giá vàng thế giới quy đổi ra đồng, chưa gồm thuế phí, là

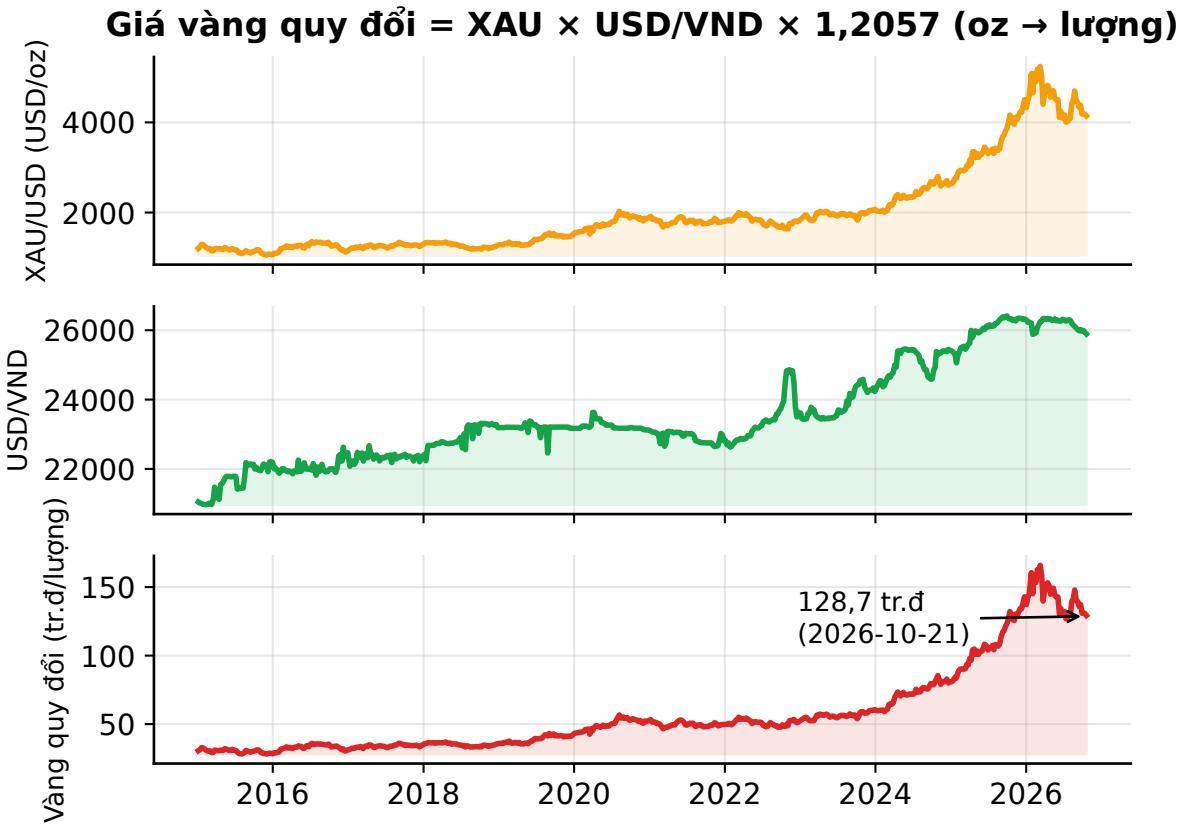
$$G_t = \frac{\text{XAU}_t \times (\text{USD/VND})_t \times 1{,}2057}{10^6} \quad \text{(triệu đồng/lượng)}, \tag{5.1}$$

với hệ số $1{,}2057 \approx 37{,}5/31{,}1035$ đổi giá mỗi ounce sang giá mỗi lượng. Giá vàng miếng SJC bằng giá quy đổi cộng một *phần bù* do cung cầu trong nước quyết định: $P_t^{\text{SJC}} = G_t + \Delta_t$. Ngày 2026-10-21, XAU/USD $= 4.125$ USD/ounce và USD/VND $= 25.872$, nên $G_t \approx 128{,}69$ triệu đồng/lượng. Công thức (5.1) chính là công cụ `quy_doi_vang` của tác tử, và là lý do hai cạnh XAU_USD $\rightarrow$ Vang_Quy_Doi, USD_VND $\rightarrow$ Vang_Quy_Doi trong KG đều mang dấu dương.



1 lượng $= 37{,}5$ g, 1 troy ounce $= 31{,}1035$ g $\Rightarrow 37{,}5/31{,}1035 = 1{,}2057$.
Ngày 2026-10-21: XAU 4.125 USD/oz, USD/VND 25.872 $\Rightarrow$ **128,69 triệu đồng/lượng** (chưa gồm thuế, phí và phần bù trong nước). Công cụ `quy_doi_vang` của tác tử tính đúng công thức này.

Hình 5.5. Cấu thành giá vàng miếng SJC: giá thế giới, tỷ giá và hệ số đổi đơn vị tạo nên giá quy đổi; phần bù trong nước phụ thuộc chính sách



Hình 5.6. XAU/USD, USD/VND và giá vàng quy đổi theo (5.1)

Phần bù Δ_t là đại lượng chịu tác động của chính sách, và là nơi KG cần tri thức pháp luật:

- **Nghị định 24/2012/NĐ-CP** [30] quy định Nhà nước độc quyền sản xuất vàng miếng, độc quyền xuất nhập khẩu vàng nguyên liệu để sản xuất vàng miếng; NHNN chọn SJC làm thương hiệu vàng miếng quốc gia. Từ năm 2014 NHNN gần như không nhập vàng nguyên liệu, nguồn cung bị giới hạn trong khi nhu cầu tích trữ tăng, nên phần bù có lúc lên tới 15 – 18 triệu đồng/lượng đầu năm 2024.
- **Can thiệp năm 2024**: NHNN đấu thầu vàng miếng từ tháng 4, rồi bán trực tiếp qua bốn ngân hàng thương mại nhà nước và Công ty SJC từ tháng 6; phần bù thu hẹp còn khoảng 3 – 5 triệu đồng/lượng.
- **Nghị định 232/2025/NĐ-CP** [31] sửa đổi Nghị định 24, có hiệu lực từ tháng 10/2025, bãi bỏ cơ chế độc quyền sản xuất vàng miếng và cho phép doanh nghiệp, ngân hàng đủ điều kiện sản xuất vàng miếng, nhập khẩu vàng nguyên liệu theo hạn mức do NHNN quản lý.

Trong KG, ba sự kiện này là ba cạnh vào nút Chenh_Lech_SJC: Nghị định 24 làm phần bù tăng (dấu $+$), hai biện pháp còn lại làm phần bù giảm (dấu $-$). Vàng nhẫn 9999 không chịu cơ chế độc quyền nên chỉ nối với giá quy đổi, không nối với phần bù.

5.2 Đồ thị tri thức tài chính `VietnamFinanceKG`

5.2.1 Bản thể học

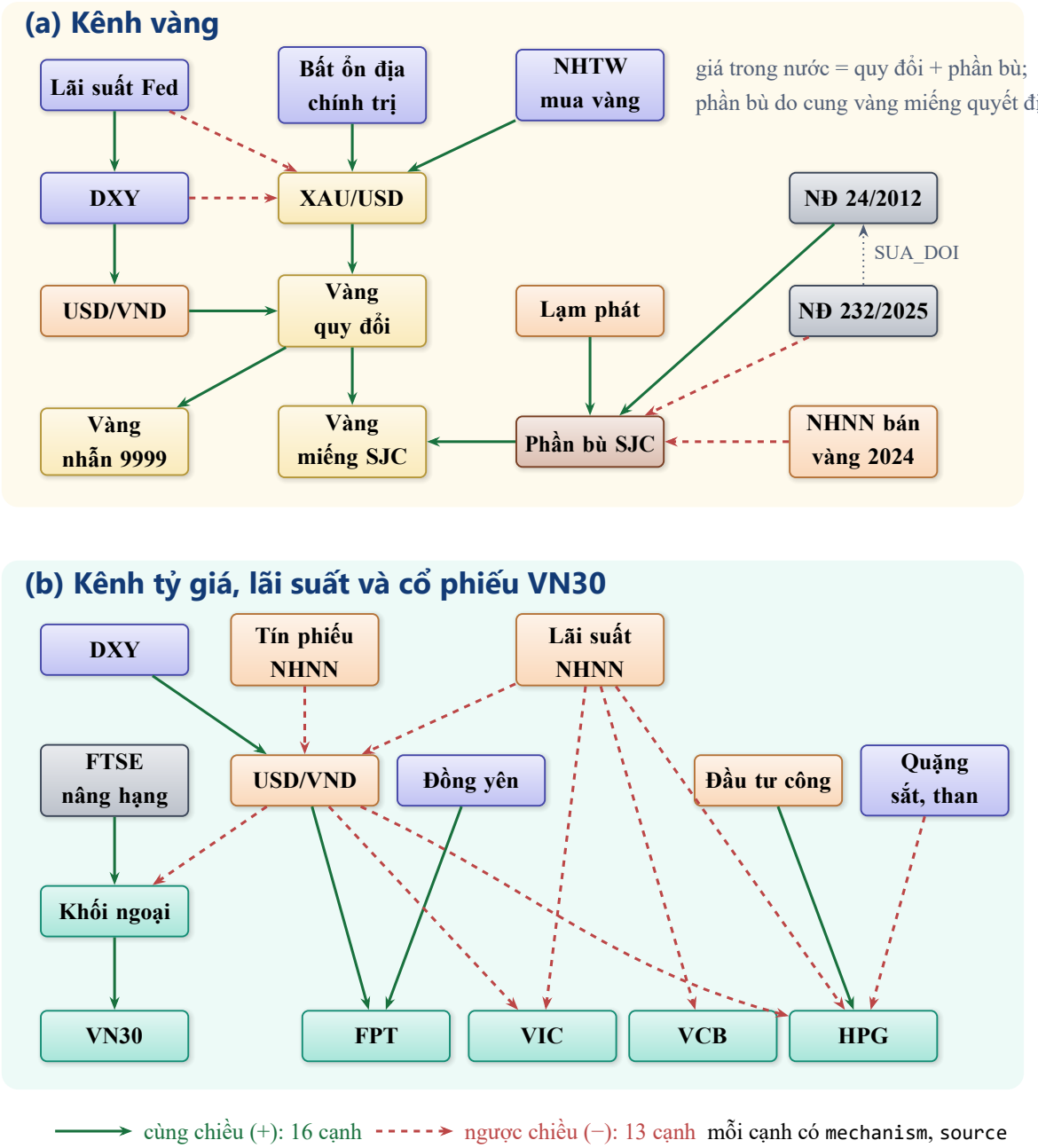
Bản thể học tài chính nhỏ hơn bản thể học y tế nhưng có cấu trúc khác: 16 lớp và chỉ 6 vị từ. Lớp gốc FinancialEntity tách thành yếu tố vĩ mô (MacroFactor, với ba lớp con PolicyTool, GlobalFactor, MarketFlow), tài sản (Asset $\rightarrow$ Commodity $\rightarrow$ GoldAsset, Stock, Index), văn bản pháp luật, sự kiện, chỉ báo, ngành và sàn giao dịch. Hai vị từ quan trọng nhất mang *dấu*: TAC_DONG_CUNG_CHIEU ($+1$, biến nguồn tăng thì biến đích có xu hướng tăng) và TAC_DONG_NGUOC_CHIEU (-1). Bốn vị từ còn lại là quan hệ cấu trúc (thành viên chỉ số, ngành, sửa đổi văn bản, phạm vi điều chỉnh) và không tham gia lan truyền tác động. THUOC_VE được khai báo bắc cầu, nên suy diễn RDFS (Hình 2.6) tự sinh "FPT thuộc về HOSE" từ hai cạnh "FPT thuộc VN30" và "VN30 thuộc HOSE".

5.2.2 Thực thể và quan hệ nhân quả

KG có 31 thực thể và 40 cạnh, trong đó 16 cạnh cùng chiều và 13 cạnh ngược chiều. Cả 40 cạnh đều có thuộc tính `source` trỏ về tài liệu trong kho văn bản (`f01` – `f09`) và thuộc tính `mechanism` giải thích cơ chế bằng một câu, nên mỗi kết luận của tác tử đều truy được về văn bản gốc. Thực thể có chuỗi giá thật mang thuộc tính `series`, nối KG với dữ liệu thị trường; nhờ đó Mục 5.3.3 kiểm chứng được dấu cạnh. Hình 5.7 tách đồ thị nhân quả thành hai kênh để dễ đọc, Hình 5.8 vẽ toàn bộ KG.

```python
# tl2/kg_finance.py
def finance_ontology():
    o = Ontology()
    for c, parent in [
        ("Thing", None), ("FinancialEntity", "Thing"),
        ("MacroFactor", "FinancialEntity"), ("PolicyTool", "MacroFactor"),
        ↪ ("GlobalFactor", "MacroFactor"),
        ("MarketFlow", "MacroFactor"), ("Event", "FinancialEntity"),
        ("Regulation", "FinancialEntity"), ("Asset", "FinancialEntity"),
        ↪ ("Commodity", "Asset"),
        ("GoldAsset", "Commodity"), ("Indicator", "FinancialEntity"),
        ("Stock", "Asset"), ("Index", "Asset"), ("Exchange",
        ↪ "FinancialEntity"), ("Sector", "FinancialEntity"),
    ]:
        o.add_class(c, parent)
    P = o.add_predicate
    P("TAC_DONG_CUNG_CHIEU", "FinancialEntity", "FinancialEntity", "tác động
    ↪ cùng chiều (+)")
    P("TAC_DONG_NGUOC_CHIEU", "FinancialEntity", "FinancialEntity", "tác
    ↪ động ngược chiều (−)")
    P("THUOC_VE", "FinancialEntity", "FinancialEntity", "thuộc về",
    ↪ transitive=True)
    P("THUOC_NGANH", "Stock", "Sector", "thuộc ngành")
    P("SUA_DOI", "Regulation", "Regulation", "sửa đổi, bổ sung")
    P("QUY_DINH_VE", "Regulation", "Asset", "quy định về")
    return o
```

Đoạn mã 5.1. Bản thể học tài chính: 16 lớp và 6 vị từ, hai vị từ nhân quả mang dấu



Hình 5.7. Đồ thị nhân quả của `VietnamFinanceKG`: (a) kênh vàng, (b) kênh tỷ giá – lãi suất – cổ phiếu; xanh liền là cùng chiều, đỏ đứt là ngược chiều

Đoạn mã 5.2. Xây `VietnamFinanceKG`: thực thể kèm bí danh, chuỗi giá, và các cạnh nhân quả có nguồn, có cơ chế

```
# tl2/kg_finance.py
def build_finance_kg():
    kg = KnowledgeGraph("VietnamFinanceKG", finance_ontology())
    E = kg.add_entity
    E("Lai_Suat_Fed", "GlobalFactor", "Lãi suất Fed", ["fed", "lãi suất mỹ",
     ↪  "lãi suất fed"])
```

```
    E("DXY", "GlobalFactor", "Chỉ số USD (DXY)", ["dxy", "đồng usd mạnh", "đô la mỹ", "đồng đô"], series="DXY")
    E("Bat_On_Dia_Chinh_Tri", "GlobalFactor", "Bất ổn địa chính trị", ["địa chính trị", "chiến tranh", "bất ổn"])
    E("NHTW_Mua_Vang", "GlobalFactor", "Ngân hàng trung ương mua vàng", ["ngân hàng trung ương mua vàng"])
    E("Dong_Yen", "GlobalFactor", "Sức mạnh đồng yên (JPY)", ["yên", "jpy", "đồng yên"])
    E("Gia_Quang_Sat", "GlobalFactor", "Giá quặng sắt, than cốc", ["quặng sắt", "than cốc", "than mỡ"])
    E("USD_VND", "MacroFactor", "Tỷ giá USD/VND", ["tỷ giá", "usd/vnd", "usdvnd", "đô la", "usd"], series="USDVND")
    E("Lam_Phat", "MacroFactor", "Lạm phát", ["lạm phát"])
    E("Lai_Suat_NHNN", "PolicyTool", "Lãi suất điều hành NHNN", ["lãi suất", "lãi suất điều hành", "tái cấp vốn", "nhnn"],
      refinancing_rate=4.5)
    E("Tin_Phieu_NHNN", "PolicyTool", "Phát hành tín phiếu NHNN", ["tín phiếu"])
    E("Dau_Tu_Cong", "MacroFactor", "Giải ngân đầu tư công", ["đầu tư công", "hạ tầng", "cao tốc"])
    E("Khoi_Ngoai", "MarketFlow", "Khối ngoại mua ròng", ["khối ngoại", "nhà đầu tư nước ngoài", "vốn ngoại"])
    E("Nang_Hang_FTSE", "Event", "FTSE nâng hạng thị trường mới nổi thứ cấp", ["nâng hạng", "ftse"], effective="9/2026")
    E("ND_24_2012", "Regulation", "Nghị định 24/2012/NĐ-CP", ["nghị định 24", "24/2012", "độc quyền vàng"], year=2012)
    E("ND_232_2025", "Regulation", "Nghị định 232/2025/NĐ-CP", ["nghị định 232", "232/2025"], year=2025)
    E("NHNN_Ban_Vang_2024", "PolicyTool", "NHNN đấu thầu và bán vàng miếng (2024)", ["đấu thầu vàng", "bán vàng"], year=2024)
    E("XAU_USD", "GoldAsset", "Giá vàng thế giới (XAU/USD)", ["giá vàng thế giới", "xau", "vàng thế giới"], series="XAU")
    E("Vang_Quy_Doi", "GoldAsset", "Giá vàng thế giới quy đổi (VND/lượng)", ["giá quy đổi", "quy đổi"], series="GOLD_VND",
      factor=1.2057)
    E("Chenh_Lech_SJC", "Indicator", "Phần bù vàng miếng SJC so với thế giới", ["chênh lệch", "phần bù"])
    E("Vang_SJC", "GoldAsset", "Vàng miếng SJC", ["vàng sjc", "sjc", "vàng miếng", "vàng"])
    E("Vang_Nhan", "GoldAsset", "Vàng nhẫn 9999", ["vàng nhẫn", "9999"])
    E("FPT", "Stock", "FPT (Tập đoàn FPT)", ["fpt"], series="FPT")
    E("HPG", "Stock", "HPG (Hòa Phát)", ["hpg", "hòa phát", "thép"], series="HPG")
    E("VCB", "Stock", "VCB (Vietcombank)", ["vcb", "vietcombank"], series="VCB")
    E("VIC", "Stock", "VIC (Vingroup)", ["vic", "vingroup", "vinhomes", "vinfast"], series="VIC")
    E("VN30", "Index", "Chỉ số VN30", ["vn30"], series="VN30ETF")
```

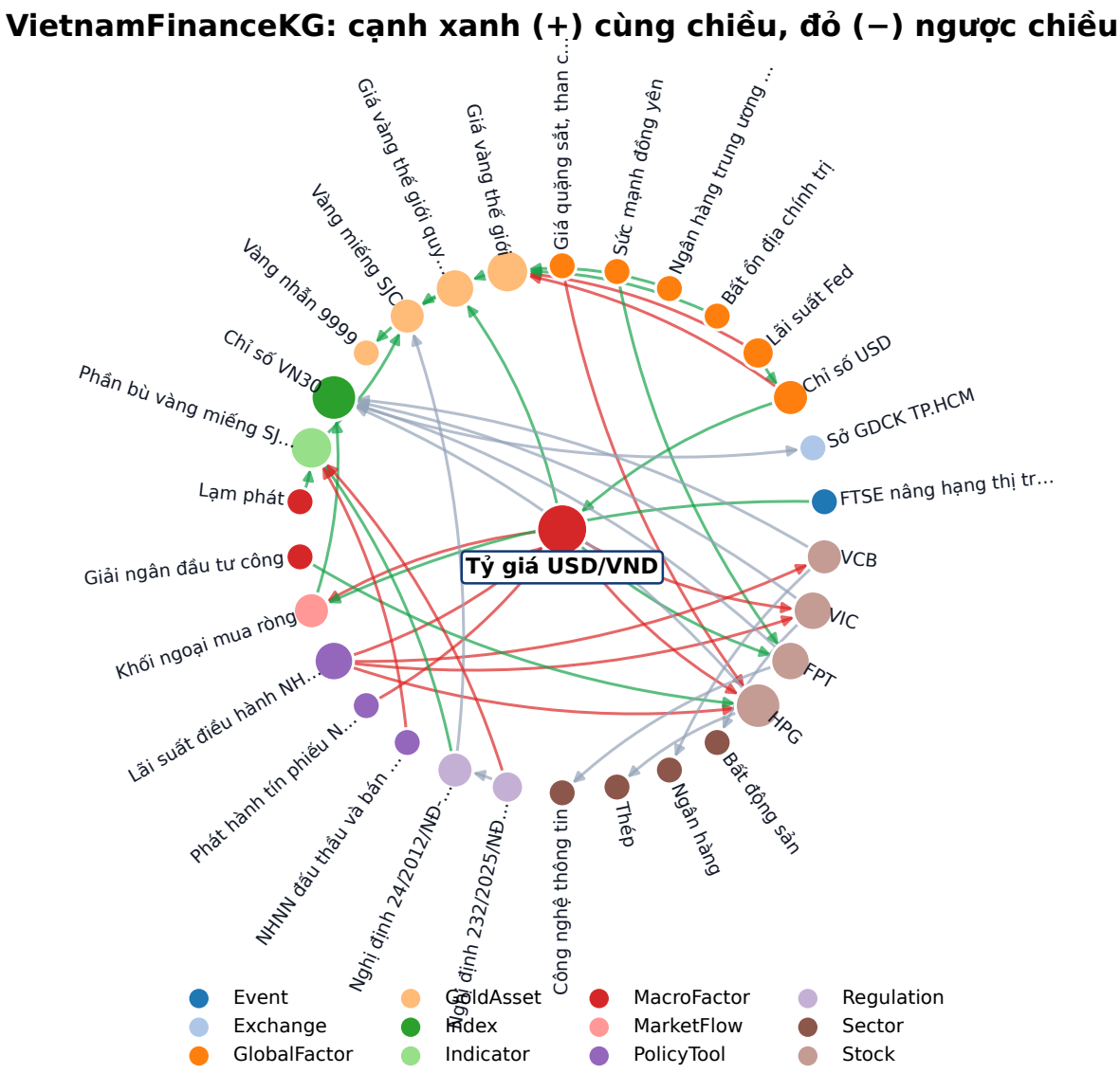
```
    E("HOSE", "Exchange", "Sở GDCK TP.HCM (HOSE)", ["hose"])
    E("Nganh_CNTT", "Sector", "Công nghệ thông tin", ["công nghệ"])
    E("Nganh_Thep", "Sector", "Thép", ["ngành thép"])
    E("Nganh_Ngan_Hang", "Sector", "Ngân hàng", ["ngân hàng"])
    E("Nganh_BDS", "Sector", "Bất động sản", ["bất động sản"])

    def up(s, o, src, why):
        kg.add_fact(s, "TAC_DONG_CUNG_CHIEU", o, source=src, mechanism=why)

    def down(s, o, src, why):
        kg.add_fact(s, "TAC_DONG_NGUOC_CHIEU", o, source=src, mechanism=why)

    # kênh vàng
    down("Lai_Suat_Fed", "XAU_USD", "f02-gia-vang-the-gioi", "chi phí cơ hội
    ↪ nắm giữ vàng tăng")
    up("Lai_Suat_Fed", "DXY", "f02-gia-vang-the-gioi", "lãi suất USD cao làm
    ↪ USD mạnh lên")
    down("DXY", "XAU_USD", "f02-gia-vang-the-gioi", "vàng định giá bằng USD,
    ↪ biến động ngược chiều DXY")
    up("Bat_On_Dia_Chinh_Tri", "XAU_USD", "f02-gia-vang-the-gioi", "nhu cầu
    ↪ tài sản trú ẩn")
    up("NHTW_Mua_Vang", "XAU_USD", "f02-gia-vang-the-gioi", "cầu vàng của
    ↪ ngân hàng trung ương")
    up("XAU_USD", "Vang_Quy_Doi", "f02-gia-vang-the-gioi", "XAU/USD ×
    ↪ USD/VND × 1,2057")
    up("USD_VND", "Vang_Quy_Doi", "f02-gia-vang-the-gioi", "XAU/USD ×
    ↪ USD/VND × 1,2057")
    up("Vang_Quy_Doi", "Vang_SJC", "f02-gia-vang-the-gioi", "giá trong nước
    ↪ = quy đổi + phần bù")
    up("Vang_Quy_Doi", "Vang_Nhan", "f01-vang-mieng-sjc", "vàng nhẫn bám sát
    ↪ giá quy đổi")
    up("Chenh_Lech_SJC", "Vang_SJC", "f01-vang-mieng-sjc", "giá trong nước =
    ↪ quy đổi + phần bù")
    up("ND_24_2012", "Chenh_Lech_SJC", "f01-vang-mieng-sjc", "độc quyền,
    ↪ không nhập vàng nguyên liệu, cung giới hạn")
    down("NHNN_Ban_Vang_2024", "Chenh_Lech_SJC", "f01-vang-mieng-sjc", "tăng
    ↪ cung, chênh lệch thu hẹp")
    down("ND_232_2025", "Chenh_Lech_SJC", "f01-vang-mieng-sjc", "bãi bỏ độc
    ↪ quyền, tăng cung hợp pháp")
    up("Lam_Phat", "Chenh_Lech_SJC", "f01-vang-mieng-sjc", "nhu cầu tích trữ
    ↪ tăng khi lạm phát")
    # kênh tỷ giá và chính sách tiền tệ
    up("DXY", "USD_VND", "f03-ty-gia", "USD mạnh lên toàn cầu gây áp lực tỷ
    ↪ giá")
    down("Lai_Suat_NHNN", "USD_VND", "f04-chinh-sach-tien-te", "lãi suất VND
    ↪ thấp hơn USD gây áp lực tỷ giá")
    down("Tin_Phieu_NHNN", "USD_VND", "f04-chinh-sach-tien-te", "hút thanh
    ↪ khoản VND, giảm đầu cơ USD")
```

```
    down("USD_VND", "Khoi_Ngoai", "f03-ty-gia", "lợi nhuận quy đổi ra USD giảm, khối ngoại bán ròng")
    up("Nang_Hang_FTSE", "Khoi_Ngoai", "f09-vn30", "thu hút vốn ngoại")
    up("Khoi_Ngoai", "VN30", "f09-vn30", "dòng vốn vào cổ phiếu vốn hóa lớn")
    # kênh doanh nghiệp
    up("USD_VND", "FPT", "f05-fpt", "doanh thu xuất khẩu phần mềm bằng ngoại tệ, chi phí bằng VND")
    up("Dong_Yen", "FPT", "f05-fpt", "Nhật Bản là thị trường xuất khẩu lớn nhất")
    down("USD_VND", "HPG", "f06-hpg", "nhập quặng sắt, than cốc bằng USD; lỗ tỷ giá nợ vay")
    down("Gia_Quang_Sat", "HPG", "f06-hpg", "giá nguyên liệu tăng làm co biên lợi nhuận")
    up("Dau_Tu_Cong", "HPG", "f06-hpg", "tăng nhu cầu thép xây dựng")
    down("Lai_Suat_NHNN", "HPG", "f06-hpg", "lãi suất cao làm giảm nhu cầu xây dựng, bất động sản")
    down("Lai_Suat_NHNN", "VCB", "f07-vcb", "lãi suất thấp, tín dụng tăng làm lợi nhuận ngân hàng tăng")
    down("Lai_Suat_NHNN", "VIC", "f08-vic", "lãi suất vay mua nhà và chi phí tài chính")
    down("USD_VND", "VIC", "f08-vic", "gánh nặng nợ vay ngoại tệ tăng")
    # cấu trúc
    for s, sec in (("FPT", "Nganh_CNTT"), ("HPG", "Nganh_Thep"), ("VCB", "Nganh_Ngan_Hang"), ("VIC", "Nganh_BDS")):
        kg.add_fact(s, "THUOC_VE", "VN30", source="f09-vn30")
        kg.add_fact(s, "THUOC_NGANH", sec, source="f09-vn30")
    kg.add_fact("VN30", "THUOC_VE", "HOSE", source="f09-vn30")
    kg.add_fact("ND_232_2025", "SUA_DOI", "ND_24_2012", source="f01-vang-mieng-sjc")
    kg.add_fact("ND_24_2012", "QUY_DINH_VE", "Vang_SJC", source="f01-vang-mieng-sjc")
    return kg
```



Hình 5.8. Toàn cảnh `VietnamFinanceKG`: 31 thực thể, 40 quan hệ; màu nút theo lớp, màu cạnh theo dấu tác động

Khi xuất ra RDF, KG có 402 bộ ba; bộ suy diễn RDFS thêm 111 bộ ba (kiểu kế thừa từ lớp cha, quan hệ bắc cầu) để tổng số đạt 513. Chẳng hạn "FPT là một Stock" kéo theo "FPT là một Asset" và "FPT là một FinancialEntity". Nhờ vậy truy vấn SPARQL hỏi "mọi tài sản chịu tác động ngược chiều của tỷ giá" không cần liệt kê từng lớp con.

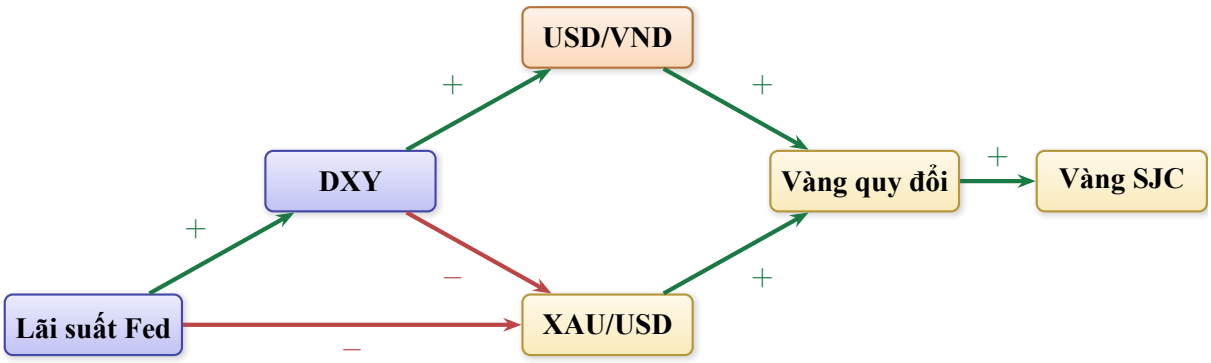
5.3 Suy luận tác động đa bước

5.3.1 Dấu của đường đi và tác động ròng

Với một biến nguồn u (chẳng hạn lãi suất Fed) và một biến đích v (vàng SJC), hàm `impact_paths` liệt kê mọi đường đơn $\pi = (u = x_0 \to x_1 \to \cdots \to x_k = v)$ dài không quá 4 bước trên đồ thị con nhân quả. Dấu của đường là tích dấu các cạnh, và tác động ròng là tổng dấu các đường có trọng số nghịch đảo độ dài:

$$s(\pi) \;=\; \prod_{j=1}^{k} s(x_{j-1}, x_j), \qquad N(u, v) \;=\; \sum_{\pi:\, u \rightsquigarrow v} \frac{s(\pi)}{|\pi|}. \tag{5.2}$$

Trọng số $1/|\pi|$ thể hiện giả định rằng tác động suy giảm qua mỗi bước trung gian: một kênh trực tiếp đáng tin hơn một chuỗi bốn mắt xích. Khi mọi đường cùng dấu, kết luận là "tích cực" hoặc "tiêu cực"; khi các đường mâu thuẫn, bộ suy luận *không* gộp thành một chiều mà trả về "hỗn hợp" kèm số kênh mỗi dấu, để tác tử nói rõ kết quả phụ thuộc kênh truyền dẫn nào mạnh hơn.



Đường	Dấu	Bước	Đóng góp
Fed → XAU → quy đổi → SJC	(−)	3	−1/3
Fed → DXY → XAU → quy đổi → SJC	(−)	4	−1/4
Fed → DXY → USD/VND → quy đổi → SJC	(+)	4	+1/4
Ròng $\sum_\pi \mathrm{sign}(\pi)/\lvert\pi\rvert$			**−1/3**

Kết luận của agent: "hỗn hợp: 1 kênh (+), 2 kênh (−), ròng (−)". Fed tăng lãi suất kéo giá vàng thế giới xuống, nhưng tỷ giá tăng bù một phần cho giá quy đổi; agent nói rõ cả hai kênh thay vì kết luận một chiều.

Hình 5.9. Ba đường nhân quả từ lãi suất Fed tới vàng SJC mà `impact_paths` tìm được, dấu của từng đường và tác động ròng

```python
# tl2/kg_finance.py
def impact_paths(kg, factor, target, max_hops=4):
    """Liệt kê mọi đường nhân quả factor ⇝ target dài ≤ max_hops, kèm dấu tổng hợp.

    Tác động ròng được ước lượng bằng tổng có trọng số các dấu, đường ngắn nặng hơn: Σ sign(π) / |π|.
    Khi các đường mâu thuẫn dấu, tác tử phải nói rõ là "phụ thuộc kênh truyền dẫn" thay vì kết luận một chiều.
    """
    C = causal_graph(kg)
    if factor not in C or target not in C:
        return {"paths": [], "net": 0.0, "verdict": "không có đường nhân quả"}
    paths = []
    for path in nx.all_simple_paths(C, factor, target, cutoff=max_hops):
        sign, steps = 1, []
        for u, v in zip(path, path[1:]):
            e = C.edges[u, v]
            sign *= e["sign"]
            steps.append({"from": u, "to": v, "sign": e["sign"], "mechanism": e["mechanism"], "source": e["source"]})
        paths.append({"nodes": path, "sign": sign, "hops": len(path) - 1, "steps": steps})
    paths.sort(key=lambda p: p["hops"])
    net = sum(p["sign"] / p["hops"] for p in paths)
    pos, neg = sum(p["sign"] > 0 for p in paths), sum(p["sign"] < 0 for p in paths)
    verdict = ("không có đường nhân quả" if not paths else "tích cực (+)" if neg == 0 else "tiêu cực (−)" if pos == 0
               else f"hỗn hợp: {pos} kênh (+), {neg} kênh (−), ròng {'+' if net > 0 else '−'}")
    return {"paths": paths, "net": net, "verdict": verdict}
```

Đoạn mã 5.3. Suy luận đa bước: liệt kê đường nhân quả, tính dấu từng đường và tác động ròng (5.2)

Bảng 5.2. Kết quả suy luận đa bước cho bảy cặp nguồn – đích

Câu hỏi tác động	Số đường	Ròng	Kết luận của bộ suy luận
`USD_VND` → `FPT`	1	+1,00	tích cực (+)
`USD_VND` → `HPG`	1	−1,00	tiêu cực (−)
`Lai_Suat_Fed` → `Vang_SJC`	3	−0,33	hỗn hợp: 1 kênh (+), 2 kênh (−), ròng −
`Lai_Suat_Fed` → `FPT`	1	+0,33	tích cực (+)
`ND_232_2025` → `Vang_SJC`	1	−0,50	tiêu cực (−)
`DXY` → `VN30`	1	−0,33	tiêu cực (−)
`Lai_Suat_NHNN` → `HPG`	2	−0,50	hỗn hợp: 1 kênh (+), 1 kênh (−), ròng −

Dấu một đường π là tích dấu các cạnh; ròng $= \sum_{\pi}$ dấu$(\pi)/|\pi|$ với $|\pi|$ là số bước, nên đường ngắn có trọng số lớn hơn.

Bảng 5.2 có hai trường hợp "hỗn hợp" đáng phân tích:

- **Fed → vàng SJC**: hai đường âm (Fed → XAU → quy đổi → SJC, và Fed → DXY → XAU → quy đổi → SJC) và một đường dương (Fed → DXY → USD/VND → quy đổi → SJC). Đường dương có thật về kinh tế: Fed tăng lãi suất làm USD mạnh, tỷ giá tăng, và giá vàng tính bằng đồng được đỡ một phần. Tác động ròng $N = -1/3 - 1/4 + 1/4 = -0{,}3333$: chiều chi phí cơ hội của vàng thắng, nhưng nhà đầu tư trong nước chịu giảm ít hơn nhà đầu tư nắm vàng tính bằng USD.
- **Lãi suất NHNN → HPG**: một đường trực tiếp âm (lãi suất cao làm giảm cầu xây dựng) và một đường hai bước dương (lãi suất VND cao làm giảm áp lực tỷ giá, mà tỷ giá thấp có lợi cho doanh nghiệp nhập nguyên liệu). Ròng $-1 + 1/2 = -0{,}5000$.

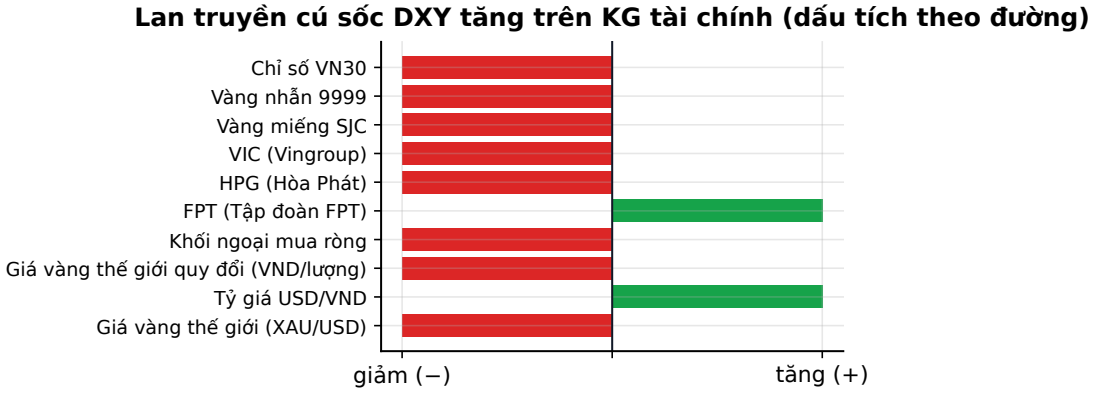
5.3.2 Lan truyền cú sốc

Câu hỏi ngược lại, "một cú sốc ở u ảnh hưởng tới những gì", được trả lời bằng hàm `downstream`: duyệt theo chiều rộng từ u tối đa 3 bước, mỗi thực thể nhận dấu của đường ngắn nhất tới nó. Hình 5.10 và Bảng 5.3 là kết quả cho cú sốc DXY tăng: mười thực thể bị ảnh hưởng. FPT là cổ phiếu duy nhất có lợi (qua

tỷ giá); VN30, HPG, VIC và cả bốn nút vàng đều chịu chiều giảm; VCB không nằm trong danh sách vì KG không có kênh nối VCB với tỷ giá.

```python
# tl2/kg_finance.py
def downstream(kg, factor, max_hops=3):
    """Mọi thực thể chịu ảnh hưởng của một cú sốc, theo dấu của đường ngắn nhất (lan truyền theo chiều rộng)."""
    C = causal_graph(kg)
    out, frontier, seen = {}, [(factor, 1, [factor])], {factor}
    for _ in range(max_hops):
        nxt = []
        for u, s, path in frontier:
            for v in C.successors(u):
                if v in seen:
                    continue
                seen.add(v)
                sv = s * C.edges[u, v]["sign"]
                out[v] = {"sign": sv, "path": path + [v]}
                nxt.append((v, sv, path + [v]))
        frontier = nxt
    return out
```

Đoạn mã 5.4. Lan truyền cú sốc theo chiều rộng, giữ dấu của đường ngắn nhất



Hình 5.10. Lan truyền cú sốc "DXY tăng" trong `VietnamFinanceKG`: xanh là tăng, đỏ là giảm

Bảng 5.3. Chiều tác động lên các thực thể khi DXY tăng, theo đường ngắn nhất

Biến bị ảnh hưởng	Chiều khi DXY tăng
Giá vàng thế giới (XAU/USD)	giảm
Tỷ giá USD/VND	tăng
Giá vàng thế giới quy đổi (VND/lượng)	giảm
Khối ngoại mua ròng	giảm
FPT (Tập đoàn FPT)	tăng
HPG (Hòa Phát)	giảm
VIC (Vingroup)	giảm
Vàng miếng SJC	giảm
Vàng nhẫn 9999	giảm
Chỉ số VN30	giảm

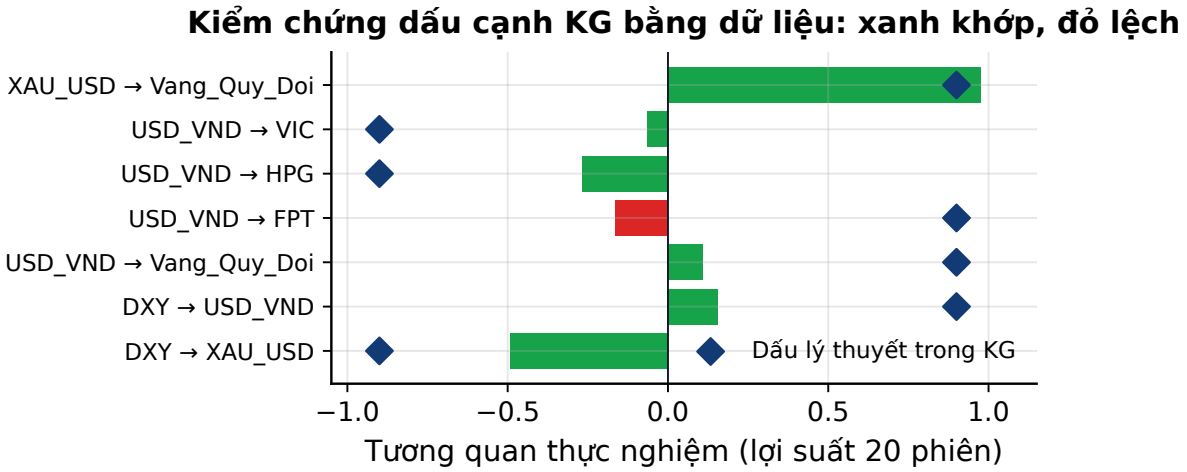
5.3.3 Kiểm chứng dấu cạnh bằng dữ liệu

Một KG tài chính viết tay dễ chứa những "hiểu biết thông thường" sai. Bài kiểm chứng mọi cạnh nhân quả có chuỗi giá ở cả hai đầu: tính tương quan Pearson giữa lợi suất 20 phiên của biến nguồn và biến đích trên toàn giai đoạn, rồi so dấu với dấu lý thuyết. Kết quả (Bảng 5.4, Hình 5.11): 6 trên 7 cạnh khớp dấu. Cạnh XAU_USD → Vang_Quy_Doi gần như tất định (0,9755); cạnh DXY → XAU mạnh và đúng chiều; các cạnh tỷ giá → HPG, VIC yếu nhưng đúng dấu.

Bảng 5.4. Đối chiếu dấu lý thuyết của cạnh KG với tương quan thực nghiệm

Cạnh trong KG	Dấu lý thuyết	Tương quan 20 phiên	Đối chiếu
DXY → XAU_USD	−	−0,490	khớp
DXY → USD_VND	+	+0,154	khớp
USD_VND → Vang_Quy_Doi	+	+0,108	khớp
USD_VND → FPT	+	−0,164	không khớp
USD_VND → HPG	−	−0,267	khớp
USD_VND → VIC	−	−0,063	khớp
XAU_USD → Vang_Quy_Doi	+	+0,975	khớp

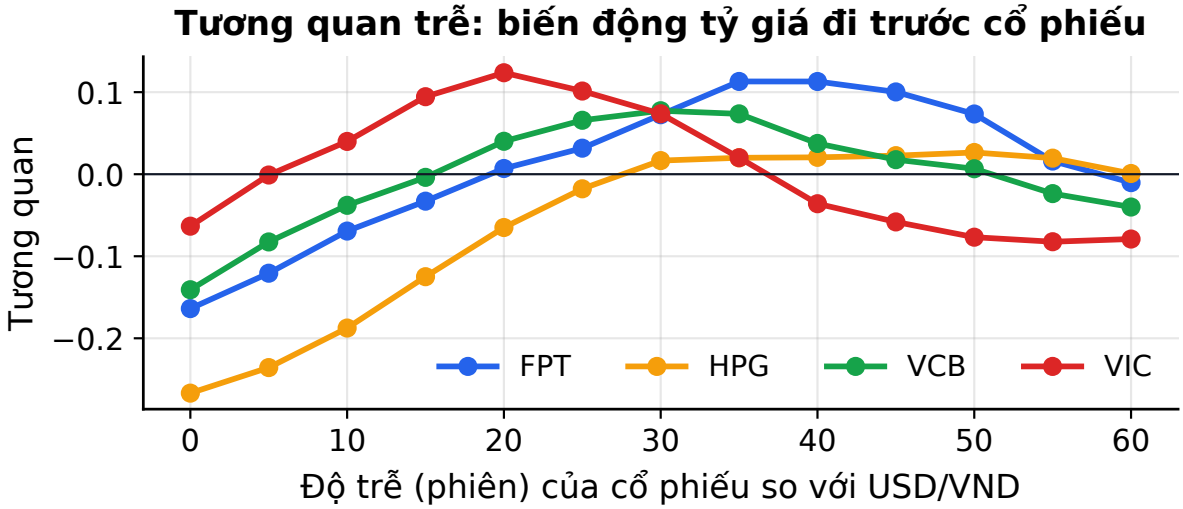
Tương quan Pearson của lợi suất cộng dồn 20 phiên trên toàn bộ giai đoạn.



Hình 5.11. Tương quan thực nghiệm (thanh) và dấu lý thuyết trong KG (hình thoi) của bảy cạnh có chuỗi giá ở hai đầu

Cạnh không khớp là USD/VND → FPT: lý thuyết cho dấu dương, dữ liệu cho tương quan cùng kỳ −0,1638. Sai lệch này không có nghĩa cạnh sai, mà cho thấy hai cơ chế chạy ở hai thang thời gian khác nhau. Trong ngắn hạn, tỷ giá tăng nhanh đi cùng dòng vốn ngoại rút khỏi thị trường và tâm lý né rủi ro, kéo mọi cổ phiếu lớn giảm cùng lúc, kể cả FPT. Lợi ích cơ bản của doanh thu ngoại tệ chỉ hiện ra khi doanh nghiệp công bố kết quả kinh doanh, tức là vài tháng sau. Tương quan trễ (Hình 5.12) xác nhận cách giải thích này: tương quan giữa lợi suất 20 phiên của tỷ giá hôm nay và của FPT k phiên sau tăng dần theo k, đổi dấu quanh $k = 20$ và đạt cực đại 0,1130 ở độ trễ 35 phiên, trong khi với HPG,

đường tương quan chỉ tiến về 0 mà không đổi dấu đáng kể.



Hình 5.12. Tương quan giữa lợi suất 20 phiên của USD/VND và của cổ phiếu sau k phiên

Nhận xét: Cạnh KG mã hóa cơ chế cơ bản, còn tương quan cùng kỳ đo phản ứng của dòng tiền. Bài giữ nguyên cạnh USD/VND → FPT vì có căn cứ văn bản; đây là ví dụ cho thấy KG tài chính cần được đối chiếu với dữ liệu thay vì chỉ dựa vào hiểu biết thông thường, và là lý do Chương 7 đề xuất gắn tương quan thực nghiệm cùng độ trễ vào thuộc tính của từng cạnh. Một lưu ý về phương pháp: lợi suất 20 phiên tính trên cửa sổ chồng lấn nên các quan sát liền nhau phụ thuộc nhau; độ lớn tương quan dùng được để so chiều, nhưng không nên đọc như kiểm định ý nghĩa thống kê.

5.4 Dự báo chuỗi thời gian bằng BiLSTM

5.4.1 Phát biểu bài toán

Mô hình không dự báo trực tiếp giá mà dự báo *lợi suất log* phiên kế tiếp, $r_{t+1} = \ln(P_{t+1}/P_t)$, rồi suy ra giá $\hat{P}_{t+1} = P_t\, e^{\hat{r}_{t+1}}$. Chuỗi giá không dừng: một mô hình hồi quy trên giá đạt R^2 rất cao chỉ bằng cách chép giá hôm qua, và con số đó không nói gì về khả năng dự báo. Lợi suất gần dừng hơn, và cho phép so sánh với mốc khó nhất: *dự báo ngây thơ* "giá không đổi", tức $\hat{r}_{t+1} = 0$. Với mốc này,

$\text{RMSE}_{\text{ngây thơ}} = \sqrt{\frac{1}{n}\sum r_{t+1}^2}$, và chỉ số chính của bài là tỉ số $\text{RMSE}/\text{RMSE}_{\text{ngây thơ}}$: dưới 1 mới là có năng lực dự báo.

Mỗi phiên có 7 đặc trưng với cổ phiếu (6 với vàng): lợi suất log của chính tài sản; lợi suất của bốn biến ngoại sinh USD/VND, XAU, DXY, ETF VN30 (bỏ XAU khi dự báo vàng quy đổi vì XAU là thành phần của chính nó); độ lệch của giá so với trung bình trượt 20 phiên; và độ biến động 20 phiên. Mỗi mẫu là một cửa sổ 30 phiên liên tiếp, tức một tenxơ 30×7.

```python
# tl2/data.py
def forecast_features(df, target, keep_last=False):
    """Đặc trưng mỗi ngày: lợi suất log của đích và 4 biến ngoại sinh, độ lệch so với MA20, độ biến động 20 phiên.

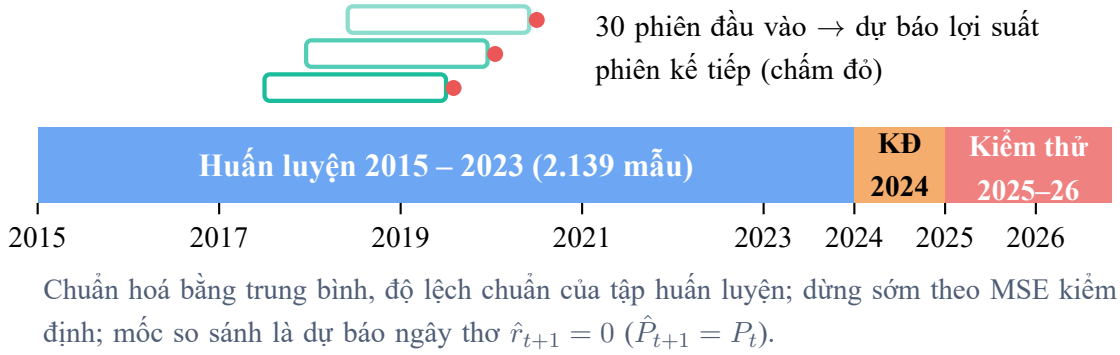
    keep_last=True giữ phiên mới nhất (chưa có nhãn y) cho suy luận; huấn luyện cần nhãn nên bỏ phiên đó.
    """
    r = np.log(df).diff()
    F = pd.DataFrame({"r_target": r[target]})
    for s in EXOG:
        if s != target and not (target == "GOLD_VND" and s == "XAU"):
            F[f"r_{s}"] = r[s]
    F["dev_ma20"] = df[target] / df[target].rolling(20).mean() - 1
    F["vol20"] = r[target].rolling(20).std()
    F["y"] = r[target].shift(-1)                                  # lợi suất phiên kế tiếp
    F["price"] = df[target]
    return F.dropna(subset=F.columns.drop("y")) if keep_last else F.dropna()
```

Đoạn mã 5.5. Đặc trưng dự báo: lợi suất của đích và biến ngoại sinh, độ lệch MA20, độ biến động 20 phiên; `keep_last` giữ phiên mới nhất khi suy luận

```python
# tl2/data.py
def make_windows(F, cols, window):
    X = F[cols].to_numpy(np.float32)
    y = F["y"].to_numpy(np.float32)
    idx = np.arange(window - 1, len(F))
    Xw = np.stack([X[i - window + 1:i + 1] for i in idx])
    return Xw, y[idx], F.index[idx]
```

Đoạn mã 5.6. Cắt chuỗi đặc trưng thành các cửa sổ trượt cho mô hình tuần tự

Dữ liệu được chia *theo thời gian*, không xáo trộn (Hình 5.13): huấn luyện trước 2024-01-01, kiểm định năm 2024, kiểm thử từ 2025-01-01 tới hết dữ liệu. Trung bình và độ lệch chuẩn để chuẩn hóa chỉ tính trên tập huấn luyện. Với FPT, ba tập có 2.139, 247 và 427 cửa sổ.



Chuẩn hoá bằng trung bình, độ lệch chuẩn của tập huấn luyện; dừng sớm theo MSE kiểm định; mốc so sánh là dự báo ngây thơ $\hat{r}_{t+1} = 0$ ($\hat{P}_{t+1} = P_t$).

Hình 5.13. Chia dữ liệu theo thời gian và cửa sổ trượt 30 phiên: mô hình không bao giờ nhìn thấy tương lai của mẫu nó dự báo

5.4.2 Kiến trúc LSTM hai chiều

LSTM [32] giải quyết bài toán gradient biến mất của RNN bằng một ô nhớ c_t và ba cổng điều khiển luồng thông tin. Với đầu vào x_t và trạng thái ẩn trước h_{t-1}:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \qquad \text{cổng quên} \tag{5.3}$$

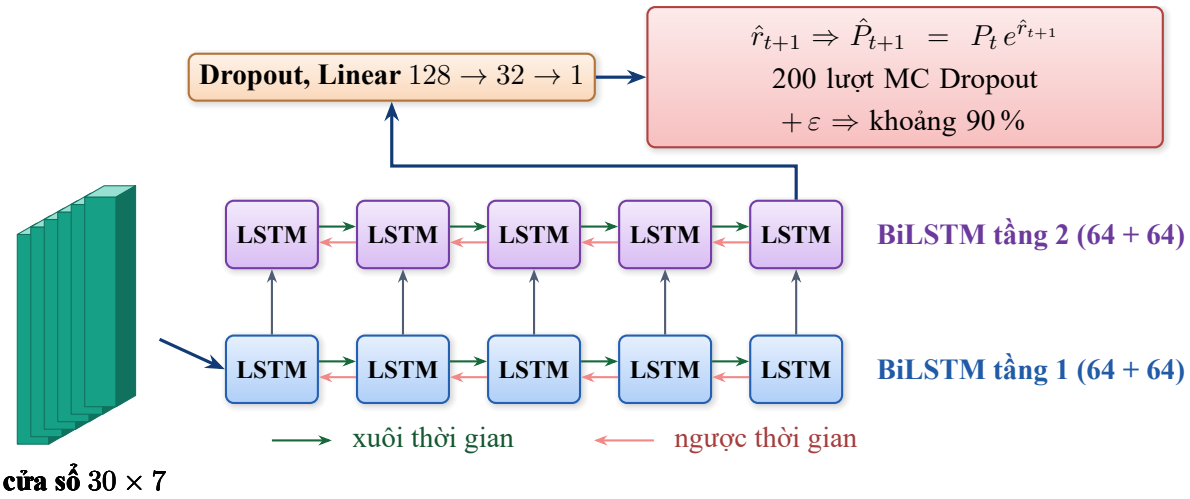
$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)\,, \quad \tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c) \qquad \text{cổng vào, ứng viên} \tag{5.4}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \qquad \text{cập nhật ô nhớ} \tag{5.5}$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)\,, \quad h_t = o_t \odot \tanh(c_t) \qquad \text{cổng ra} \tag{5.6}$$

LSTM hai chiều [33] chạy thêm một LSTM ngược thời gian trên *cùng cửa sổ* và ghép hai trạng thái, $h_t = [\overrightarrow{h}_t; \overleftarrow{h}_t]$. Vì cửa sổ chỉ gồm các phiên đã qua, chiều ngược đọc từ phiên t lùi về phiên $t-29$ và không làm rò rỉ tương lai; tác dụng của nó là để trạng thái cuối mang thông tin của cả đầu lẫn cuối cửa sổ. Mạng gồm hai tầng BiLSTM 64 chiều mỗi hướng, lấy trạng thái ở bước cuối (128 chiều) qua Dropout và hai lớp tuyến tính $128 \to 32 \to 1$; tổng cộng 140.865 tham số

với 7 đặc trưng, so với 54.081 tham số của LSTM một chiều cùng cấu hình.



Đặc trưng: lợi suất log mục tiêu, lợi suất USD/VND, XAU, DXY, VN30, độ lệch MA20, độ biến động 20 phiên.

Hình 5.14. Bộ dự báo BiLSTM: tenxơ cửa sổ 30×7, hai tầng LSTM xuôi và ngược thời gian, đầu tuyến tính cho lợi suất phiên kế tiếp và MC Dropout cho khoảng dự báo

```python
# tl2/dl_models.py
class SeqForecaster(nn.Module):
    """LSTM một chiều hoặc hai chiều; đầu ra là lợi suất log phiên t+1."""

    def __init__(self, d_in, hidden=64, layers=2, dropout=0.2, bidirectional=True):
        super().__init__()
        self.rnn = nn.LSTM(d_in, hidden, num_layers=layers, batch_first=True,
                           dropout=dropout, bidirectional=bidirectional)
        k = 2 if bidirectional else 1
        self.head = nn.Sequential(nn.Dropout(dropout), nn.Linear(k * hidden, 32), nn.ReLU(), nn.Linear(32, 1))

    def forward(self, x):
        out, _ = self.rnn(x)
        return self.head(out[:, -1]).squeeze(-1)
```

Đoạn mã 5.7. Mô hình tuần tự dùng chung cho BiLSTM và LSTM một chiều

Bảng 5.5. Cấu hình bộ dự báo

Siêu tham số	Giá trị
Cửa sổ quan sát (phiên)	30
Kích thước ẩn mỗi chiều	64
Số lớp LSTM	2
Dropout	0,2
Đặc trưng đầu vào (cổ phiếu / vàng)	7 / 6
Mẫu huấn luyện / kiểm định / kiểm thử (FPT)	2139 / 247 / 427
Bắt đầu kiểm định / kiểm thử	2024-01-01 / 2025-01-01
Số mẫu MC Dropout	200

5.4.3 Huấn luyện

Hàm mất mát là Huber [34] với $\delta = 1$ trên lợi suất đã chuẩn hóa:

$$\mathcal{L}_\delta(e) = \begin{cases} \frac{1}{2}e^2 & |e| \le \delta \\ \delta\left(|e| - \frac{1}{2}\delta\right) & |e| > \delta \end{cases} \tag{5.7}$$

Lợi suất có đuôi dày: vài phiên giảm sàn hay tăng trần có thể chi phối MSE, trong khi Huber chỉ phạt tuyến tính các sai số lớn. Tối ưu bằng AdamW [35] (tốc độ học 10^{-3}, suy giảm trọng số 10^{-4}), lô 128, cắt chuẩn gradient ở 1,0 để tránh bùng nổ gradient trong LSTM, tối đa 80 epoch và dừng sớm khi mất mát kiểm định không giảm sau 10 epoch. Huấn luyện chạy trên GPU, mỗi mô hình mất vài giây (Bảng 5.6). Đối chứng gồm LSTM một chiều cùng cấu hình, hồi quy Ridge trên 30 trễ của mọi đặc trưng, và dự báo ngây thơ.

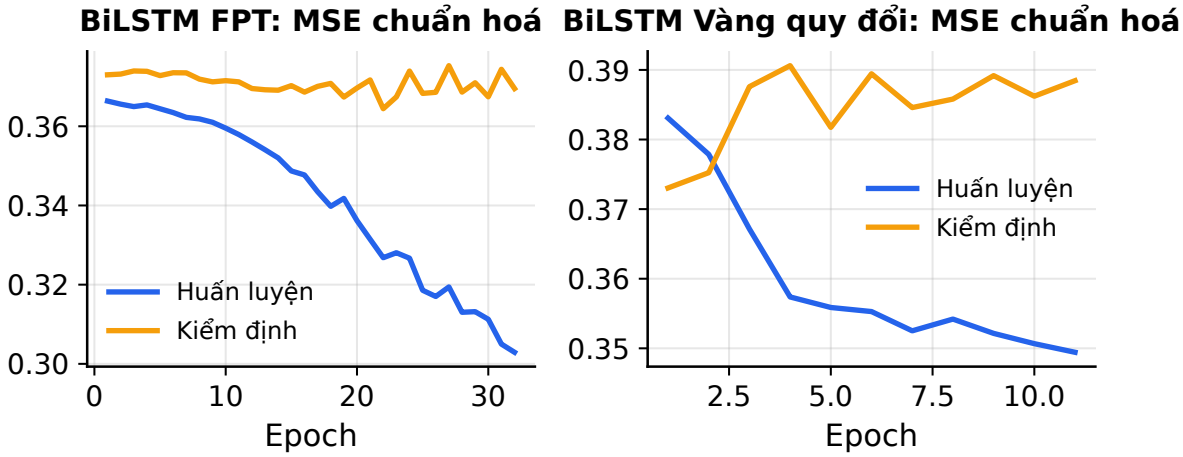
Đoạn mã 5.8. Vòng huấn luyện: Huber, AdamW, cắt gradient, dừng sớm theo mất mát kiểm định

```
# scripts/train_forecast.py
def train_rnn(Xtr, ytr, Xva, yva, bidirectional, dev):
    seed_all()
    m = SeqForecaster(Xtr.shape[2], HIDDEN, LAYERS, DROPOUT,
     ↪  bidirectional).to(dev)
    opt = torch.optim.AdamW(m.parameters(), lr=LR, weight_decay=1e-4)
```

```
    loss_fn = nn.HuberLoss(delta=1.0)
    T = lambda a: torch.tensor(a, dtype=torch.float32, device=dev)  # noqa: E731
    xtr, ytr_t, xva, yva_t = T(Xtr), T(ytr), T(Xva), T(yva)
    best, state, wait, hist = 1e9, None, 0, []
    for ep in range(1, EPOCHS + 1):
        m.train()
        perm = torch.randperm(len(xtr), device=dev)
        tot = 0.0
        for i in range(0, len(xtr), BATCH):
            b = perm[i:i + BATCH]
            opt.zero_grad()
            loss = loss_fn(m(xtr[b]), ytr_t[b])
            loss.backward()
            nn.utils.clip_grad_norm_(m.parameters(), 1.0)
            opt.step()
            tot += loss.item() * len(b)
        m.eval()
        with torch.no_grad():
            v = loss_fn(m(xva), yva_t).item()
        hist.append({"epoch": ep, "train": tot / len(xtr), "val": v})
        if v < best - 1e-6:
            best, wait = v, 0
            state = {k: x.detach().clone() for k, x in m.state_dict().items()}
        else:
            wait += 1
            if wait >= PATIENCE:
                break
    m.load_state_dict(state)
    m.eval()
    return m, hist
```

```python
# scripts/train_forecast.py
def evaluate(r_true, r_pred, p_prev):
    p_true, p_hat = p_prev * np.exp(r_true), p_prev * np.exp(r_pred)
    rmse = float(np.sqrt(np.mean((r_true - r_pred) ** 2)))
    rmse_naive = float(np.sqrt(np.mean(r_true ** 2)))
    nz = r_true != 0
    return {"rmse_ret": rmse, "rmse_naive": rmse_naive, "rmse_ratio": rmse /
    ↪  rmse_naive,
            "mae_price": float(np.mean(np.abs(p_true - p_hat))),
            ↪  "mae_price_naive": float(np.mean(np.abs(p_true - p_prev))),
            "dir_acc": float(np.mean(np.sign(r_pred[nz]) ==
            ↪  np.sign(r_true[nz]))),
            "r2_ret": float(1 - np.sum((r_true - r_pred) ** 2) /
            ↪  np.sum((r_true - r_true.mean()) ** 2))}
```

Đoạn mã 5.9. Đánh giá trên tập kiểm thử: RMSE lợi suất, tỉ số với dự báo ngây thơ, MAE giá, độ chính xác hướng



Hình 5.15. Đường cong học của BiLSTM cho FPT và vàng quy đổi

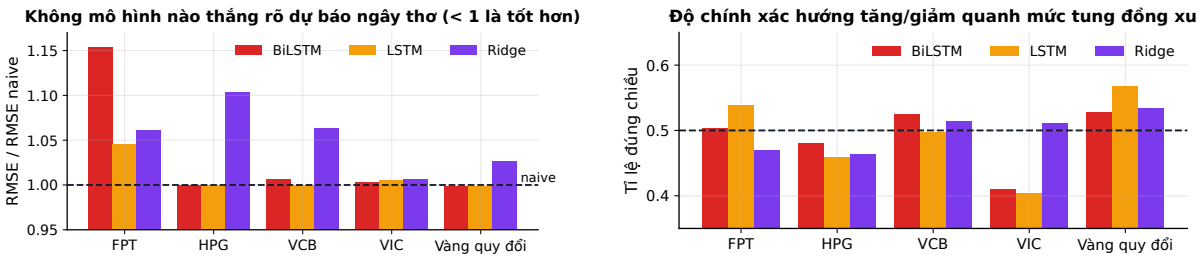
Đường cong học (Hình 5.15) cho thấy dấu hiệu đầu tiên của kết quả: mất mát huấn luyện giảm đều, còn mất mát kiểm định gần như đi ngang hoặc tăng ngay từ những epoch đầu. Với HPG, VIC và vàng quy đổi, trọng số tốt nhất là của epoch thứ nhất và quá trình dừng ở epoch 11; với FPT, mất mát kiểm định tốt nhất (epoch 22) chỉ thấp hơn epoch đầu khoảng 2%. Phần mô hình học thêm từ tập huấn luyện gần như không tổng quát hóa sang năm 2024.

5.4.4 Kết quả và diễn giải

Bảng 5.6. Kết quả dự báo trên tập kiểm thử của năm tài sản và bốn mô hình

Tài sản	Mô hình	RMSE ($\times 10^{-3}$)	RMSE / ngây thơ	MAE giá	Đúng chiều (%)	R^2 lợi suất	Epoch	Huấn luyện (s)
FPT	BiLSTM	23,44	1,1535	1.374	50,3	−0,3373	32	4,61
	LSTM	21,23	1,0448	1.244	53,9	−0,0971	29	1,48
	Ridge (30 trễ)	21,55	1,0605	1.304	46,9	−0,1304	–	–
	Giữ nguyên giá	20,32	**1,0000**	1.178	–	−0,0050	–	–
HPG	BiLSTM	18,26	1,0000	274	48,0	+0,0001	11	0,80
	LSTM	18,25	**0,9998**	273	45,8	+0,0004	11	0,54
	Ridge (30 trễ)	20,15	1,1038	325	46,3	−0,2184	–	–
	Giữ nguyên giá	18,26	1,0000	272	–	−0,0000	–	–
VCB	BiLSTM	26,06	1,0068	750	52,5	−0,0152	16	1,17
	LSTM	25,89	**1,0000**	726	49,7	−0,0016	14	0,69
	Ridge (30 trễ)	27,52	1,0630	911	51,4	−0,1318	–	–
	Giữ nguyên giá	25,89	1,0000	721	–	−0,0016	–	–
VIC	BiLSTM	32,37	1,0034	2.630	41,0	−0,0388	11	0,79
	LSTM	32,42	1,0048	2.634	40,4	−0,0417	11	0,54
	Ridge (30 trễ)	32,46	1,0062	2.708	51,1	−0,0446	–	–
	Giữ nguyên giá	32,26	**1,0000**	2.610	–	−0,0318	–	–
Vàng TG quy đổi	BiLSTM	15,96	0,9988	1,5	52,8	−0,0015	11	0,84
	LSTM	15,96	**0,9988**	1,5	56,7	−0,0014	11	0,61
	Ridge (30 trễ)	16,40	1,0266	1,5	53,4	−0,0580	–	–
	Giữ nguyên giá	15,98	1,0000	1,5	–	−0,0039	–	–

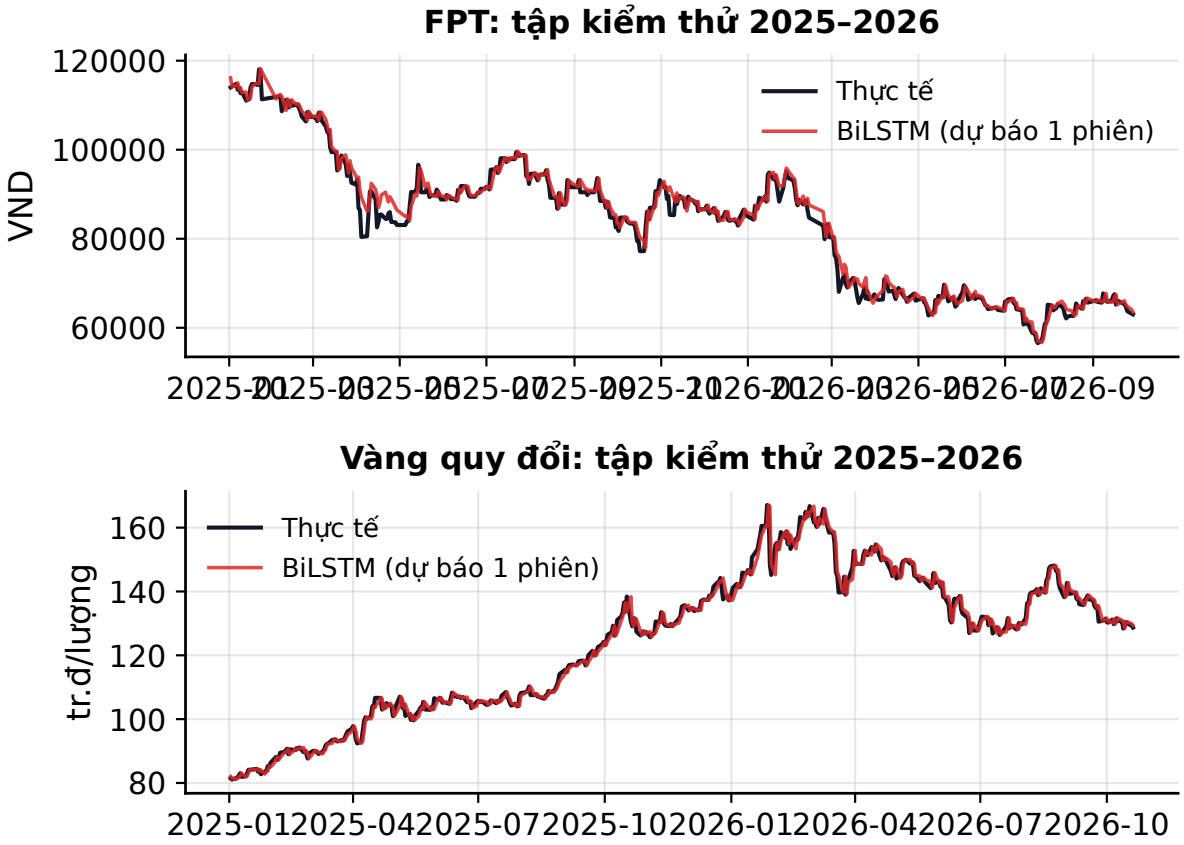
Kiểm thử từ 2025-01-01; RMSE trên lợi suất log phiên kế tiếp; giữ nguyên giá = dự báo lợi suất 0. MAE giá: đồng/cổ phiếu, vàng: triệu đồng/lượng. Chữ đậm: tỉ số nhỏ nhất của mỗi tài sản (dưới 1 mới là thắng mô hình ngây thơ).



(a) Tỉ số RMSE so với dự báo ngây thơ **(b)** Tỉ lệ đoán đúng chiều tăng/giảm

Hình 5.16. So sánh BiLSTM, LSTM và Ridge trên năm tài sản

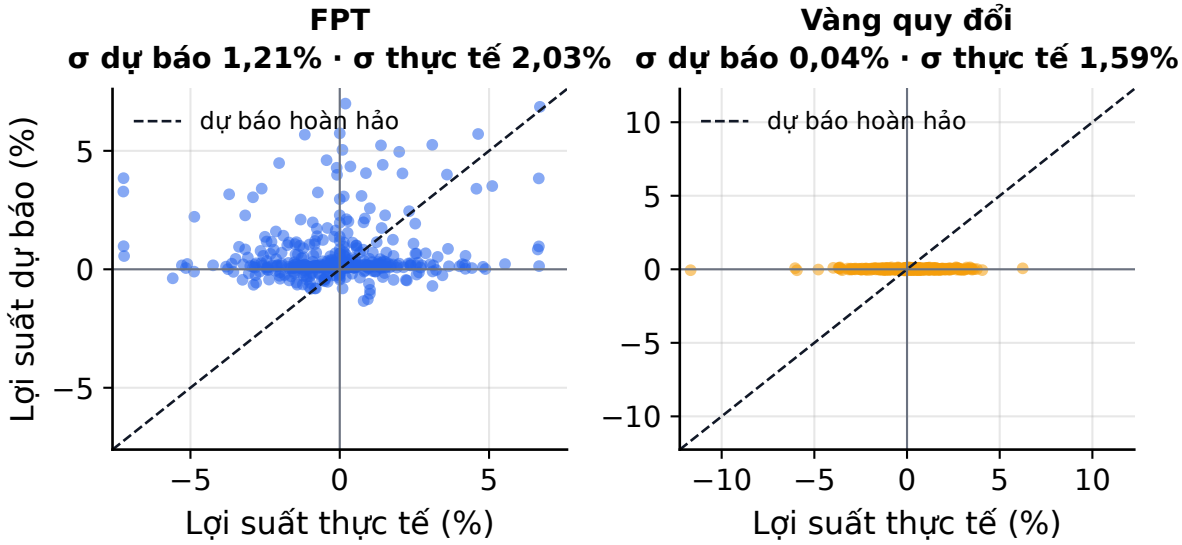
Kết quả cần được nói thẳng. Chỉ 5 trên 15 mô hình có tỉ số RMSE dưới 1, và mô hình tốt nhất (LSTM trên vàng quy đổi) chỉ giảm sai số 0,12% so với dự báo ngây thơ, một mức không có ý nghĩa thực tế. BiLSTM của FPT, mô hình phức tạp nhất trên tài sản biến động mạnh nhất trong nhóm, kém mốc ngây thơ tới tỉ số 1,154. Ridge trên 30 trễ kém dự báo ngây thơ ở mọi tài sản: với hơn 200 đặc trưng tuyến tính, mô hình khớp nhiễu của tập huấn luyện. Tỉ lệ đoán đúng chiều dao động quanh 50%, cao nhất là LSTM trên vàng (56,73%) và thấp nhất là hai mô hình LSTM trên VIC (khoảng 41%, tức tệ hơn tung đồng xu). Dự báo ngây thơ không có cột đúng chiều vì lợi suất 0 không có dấu.



Hình 5.17. Giá thực tế và giá dự báo một phiên của BiLSTM trên tập kiểm thử

Hình 5.17 dễ gây hiểu lầm nếu đọc thiếu cẩn thận: đường dự báo bám sát đường thực tế, nhưng đó là vì $\hat{P}_{t+1} = P_t e^{\hat{r}_{t+1}}$ và lợi suất một phiên chỉ vài phần trăm, nên bất kỳ dự báo nào gần giá hôm qua cũng trông khớp. Hình 5.18 đặt lợi suất dự báo cạnh lợi suất thực tế và cho thấy hai kiểu thất bại khác nhau. Với vàng quy đổi, BiLSTM *co hẳn về 0*: độ lệch chuẩn của dự báo chỉ 0,0382% so với 1,594% của thực tế, tức mô hình tự học ra dự báo ngây thơ, và tỉ số RMSE

vì vậy sát 1. Với FPT, BiLSTM làm điều ngược lại: dự báo dao động mạnh (độ lệch chuẩn 1,212% so với 2,027%) và lệch dương trung bình 0,5497% mỗi phiên, nhưng các dao động đó không trùng chiều với thực tế, nên mỗi lần đoán mạnh là một lần cộng thêm sai số; đó là lý do tỉ số RMSE của nó tệ nhất bảng. Cả hai kiểu đều dẫn tới cùng kết luận: lợi suất ngày mai gần như không dự đoán được từ thông tin công khai của 30 phiên trước, và mô hình nào "dám" dự báo xa khỏi 0 thì bị phạt.



Hình 5.18. Lợi suất dự báo và lợi suất thực tế trên tập kiểm thử: FPT dự báo mạnh nhưng sai chiều, vàng quy đổi co về 0

Kết quả này phù hợp với giả thuyết thị trường hiệu quả dạng yếu [36]: nếu giá đã phản ánh mọi thông tin trong lịch sử giá, không mô hình nào chỉ dùng lịch sử giá có thể dự báo lợi suất tốt hơn bước ngẫu nhiên. Thị trường Việt Nam được xem là kém hiệu quả hơn thị trường phát triển, nhưng ở tầm một phiên và với các cổ phiếu thanh khoản cao nhất VN30, phần dự báo được, nếu có, nhỏ hơn nhiều. Bài không coi đây là thất bại cần che đi, mà là một kết quả thực nghiệm quyết định cách tác tử dùng mô hình.

5.4.5 Khoảng dự báo cho tác tử

Công cụ `du_bao_bilstm` của tác tử không chỉ trả một con số. Lớp `Forecaster` lấy cửa sổ 30 phiên mới nhất và chạy 200 lượt với Dropout bật, mỗi lượt là một mẫu từ phân phối hậu nghiệm xấp xỉ của trọng số theo MC Dropout [37]. Tuy

nhiên MC Dropout chỉ đo *bất định của mô hình*; với chuỗi lợi suất, phần này rất nhỏ so với nhiễu vốn có của giá. Phiên bản đầu của công cụ dùng riêng MC Dropout và báo "xác suất tăng 100%" với khoảng 90% rộng chưa tới $\pm 0{,}2\%$ quanh giá hiện tại, một kết luận tự tin sai hoàn toàn. Bản cuối cộng vào mỗi mẫu một nhiễu phần dư có độ lệch chuẩn bằng RMSE trên tập kiểm thử:

$$r^{(s)} = f_{\theta^{(s)}}(\mathbf{x}_{t-29:t}) + \varepsilon^{(s)}, \quad \varepsilon^{(s)} \sim \mathcal{N}\left(0, \mathrm{RMSE}^2_{\text{kiểm thử}}\right), \quad s = 1, \ldots, 200, \tag{5.8}$$

rồi lấy phân vị 5% và 95% của $P_t e^{r^{(s)}}$ làm khoảng dự báo 90% và tỉ lệ $r^{(s)} > 0$ làm xác suất tăng. Bảng 5.7 đặt hai nguồn bất định cạnh nhau: với FPT, độ lệch chuẩn của MC Dropout chỉ 0,1007% trong khi RMSE kiểm thử là 2,344%, tức nhiễu của dữ liệu lớn hơn bất định của mô hình hơn hai mươi lần. Xác suất tăng sau khi sửa nằm trong khoảng 50 – 60%, đúng với mức hiểu biết thật của mô hình.

Bảng 5.7. Dự báo phiên tới của năm tài sản: bất định mô hình so với sai số dự báo thực tế

Tài sản	Phiên gần nhất	Giá gần nhất	Lợi suất dự báo (%)	σ MC Dropout (%)	RMSE kiểm thử (%)	Khoảng dự báo 90%	P(tăng) (%)
FPT	2026-09-30	63.000	+0,50	0,101	2,34	61.104 – 65.663	59,5
HPG	2026-09-30	20.150	−0,01	0,015	1,83	19.616 – 20.733	51,0
VCB	2026-09-30	58.000	−0,09	0,020	2,61	55.764 – 60.356	50,5
VIC	2026-09-30	227.900	−0,08	0,015	3,24	217.116 – 239.598	51,0
Vàng TG quy đổi	2026-10-21	128,7	+0,06	0,009	1,60	125,8 – 132,0	52,5

Mỗi tài sản 200 mẫu MC Dropout, mỗi mẫu cộng nhiễu $\mathcal{N}(0, \mathrm{RMSE}_{\text{kiểm thử}})$; giá: đồng/cổ phiếu, vàng: triệu đồng/lượng.

Văn bản công cụ trả cho tác tử luôn đi kèm hai chỉ số kiểm thử: tỉ lệ đoán đúng chiều và tỉ số RMSE so với dự báo ngây thơ, cùng câu "không vượt dự báo giữ nguyên giá" khi tỉ số không dưới 1. Như vậy câu trả lời cuối cùng của tác tử tự mang theo mức độ tin cậy của chính nó.

5.5 Tình huống minh họa: tác tử trả lời nhà đầu tư

```python
# tl2/dl_models.py
def predict(self, n_samples=200):
    df = gold_frame() if self.target == "GOLD_VND" else market_frame()
    F = forecast_features(df, self.target, keep_last=True)
    cols, w = self.meta["cols"], self.meta["window"]
    x = (F[cols].to_numpy(np.float32)[-w:] - self.meta["mean"]) /
    ↪  self.meta["std"]
    xt = torch.tensor(x[None], dtype=torch.float32,
    ↪  device=self.dev).repeat(n_samples, 1, 1)
    self.model.train()                                    # bật dropout:
    ↪  mỗi lượt là một mẫu bất định mô hình
    torch.manual_seed(0)                                  # cùng đầu vào
    ↪  cho cùng khoảng dự báo ở app, tác tử, báo cáo
    with torch.no_grad():
        r_mc = self.model(xt).cpu().numpy() * self.meta["y_std"]
    self.model.eval()
    r = r_mc + np.random.default_rng(0).normal(0.0,
    ↪  self.meta["test"]["rmse_ret"], n_samples)
    p0 = float(F["price"].iloc[-1])
    prices = p0 * np.exp(r)
    return {"model": "BiLSTM (PyTorch)", "target": self.target, "last_date":
    ↪  F.index[-1].strftime("%Y-%m-%d"),
            "last_price": p0, "pred_price": float(p0 *
            ↪  np.exp(np.mean(r_mc))),
            "pred_return_pct": float(100 * np.mean(r_mc)), "mc_std_pct":
            ↪  float(100 * np.std(r_mc)),
            "interval90": [float(np.percentile(prices, 5)),
            ↪  float(np.percentile(prices, 95))],
            "prob_up": float(np.mean(r > 0)), "test_dir_acc":
            ↪  self.meta["test"]["dir_acc"],
            "test_rmse_vs_naive": self.meta["test"]["rmse_ratio"]}
```

Đoạn mã 5.10. Công cụ dự báo của tác tử: MC Dropout cộng nhiễu phần dư cho khoảng dự báo 90% và xác suất tăng

5.5.1 Câu hỏi tác động vĩ mô

Câu hỏi "Tỷ giá USD/VND tăng mạnh thì cổ phiếu FPT và giá vàng SJC ảnh hưởng thế nào?" cần cả ba lớp. Liên kết thực thể nhận ra USD_VND, FPT và Vang_SJC; bộ lập kế hoạch gọi `kg_tai_chinh` để suy luận hai cặp tác động, `du_lieu_thi_truong` để lấy giá FPT gần nhất, `quy_doi_vang` để tính giá quy đổi theo (5.1), và `truy_xuat_rag` để lấy đoạn văn bản giải thích cơ chế. Khung dưới đây là vết suy luận thật của tác tử, do `build.py` sinh từ `agent.json`.

Vết suy luận ReAct thật cho câu hỏi tài chính (kịch bản 9)

CÂU HỎI *Tỷ giá USD/VND tăng mạnh thì cổ phiếu FPT và giá vàng SJC ảnh hưởng thế nào?*

THOUGHT Miền finance; thực thể liên kết: Tỷ giá USD/VND, Vàng miếng SJC, FPT (Tập đoàn FPT). Kế hoạch: kg_tai_chinh → du_lieu_thi_truong → quy_doi_vang → truy_xuat_rag.

THOUGHT Câu hỏi về tác động của biến số vĩ mô; cần suy luận đa bước trên đồ thị nhân quả.

ACTION `kg_tai_chinh`(USD_VND, Vang_SJC, FPT) – loại KG, 0,7 ms

OBSERV. Tỷ giá USD/VND tăng → Vàng miếng SJC: tích cực (+) (1 đường). Kênh chính: Tỷ giá USD/VND → Giá vàng thế giới quy đổi (VND/lượng) → Vàng miếng SJC. Cơ chế: XAU/USD × USD/VND × 1,2057; giá trong nước = quy đổi + phần bù. | Tỷ giá USD/VND tăng → FPT (Tập đoàn FPT): tích cực (+) (1 đường). Kênh chính: Tỷ giá USD/VND → FPT (Tập đoàn…

THOUGHT Lấy giá gần nhất của cổ phiếu được hỏi.

ACTION `du_lieu_thi_truong`(FPT) – loại CALC, 0,5 ms

OBSERV. FPT đóng cửa 63.000 đồng ngày 30/09/2026, thay đổi 20 phiên −2,0%.

THOUGHT Câu hỏi về mức giá vàng; tính giá thế giới quy đổi từ dữ liệu mới nhất.

ACTION `quy_doi_vang`( ) – loại CALC, 0,2 ms

OBSERV. Ngày 21/10/2026: XAU/USD = 4.125,4 USD/ounce, USD/VND = 25.872; giá thế giới quy đổi = 4.125,4 × 25.872 × 1,2057 ≈ 128,7 triệu đồng/lượng, chưa gồm thuế phí và phần bù vàng miếng trong nước.

THOUGHT Truy xuất văn bản gốc trong kho tri thức để trích dẫn căn cứ.

ACTION `truy_xuat_rag`(Tỷ giá USD/VND tăng mạnh thì cổ phiếu FPT và giá vàng SJC ảnh hưởng thế nào? Tỷ…) – loại RAG, 519,4 ms

OBSERV. Doanh thu xuất khẩu phần mềm của FPT được thanh toán chủ yếu bằng USD, JPY và EUR, trong khi phần lớn chi phí là lương kỹ sư tại Việt Nam trả bằng đồng. Vì vậy khi USD/VND tăng, doanh thu quy đổi tăng nhanh hơn chi phí và biên lợi nhuận được cải thiện. FPT được

xem là doanh nghiệp hưởng lợi khi tỷ giá tăng. Ngược lại, đồng yên…

VERIFY trích dẫn hợp lệ: 0; trung thực số học: 1,00; dùng **bản theo khuôn**; tổng thời gian 8,50 s

Câu trả lời cuối của tác tử (bản tổng hợp theo khuôn, có trích dẫn [n])

Tổng hợp từ các công cụ:
- Tỷ giá USD/VND tăng → Vàng miếng SJC: tích cực (+) (1 đường). Kênh chính: Tỷ giá USD/VND → Giá vàng thế giới quy đổi (VND/lượng) → Vàng miếng SJC. Cơ chế: XAU/USD × USD/VND × 1,2057; giá trong nước = quy đổi + phần bù. [1]
- Tỷ giá USD/VND tăng → FPT (Tập đoàn FPT): tích cực (+) (1 đường). Kênh chính: Tỷ giá USD/VND → FPT (Tập đoàn FPT). Cơ chế: doanh thu xuất khẩu phần mềm bằng ngoại tệ, chi phí bằng VND. [2]
- FPT đóng cửa 63.000 đồng ngày 30/09/2026, thay đổi 20 phiên −2,0%. [3]
- Ngày 21/10/2026: XAU/USD = 4.125,4 USD/ounce, USD/VND = 25.872; giá thế giới quy đổi = 4.125,4 × 25.872 × 1,2057 ≈ 128,7 triệu đồng/lượng, chưa gồm thuế phí và phần bù vàng miếng trong nước. [4]
- Doanh thu xuất khẩu phần mềm của FPT được thanh toán chủ yếu bằng USD, JPY và EUR, trong khi phần lớn chi phí là lương kỹ sư tại Việt Nam trả bằng đồng. Vì vậy khi USD/VND tăng, doanh thu quy đổi tăng nhanh hơn chi phí và biên lợi nhuận được cải thiện. FPT được xem là doanh nghiệp hưởng lợi khi…
- Khi USD/VND tăng, doanh nghiệp có doanh thu bằng ngoại tệ và chi phí bằng đồng Việt Nam được hưởng lợi, ví dụ doanh nghiệp xuất khẩu phần mềm, thủy sản, dệt may. Ngược lại doanh nghiệp nhập khẩu nguyên liệu bằng USD hoặc có nợ vay ngoại tệ lớn bị bất lợi do giá vốn tăng và phát sinh lỗ chênh lệch…
- Tỷ giá USD/VND chịu áp lực tăng khi chỉ số DXY tăng, khi chênh lệch lãi suất giữa USD và VND mở rộng theo hướng có lợi cho USD, khi nhập khẩu tăng nhanh hơn xuất khẩu, hoặc khi dòng vốn ngoại rút ra. Ngược lại, thặng dư thương mại, kiều hối và vốn FDI giải ngân giúp ổn định tỷ giá. Việt Nam là…

Thông tin chỉ mang tính tham khảo, không phải khuyến nghị đầu tư.

Bản nháp của Qwen2.5-1.5B cho câu hỏi này không sai số liệu nào (trung thực số học 1,00) nhưng không ghi số hiệu bằng chứng [n] nào, nên bộ kiểm chứng từ chối và trả bản tổng hợp theo khuôn; Mục 6.4 phân tích hiện tượng này trên toàn bộ kịch bản. Bản cuối đặt cạnh nhau hai loại bằng chứng: kết luận suy luận từ KG (FPT và vàng SJC cùng chịu tác động cùng chiều của tỷ giá, mỗi kết luận kèm kênh truyền dẫn và cơ chế) và số liệu tính từ dữ liệu (giá FPT, giá vàng

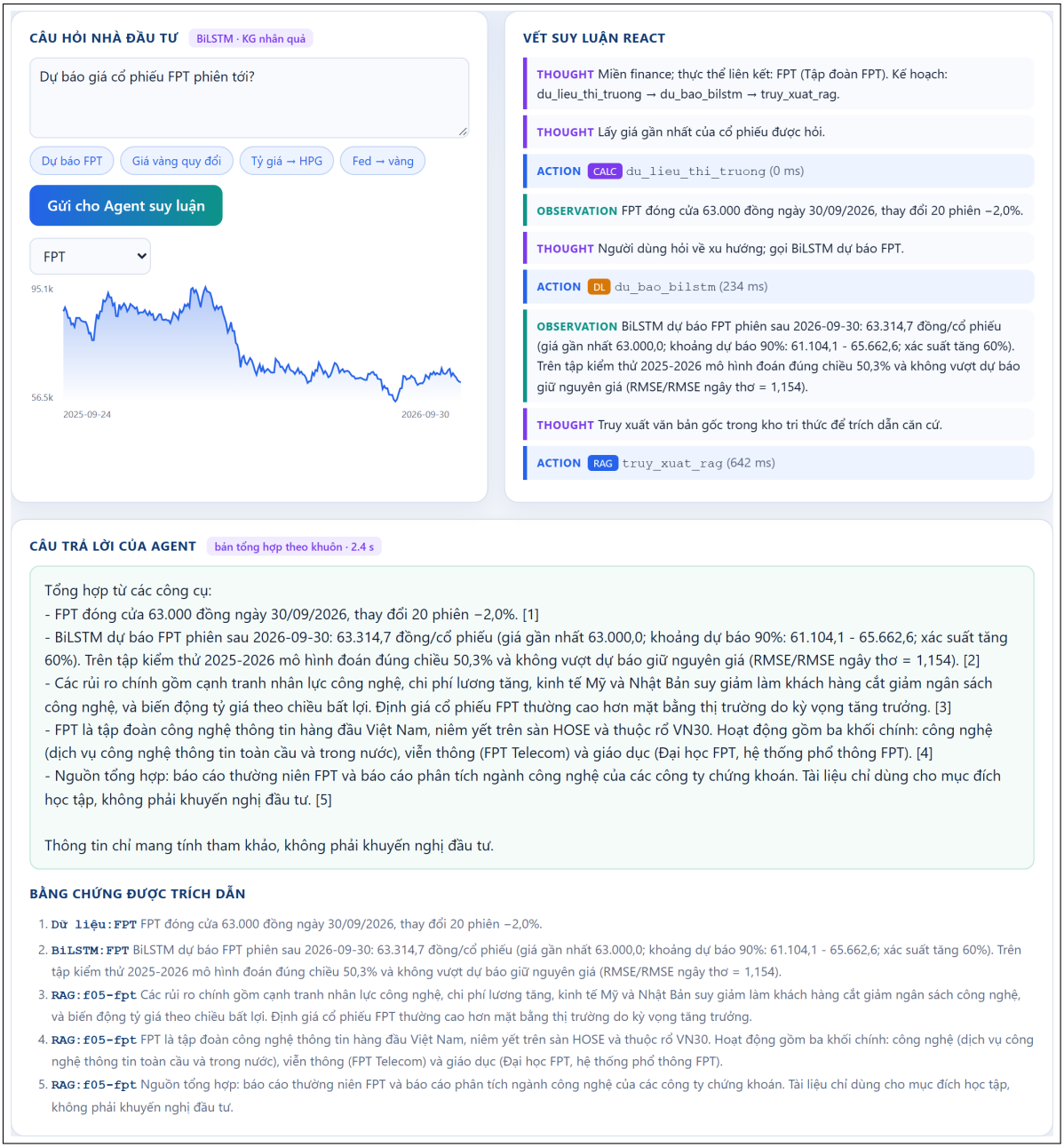
quy đổi), mỗi dòng mang số hiệu [n] trỏ về bằng chứng gốc. Hình 5.19 là cùng câu hỏi trên Web App.



Hình 5.19. Trợ lý tài chính trên Web App: câu hỏi tác động tỷ giá, vết suy luận và câu trả lời có trích dẫn

5.5.2 Câu hỏi dự báo

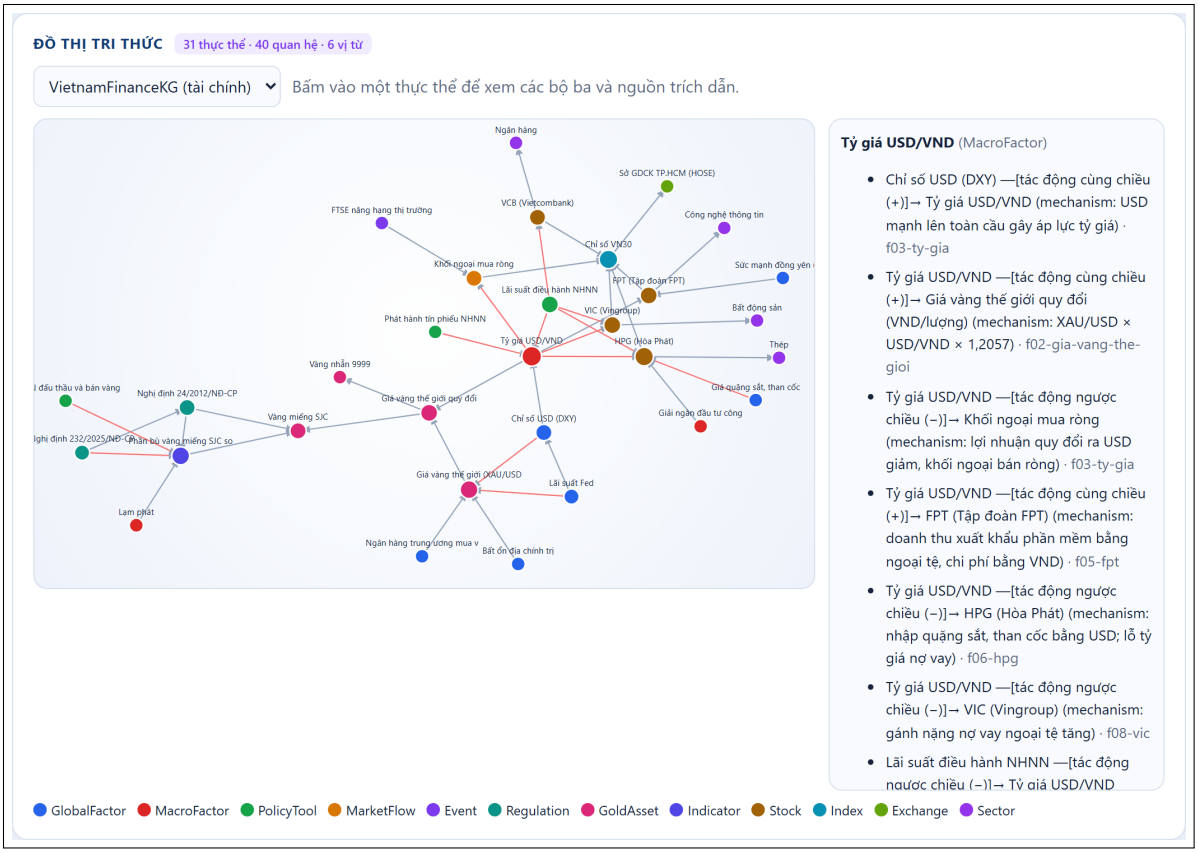
Với "Dự báo giá cổ phiếu FPT phiên tới?", bộ lập kế hoạch nhận ra ý định dự báo và gọi `du_bao_bilstm` sau `du_lieu_thi_truong` (Hình 5.20). Câu trả lời nêu giá dự báo, khoảng dự báo 90%, xác suất tăng, và ngay sau đó là hai chỉ số kiểm thử cho thấy mô hình không vượt dự báo ngây thơ. Một nhà đầu tư đọc câu trả lời này biết chính xác nên tin con số tới đâu, điều mà một câu trả lời chỉ có "giá dự kiến 63.315 đồng" không làm được.



Hình 5.20. Câu hỏi dự báo FPT: tác tử gọi BiLSTM và trả kèm khoảng dự báo, xác suất tăng và độ tin cậy trên tập kiểm thử

5.5.3 Khám phá đồ thị tri thức trên giao diện

Trang "Đồ thị tri thức" của Web App vẽ `VietnamFinanceKG` bằng SVG thuần, tô màu nút theo lớp, cạnh ngược chiều màu đỏ (Hình 5.21). Người dùng nhấn vào một nút để xem lớp của nó và mọi cạnh vào ra kèm tài liệu nguồn; đây là cách nhanh nhất để kiểm tra vì sao tác tử kết luận một chiều tác động.



Hình 5.21. Trang khám phá `VietnamFinanceKG` trên Web App

5.6 Tổng kết chương

- KG tài chính với 31 thực thể và 40 cạnh có dấu, có nguồn, đủ để suy luận đa bước và lan truyền cú sốc; khi các kênh mâu thuẫn, bộ suy luận trả "hỗn hợp" thay vì ép một chiều.
- 6 trên 7 cạnh kiểm chứng được khớp dấu với dữ liệu; cạnh tỷ giá → FPT chỉ khớp khi xét độ trễ khoảng 35 phiên.
- BiLSTM không thắng có ý nghĩa dự báo ngây thơ ở tầm một phiên, phù hợp giả thuyết thị trường hiệu quả dạng yếu; giá trị của nó với tác tử nằm ở

khoảng dự báo trung thực và các chỉ số tin cậy đi kèm, không ở con số điểm.

- Khoảng dự báo chỉ dựa trên MC Dropout tự tin sai; cộng nhiễu phần dư theo (5.8) đưa xác suất tăng về đúng mức 50 – 60%.

Chương 6

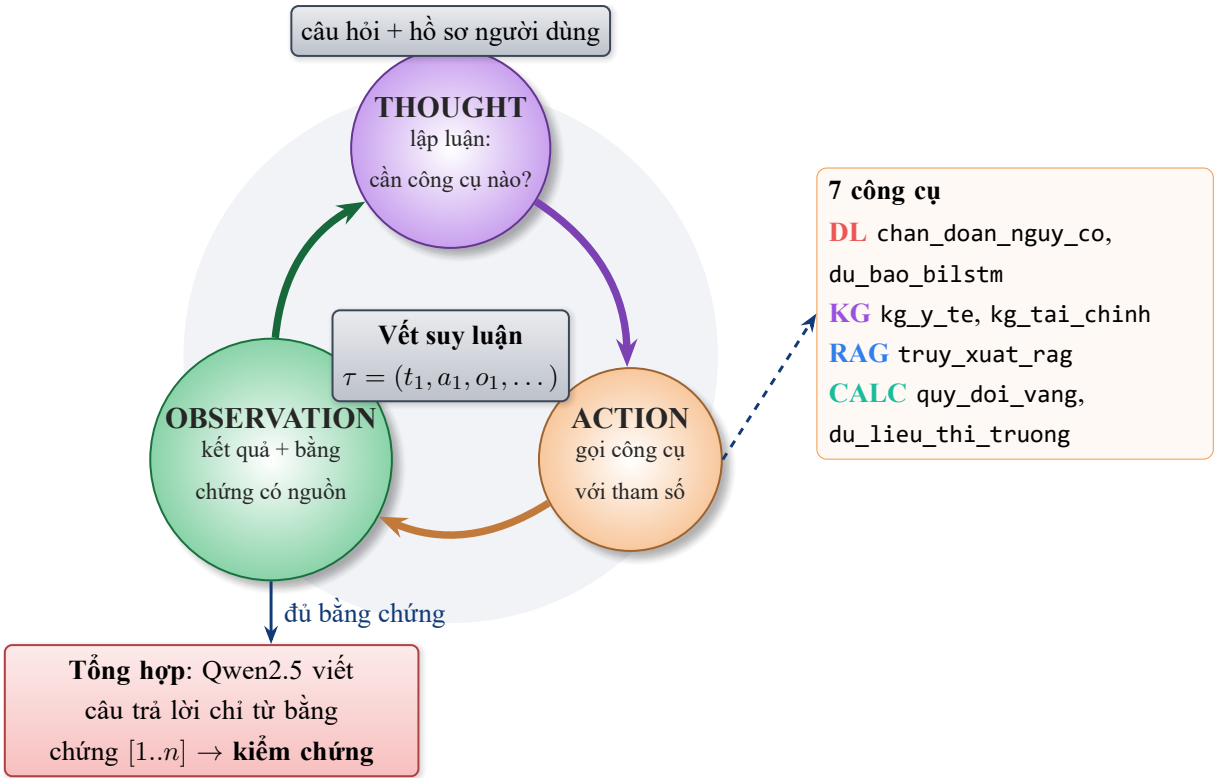
Triển khai hệ thống tác tử thông minh

Hai chương trước dựng các thành phần riêng lẻ: KG y tế và MLP nguy cơ, KG tài chính và BiLSTM, cùng kho văn bản và bộ truy xuất của Chương 3. Chương này ghép chúng thành một tác tử duy nhất nhận câu hỏi tiếng Việt tự do, tự quyết định miền và công cụ cần dùng, ghi lại toàn bộ vết suy luận, rồi viết câu trả lời có trích dẫn và tự kiểm chứng trước khi trả cho người dùng. Phần cuối chương đánh giá tác tử trên 16 kịch bản và so với chính LLM đó khi trả lời một mình.

6.1 Mô hình ReAct

6.1.1 Suy luận đan xen hành động

ReAct [6] kết hợp hai dòng nghiên cứu: suy luận từng bước bằng chuỗi ý nghĩ [38] và mô hình ngôn ngữ biết gọi công cụ [39]. Thay vì trả lời ngay, tác tử lặp ba bước: **Thought** (nói ra mình cần biết gì), **Action** (gọi một công cụ với tham số cụ thể) và **Observation** (đọc kết quả công cụ trả về). Vết Thought – Action – Observation vừa là bộ nhớ làm việc của tác tử, vừa là bản giải trình cho người dùng: mỗi kết luận cuối cùng đều truy được về một quan sát cụ thể.



Hình 6.1. Vòng lặp ReAct: Thought → Action → Observation lặp quanh bộ nhớ vết suy luận, kết thúc bằng bước tổng hợp và kiểm chứng

Đối với bài toán y tế và tài chính, ưu điểm quan trọng nhất của ReAct là *tách tri thức khỏi mô hình ngôn ngữ*. LLM không cần nhớ ngưỡng eGFR của metformin hay hệ số 1,2057; nó chỉ cần đọc các con số đó trong quan sát. Điều này trực tiếp giảm hiện tượng ảo giác [8], vốn nguy hiểm nhất đúng ở hai miền này, như Mục 6.4 sẽ cho thấy bằng ví dụ thật.

6.1.2 Bộ công cụ

Tác tử có 7 công cụ thuộc bốn loại (Bảng 6.1): học sâu (DL), đồ thị tri thức (KG), truy xuất văn bản (RAG) và tính toán xác định (CALC). Mỗi công cụ trả về hai thứ: một đối tượng quan sát có cấu trúc cho bộ lập kế hoạch và giao diện, và một danh sách bằng chứng dạng `(nguồn, văn bản)` để LLM trích dẫn. Nhãn nguồn ghi rõ loại và gốc của bằng chứng: `KG:f05-fpt`, `RAG:h03-ruou-bia`, `BiLSTM:FPT`, `Dữ liệu:FPT`.

Bảng 6.1. Bảy công cụ của tác tử, số lần được gọi và độ trễ trung vị trên 16 kịch bản

Công cụ	Loại	Chức năng	Lần gọi	Trễ TV (ms)
`chan_doan_nguy_co`	DL	MLP PyTorch ước lượng P(đái tháo đường) từ hồ sơ lâm sàng	2	10,0
`kg_y_te`	KG	Truy vấn VietnamDiabetesKG: tương tác thuốc, dinh dưỡng, chống chỉ định	8	0,1
`kg_tai_chinh`	KG	Đường nhân quả đa bước trên VietnamFinanceKG kèm dấu tác động	5	0,3
`du_bao_bilstm`	DL	BiLSTM dự báo giá phiên kế tiếp, khoảng dự báo MC Dropout + phần dư	2	75,0
`quy_doi_vang`	CALC	Giá vàng thế giới quy đổi VND/lượng từ XAU/USD và USD/VND	4	0,2
`du_lieu_thi_truong`	CALC	Giá đóng cửa gần nhất và biến động 20 phiên	4	0,3
`truy_xuat_rag`	RAG	Hybrid BM25 + e5 (RRF) rồi rerank bge-m3 trên kho tài liệu	16	590,0

Lần gọi và trung vị độ trễ đo trên 16 kịch bản đánh giá; DL: học sâu, KG: đồ thị tri thức, RAG: truy xuất văn bản, CALC: tính toán trên dữ liệu thị trường.

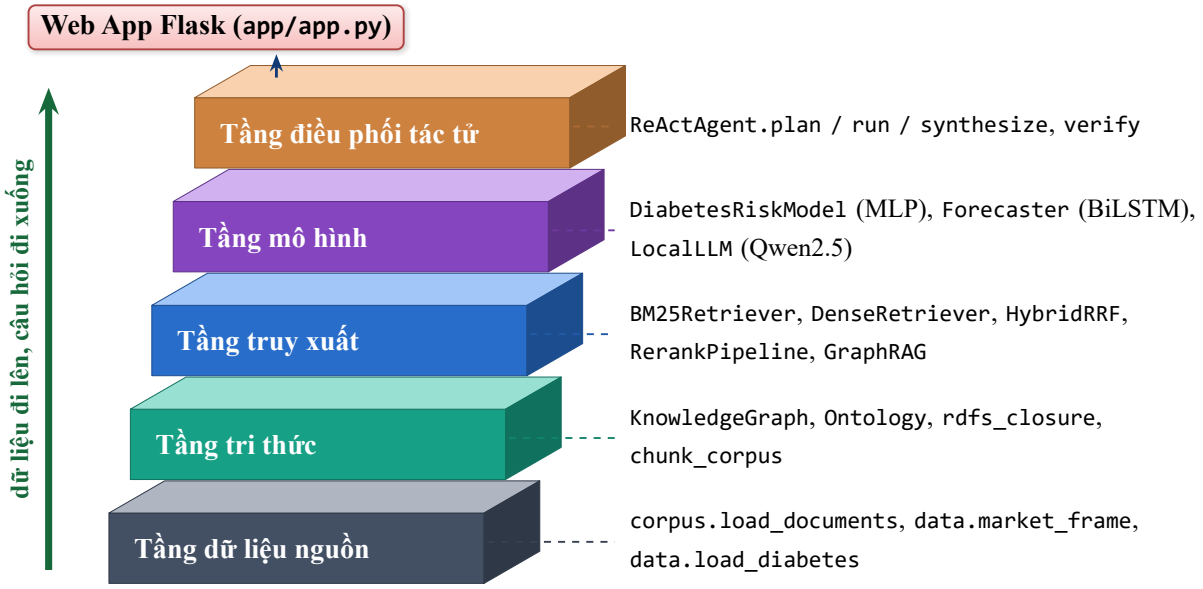
```python
# tl2/agent.py
@dataclass
class Tool:
    name: str
    kind: str          # DL | KG | RAG | CALC
    description: str
    fn: callable
```

Đoạn mã 6.1. Đặc tả một công cụ: tên, loại, mô tả và hàm thực thi

6.2 Kiến trúc hệ thống

6.2.1 Năm tầng

Hệ thống được tổ chức thành năm tầng (Hình 6.2). Tầng giao diện (Web App) chỉ gửi câu hỏi và hồ sơ qua HTTP. Tầng điều phối (`ReActAgent`) phân tích câu hỏi, lập kế hoạch và chạy vòng lặp. Tầng công cụ bọc các mô hình và đồ thị thành bảy hàm có cùng giao diện. Tầng tri thức gồm hai KG, kho văn bản đã chia đoạn và chỉ mục truy xuất. Tầng mô hình gồm MLP, BiLSTM, bộ nhúng e5, bộ xếp hạng lại bge và LLM Qwen2.5-1.5B [40], tất cả chạy cục bộ, không gọi dịch vụ ngoài.



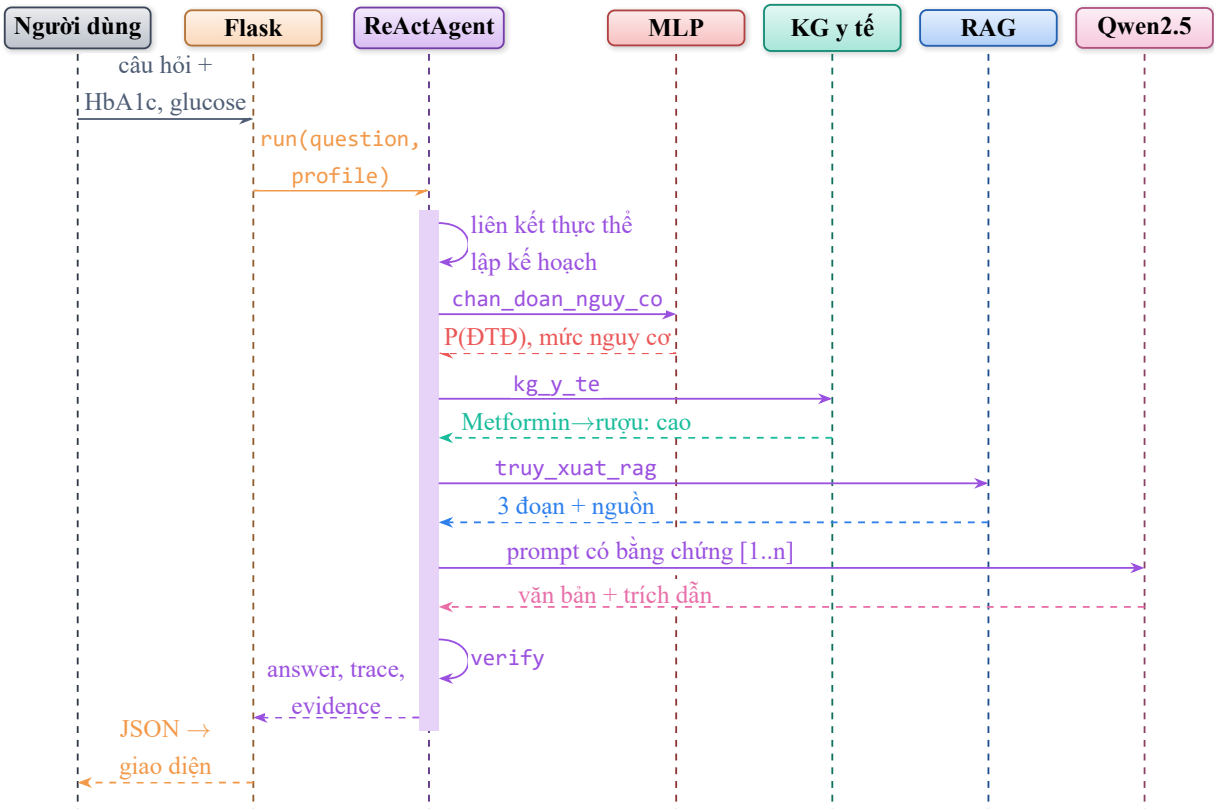
Hình 6.2. Kiến trúc năm tầng của tác tử và lớp, hàm hiện thực tương ứng trong gói `tl2`

6.2.2 Luồng xử lý một câu hỏi

Hình 6.3 là biểu đồ tuần tự của một câu hỏi y tế có kèm hồ sơ xét nghiệm. Năm bước chính:

1. **Liên kết thực thể**: `EntityLinker` (Chương 3) quét câu hỏi bằng bí danh của mọi thực thể trong hai KG, ưu tiên bí danh dài nhất và so khớp cả dạng bỏ dấu, nên "tieu duong" và "tiểu đường" đều được nhận.
2. **Nhận miền**: miền là KG có nhiều thực thể được liên kết hơn; câu hỏi không liên kết được thực thể nào mặc định là y tế.
3. **Lập kế hoạch**: chọn chuỗi công cụ theo loại thực thể và ý định (Mục 6.3.1).

4. **Thực thi**: gọi lần lượt từng công cụ, mỗi lần ghi một cặp Thought – Action và một Observation.
5. **Tổng hợp và kiểm chứng**: LLM viết câu trả lời từ bằng chứng; bộ kiểm chứng quyết định dùng bản của LLM hay bản tổng hợp theo khuôn (Mục 6.3.3).



Hình 6.3. Biểu đồ tuần tự cho một câu hỏi y tế có hồ sơ xét nghiệm: trình duyệt, Flask, tác tử, các công cụ, LLM và bộ kiểm chứng

6.3 Hiện thực tác tử

6.3.1 Bộ lập kế hoạch tất định

Một quyết định thiết kế quan trọng là *không* để LLM tự chọn công cụ. Với 1,5 tỉ tham số, Qwen2.5 sinh lời gọi công cụ không ổn định: sai tên, sai tham số hoặc bỏ qua công cụ cần thiết. Thay vào đó bộ lập kế hoạch dùng luật trên kết quả liên kết thực thể và từ khóa ý định (Đoạn mã 6.2):

- **Y tế**: có chỉ số xét nghiệm (HbA1c, glucose, BMI, trích từ câu hỏi bằng biểu thức chính quy hoặc gửi kèm từ giao diện) thì gọi `chan_doan_nguy_co`; có

thực thể y tế thì gọi `kg_y_te`.

- **Tài chính**: có biến vĩ mô (tỷ giá, lãi suất, nghị định) thì gọi `kg_tai_chinh`; có mã cổ phiếu thì gọi `du_lieu_thi_truong`; hỏi mức giá vàng thì gọi `quy_doi_vang`; có từ khóa xu hướng ("dự báo", "phiên tới", "nên mua") thì gọi `du_bao_bilstm`.
- **Luôn luôn**: kết thúc bằng `truy_xuat_rag` để có văn bản gốc làm căn cứ trích dẫn.

Đây là biến thể "lập kế hoạch rồi thực thi" của ReAct: chuỗi hành động được quyết định một lần từ đầu thay vì sau mỗi quan sát. Đổi lại tính linh hoạt, bộ lập kế hoạch đạt độ chính xác chọn công cụ gần tuyệt đối (Mục 6.4) và hoàn toàn tái lập được, điều cần thiết khi đánh giá.

Đoạn mã 6.2. Bộ lập kế hoạch: nhận miền từ thực thể liên kết, chọn chuỗi công cụ theo loại thực thể và ý định

```python
# tl2/agent.py
def plan(self, question, profile=None):
    ents = self.linker.link(question)
    kgs = [kg for _, kg, _ in ents]
    n_h, n_f = kgs.count(self.hkg.name), kgs.count(self.fkg.name)
    domain = "finance" if n_f > n_h else "health"
    profile = {**self.parse_profile(question), **(profile or {})}
    steps = []
    q = strip_accents(question)
    if domain == "health":
        if any(k in profile for k in ("HbA1c_level", "blood_glucose_level",
        ↪  "bmi")):
            steps.append(("chan_doan_nguy_co", profile,
                          "Người dùng cung cấp chỉ số xét nghiệm; cần mô
                          ↪  hình học sâu ước lượng nguy cơ bệnh."))
        if n_h:
            steps.append(("kg_y_te", ents, "Câu hỏi nhắc tới thuốc, chất
            ↪  hoặc món ăn; tra đồ thị tri thức để lấy "
                                           "ràng buộc tương tác và khuyến
                                           ↪  nghị dinh dưỡng có nguồn."))
    else:
        fin = [(e, self.fkg.G.nodes[e]["type"]) for e, kg, _ in ents if kg
        ↪  == self.fkg.name]
        if any(t in FACTOR_TYPES or self.fkg.onto.is_a(t, "MacroFactor") for
        ↪  _, t in fin):
            steps.append(("kg_tai_chinh", ents, "Câu hỏi về tác động của
            ↪  biến số vĩ mô; cần suy luận đa bước trên "
```

```
                                                "đồ thị nhân quả."))
        stocks = [e for e, t in fin if t == "Stock"]
        if stocks:
            steps.append(("du_lieu_thi_truong", stocks, "Lấy giá gần nhất
            ↪ của cổ phiếu được hỏi."))
        golds = [e for e, _ in fin if e in GOLD_NODES]
        if golds and re.search(r"quy doi|bao nhieu|gia vang|hom nay|hien
        ↪ nay", q):
            steps.append(("quy_doi_vang", None, "Câu hỏi về mức giá vàng;
            ↪ tính giá thế giới quy đổi từ dữ liệu mới nhất."))
        if re.search(r"du bao|du doan|xu huong|phien toi|tuan toi|sap
        ↪ toi|tang hay giam|nen mua", q):
            for e in stocks[:2] + (["GOLD_VND"] if golds else []):
                steps.append(("du_bao_bilstm", e, f"Người dùng hỏi về xu
                ↪ hướng; gọi BiLSTM dự báo {e}."))
    rag_q = question + " " + " ".join(self._label(e, kg) for e, kg, _ in
    ↪ ents[:4])
    steps.append(("truy_xuat_rag", rag_q, "Truy xuất văn bản gốc trong kho
    ↪ tri thức để trích dẫn căn cứ."))
    return domain, ents, profile, steps
```

```
# tl2/agent.py
@staticmethod
def parse_profile(text):
    t = text.lower().replace(",", ".")
    pats = {"HbA1c_level": r"hba1c\D{0,8}(\d+(?:\.\d+)?)", "bmi":
    ↪ r"bmi\D{0,8}(\d+(?:\.\d+)?)",
            "blood_glucose_level": r"(?:glucose|đường
            ↪ huyết)[^\d]{0,20}(\d{2,3}(?:\.\d+)?)",
            "age": r"(\d{2})\s*tuổi"}
    p = {k: float(m.group(1)) for k, rx in pats.items() if (m :=
    ↪ re.search(rx, t))}
    if re.search(r"\b(nữ|phụ nữ|bà)\b", t):
        p["gender"] = "Female"
    elif re.search(r"\b(nam|đàn ông|ông)\b", t):
        p["gender"] = "Male"
    if "huyết áp" in t:
        p["hypertension"] = 1
    if "hút thuốc" in t:
        p["smoking_history"] = "current"
    return p
```

Đoạn mã 6.3. Trích hồ sơ xét nghiệm từ câu hỏi tiếng Việt tự do

6.3.2 Vòng lặp và các công cụ

Hàm `run` thực thi kế hoạch, đo thời gian mỗi công cụ và ghi vết (Đoạn mã 6.4). Công cụ phức tạp nhất là `kg_tai_chinh`: nó tách thực thể thành *biến nguồn* (yếu tố vĩ mô, chính sách, sự kiện) và *biến đích* (cổ phiếu, vàng, chỉ số), gọi `impact_paths` cho mọi cặp và diễn đạt đường ngắn nhất thành một câu có kênh truyền dẫn và cơ chế (Đoạn mã 6.5). Khi câu hỏi chỉ nêu biến nguồn, công cụ xét mặc định bốn cổ phiếu và vàng SJC.

```python
# tl2/agent.py
def run(self, question, profile=None, synthesize=True):
    t_all = time.perf_counter()
    domain, ents, profile, plan = self.plan(question, profile)
    trace, evidence, obs_all = [], [], {}
    trace.append({"type": "THOUGHT", "text": f"Miền {domain}; thực thể liên kết: "
                  + (", ".join(self._label(e, kg) for e, kg, _ in ents) or "không có")
                  + f". Kế hoạch: {' → '.join(s[0] for s in plan)}."})
    for tool, arg, why in plan:
        trace.append({"type": "THOUGHT", "text": why})
        t0 = time.perf_counter()
        obs, ev = self.tools[tool].fn(arg)
        ms = 1000 * (time.perf_counter() - t0)
        obs_all[tool] = obs
        trace.append({"type": "ACTION", "tool": tool, "kind": self.tools[tool].kind, "ms": ms,
                      "args": arg if tool != "kg_y_te" and tool != "kg_tai_chinh" else [e for e, _, _ in arg]})
        trace.append({"type": "OBSERVATION", "tool": tool, "text": " | ".join(t for _, t in ev)[:600] or "rỗng"})
        evidence += ev
    result = {"question": question, "domain": domain, "entities": [(e, kg) for e, kg, _ in ents],
              "profile": profile, "trace": trace, "evidence": evidence, "observations": obs_all}
    if synthesize:
        result.update(self.synthesize(question, domain, evidence, obs_all))
    result["total_ms"] = 1000 * (time.perf_counter() - t_all)
    return result
```

Đoạn mã 6.4. Vòng lặp ReAct: mỗi bước một Thought, một Action đo thời gian và một Observation

```python
# tl2/agent.py
def _t_fkg(self, ents):
    ids = [e for e, kg, _ in ents if kg == self.fkg.name]
    types = {e: self.fkg.G.nodes[e]["type"] for e in ids}
    factors = [e for e in ids if self.fkg.onto.is_a(types[e], "MacroFactor")
    ↪  or types[e] in FACTOR_TYPES]
    targets = [e for e in ids if types[e] in TARGET_TYPES] or (["FPT", "HPG",
    ↪  "VCB", "VIC", "Vang_SJC"]
                                                            if factors
                                                            ↪  else [])
    obs, ev = [], []
    for a in factors:
        for b in targets:
            if a == b:
                continue
            r = impact_paths(self.fkg, a, b)
            if not r["paths"]:
                continue
            best = r["paths"][0]
            chain = " → ".join(self.fkg.label(n) for n in best["nodes"])
            why = "; ".join(s["mechanism"] for s in best["steps"])
            obs.append({"factor": a, "target": b, "verdict": r["verdict"],
            ↪  "n_paths": len(r["paths"]),
                        "chain": chain})
            src = best["steps"][-1]["source"]
            ev.append((f"KG:{src}", f"{self.fkg.label(a)} tăng →
            ↪  {self.fkg.label(b)}: {r['verdict']} "
                                    f"({len(r['paths'])} đường). Kênh chính:
                                    ↪  {chain}. Cơ chế: {why}."))
    return obs, ev
```

Đoạn mã 6.5. Công cụ `kg_tai_chinh`: suy luận đa bước cho mọi cặp nguồn – đích và diễn đạt thành bằng chứng có nguồn

6.3.3 Tổng hợp có căn cứ và bộ kiểm chứng

Bằng chứng từ mọi công cụ được đánh số $[1], \ldots, [n]$ và đưa vào lời nhắc của LLM cùng chỉ dẫn hệ thống: chỉ dùng thông tin trong bằng chứng, mỗi ý quan trọng phải kèm số hiệu, không bịa số liệu (Đoạn mã 6.6, 6.7). Đây là cùng lời nhắc đã dùng cho GraphRAG ở Chương 3, nên kết quả hai chương so sánh được với nhau. LLM sinh tham lam (greedy) để mọi lần chạy cho cùng một câu trả lời.

```
# tl2/llm.py
SYSTEM_GROUNDED = (
    "Bạn là trợ lý tiếng Việt của một hệ thống RAG. Chỉ được dùng thông tin trong phần BẰNG CHỨNG. "
    "Mỗi ý quan trọng phải kèm số hiệu bằng chứng trong ngoặc vuông, ví dụ [1]. "
    "Nếu bằng chứng không đủ để trả lời, nói rõ 'không đủ thông tin'. Không bịa số liệu. Trả lời ngắn gọn, mạch lạc."
)
```

Đoạn mã 6.6. Chỉ dẫn hệ thống cho LLM: chỉ dùng bằng chứng, trích dẫn [n], không bịa số liệu

```
# tl2/llm.py
def grounded_prompt(question, evidence):
    """evidence: list[(nhãn nguồn, văn bản)] -> prompt đánh số [1]..[n]."""
    lines = [f"[{i}] ({src}) {txt}" for i, (src, txt) in enumerate(evidence, 1)]
    return "BẰNG CHỨNG:\n" + "\n".join(lines) + f"\n\nCÂU HỎI: {question}\n\nTRẢ LỜI (tiếng Việt, có trích dẫn [n]):"
```

Đoạn mã 6.7. Lời nhắc có căn cứ: bằng chứng đánh số, câu hỏi, yêu cầu trích dẫn

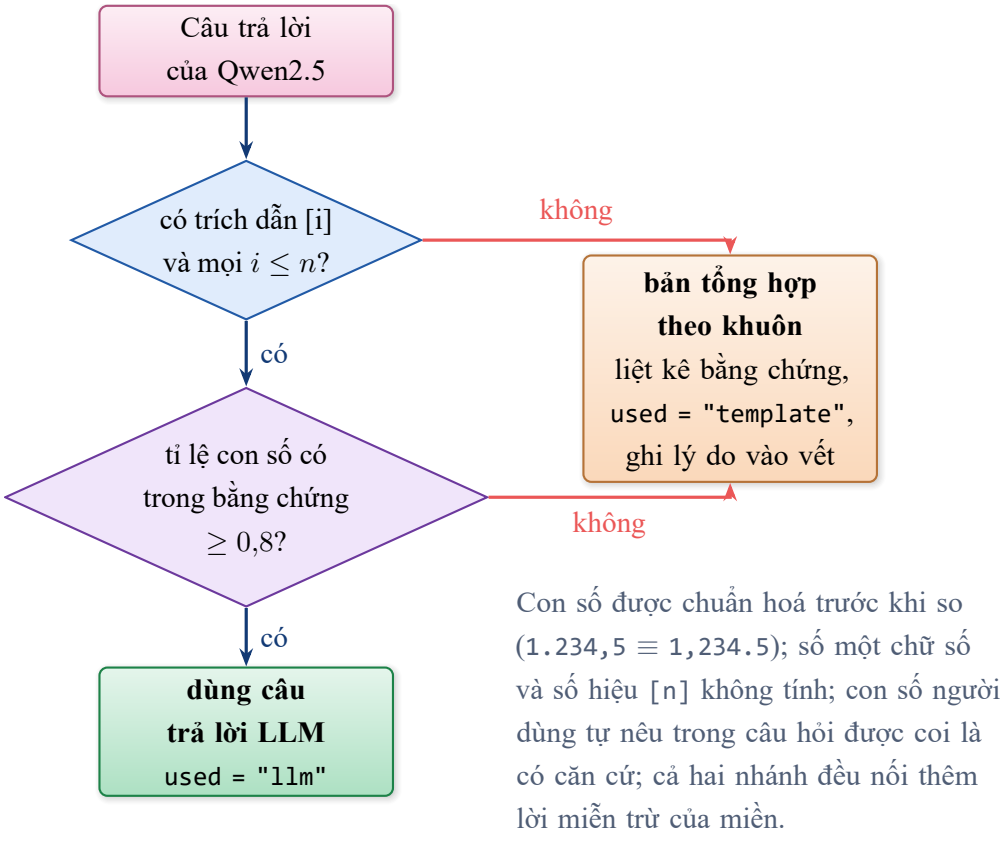
Mô hình nhỏ không phải lúc nào cũng tuân thủ chỉ dẫn, vì vậy câu trả lời đi qua bộ kiểm chứng trước khi tới người dùng (Hình 6.4). Bộ kiểm chứng kiểm ba điều kiện:

1. có ít nhất một trích dẫn $[n]$;
2. mọi trích dẫn trỏ tới bằng chứng có thật ($1 \leq n \leq$ số bằng chứng);

3. *độ trung thực số học* không dưới 0,8, với

$$\text{Faith} = \frac{\left|\{\text{con số trong câu trả lời có trong bằng chứng hoặc câu hỏi}\}\right|}{\left|\{\text{con số trong câu trả lời}\}\right|}. \tag{6.1}$$

Con số được chuẩn hóa trước khi so (bỏ dấu ngăn nghìn và dấu thập phân, nên 1.234,5 và 1,234.5 là một), số một chữ số và số hiệu trích dẫn không tính. Con số người dùng tự nêu (liều 500 mg, HbA1c 7,3) được coi là có căn cứ, vì nhắc lại dữ kiện của câu hỏi không phải bịa. Không đạt một trong ba điều kiện, tác tử trả bản tổng hợp theo khuôn: liệt kê nguyên văn từng bằng chứng kèm số hiệu. Bản khuôn kém tự nhiên nhưng đúng theo cấu tạo, và vết suy luận ghi rõ lý do bản LLM bị từ chối.



Hình 6.4. Bộ kiểm chứng: trích dẫn phải trỏ tới bằng chứng có thật và con số phải có trong bằng chứng; không đạt thì dùng bản tổng hợp theo khuôn

6.4 Đánh giá tác tử

```python
# tl2/agent.py
def synthesize(self, question, domain, evidence, obs):
    template = self.template_answer(domain, evidence)
    if self.llm is None:
        return {"answer": template + f"\n\n{DISCLAIMER[domain]}",
                "verification": {"used": "template", "reason": "không dùng LLM"}}
    text, n_tok, sec = self.llm.chat(grounded_prompt(question, evidence))
    ver = verify(text, evidence, question)
    ok = ver["citations_valid"] and ver["has_citation"] and ver["numeric_faithfulness"] >= 0.8
    ver.update(used="llm" if ok else "template", llm_tokens=n_tok, llm_s=sec, llm_text=text)
    if not ok:
        ver["reason"] = "câu trả lời LLM không qua kiểm chứng (trích dẫn hoặc con số không có trong bằng chứng)"
    answer = (text if ok else template) + f"\n\n{DISCLAIMER[domain]}"
    return {"answer": answer, "verification": ver}
```

Đoạn mã 6.8. Tổng hợp: LLM viết từ bằng chứng, bộ kiểm chứng chọn bản LLM hoặc bản khuôn

```python
# tl2/agent.py
def verify(answer, evidence, question=""):
    """Kiểm chứng: trích dẫn [n] hợp lệ; tỷ lệ con số trong câu trả lời có xuất hiện trong bằng chứng.

    Con số người dùng tự nêu trong câu hỏi (liều 500 mg, HbA1c 7,2) được coi là có căn cứ.
    """
    cites = [int(x) for x in re.findall(r"\[(\d+)\]", answer)]
    ev_text = " ".join(t for _, t in evidence) + " " + question
    ev_nums = {_norm_num(n) for n in NUM.findall(ev_text)}
    nums = [n for n in NUM.findall(re.sub(r"\[\d+\]", "", answer)) if len(_norm_num(n)) >= 2]
    found = [n for n in nums if _norm_num(n) in ev_nums]
    return {"has_citation": bool(cites), "citations_valid": all(1 <= c <= len(evidence) for c in cites),
            "n_citations": len(set(cites)), "numbers": len(nums),
            "numeric_faithfulness": 1.0 if not nums else len(found) / len(nums),
            "unsupported_numbers": [n for n in nums if n not in found]}
```

Đoạn mã 6.9. Bộ kiểm chứng: trích dẫn hợp lệ và độ trung thực số học (6.1)

6.4.1 Thiết kế thí nghiệm

Bộ đánh giá gồm 16 kịch bản, 8 y tế và 8 tài chính, bao phủ mọi công cụ và các loại câu hỏi đã phân tích ở hai chương trước. Mỗi kịch bản có ba thứ được viết trước khi chạy: câu hỏi, *tập công cụ kỳ vọng* và một danh sách *dữ kiện bắt buộc*, mỗi dữ kiện là một nhóm cụm từ đồng nghĩa (chẳng hạn {"tránh", "không nên", "không uống"}). Các chỉ số:

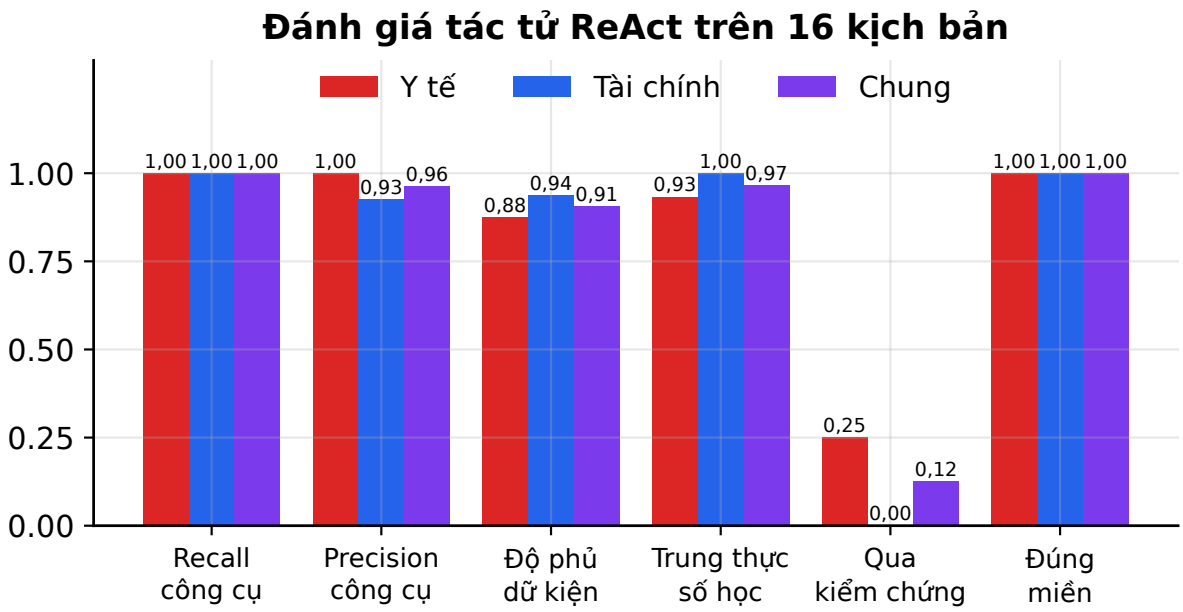
- **Precision và recall công cụ**: so tập công cụ đã gọi với tập kỳ vọng.
- **Độ phủ dữ kiện**: tỉ lệ nhóm dữ kiện bắt buộc có ít nhất một cụm từ xuất hiện trong câu trả lời cuối (so khớp không dấu), cùng hàm đã dùng cho GraphRAG.
- **Trung thực số học** (6.1) và **tỉ lệ bản LLM qua kiểm chứng**.
- **Đối chứng "chỉ LLM"**: cùng Qwen2.5-1.5B trả lời cùng câu hỏi, không có công cụ, với chỉ dẫn "trả lời ngắn gọn, chính xác".

6.4.2 Kết quả tổng hợp

Bảng 6.2. Kết quả tổng hợp của tác tử theo miền

Chỉ số	Y tế	Tài chính	Chung
Recall công cụ	1,000	1,000	1,000
Precision công cụ	1,000	0,927	0,964
Nhận đúng miền	1,000	1,000	1,000
Độ phủ dữ kiện (agent)	0,875	0,938	0,906
Độ phủ dữ kiện (chỉ LLM)	0,000	0,312	0,156
Trung thực số học bản nháp LLM (agent)	0,933	1,000	0,967
Trung thực số học (chỉ LLM)	0,958	0,896	0,927
Tỉ lệ bản LLM qua kiểm chứng	0,250	0,000	0,125
Tổng thời gian TB (s)	6,95	4,61	5,78
Thời gian sinh của LLM TB (s)	6,28	4,05	5,17

Trung thực số học: tỉ lệ con số trong văn bản có mặt trong bằng chứng hoặc câu hỏi.



Hình 6.5. Các chỉ số của tác tử theo miền

Bảng 6.2 và Hình 6.5 cho ba kết luận. Thứ nhất, bộ lập kế hoạch tất định gọi đủ mọi công cụ cần thiết (recall 1,000) và nhận đúng miền ở mọi kịch bản. Precision tài chính dưới 1 vì hai kịch bản (9 và 15) gọi thêm `quy_doi_vang` khi câu hỏi nhắc tới giá vàng mà tập kỳ vọng không có; lời gọi thừa này vô hại và thực ra bổ sung thông tin. Thứ hai, độ phủ dữ kiện của tác tử là 90,62%, so với 15,62% khi LLM trả lời một mình: công cụ nâng độ phủ thêm 75,00 điểm phần trăm. Thứ ba, và đáng bàn nhất, chỉ 12,50% bản nháp của LLM qua kiểm chứng.

Bảng 6.3. Kết quả từng kịch bản

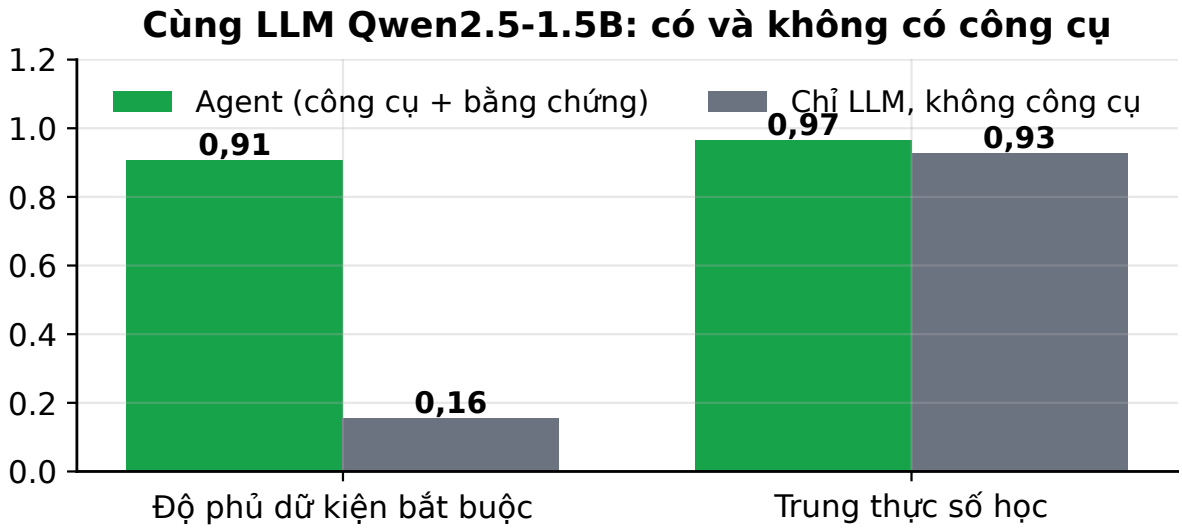
STT	Miền	Câu hỏi	Công cụ đã gọi	Recall	Độ phủ	Chỉ LLM	Trả lời
1	Y tế	Tôi bị tiểu đường típ 2 đang uống Metformin 500mg, tối nay đi tiệc có uống…	KG-YT, RAG	1,00	1,00	0,00	khuôn
2	Y tế	Bị tiểu đường có ăn thịt gà được không?	KG-YT, RAG	1,00	1,00	0,00	khuôn
3	Y tế	Tôi nam 52 tuổi, HbA1c 7.3, đường huyết lúc đói 138, BMI 26.5. Tôi có bị…	MLP, KG-YT, RAG	1,00	1,00	0,00	khuôn
4	Y tế	Tôi nữ 45 tuổi, HbA1c 5.9, đường huyết 110, BMI 24. Tôi có nguy cơ gì và…	MLP, KG-YT, RAG	1,00	0,50	0,00	khuôn
5	Y tế	Người tiểu đường ăn cơm trắng thế nào cho đúng?	KG-YT, RAG	1,00	1,00	0,00	khuôn
6	Y tế	Bệnh nhân suy thận nặng có dùng metformin được không?	KG-YT, RAG	1,00	0,50	0,00	LLM
7	Y tế	Đang tiêm insulin mà bị hạ đường huyết thì xử trí thế nào?	KG-YT, RAG	1,00	1,00	0,00	khuôn
8	Y tế	Tiểu đường kèm suy tim nên ưu tiên thuốc gì?	KG-YT, RAG	1,00	1,00	0,00	LLM
9	TC	Tỷ giá USD/VND tăng mạnh thì cổ phiếu FPT và giá vàng SJC ảnh hưởng thế nào?	Giá, KG-TC, Vàng, RAG	1,00	1,00	0,00	khuôn
10	TC	Giá vàng thế giới quy đổi hôm nay khoảng bao nhiêu triệu đồng một lượng?	Vàng, RAG	1,00	1,00	0,50	khuôn
11	TC	Dự báo giá cổ phiếu FPT phiên tới?	BiLSTM, Giá, RAG	1,00	1,00	0,50	khuôn
12	TC	Tỷ giá tăng thì Hòa Phát bị ảnh hưởng ra sao?	Giá, KG-TC, RAG	1,00	1,00	0,50	khuôn
13	TC	Lãi suất điều hành NHNN giảm thì VCB và VIC hưởng lợi thế nào?	Giá, KG-TC, RAG	1,00	1,00	1,00	khuôn
14	TC	Nghị định 232/2025 thay đổi gì với vàng miếng SJC?	KG-TC, RAG	1,00	1,00	0,00	khuôn
15	TC	Fed tăng lãi suất thì giá vàng thế giới và tỷ giá biến động ra sao?	KG-TC, Vàng, RAG	1,00	0,50	0,00	khuôn
16	TC	Xu hướng giá vàng phiên tới thế nào, có nên mua vàng không?	BiLSTM, Vàng, RAG	1,00	1,00	0,00	khuôn

Recall: tỉ lệ công cụ kỳ vọng đã được gọi. Độ phủ: tỉ lệ dữ kiện bắt buộc có trong câu trả cuối; chỉ LLM: cùng Qwen2.5-1.5B trả lời không có công cụ. Trả lời: bản LLM qua kiểm chứng hay bản tổng hợp theo khuôn.

6.4.3 Vì sao LLM trả lời một mình là nguy hiểm

Độ phủ thấp của đối chứng "chỉ LLM" không chỉ là thiếu thông tin. Đọc từng câu trả lời cho thấy các lỗi có hậu quả thật:

- Kịch bản 1, câu hỏi về bia khi đang dùng metformin: LLM trả lời "Dùng Metformin không ảnh hưởng đến việc uống bia", bỏ qua hoàn toàn nguy cơ nhiễm toan lactic mà Quyết định 5481/QĐ-BYT cảnh báo.
- Kịch bản 8, đái tháo đường kèm suy tim: LLM liệt kê đầu tiên nhóm thiazolidinedione (pioglitazone), là nhóm thuốc gây giữ nước và *chống chỉ định* ở người suy tim, và không nhắc tới nhóm SGLT2 được ưu tiên. Tác tử trả lời đúng "ưu tiên SGLT2 (empagliflozin)" với một trích dẫn tới KG.
- Kịch bản 10, giá vàng quy đổi hôm nay: LLM tự nhận thời điểm là "tháng 10/2023" và trả "645 – 650 USD/ounce, tương đương 21,89 – 22,17 triệu đồng mỗi lượng 1 gram", sai cả mức giá (giá thật 4.125 USD/ounce), cả đơn vị (lượng là 37,5 g). Mô hình không có cách nào biết giá hôm nay; tác tử thì tính đúng 128,69 triệu đồng/lượng từ dữ liệu mới nhất.
- Kịch bản 13, lãi suất điều hành và VCB, VIC: LLM gọi VIC là "ngân hàng thương mại".



Hình 6.6. Độ phủ dữ kiện theo từng kịch bản: tác tử có công cụ so với LLM trả lời một mình

Trung thực số học của đối chứng (0,9271) trông không thấp, nhưng phải đọc đúng cách: 11 trên 16 câu trả lời chỉ LLM không có con số hai chữ số nào, và theo định nghĩa (6.1) một câu trả lời không có con số được tính trung thực tuyệt đối. Khi LLM thật sự đưa ra con số, như ở kịch bản 10, tỉ lệ rơi xuống 0,17. Đây là ví dụ cho thấy một chỉ số trung thực phải được đọc cùng độ phủ, không đọc

riêng.

6.4.4 Vì sao bản nháp LLM bị từ chối

Trong 16 kịch bản, chỉ 2 bản nháp qua kiểm chứng (kịch bản 6 và 8, cả hai đều ngắn, một ý, một trích dẫn). 13 bản nháp bị từ chối vì *không có trích dẫn nào*, dù hầu hết trung thực về số liệu (chỉ một bản dưới 1,0) và nhiều bản đúng về nội dung. Một bản nháp còn lại (kịch bản 7) có ba trích dẫn nhưng một phần ba số con số không tìm thấy trong bằng chứng. Nguyên nhân chính vì vậy không phải LLM bịa số, mà là mô hình 1,5B tham số không tuân thủ yêu cầu định dạng "kèm số hiệu [n]" khi bằng chứng dài và nhiều nguồn.

Bản nháp của Qwen2.5-1.5B cho kịch bản 9 (bị từ chối vì không có trích dẫn)

Khi tỷ giá USD/VND tăng mạnh:
FPT:
- Tăng tích cực (+)
- Doanh thu xuất khẩu phần mềm bằng ngoại tệ, chi phí bằng VND
- Biên lợi nhuận được cải thiện
Giá vàng SJC:
- Tăng tích cực (+)
- Giá quy đổi tăng nhanh hơn chi phí
- Có thể tăng giá trong nước
Giải thích:
- FPT là doanh nghiệp xuất khẩu phần mềm, do tỷ giá USD/VND tăng, doanh thu quy đổi tăng nhanh hơn chi phí, nên biên lợi nhuận được cải thiện.
- Giá vàng miếng SJC cũng tăng do giá quy đổi tăng nhanh hơn chi phí.
- Tuy nhiên, giá vàng miếng trong nước còn phụ thuộc vào các yếu tố khác như giá nguyên vật liệu, công nghệ, thị trường xuất khẩu... nên không chắc chắn tăng mạnh hoàn toàn.

Bản nháp trên minh họa thêm một giới hạn của bộ kiểm chứng: nó không có con số nào nên đạt trung thực số học tuyệt đối, nhưng câu "giá vàng miếng SJC cũng tăng do giá quy đổi tăng nhanh hơn chi phí" đã trộn cơ chế của FPT vào vàng, và câu cuối viện dẫn "giá nguyên vật liệu, công nghệ, thị trường xuất khẩu" không có trong bằng chứng nào. Kiểm chứng con số bắt được lỗi bịa số liệu, nhưng không bắt được lỗi trộn ngữ nghĩa; việc thiếu trích dẫn vô tình chặn được bản nháp này. Một bộ kiểm chứng mạnh hơn cần đối chiếu từng câu với

bằng chứng được trích, chẳng hạn bằng mô hình suy diễn ngôn ngữ tự nhiên (NLI); Chương 7 bàn hướng này.

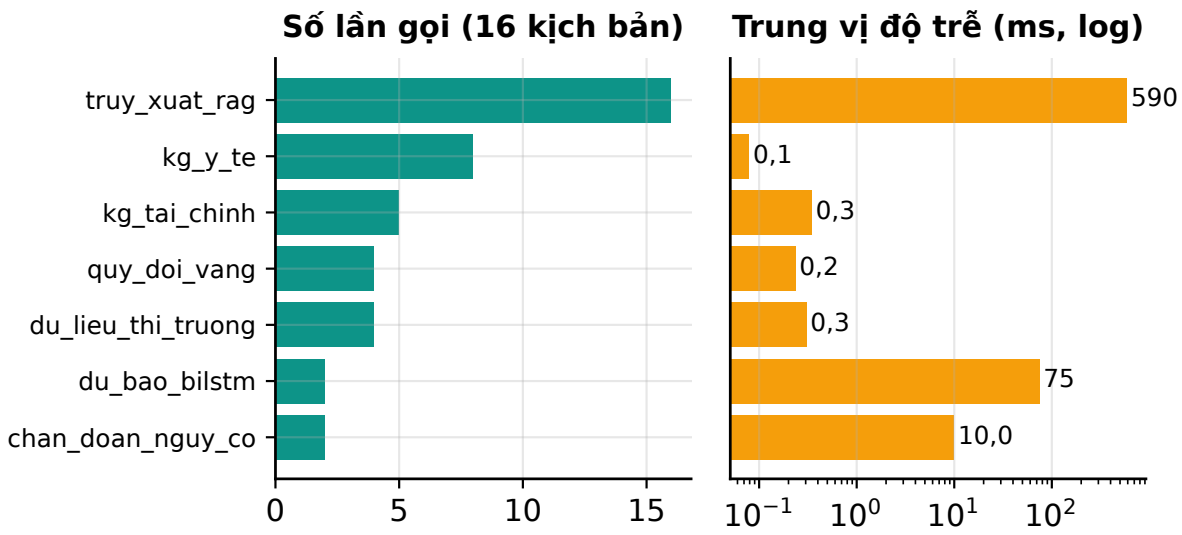
6.4.5 Ba kịch bản chưa đạt độ phủ đầy đủ

Ba kịch bản có độ phủ 0,50, mỗi kịch bản chỉ ra một giới hạn khác nhau:

- **Kịch bản 4** (HbA1c 5,9%): câu trả lời không có cụm "tiền đái tháo đường". KG đã có tri thức cần thiết (cạnh HbA1c_Tien $\xrightarrow{\text{TIEU_CHUAN_CHAN_DOAN}}$ Tien_DTD theo Bảng 4.1), nhưng công cụ `kg_y_te` chỉ tra theo thực thể có tên trong câu hỏi, không đối chiếu chỉ số xét nghiệm của hồ sơ với ngưỡng chẩn đoán. Đây là một bước nối giữa tầng học sâu và tầng đồ thị nên được bổ sung.
- **Kịch bản 6** (suy thận nặng): bản LLM qua kiểm chứng, nói đúng "không nên dùng metformin khi eGFR $<$ 30", nhưng dùng "không nên" thay cho "chống chỉ định", nên bị tính thiếu một dữ kiện. Đây là giới hạn của độ phủ đo bằng so khớp cụm từ.
- **Kịch bản 15** (Fed và tỷ giá): USD_VND là MacroFactor nên bộ lập kế hoạch xếp nó vào biến nguồn, không xét cặp Fed $\rightarrow$ USD/VND. Một biến có thể vừa là nguyên nhân vừa là kết quả; bộ lập kế hoạch cần xét vai trò theo vị trí trong câu, không chỉ theo lớp của thực thể.

6.4.6 Độ trễ

Tổng thời gian trung bình một câu hỏi là 5.782 ms, trong đó LLM chiếm phần lớn (5,167 giây cho bước tổng hợp). Hình 6.7 so sánh các công cụ: truy vấn KG và tính toán chưa tới 1 ms, MLP khoảng 9,992 ms, BiLSTM với 200 mẫu MC Dropout khoảng 74,97 ms, và truy xuất RAG khoảng 589,99 ms vì phải nhúng câu hỏi và chạy bộ xếp hạng lại trên 30 ứng viên. Nếu bỏ LLM và chỉ dùng bản khuôn, tác tử trả lời trong chưa tới một giây.



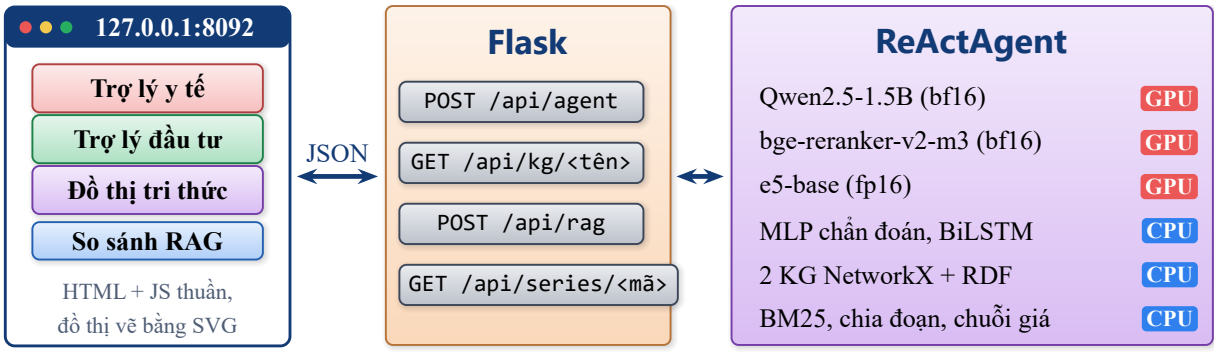
Hình 6.7. Số lần gọi và độ trễ trung vị của từng công cụ

6.5 Ứng dụng Web

6.5.1 Kiến trúc triển khai

Web App gồm một máy chủ Flask và một trang HTML với JavaScript thuần, không dùng thư viện giao diện ngoài; đồ thị tri thức và biểu đồ giá được vẽ bằng SVG ngay trên trình duyệt. Máy chủ nạp tác tử một lần khi khởi động và cung cấp bốn API (Hình 6.8):

- `POST /api/agent`: chạy tác tử, trả vết suy luận, bằng chứng, câu trả lời và kết quả kiểm chứng (Đoạn mã 6.10);
- `GET /api/kg/<tên>`: trả đồ thị kèm tọa độ Kamada-Kawai tính sẵn ở máy chủ (Đoạn mã 6.11);
- `POST /api/rag`: chạy ba bộ truy xuất trên cùng câu hỏi để so sánh;
- `GET /api/series/<mã>`: trả 250 phiên giá gần nhất cho biểu đồ.



Hình 6.8. Kiến trúc triển khai: trình duyệt, máy chủ Flask với bốn API, tác tử và phân bổ CPU/GPU cho từng thành phần

Mọi mô hình học sâu chạy trên GPU trong cùng tiến trình (Bảng 1): LLM bf16, bộ nhúng e5 fp16, bộ xếp hạng lại bf16, MLP và BiLSTM; BM25, KG và các công cụ tính toán chạy trên CPU.

```python
# app/app.py
@app.post("/api/agent")
def api_agent():
    body = request.get_json(force=True)
    profile = {k: body[k] for k in ("HbA1c_level", "blood_glucose_level",
    ↪  "bmi", "age", "gender") if body.get(k)}
    r = AGENT.run(body["question"], profile or None)
    return jsonify({k: r[k] for k in ("question", "domain", "trace",
    ↪  "evidence", "answer", "verification",
                                      "total_ms", "profile")} |
                                      ↪ {"observations":
                                      ↪ _jsonable(r["observations"])})
```

Đoạn mã 6.10. API tác tử: nhận câu hỏi và hồ sơ tùy chọn, trả vết suy luận và câu trả lời đã kiểm chứng

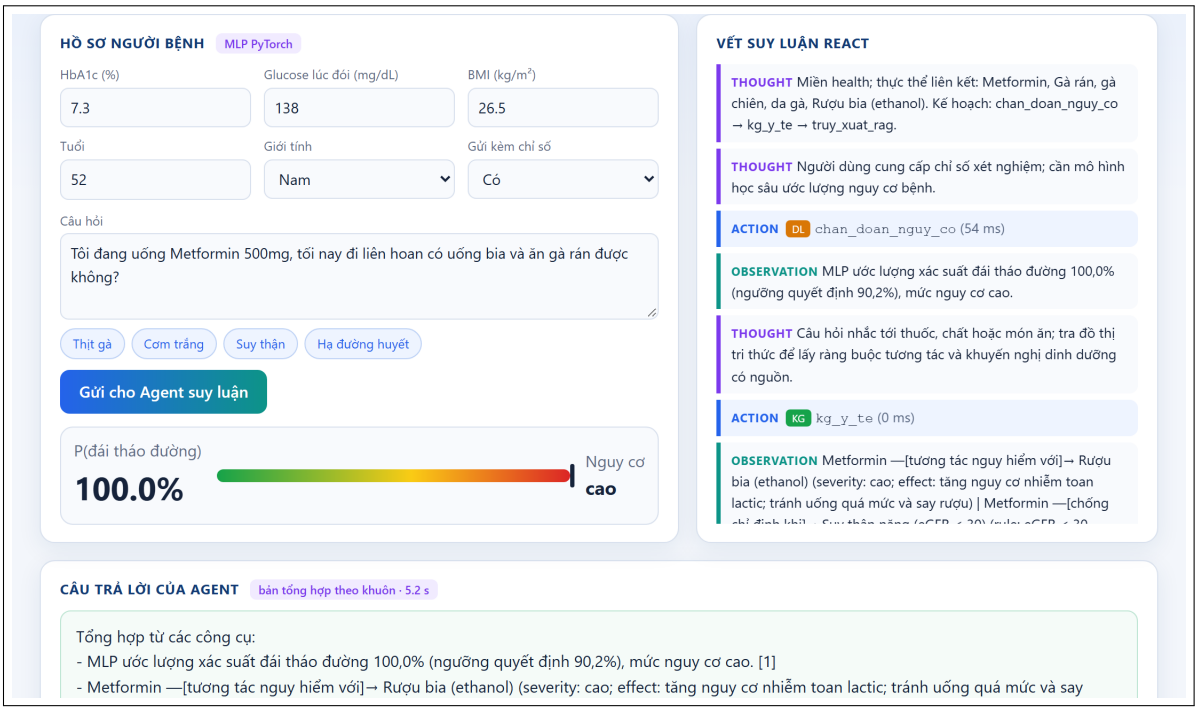
6.5.2 Giao diện

Trang có bốn thẻ: Trợ lý Y tế, Trợ lý Tài chính, Khám phá KG và Tra cứu RAG (Hình 6.9). Hai thẻ trợ lý có cùng bố cục: khung câu hỏi với các câu mẫu, khung vết suy luận tô màu theo loại bước, khung câu trả lời ghi rõ bản LLM hay bản khuôn cùng thời gian xử lý, và danh sách bằng chứng được trích. Thẻ y tế có

```python
# app/app.py
def kg_payload(name):
    """Đồ thị kèm toạ độ tính sẵn (Kamada-Kawai) để trình duyệt vẽ SVG mà không cần thư viện ngoài."""
    if name in LAYOUT:
        return LAYOUT[name]
    kg = AGENT.hkg if name == "health" else AGENT.fkg
    U = nx.Graph(kg.G.to_undirected())
    pos = nx.kamada_kawai_layout(U)
    xs, ys = [p[0] for p in pos.values()], [p[1] for p in pos.values()]
    sx = lambda v: 40 + 920 * (v - min(xs)) / (max(xs) - min(xs))  # noqa: E731
    sy = lambda v: 30 + 540 * (v - min(ys)) / (max(ys) - min(ys))  # noqa: E731
    LAYOUT[name] = {
        "nodes": [{"id": n, "label": d["label"], "type": d["type"], "x": sx(pos[n][0]), "y": sy(pos[n][1]),
                   "deg": U.degree(n)} for n, d in kg.G.nodes(data=True)],
        "edges": [{"s": s, "o": o, "p": p, "label": kg.onto.predicates[p].label, "source": d.get("source"),
                   "text": kg.verbalize((s, p, o, d))} for s, p, o, d in kg.facts()],
        "stats": {k: v for k, v in kg.stats().items() if not isinstance(v, dict)}}
    return LAYOUT[name]
```

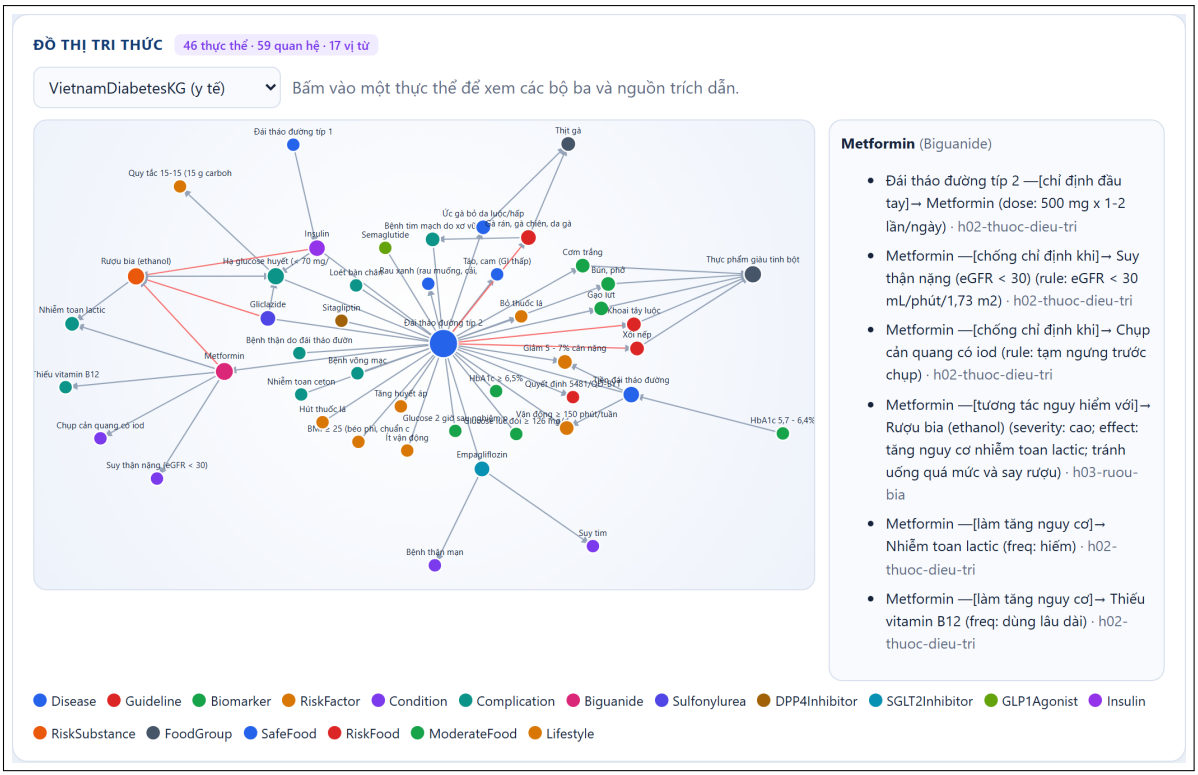
Đoạn mã 6.11. API đồ thị tri thức: bố cục Kamada-Kawai tính sẵn, cạnh diễn đạt thành câu kèm nguồn

thêm biểu mẫu hồ sơ xét nghiệm và đồng hồ nguy cơ; thẻ tài chính có biểu đồ giá 250 phiên. Ảnh chụp của hai thẻ trợ lý đã được trình bày ở Hình 4.15, 4.16, 5.19 và 5.20.

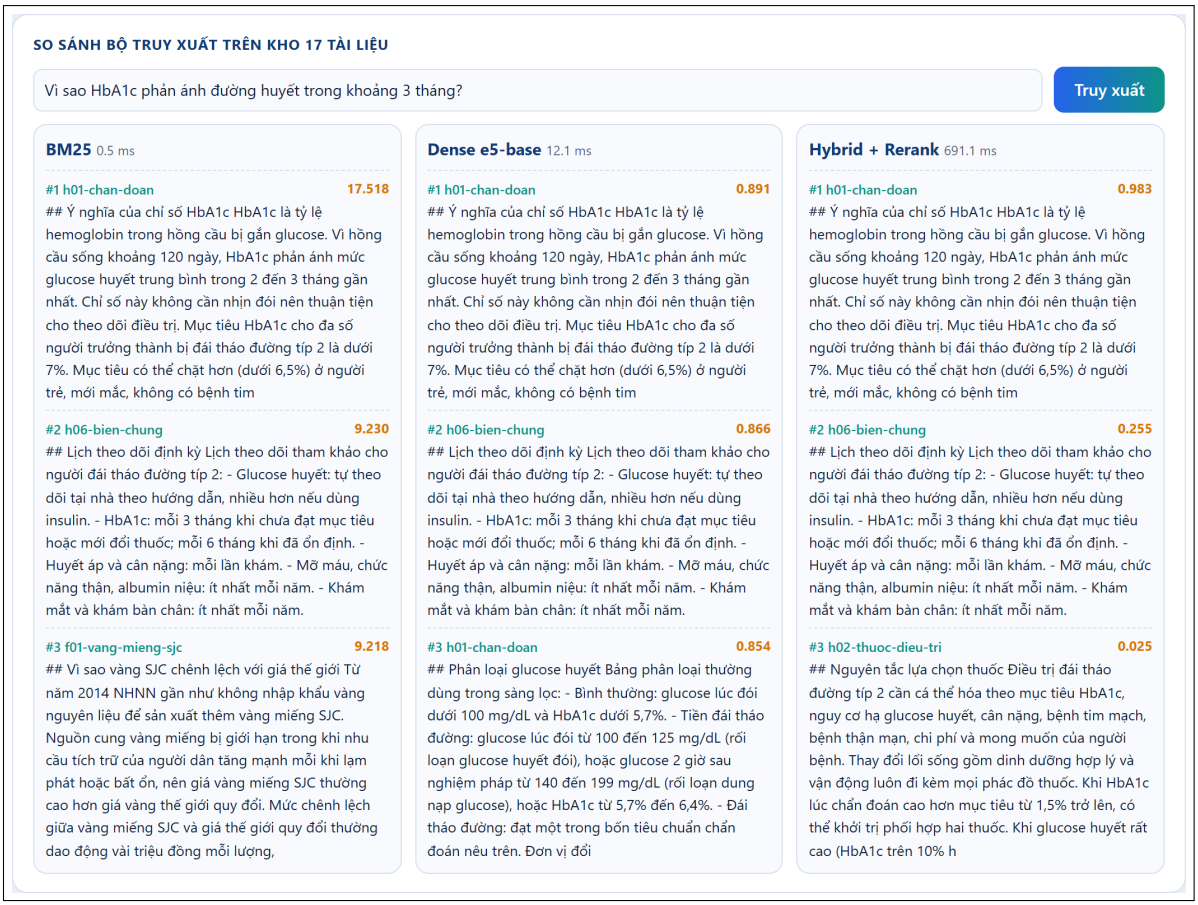


Hình 6.9. Màn hình chính của Web App: bốn thẻ chức năng và trợ lý y tế với hồ sơ xét nghiệm

Thẻ Khám phá KG (Hình 6.10) cho phép chọn một trong hai KG, nhấn vào nút để xem các cạnh vào ra kèm nguồn. Thẻ Tra cứu RAG (Hình 6.11) chạy BM25, Dense e5-base và Hybrid + Rerank trên cùng câu hỏi, hiển thị ba đoạn đứng đầu của mỗi bộ cùng điểm số và thời gian, để người dùng tự thấy kết quả của Chương 3: BM25 nhạy với từ khóa, dense hiểu đồng nghĩa, và kết hợp có xếp hạng lại cho kết quả tốt nhất.



Hình 6.10. Thẻ Khám phá KG với `VietnamDiabetesKG`: nút tô theo lớp, cạnh nguy cơ màu đỏ, khung bên liệt kê các cạnh của nút đang chọn



Hình 6.11. Thẻ Tra cứu RAG: ba bộ truy xuất trên cùng một câu hỏi

6.6 Tổng kết chương

- Tác tử ReAct với bộ lập kế hoạch tất định gọi đủ công cụ ở mọi kịch bản và nâng độ phủ dữ kiện từ 15,62% lên 90,62% so với cùng LLM trả lời một mình.
- LLM trả lời một mình mắc lỗi nguy hiểm ở cả hai miền (khẳng định metformin không ảnh hưởng khi uống bia, đề xuất thuốc chống chỉ định cho người suy tim); công cụ và trích dẫn loại bỏ được các lỗi này.
- Bộ kiểm chứng từ chối phần lớn bản nháp vì thiếu trích dẫn; bản khuôn giữ cho câu trả lời luôn truy vết được, nhưng kiểm chứng con số không bắt được lỗi trộn ngữ nghĩa.
- Ba kịch bản thiếu dữ kiện chỉ ra ba hướng cải tiến cụ thể: luật tiền đái tháo đường trong KG, đo độ phủ theo ngữ nghĩa, và lập kế hoạch theo vai trò của thực thể trong câu.

Chương 7

Kết luận và hướng phát triển

7.1 Kết luận

Tiểu luận đã nghiên cứu đồ thị tri thức, sinh tăng cường truy xuất và tác tử thông minh, rồi hiện thực hóa toàn bộ thành một hệ thống chạy được cho hai bài toán của Việt Nam: tư vấn cho người bệnh đái tháo đường típ 2 và phân tích tác động vĩ mô lên cổ phiếu VN30 và vàng SJC. Mọi con số trong báo cáo được sinh tự động từ kết quả thí nghiệm, và mọi đoạn mã được trích từ chính mã nguồn đã chạy ra các con số đó. Kết quả chính:

1. **Đồ thị tri thức.** Hai KG có bản thể học riêng, ràng buộc miền – miền giá trị và nguồn văn bản cho từng cạnh: `VietnamDiabetesKG` (46 thực thể, 59 quan hệ) và `VietnamFinanceKG` (31 thực thể, 40 quan hệ). Suy diễn RDFS làm 7 truy vấn SPARQL trả đúng kết quả, trong đó có truy vấn chỉ có kết quả *sau* suy diễn. Với câu hỏi có điều kiện số trên dữ liệu bảng, truy vấn KG trả đúng 125/125 hồ sơ, trong khi tìm kiếm văn bản TF-IDF chỉ đưa 2 hồ sơ đúng vào top-20.
2. **Truy xuất.** Trên 114 câu hỏi, chia đoạn theo cấu trúc kết hợp BM25, e5 và xếp hạng lại đạt Hit@1 97,37%, so với 71,05% của chia đoạn cố định với dense thuần. Hai kỹ thuật RAG nâng cao phổ biến lại *làm giảm* chất lượng khi dùng LLM 1,5B: HyDE hạ Hit@1 từ 84,21% xuống 71,05%, Multi-Query xuống 78,07%. GraphRAG cục bộ nâng độ phủ đáp án từ 62,50% lên 81,94%.
3. **Học sâu.** MLP nguy cơ đái tháo đường đạt ROC-AUC 97,37% và PR-AUC 86,65% trên tập kiểm thử. BiLSTM dự báo lợi suất không thắng có ý nghĩa dự báo ngây thơ (5/15 mô hình có tỉ số RMSE dưới 1, tốt nhất chỉ giảm 0,12% sai số), phù hợp giả thuyết thị trường hiệu quả dạng yếu.
4. **Tác tử.** Tác tử ReAct với 7 công cụ gọi đủ công cụ cần thiết ở mọi kịch bản và nâng độ phủ dữ kiện từ 15,62% (LLM trả lời một mình) lên 90,62%, đồng thời loại bỏ các lỗi nguy hiểm của LLM như khẳng định metformin không

ảnh hưởng khi uống bia hay đề xuất thuốc chống chỉ định cho người suy tim.

5. **Ứng dụng.** Web App Flask chạy toàn bộ mô hình cục bộ trên một GPU 8 GB, hiển thị vết suy luận, câu trả lời có trích dẫn, đồ thị tri thức tương tác và so sánh ba bộ truy xuất.

Bài học chung của tiểu luận là các thành phần bổ trợ nhau đúng như lý thuyết dự đoán: KG cho ràng buộc chính xác và suy luận nhiều bước, RAG cho văn bản gốc để giải thích và trích dẫn, mô hình học sâu cho đánh giá định lượng từ dữ liệu của từng người, còn LLM chỉ làm việc nó làm tốt nhất là diễn đạt. Khi LLM được giao thêm việc nhớ tri thức, tự viết truy vấn giả định (HyDE) hay tự chọn công cụ, chất lượng giảm rõ rệt với mô hình nhỏ.

7.2 Hạn chế

- **LLM nhỏ không tuân thủ định dạng trích dẫn.** Chỉ 12,50% bản nháp qua kiểm chứng; 13 bản bị từ chối vì không có số hiệu [n]. Câu trả lời cuối vì vậy thường là bản tổng hợp theo khuôn: đúng và truy vết được, nhưng kém tự nhiên.
- **Bộ kiểm chứng chỉ kiểm con số và trích dẫn.** Nó không phát hiện được câu trộn cơ chế hay viện dẫn ngoài bằng chứng khi câu đó không chứa con số (Mục 6.4).
- **Xác suất của MLP chưa hiệu chỉnh.** Trọng số lớp dương làm xác suất bị đẩy cao so với tần suất thực; con số "xác suất mắc bệnh" chỉ nên đọc như điểm xếp hạng nguy cơ.
- **KG viết tay và nhỏ.** Hai KG có vài chục thực thể, cạnh được viết từ kho văn bản thay vì trích xuất tự động; dấu cạnh tài chính cố định trong khi tương quan thực tế đổi dấu theo giai đoạn, như cạnh USD/VND → FPT chỉ khớp dữ liệu khi xét độ trễ.
- **Bộ lập kế hoạch dựa trên luật.** Nó tái lập được và chính xác trên tập kịch bản, nhưng không thích nghi với câu hỏi ngoài mẫu, và xét vai trò nguyên nhân – kết quả theo lớp của thực thể thay vì theo câu.
- **Quy mô đánh giá.** 114 câu hỏi truy xuất, 12 câu hỏi GraphRAG và 16 kịch bản tác tử đủ để so sánh các phương án trong bài, nhưng chưa đủ để kết luận

có ý nghĩa thống kê cho những chênh lệch nhỏ.

7.3 Hướng phát triển

- **LLM tuân thủ trích dẫn tốt hơn**: dùng mô hình 7B lượng tử hóa 4-bit vừa với GPU 8 GB, hoặc tinh chỉnh LoRA mô hình 1,5B trên các cặp (bằng chứng, câu trả lời có trích dẫn) sinh từ chính bản tổng hợp theo khuôn.
- **Kiểm chứng theo ngữ nghĩa**: đối chiếu từng câu của câu trả lời với bằng chứng được trích bằng một mô hình suy diễn ngôn ngữ tự nhiên (NLI) đa ngữ, thay cho chỉ kiểm con số.
- **Hiệu chỉnh xác suất**: áp dụng temperature scaling hoặc hồi quy isotonic trên tập kiểm định để xác suất của MLP phản ánh đúng tần suất mắc bệnh.
- **Nối tầng học sâu với KG**: đối chiếu chỉ số xét nghiệm của hồ sơ với các ngưỡng chẩn đoán đã có trong KG, để tác tử tự kết luận "tiền đái tháo đường" thay vì chỉ báo điểm nguy cơ.
- **KG tài chính có bằng chứng thực nghiệm**: gắn tương quan, độ trễ và khoảng thời gian hiệu lực vào thuộc tính của từng cạnh, cập nhật định kỳ từ dữ liệu, để tác tử nói "tác động cùng chiều thể hiện sau khoảng hai tháng" thay vì một dấu cố định.
- **Mở rộng KG tự động**: dùng LLM trích xuất bộ ba từ văn bản mới của Bộ Y tế, NHNN và báo cáo doanh nghiệp, kiểm tra bằng ràng buộc bản thể học trước khi nạp, và chuyển lưu trữ từ NetworkX sang cơ sở dữ liệu đồ thị như Neo4j khi quy mô tăng.
- **Bộ lập kế hoạch lai**: giữ luật làm mặc định, nhưng cho LLM đề xuất thêm công cụ khi luật không khớp, với bộ kiểm tra tên và tham số trước khi thực thi.

Tài liệu tham khảo

[1] Aidan Hogan et al. “Knowledge Graphs”. In: *ACM Computing Surveys* 54.4 (2021), 71:1–71:37.

[2] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 33. 2020, pp. 9459–9474.

[3] Tim Berners-Lee, James Hendler, and Ora Lassila. “The Semantic Web”. In: *Scientific American* 284.5 (2001), pp. 34–43.

[4] Ashish Vaswani et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30. 2017.

[5] Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. “Unifying Large Language Models and Knowledge Graphs: A Roadmap”. In: *IEEE Transactions on Knowledge and Data Engineering* 36.7 (2024), pp. 3580–3599.

[6] Shunyu Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *International Conference on Learning Representations (ICLR)*. 2023.

[7] Darren Edge et al. *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*. 2024. arXiv: 2404.16130.

[8] Ziwei Ji et al. “Survey of Hallucination in Natural Language Generation”. In: *ACM Computing Surveys* 55.12 (2023), 248:1–248:38.

[9] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. “Exploring Network Structure, Dynamics, and Function using NetworkX”. In: *Proceedings of the 7th Python in Science Conference (SciPy)*. 2008, pp. 11–15.

[10] Richard Cyganiak, David Wood, and Markus Lanthaler. *RDF 1.1 Concepts and Abstract Syntax*. W3C Recommendation. World Wide Web Consortium (W3C), 2014.

[11] Thomas R. Gruber. “A Translation Approach to Portable Ontology Specifications”. In: *Knowledge Acquisition* 5.2 (1993), pp. 199–220.

[12] W3C OWL Working Group. *OWL 2 Web Ontology Language Document Overview (Second Edition)*. W3C Recommendation. World Wide Web Consortium (W3C), 2012.

[13] Steve Harris and Andy Seaborne. *SPARQL 1.1 Query Language*. W3C Recommendation. World Wide Web Consortium (W3C), 2013.

[14] Mohammed Mustafa. *Diabetes Prediction Dataset*. Bộ dữ liệu công khai trên Kaggle. 2023.

[15] Yunfan Gao et al. *Retrieval-Augmented Generation for Large Language Models: A Survey*. 2023. arXiv: 2312.10997.

[16] Stephen Robertson and Hugo Zaragoza. “The Probabilistic Relevance Framework: BM25 and Beyond”. In: *Foundations and Trends in Information Retrieval* 3.4 (2009), pp. 333–389.

[17] Vladimir Karpukhin et al. “Dense Passage Retrieval for Open-Domain Question Answering”. In: *Proceedings of EMNLP*. 2020, pp. 6769–6781.

[18] Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. *Multilingual E5 Text Embeddings: A Technical Report*. 2024. arXiv: 2402.05672.

[19] Nils Reimers and Iryna Gurevych. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”. In: *Proceedings of EMNLP-IJCNLP*. 2019, pp. 3982–3992.

[20] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods”. In: *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2009, pp. 758–759.

[21] Rodrigo Nogueira and Kyunghyun Cho. *Passage Re-ranking with BERT*. 2019. arXiv: 1901.04085.

[22] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. *BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation*. 2024. arXiv: 2402.03216.

[23] Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. "Precise Zero-Shot Dense Retrieval without Relevance Labels". In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*. 2023, pp. 1762–1777.

[24] Vincent D. Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. "Fast Unfolding of Communities in Large Networks". In: *Journal of Statistical Mechanics: Theory and Experiment* 2008.10 (2008), P10008.

[25] International Diabetes Federation. *IDF Diabetes Atlas*. 10th ed. Brussels: International Diabetes Federation, 2021.

[26] Bộ Y tế. *Quyết định số 5481/QĐ-BYT ngày 30/12/2020 ban hành tài liệu chuyên môn "Hướng dẫn chẩn đoán và điều trị đái tháo đường típ 2"*. Hà Nội. 2020.

[27] American Diabetes Association Professional Practice Committee. "Standards of Care in Diabetes—2024". In: *Diabetes Care* 47.Supplement 1 (2024).

[28] Diabetes Prevention Program Research Group. "Reduction in the Incidence of Type 2 Diabetes with Lifestyle Intervention or Metformin". In: *New England Journal of Medicine* 346.6 (2002), pp. 393–403.

[29] Fiona S. Atkinson, Kaye Foster-Powell, and Jennie C. Brand-Miller. "International Tables of Glycemic Index and Glycemic Load Values: 2008". In: *Diabetes Care* 31.12 (2008), pp. 2281–2283.

[30] Chính phủ nước Cộng hòa xã hội chủ nghĩa Việt Nam. *Nghị định số 24/2012/NĐ-CP ngày 03/4/2012 về quản lý hoạt động kinh doanh vàng*. Hà Nội. 2012.

[31] Chính phủ nước Cộng hòa xã hội chủ nghĩa Việt Nam. *Nghị định số 232/2025/NĐ-CP sửa đổi, bổ sung một số điều của Nghị định số 24/2012/NĐ-CP về quản lý hoạt động kinh doanh vàng*. Hà Nội. 2025.

[32] Sepp Hochreiter and Jürgen Schmidhuber. “Long Short-Term Memory”. In: *Neural Computation* 9.8 (1997), pp. 1735–1780.

[33] Mike Schuster and Kuldip K. Paliwal. “Bidirectional Recurrent Neural Networks”. In: *IEEE Transactions on Signal Processing* 45.11 (1997), pp. 2673–2681.

[34] Peter J. Huber. “Robust Estimation of a Location Parameter”. In: *The Annals of Mathematical Statistics* 35.1 (1964), pp. 73–101.

[35] Ilya Loshchilov and Frank Hutter. “Decoupled Weight Decay Regularization”. In: *International Conference on Learning Representations (ICLR)*. 2019.

[36] Eugene F. Fama. “Efficient Capital Markets: A Review of Theory and Empirical Work”. In: *The Journal of Finance* 25.2 (1970), pp. 383–417.

[37] Yarin Gal and Zoubin Ghahramani. “Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning”. In: *Proceedings of the 33rd International Conference on Machine Learning (ICML)*. 2016, pp. 1050–1059.

[38] Jason Wei et al. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 35. 2022.

[39] Timo Schick et al. “Toolformer: Language Models Can Teach Themselves to Use Tools”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 36. 2023.

[40] Qwen Team. *Qwen2.5 Technical Report*. 2024. arXiv: 2412.15115.